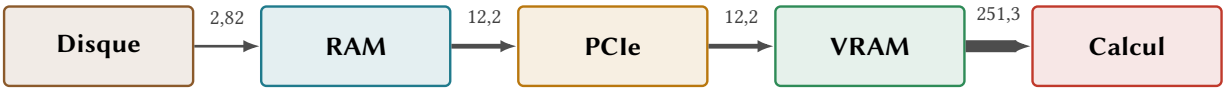
troncons

Faire tourner un modèle bien plus grand que la mémoire : compresser à l'extrême, décompresser à l'instant exact du besoin

Mémoire de recherche — parties I et II : les fondations mesurées



Débits mesurés sur la machine de Stone, en Go/s (disque : lectures aléatoires de 16 Mio ; PCIe : DMA hôte vers carte ; VRAM : lecture par shader). L'épaisseur des flèches croît avec le débit.

Stone

Toronto, le 15 octobre 2026

Version de travail. Les chapitres d'architecture, d'implémentation et de résultats sont à venir ; chaque figure se régénère depuis les données de mesure commitées dans le dépôt `troncons`.

Résumé

Le but de *troncons* est de faire tourner, sur un portable ordinaire (16 Go de RAM, une RTX 4060 de 8 Go, un disque NVMe de 1 To), un modèle de langue plusieurs fois plus gros que toute sa mémoire, proprement : des poids stockés compressés à l'extrême sur le disque, décompressés seulement pour le calcul qui en a besoin, à l'instant exact où il en a besoin, puis libérés. Au pied de la lettre, « sans RAM » est impossible ; la lecture retenue est que *la mémoire occupée ne dépend plus de la taille du modèle*, seulement d'une fenêtre de travail bornée.

Cette première partie pose les fondations mesurées. Chaque étage entre le disque et les unités de calcul a été mesuré sur la machine cible, avec des outils écrits pour cela, chacun avec un témoin qu'on a forcé à échouer : le disque livre 2,82 Go/s (47 % de son annonce, parce que le volume est chiffré par BitLocker derrière Intel RST VMD), le lien hôte-carte 12,2 Go/s, la mémoire vidéo 251,3 Go/s. On en démontre le toit du problème, $t \geq \max_i O_i/W_i$, et ses corollaires : un facteur de croisement $r^\star = 4{,}33$ au-delà duquel la compression est perdue si le processeur décompresse, et aucun croisement si la carte graphique décompresse (architecture B).

Quatre leviers sont ensuite étudiés et mesurés. *La quantification extrême* : sur la source gaussienne, un treillis à décalage atteint une erreur quadratique de 0,068 à 2 bits et ferme 90 % de l'écart entre le scalaire optimal (0,1175) et la borne de Shannon (0,0625) ; nous avons écrit et mesuré le premier décodeur de treillis portable (Vulkan, 124 Go/s équivalents), et démontré une règle de partage des bits selon l'étage de mémoire (avec les débits mesurés, une matrice résidente en VRAM mérite 2,18 bits de plus qu'une matrice lue par le lien, 3,24 de plus qu'une matrice lue du disque). *Le chargement à la demande* : pour un mélange d'experts, le taux de succès du cache décide de tout ; on dérive le succès qu'implique toute mesure publiée. *Le décodage spéculatif* : on démontre $\mathscr{E}(\alpha,k) = (1-\alpha^{k+1})/(1-\alpha)$ jetons par lecture des poids, et que pour des experts fins lus du disque la spéculation ne paie que si $\mathscr{E}(\alpha,k) > U(k+1)/k_e$, ce qu'aucun routage indépendant ne satisfait. *Les modèles visés* : 49 modèles ouverts inventoriés dans leurs en-têtes ; gpt-oss-120b (63,4 Go, MXFP4 natif) est la première cible « ultra lourde », téléchargée et vérifiée.

Le codage sans perte les complète. Les codes de 4 bits des modèles de la machine portent 3,86 bits sur 4, ce qu'un calcul exact retrouve au centième pour toute source en cloche sous un min-max par petits groupes, ce qui réfute pour ces formats les « 0,6 à 1 bit » publiés. Le gain du MXFP4 est dans ses échelles : avec leurs échelles codées par la médiane de leur ligne, et décodés exactement, les experts de gpt-oss-120b atteignent un taux de 1,1026 là où zstd prend 1,04. Notre décodeur rANS entrelacé, démontré exact, tourne à 70 Go/s sous Vulkan et 82 sous CUDA ; fondu au produit matrice-vecteur, il est 3,7 fois plus lent : un tronçon se décode donc une fois, à son arrivée en mémoire vidéo, et rend tout son taux au disque.

Une seule équation contient ensuite les quatre leviers : le temps d'un jeton d'un mélange d'experts servi par la carte, étage par étage, sous un modèle de localité à une constante. Ses élasticités, démontrées, disent que sur cette machine le disque tient le maximum (157 ms par jeton contre 58 pour le lien et 36 pour la carte, gpt-oss-120b) : chaque bit retiré et chaque octet de débit gagné valent presque un pour un, et des noyaux plus rapides rapportent moins de 2 %. Trois architectures candidates y sont calculées. A, exacte, sert le modèle de 63,4 Go avec 1,9 Go de RAM de pointe. B repose sur un mécanisme neuf, le raffinement exact d'un code 4 bits natif en une base et un complément, qui ne

coûte rien en entropie (démontré) et dont la base perd 0,27 bit de moins que la troncature à erreur égale (mesuré) ; compter les compléments a fait barrer un gain de ×1,42 annoncé plus tôt. L'arbitre a décidé de construire A d'abord, dans un format qui prépare B. Les premières mesures sur gpt-oss-20b donnent une réutilisation des experts d'un jeton au suivant $\rho = 0{,}577$, sous laquelle la spéculation paierait dès un taux d'acceptation de 0,42.

Le mémoire existe en deux versions complètes, anglaise et française ; chaque figure se régénère depuis les données de mesure commitées.

Abstract

The aim of *troncons* is to run, on one ordinary laptop (16 GB of RAM, an 8 GB RTX 4060, a 1 TB NVMe disk), a language model several times larger than all of its memory, cleanly: weights stored compressed to the extreme on disk, decompressed only for the computation that needs them, at the exact moment it needs them, then released. Read literally, « without RAM » is impossible; the reading retained here is that *the memory in use no longer depends on the size of the model*, only on a bounded working window.

This first part lays the measured foundations. Every stage between the disk and the compute units was measured on the target machine with tools written for the purpose, each with a control case that was forced to fail: the disk delivers 2,82 Go/s (47 % of its advertised rate, because the volume is encrypted by BitLocker behind Intel RST VMD), the host-to-device link 12,2 Go/s, the video memory 251,3 Go/s. From these we prove the roof of the problem, $t \geq \max_i O_i/W_i$, and its corollaries: a crossing factor $r^\star = 4.33$ beyond which compression is wasted if the processor decompresses, and no crossing at all if the graphics card decompresses (architecture B).

Four levers are then studied and measured. *Extreme quantization*: on the Gaussian source, a shift-register trellis reaches a mean squared error of 0.068 at 2 bits, closing 90 % of the gap between the optimal scalar quantizer (0.1175) and the Shannon bound (0.0625); we wrote and measured the first portable trellis decoder (Vulkan, 124 Go/s equivalent), and proved a bit-allocation rule by memory stage (with the measured throughputs, a VRAM-resident matrix deserves 2.18 more bits than one read through the link, 3.24 more than one read from disk). *On-demand loading*: what decides everything for a mixture of experts is the cache hit rate; we derive the hit rate implied by any published measurement. *Speculative decoding*: we prove $\mathscr{E}(\alpha,k) = (1-\alpha^{k+1})/(1-\alpha)$ tokens per read of the weights, and that for fine-grained experts read from disk speculation pays only if $\mathscr{E}(\alpha,k) > U(k+1)/k_e$, which no independent routing satisfies. *The target models*: 49 open models inventoried from their headers; gpt-oss-120b (63.4 GB, native MXFP4) is the first « ultra heavy » target, now downloaded and verified.

Lossless coding completes them. The 4-bit codes of the machine's models carry 3.86 bits out of 4, which an exact calculation recovers to the hundredth from any bell-shaped source under a small-group min-max, refuting for these formats the published "0.6 to 1 bit". The gain of MXFP4 lies in its scales: with those coded by the median of their row, and decoded exactly, the experts of gpt-oss-120b reach a rate of 1.1026 where zstd takes 1.04. Our interleaved rANS decoder, proved correct, runs at 70 Go/s under Vulkan and 82 under CUDA; fused into the matrix–vector product it is 3.7 times slower, so a chunk is decoded once, on arrival in video memory, and gives its whole rate back to the disk.

One equation then contains the four levers: the time of a token of a mixture of experts served by the GPU, stage by stage, under a one-constant locality model. Its proven elasticities say that on this machine the disk holds the maximum (157 ms per token against 58 for the link and 36 for the GPU, gpt-oss-120b): every bit removed and every byte of throughput gained is worth almost one for one, and faster GPU kernels return under 2 %. Three candidate architectures are computed on it. A, exact, serves the 63.4 GB model with 1.9 GB of peak RAM. B rests on a new mechanism, the exact refinement of a native 4-bit code into a base and a complement, which costs nothing in entropy (proven) and whose base loses 0.27 bit less than truncation at equal error (measured); counting the complements struck out an earlier ×1.42 gain. The arbiter decided to build A first, in a format that

prepares B. First measurements on gpt-oss-20b show a token-to-token expert reuse $\rho = 0.577$, under which speculation would pay from an acceptance rate of 0.42.

The thesis exists in two complete versions, English and French; every figure is regenerated from the committed measurement data.

Table des matières

Résumé iii

Abstract v

Table des figures xiii

Liste des tableaux xv

Liste des listings xvii

Notations xix

Première partie Le problème 1

1 Introduction et motivation 3
1.1 La mission . . . 3
1.2 D'où l'on part . . . 3
1.3 Ce que veut dire « sans RAM ni rien » . . . 4
1.4 La question de recherche . . . 4
1.5 Ce que cette première partie apporte . . . 5
1.6 La méthode . . . 5
1.7 Plan du mémoire . . . 6

2 État de l'art 7
2.1 Le mur de la mémoire, au lot de un . . . 7
2.2 La fenêtre de travail : une idée de 1968 . . . 8
2.3 Faire tourner un modèle plus gros que la mémoire . . . 9
2.4 La quantification extrême . . . 9
2.5 La compression sans perte . . . 10
2.6 La spéculation . . . 10
2.7 Ce que chaque famille change dans le toit . . . 10
2.8 Positionnement . . . 10

3 La machine et sa physique 13
3.1 La machine . . . 13
3.2 La méthode de mesure . . . 14
3.3 Le disque . . . 14
3.3.1 L'instrument . . . 14
3.3.2 Les résultats . . . 15
3.3.3 Pourquoi si loin de l'annonce . . . 16
3.3.4 Le coût de BitLocker, mesuré par un second chemin . . . 16

3.3.5 IoRing, BypassIO, DirectStorage . . . 16
3.4 La RAM . . . 17
3.5 De l'hôte à la carte . . . 18
3.5.1 L'instrument, et le piège que le témoin a levé . . . 18
3.5.2 Les résultats . . . 18
3.6 La carte . . . 19
3.6.1 Thermique, et la découverte de la batterie . . . 20
3.7 La base de comparaison : llama.cpp . . . 21
3.8 Le toit . . . 22
3.8.1 Le modèle . . . 22
3.8.2 Le théorème du toit . . . 22
3.8.3 Les deux corollaires qui décident de l'architecture . . . 23
3.8.4 Les plafonds par étage . . . 24
3.8.5 Le budget du décodeur, et pourquoi B . . . 24
3.9 Ce qui manque dans le monde . . . 25

4 Formulation du problème 31
4.1 Le jeton comme chaîne d'étages . . . 31
4.2 Les termes pour un mélange d'experts servi par la carte . . . 32
4.3 La localité, en une constante . . . 32
4.4 Les chiffres de la machine . . . 34
4.5 Le problème, posé . . . 34
4.6 Ce qui tient le maximum : les élasticités . . . 35
4.7 Un exemple posé pas à pas . . . 36
4.8 La fenêtre de travail, chiffrée . . . 36
4.9 Le terme qui manquait à la spéculation . . . 37
4.10 Les témoins de l'équation . . . 38

Deuxième partie Les leviers, mesurés 41

5 Quantification extrême 43
5.1 Le levier . . . 43
5.2 Les formats, au bit près . . . 44
5.3 La source gaussienne et sa borne . . . 44
5.4 Le quantificateur scalaire optimal . . . 46
5.5 Le banc théorique, refait ici . . . 47
5.6 Le treillis à décalage . . . 48
5.7 Le coût du décodage sur la carte . . . 50
5.7.1 En CUDA . . . 50
5.7.2 En Vulkan et en DirectX 12 : le premier treillis portable mesuré . . . 50
5.7.3 Un treillis sans multiplication n'est pas plus rapide . . . 52
5.8 L'échelle de llama.cpp, mesurée sur Qwen3.5 . . . 52
5.9 À octets lus égaux : le meilleur rapport d'aujourd'hui . . . 53
5.10 Le partage des bits selon l'étage . . . 54
5.11 Les techniques, en une table . . . 57
5.12 Ce qui manque dans le monde . . . 57

6 Compression sans perte, à la vitesse du calcul 59
6.1 Le juge : quand le sans-perte paie . . . 59
6.2 Ce que contient un code quantifié . . . 60
6.2.1 L'estimateur, et son biais . . . 60
6.2.2 Les codes : presque uniformes, et l'ordre 0 dit tout . . . 61
6.2.3 Les métadonnées, et ce que voit un codeur d'octets . . . 61
6.3 Pourquoi si peu : l'entropie d'un min-max . . . 62
6.3.1 Une contradiction publiée, et une phrase corrigée . . . 64
6.4 MXFP4 : le gain est dans les échelles . . . 65
6.5 Le codeur rANS entrelacé . . . 67
6.5.1 Le format . . . 68
6.5.2 Exactitude . . . 69
6.5.3 Le cœur de la v4 . . . 71
6.6 Quatre décodeurs, et ce que chacun a appris . . . 72
6.7 La question clé : décoder dans le produit, ou juste avant ? . . . 74
6.8 Au-delà du sans-perte : une quantification taillée pour le codeur . . . 76
6.9 L'état de l'art, technique par technique . . . 78
6.10 Ce qui manque dans le monde . . . 78

7 Chargement à la demande 81
7.1 La loi du chargement . . . 81
7.2 Le mélange d'experts en trois grandeurs . . . 81
7.3 La granularité . . . 82
7.4 Les modèles denses . . . 83
7.5 Les mélanges d'experts . . . 83
7.6 La localité des experts d'un jeton au suivant . . . 85
7.7 Les pipelines à double tampon . . . 85
7.8 Le juge appliqué à la machine . . . 87
7.9 Ce qui manque dans le monde . . . 88

8 Sparsité et décodage spéculatif 91
8.1 Le juge, et le levier qui a la bonne pente . . . 91
8.2 Le décodage spéculatif . . . 92
8.2.1 La règle de vérification . . . 92
8.2.2 Le théorème d'amortissement . . . 93
8.2.3 Les arbres . . . 93
8.2.4 Les variantes . . . 94
8.3 Mélanges d'experts et spéculation . . . 94
8.4 La sparsité des activations . . . 96
8.4.1 La mesure : unions, fenêtre, neurones chauds . . . 97
8.4.2 Un masque commun au bloc vérifié . . . 97
8.5 Le calcul pour la machine de Stone . . . 99
8.5.1 Les taux d'acceptation, mesurés . . . 99
8.5.2 Le substitut résident, transposé . . . 99
8.5.3 Les modèles denses lus du disque . . . 99
8.6 Ce qui manque dans le monde . . . 100

9 Les modèles visés 101
9.1 Ce que « ultra lourd » veut dire ici . . . 101
9.2 L'inventaire, lu dans les en-têtes . . . 101
9.3 Octets lus par jeton . . . 102
9.4 Le juge physique . . . 104
9.5 Ce qui a changé en 2026 et sert troncons . . . 104
9.6 Le protocole de jugement . . . 105
9.7 Les téléchargements . . . 106
9.8 Ce qui manque dans le monde . . . 106

Troisième partie Vers le système 109

10 Architecture : les candidates, la décision, ce qui vient 111
10.1 Trois angles . . . 111
10.2 A : tout par la carte, octets codés de bout en bout, sans perte . . . 111
10.3 B : la précision selon l'étage . . . 113
10.3.1 Le mécanisme . . . 113
10.3.2 La mesure sur les poids réels . . . 115
10.3.3 Le chiffre qu'il a fallu barrer . . . 115
10.3.4 Où B gagne vraiment : B3 et le bouton λ . . . 116
10.4 C : lire moins souvent . . . 116
10.5 Le débat et la décision . . . 117
10.6 Les premières mesures du modèle visé . . . 119
10.7 Ce qui vient . . . 120

11 Discussion et limites 123
11.1 Ce que la première partie établit . . . 123
11.2 Ce qui est solide, ce qui est fragile . . . 123
11.3 Les idées tuées par les nombres . . . 124
11.4 Les chiffres barrés . . . 124
11.5 Les limites de la méthode . . . 124
11.6 La lecture de « sans RAM ni rien » . . . 125

12 Travaux futurs 127
12.1 Les mesures qui décident, d'abord . . . 127
12.2 Le jalon J1 : construire A . . . 127
12.3 Les questions de recherche . . . 128
12.4 Le geste qui revient à Stone . . . 128
12.5 Les téléchargements possibles . . . 128
12.6 Le mémoire lui-même . . . 129

A Protocoles de mesure 131
A.1 Les étiquettes . . . 131
A.2 Le témoin, et sa cassure . . . 131
A.3 Le seuil déclaré avant . . . 131
A.4 Les séries ABBA et le bruit . . . 132
A.5 Le second chemin . . . 132

A.6 L'alimentation et la machine partagée 132
A.7 Le juge de qualité 132
A.8 La phrase barrée 132

B Outils 133
B.1 La chaîne du mémoire 133
B.2 Les outils des études 133

C Journaux et reproduction des figures 139
C.1 Reconstruire le mémoire 139
C.2 Les journaux 139
C.3 Les données de chaque graphe 140

Bibliographie 143

Table des figures

2.1 Chronologie des travaux, par famille . . . 7

3.1 Débit du disque selon la taille de bloc et la profondeur de file . . . 15
3.2 Déchiffrement XTS-AES selon le nombre de fils . . . 17
3.3 Latence d'un accès dépendant selon la taille du tampon . . . 18
3.4 Génération du lien PCIe pendant des copies . . . 19
3.5 Produits matrice-vecteur écrits ici, sur la RTX 4060 . . . 20
3.6 Cinq minutes de GEMV, secteur contre batterie . . . 21
3.7 Le chemin des données, du disque aux unités de calcul . . . 27
3.8 Plafond calculé contre débit mesuré, par étage . . . 27
3.9 Architectures A et B . . . 28
3.10 Chronogramme : en série contre recouvert . . . 28
3.11 Le toit du Qwen3.5-9B entièrement sur disque, selon r . . . 29
3.12 Le toit des trois MoE visés, placement réel . . . 29

4.1 Les trois scénarios de localité . . . 33
4.2 Le plafond selon la part lue du disque . . . 37
4.3 La fenêtre de travail : plafond selon la RAM . . . 38

5.1 Les formats flottants, au bit près . . . 44
5.2 Le bloc MXFP4 . . . 44
5.3 Le super-bloc Q4_K . . . 45
5.4 Le super-bloc IQ2_XXS . . . 45
5.5 Distorsion contre débit sur la source gaussienne . . . 48
5.6 Le treillis à décalage et son décodeur calculé . . . 49
5.7 Le décodage du treillis en Vulkan et en DirectX 12 . . . 51
5.8 Perplexité contre octets lus par jeton, Qwen3.5 4B et 9B . . . 54
5.9 Vitesse de décodage contre octets lus par jeton . . . 55
5.10 À octets égaux : Llama 2 en treillis . . . 56
5.11 À octets égaux : Qwen3 au MMLU . . . 57

6.1 La loi des codes de 4 bits, Q4_K et MXFP4 . . . 62
6.2 Entropie des codes d'un min-max selon la taille du groupe . . . 63
6.3 Entropie des codes selon la plage de quantification . . . 64
6.4 Le codage des échelles par la médiane de ligne . . . 65
6.5 Taux sans perte réel de chaque tenseur d'experts . . . 67
6.6 Entropie des échelles E8M0, tenseur par tenseur . . . 68
6.7 La tuile du codeur rANS entrelacé . . . 69
6.8 L'entrée de la table de décodage, au bit près . . . 69
6.9 Le flux partagé : vote et comptage de bits . . . 70
6.10 Débit des décodeurs rANS sur la RTX 4060 Laptop . . . 73

6.11 Le produit MXFP4 brut, fondu au décodeur, ou en deux passes . . . 75
6.12 Où décoder : à l'arrivée du tronçon . . . 76
6.13 À erreur égale, ce que demande une ECSQ . . . 77

7.1 Une couche de mélange d'experts et sa hiérarchie de cache . . . 82
7.2 Succès des politiques de cache sur un routage synthétique . . . 86
7.3 Le succès du cache exigé, modèle par modèle . . . 87
7.4 Le plafond de gpt-oss-120b selon le succès du cache . . . 89
7.5 Les quatre réponses à un défaut de cache . . . 90

8.1 Le temps d'une passe de n jetons . . . 92
8.2 Jetons produits par lecture des poids, $\mathscr{E}(\alpha,k)$ . . . 94
8.3 Vérification en chaîne et en arbre . . . 95
8.4 L'union des masques de n jetons consécutifs . . . 98

9.1 Paramètres totaux et actifs des mélanges d'experts de 2026 . . . 102

10.1 Plafonds des architectures candidates . . . 112
10.2 Le chemin des octets de l'architecture A . . . 113
10.3 Chronogramme d'un jeton dans A . . . 113
10.4 La base et le complément d'un code E2M1 . . . 114
10.5 B3 : le débit contre l'exactitude . . . 116
10.6 Mesure F1 : bits par poids de chaque variante du format . . . 118
10.7 Un tronçon du format .trc . . . 119
10.8 Base llama.cpp sur gpt-oss-20b . . . 120
10.9 Localité mesurée sur gpt-oss-20b contre les hypothèses . . . 121

Liste des tableaux

2.1 Trois façons de n'occuper que ce qui sert . . . 8
2.2 Ce que chaque famille change dans l'inégalité du toit . . . 11
2.3 Ce que les systèmes existants couvrent du problème de troncons . . . 12

3.1 La machine de Stone, relevé du 2026-10-15 . . . 13
3.2 Débit du disque, extrait du balayage ABBA . . . 15
3.3 Transferts hôte ↔ carte, 64 Mio . . . 19
3.4 llama.cpp sur Qwen3.5-9B Q4_K_M, premier passage . . . 22
3.5 Plafonds de jetons/s par étage . . . 24

4.1 Les grandeurs de l'équation du jeton . . . 34
4.2 Ce que vaut chaque grandeur . . . 36
4.3 Les témoins de l'équation du jeton . . . 39

5.1 Plafonds selon le nombre de bits par poids . . . 43
5.2 Bits par poids, format par format . . . 46
5.3 Erreur quadratique à 2 bits sur la source gaussienne . . . 47
5.4 GEMV en CUDA, quatre formats . . . 50
5.5 L'échelle de llama.cpp sur Qwen3.5-4B et 9B . . . 53
5.6 Les techniques de quantification extrême . . . 58

6.1 Entropie des blocs K-quant de Qwen3.5-9B . . . 61
6.2 Ce que voit un codeur d'octets sur un bloc K . . . 62
6.3 Entropie des échelles E8M0 selon le contexte . . . 65
6.4 Taux sans perte réel de MXFP4, décodage exact . . . 66
6.5 Le décodeur v4 sous CUDA et sous Vulkan, en ABBA . . . 72
6.6 La question clé, mesurée . . . 75
6.7 Les codeurs sans perte de poids, au regard de la mission . . . 78

7.1 La localité lue dans des mesures publiées . . . 82
7.2 Prédire et précharger les experts . . . 84
7.3 Les politiques de cache d'experts . . . 84
7.4 La localité mesurée des experts . . . 85
7.5 Le juge sur la machine, disque à 2,4 puis 2,82 Go/s . . . 88

8.1 Valeurs de $\mathscr{E}(\alpha,k)$ . . . 95
8.2 Les variantes du décodage spéculatif . . . 96
8.3 Spéculation et MoE lus du disque . . . 97
8.4 Les méthodes de sparsité des activations . . . 97
8.5 Le coût en qualité d'un masque commun . . . 98
8.6 Taux d'acceptation gloutons mesurés . . . 99
8.7 Spéculation et modèles denses lus du disque . . . 100

9.1 Les mélanges d'experts visés, lus dans leurs en-têtes . . . 103
9.2 Octets lus par jeton : en-têtes contre formule . . . 103
9.3 Les fichiers candidats et leurs plafonds . . . 104

10.1 Plafonds de A . . . 114
10.2 Couches de base d'un code E2M1 . . . 115
10.3 Les jalons décidés . . . 117

11.1 Les idées tuées par les nombres . . . 124
11.2 Les chiffres barrés . . . 125

12.1 Les questions de recherche ouvertes . . . 128
12.2 Les téléchargements qui pourraient servir . . . 129

Liste des listings

5.1 Le Viterbi vectorisé du treillis à décalage 49
5.2 Le décodeur mul1 en WGSL 51

6.1 Boucle intérieure du décodeur rANS v4 (WGSL) 71

Notations

Les unités sont décimales : 1 Go = 10^9 octets, 1 Go/s = 10^9 octets par seconde ; les multiples binaires s'écrivent Kio, Mio, Gio. Chaque nombre du mémoire porte sa provenance : [mes.] mesuré ici sur la machine de Stone, par un outil et un journal commités ; [calc.] calculé à partir de nombres nommés ; [litt.] littérature (chiffre d'éditeur, non reproduit).

Symbole	Signification
Débit et toit	
B	octets de poids lus par jeton décodé (lot de un)
W_i	débit effectif de l'étage i (disque, RAM, PCIe, VRAM), octets/s
O_i	octets qui doivent traverser l'étage i pour un jeton
f_V, f_R, f_D	fractions des poids résidant en VRAM, en RAM, sur le disque ($f_V + f_R + f_D = 1$)
r	facteur de compression des poids stockés (octets servis / octets stockés)
$r^\star$	facteur de croisement $W_{\text{PCIe}}/W_{\text{disque}}$
t	temps par jeton, s
Quantification	
N	nombre de paramètres (actifs) lus par jeton
b	bits par poids, métadonnées comprises
R	débit, bits par échantillon (par poids)
$D(R)$	fonction distorsion-débit (erreur quadratique moyenne au débit R)
σ^2	variance de la source
$\Delta(q)$	distance en bits du quantificateur q à la borne de Shannon
L, k	largeur d'état et bits par poids d'un treillis à décalage
c_i, n_i	sensibilité et taille (poids) de la matrice i
Codage sans perte	
$H(\cdot), h(\cdot)$	entropie de Shannon (bits) d'une variable discrète ; entropie différentielle d'une densité
W_{dec}, W_g	débit de sortie du décodeur sur la carte ; débit auquel le produit matrice-vecteur consomme des poids décodés
M, L	précision des probabilités du rANS (2^{12}) et borne basse de l'état (2^{16}, au chapitre 6 seulement)
f_s, c_s	fréquence quantifiée et cumul du symbole s
$e, \tilde{e}, \delta$	échelle E8M0 d'un bloc MXFP4, médiane de sa ligne, écart $e - \tilde{e}$
Mélange d'experts et cache	
E, k_e	experts routés par couche, experts choisis par jeton

Symbole	Signification
B_t, B_e	octets toujours lus et octets d'experts routés par jeton ($B = B_t + B_e$)
h	taux de succès du cache d'experts (part des octets routés servie par la mémoire)
Décodage spéculatif	
α	taux d'acceptation d'une position proposée
k	nombre de jetons proposés par passe de vérification
$\mathscr{E}(\alpha,k)$	espérance du nombre de jetons produits par lecture complète des poids actifs
$U(n)$	espérance de la taille de l'union des experts de n jetons
$n^\star$	taille de lot où le calcul rattrape la lecture

Première partie

Le problème

Chapitre 1

Introduction et motivation

1.1 La mission

Ce mémoire répond à une demande écrite par Stone le 15 octobre 2026. La voici en ses mots exacts, fautes comprises, parce que ce sont eux qui font foi et que tout le reste n'en est qu'une lecture :

> *j'ai aussi une mission je veux de la recherche scientique et de l'implémentation prend les meilleurs avancées au monde en stockage optimisation et domaine connexes (enfin les plus utiles) et fait en sorte de tout les assemblées dans le but de produire et d'implémenter une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement sans ram ni rien en fait un moyen de compresser à l'extreme puis décompresser en un temps record au besoin à l'instant exacte du noeud bref tu sais ce qui manque et tu sais quoi faire je ne veux pas que des recherches déja existante je veux que tu fasse une vrai recherche de spécialiste puis tu implémente la solution fonctionnelle avec une team d'ingénieur si ce qu'on cherche n,existe pas alors tant mieux creer le ceci est ta mission principale on vas continuer la suite relais plus tard on doit faire tomber se levier d'abord*

Elle a une source plus ancienne, dans deux phrases du 2 et du 4 octobre 2026 :

> *est ce qu'il existe sinon on peut inventer un genre d'appel instanté en d,autre terme une nouvelle facon de compreser les données et en sommes elles apparaisse directement lorsqu'on a en besoin et disparaisse apr;es*
>
> *en fait je pense un peut à des chuncks comme dans minecraft un genre de manière d'ingénieurie ultra poussé qui peut faire disparaitre la charge d'une ai quand elle n'est plus utile et recharger une autre en instantané direcr en fait je pense au sens de faire tourner une ai uniquement lorsqu'elle est nécessaire et pas le tout en meme temps et une ai occupe de l'espace uniquement loesque active*

L'image des « chunks » de Minecraft est exacte et donne son nom au projet. Dans ce jeu, le monde est découpé en tronçons de 16×16 colonnes ; seuls ceux qui entourent le joueur sont chargés en mémoire, les autres dorment sur le disque, et le moteur charge et décharge en continu à mesure que le joueur avance. Un modèle de langue est un monde de poids que le calcul parcourt couche après couche, jeton après jeton : *troncons* veut charger, autour du point où le calcul se trouve, le tronçon de poids qui lui sert, et rien d'autre.

1.2 D'où l'on part

La machine. Un portable ordinaire : un processeur Intel i7-13620H, **16 Go de RAM sur un seul module**, une carte NVIDIA RTX 4060 Laptop de **8 Go**, un disque NVMe de 1 To dont 404 Go libres, chiffré par BitLocker. Après le système, l'écran et le contexte, il reste environ 6,5 Go de VRAM et 11 Go de RAM pour des poids, soit 17,5 Go au total. Le chapitre 3 en mesure chaque étage.

Les modèles. Les modèles ouverts de 2025–2026 qui comptent sont des mélanges d'experts de 20 milliards à 2,8 billions de paramètres (chapitre 9). Le premier visé, gpt-oss-120b, pèse 63,4 Go dans son format d'origine : **3,6 fois** toute la mémoire utile de la machine. D'autres pèsent cinq à vingt fois plus.

Ce qui existe. Les moteurs qui font tourner un modèle plus gros que la mémoire (chapitre 2) savent déplacer des poids, mais ils raisonnent sur des débits de fiche technique, sur des stations de travail à 128 ou 192 Go de RAM, sur des formats décodés par CUDA, et presque tous sous Linux. Aucun n'est fait pour un portable Windows de 16 Go dont le disque chiffré livre moins de la moitié de son débit annoncé.

1.3 Ce que veut dire « sans RAM ni rien »

Au pied de la lettre, la demande est impossible : tout calcul lit ses opérandes dans une mémoire, et une multiplication de matrices exige que la matrice soit, au moins par morceaux, dans les registres de l'unité qui calcule. La lecture retenue, à confirmer par Stone et jamais à lui substituer, est la suivante.

Définition 1.1 (Exécution à fenêtre bornée)**.** Une exécution d'un modèle de P octets de poids est *à fenêtre bornée* si sa mémoire de pointe (RAM et VRAM) est majorée par une constante $M_{\text{fenêtre}}$ qui ne dépend pas de P, mais seulement de la taille du plus grand tronçon de poids dont un calcul a besoin à la fois et de la profondeur de préchargement.

Sous cette lecture, « sans RAM » veut dire que **la mémoire occupée ne dépend plus de la taille du modèle**. La difficulté change alors de nature : elle n'est plus de faire tenir le modèle, mais de faire passer, à chaque jeton, les octets dont il a besoin par le plus lent des étages qui le séparent des unités de calcul. Le juge est la physique, chiffrée au théorème 3.2 :

$$\text{jetons/s} \;\leq\; \min_i \frac{W_i}{O_i},$$

où W_i est le débit mesuré de l'étage i et O_i le nombre d'octets qu'un jeton doit lui faire traverser. Les quatre leviers de ce mémoire agissent chacun sur un terme de cette inégalité : compresser à l'extrême réduit O_i (chapitre 5), et le codage sans perte le réduit encore d'un taux r en amont du décodeur (chapitre 6) ; décompresser sur la carte change l'étage où passent les octets compressés (section 3.8) ; un cache qui réussit garde les octets dans un étage plus rapide (chapitre 7) ; le décodage spéculatif rend plusieurs jetons par lecture (chapitre 8).

1.4 La question de recherche

> *Sur une machine grand public dont la mémoire est plusieurs fois plus petite que le modèle, quel assemblage de représentation des poids, de placement, de chargement et de calcul maximise les jetons par seconde à qualité mesurée, et que manque-t-il au monde pour l'atteindre ?*

« Je ne veux pas que des recherches déjà existantes » : l'état de l'art se lit à fond, mais le livrable est une contribution, un assemblage ou un mécanisme qui n'existe pas encore, prouvé par la mesure, et un programme qui marche. Cette première partie pose les fondations ; l'architecture, l'implémentation et les résultats viendront dans les chapitres annoncés au chapitre 10.

1.5 Ce que cette première partie apporte

1. **Un toit mesuré, et démontré.** Chaque étage de la machine est mesuré avec un témoin qui peut tomber (chapitre 3) : le disque livre 2,82 Go/s, 47 % de son annonce, parce que le volume est chiffré et passe par Intel RST VMD ; le lien hôte-carte 12,2 Go/s ; la VRAM 251,3 Go/s. On démontre la borne de débit d'une chaîne d'étages, son régime permanent par un lemme d'atelier à tâches identiques, et deux corollaires qui tranchent l'architecture : au-delà d'un facteur de compression $r^\star = 4{,}33$, compresser est perdu si le processeur décompresse, et ne l'est jamais si la carte décompresse.
2. **Le régime batterie.** Sur batterie, la machine est une autre machine : limite de puissance de la carte abaissée de 80 à 52,6 W, mémoire de la carte à 6 GHz au lieu de 8, et tout ce qui passe par le processeur divisé par 4 à 6. Aucun banc d'inférence publié ne note la source d'alimentation.
3. **Le premier décodeur de treillis portable.** Un treillis à décalage, la famille de codes la plus proche de la borne de Shannon à 2 bits, décodé dans un shader Vulkan à 124 Go/s équivalents, aussi vite que le scalaire 4 bits en lisant deux fois moins d'octets (chapitre 5).
4. **Une règle de partage des bits selon l'étage**, démontrée par le lagrangien, puis corrigée par les débits mesurés : une matrice résidente en VRAM mérite 2,18 bits de plus qu'une matrice lue par le lien, 3,24 de plus qu'une matrice lue du disque.
5. **La mesure, à octets égaux, de l'échelle de llama.cpp sur Qwen3.5** : avec le meilleur 2 bits portable, rapetisser vaut mieux que comprimer jusqu'à environ 3,2 bits par poids.
6. **Les échelles du MXFP4, codées par la médiane de leur ligne** : un codage sans perte par champs, décodé exactement, qui rend un taux de 1,1026 sur les 60,9 Go d'experts de gpt-oss-120b là où zstd prend 1,04 ; un décodeur rANS entrelacé, démontré exact, à 70 Go/s sous Vulkan et 82 sous CUDA ; la mesure que décoder dans le produit est 3,7 fois plus lent, d'où la règle de décoder à l'arrivée du tronçon ; et le calcul exact qui explique les 3,86 bits des codes de 4 bits et réfute, pour les formats par petits groupes, les entropies de « 0,6 à 1 bit » publiées (chapitre 6).
7. **Le succès de cache impliqué par une mesure publiée**, qui lit la localité dans une mesure qui ne la publie pas (chapitre 7).
8. **Le théorème d'amortissement de la spéculation** et la condition exacte sous laquelle elle paie pour un mélange d'experts lu du disque, qu'un routage indépendant ne satisfait jamais ; et la mesure que sparsité des activations et spéculation ne se composent pas sur un modèle SwiGLU (chapitre 8).
9. **Un inventaire de 49 modèles lus dans leurs en-têtes** et un protocole de jugement dont une mesure a déjà corrigé une hypothèse (chapitre 9).
10. **L'équation du jeton et ses élasticités**, démontrées : sur cette machine, le disque tient le maximum, et tout levier qui ne touche pas les octets lus du disque est de l'ornement ; la condition de la spéculation corrigée par la réutilisation des experts, $\alpha > 1 - \rho$ (chapitre 4).
11. **Le raffinement exact d'un code 4 bits natif**, qui ne coûte rien en entropie et dont la base bat la troncature de 0,27 bit ; trois architectures candidates, la correction d'un gain annoncé à tort, et la décision de construire d'abord l'architecture exacte (chapitre 10).

1.6 La méthode

Le niveau exigé tient en quelques règles, appliquées à chaque nombre (annexe A) :

- chaque mesure a son **témoin**, un cas dont la réponse est connue d'avance, et tout témoin neuf doit pouvoir échouer : on force sa condition à faux, on vérifie que l'instrument tombe, on restaure ;
- un **seuil** ne se pose pas au jugé : il dérive d'une grandeur nommée et se déclare avant la mesure ;
- un coût ne se fonde jamais sur une seule passe : **séries ABBA**, avec l'écart d'un programme à lui-même comme bruit de référence ;

- tout chiffre dont on dépend est refait par un **second chemin** indépendant ;
- quand une mesure contredit un chiffre publié, même le nôtre, **la phrase est barrée** et l'on dit ce qui a changé : jamais de retrait silencieux ; ce mémoire en garde plusieurs, ~~comme celle-ci~~, à leur place ;
- chaque nombre porte sa provenance : [mes.] mesuré ici, [calc.] calculé, [litt.] littérature non reproduite ;
- ce qui produit un chiffre publié est commité, outils et journaux, et chaque figure se régénère depuis ses données par un script commité (annexe C).

Chaque chapitre s'ouvre sur ce qui était acquis au départ, ce qu'il établit et ce qui reste, et se ferme sur ce qui pourrait être faux.

1.7 Plan du mémoire

La première partie pose le problème : l'état de l'art (chapitre 2), la machine et sa physique (chapitre 3), la formulation du problème en équations (chapitre 4). La deuxième mesure les leviers un par un : la quantification extrême (chapitre 5), la compression sans perte à la vitesse du calcul (chapitre 6), le chargement à la demande (chapitre 7), la sparsité et le décodage spéculatif (chapitre 8), les modèles visés (chapitre 9). La troisième prépare le système : ce qui vient (chapitre 10), la discussion et les limites (chapitre 11), les travaux futurs (chapitre 12). Les annexes donnent les protocoles de mesure, les outils et les journaux.

Chapitre 2

État de l'art

D'où l'on part, ce que le chapitre établit, ce qui reste

Ce chapitre rassemble, au 15 octobre 2026, ce que le monde sait faire pour exécuter un modèle plus grand que la mémoire. Chaque famille est lue à sa source primaire (article, dépôt officiel au commit nommé, notes de version) et jugée par la même règle : les octets qu'un jeton doit lire, divisés par le débit de l'étage qui les sert. Les détails, les mesures et les chiffres complets sont dans les chapitres de la deuxième partie ; on dit ici ce qui existe, ce que chaque famille gagne, ce qu'elle coûte, et ce qui manque pour la machine de Stone.

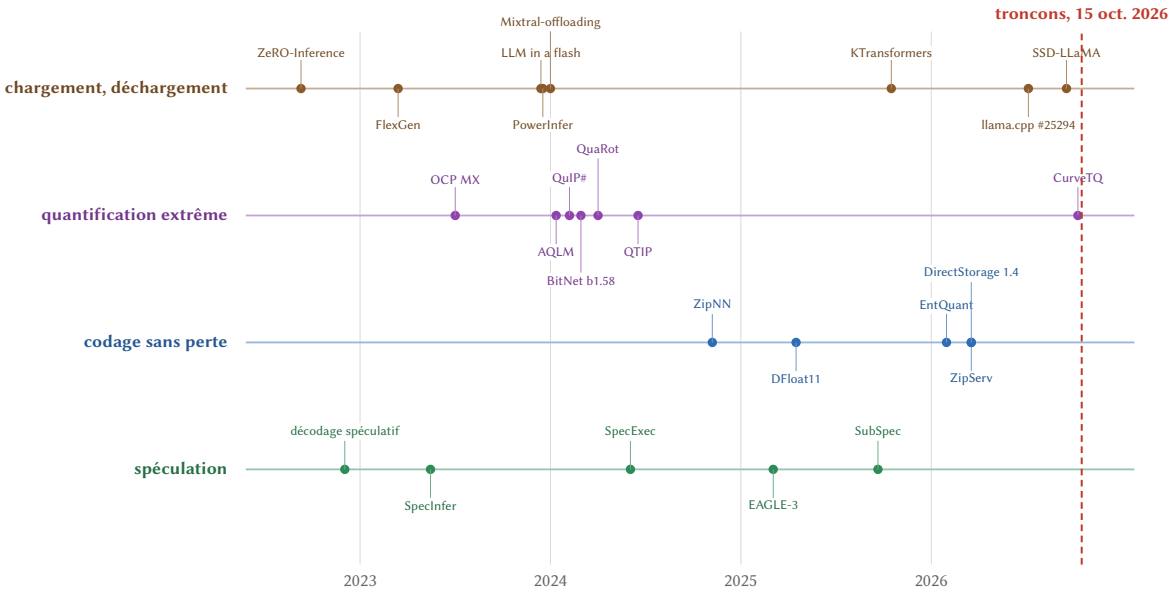


Figure 2.1 : Les travaux cités dans ce chapitre, par famille et par date de première version publique (date de l'entrée bibliographique). Quatre familles mûrissent en même temps depuis 2023 ; aucune ne vise un portable Windows de 16 Go qui lit ses poids sur un disque chiffré. Les racines théoriques sont bien plus anciennes.

Données : `memoire/sources/*.bib` (dates), `memoire/figures/schemas/etat.tex`.

2.1 Le mur de la mémoire, au lot de un

Un jeton de décodage, au lot de un, multiplie chaque matrice de poids par un seul vecteur : chaque poids est lu une fois et sert à deux opérations. L'intensité arithmétique, environ deux opérations par

poids lu, est très en dessous de ce que la carte sait calculer par octet (environ 50 opérations par octet de VRAM sur la RTX 4060, section 3.6) : le décodage est borné par la mémoire. C'est la lecture classique du modèle du toit (*roofline*) de Williams, Waterman et Patterson [1], qui borne la performance par le minimum du calcul et du produit intensité × débit. Ce mémoire en étend l'idée à une hiérarchie d'étages (disque, RAM, lien, VRAM) traversés à chaque jeton (théorème 3.2) : quand le modèle ne tient pas dans la VRAM, l'étage qui borne n'est plus la VRAM mais le lien ou le disque, 20 à 90 fois plus lents.

2.2 La fenêtre de travail : une idée de 1968

L'image de Stone, « une ai occupe de l'espace uniquement lorsque active », a un nom en informatique depuis plus d'un demi-siècle. Denning [2] définit l'*ensemble de travail* d'un programme à l'instant t comme l'ensemble des pages qu'il a référencées dans la fenêtre $(t-\tau, t]$,

$$W(t,\tau) = \{\,\text{pages référencées entre } t-\tau \text{ et } t\,\},$$

et pose que la mémoire à donner à un programme est celle de son ensemble de travail, le reste pouvant dormir sur le disque. Belady [3] avait montré deux ans plus tôt quelle politique de remplacement est optimale quand on connaît l'avenir : évincer la page dont le prochain usage est le plus lointain. Ces deux résultats bornent tout ce que le chapitre 7 mesure : l'exécution à fenêtre bornée de la définition 1.1 est la propriété d'ensemble de travail appliquée aux poids, avec une fenêtre dont la taille ne dépend pas du modèle ; et l'optimum de Belady est le plafond de tout cache d'experts.

Les moteurs de jeux vidéo ont fait de cette idée une technique de rendu. Les textures virtuelles creuses [4] découpent une texture immense en tuiles, n'envoient à la carte que les tuiles nécessaires à l'image en cours, et traduisent dans le shader l'adresse virtuelle en adresse physique, à la manière d'une table des pages ; DirectStorage [5] déplace la lecture et la décompression de ces données vers la carte. C'est exactement le « chunk » de Minecraft que Stone a en tête, et le tableau 2.1 pose la correspondance terme à terme.

Table 2.1 : Trois façons de n'occuper de la mémoire que pour ce qui sert : la mémoire virtuelle, les textures virtuelles des moteurs de jeux, et troncons. La dernière ligne est la différence qui compte.

	mémoire virtuelle	textures virtuelles	troncons
unité	la page	la tuile de texture	le tronçon : une matrice d'expert, 4,4 Mo dans gpt-oss-120b
qui demande	le processeur, à l'accès	le shader, en rendant l'image	le routeur de chaque couche, à chaque jeton
ce qui prédit	la localité temporelle	la caméra, qui bouge peu d'une image à l'autre	la réutilisation des experts entre jetons voisins (ρ, chapitre 4)
le cache	la RAM	une texture physique en VRAM	la VRAM, puis la RAM épinglée
la compression	aucune	décompression sur la carte	rANS décodé sur la carte (chapitre 6)
le défaut	le programme attend la page	une tuile plus grossière s'affiche en attendant	le jeton attend l'expert : un calcul faux n'est pas permis

La dernière ligne est la différence de fond. Un moteur de jeu peut afficher une tuile floue le temps que la bonne arrive : l'œil pardonne. Un modèle de langue ne peut pas calculer avec un expert approché sans changer sa sortie. L'équivalent le plus proche de la tuile grossière est un code progressif dont la couche de base sert tout de suite et dont le complément corrige ensuite au bit près : c'est le raffinement exact de la proposition 10.1, et le chapitre qui le porte (chapitre 10) chiffre ce qu'il coûte.

2.3 Faire tourner un modèle plus gros que la mémoire

Le déchargement des poids. La première génération charge les couches à la demande : AirLLM [6] lit une couche, la calcule, la libère, et résout la mémoire de pointe sans rien faire pour le débit ; DeepSpeed ZeRO-Inference [7] et FlexGen [8] amortissent la lecture sur de grands lots, au prix de latences de milliers de secondes. llama.cpp [9] projette le fichier en mémoire et partage les couches entre la carte et le processeur ; c'est la base de comparaison du projet.

La parcimonie des activations. Deja Vu [10], PowerInfer [11, 12] et LLM in a flash [13] ne lisent que les neurones actifs d'un jeton, prédits par de petits réseaux ; ils obtiennent des facteurs de 2 à 25 sur des modèles à ReLU, dont la majorité des neurones sont nuls. Les modèles récents utilisent SwiGLU, et la parcimonie y est faible (chapitre 8).

Les mélanges d'experts. Un MoE n'active qu'une petite part de ses paramètres par jeton (3 à 10 % pour les modèles de 2026, figure 9.1), ce qui en fait la cible naturelle : seuls les experts choisis doivent être lus. Mixtral-offloading [14] garde un cache LRU d'experts en VRAM et précharge par le routeur de la couche suivante ; ProMoE [15], HOBBIT [16], ExpertFlow [17], SeqMoE [18] et Edge0 [19] prédisent mieux ; Fiddler [20] et KTransformers [21] calculent les experts là où ils sont. Depuis 2026, des moteurs lisent les experts sur le disque avec un cache explicite : la demande #25294 de llama.cpp [22], moe-stream [23] et SSD-LLaMA [24], ce dernier compressant les experts dans la boucle de lecture. Le chapitre 7 les juge.

2.4 La quantification extrême

La quantification réduit b, les bits par poids, et multiplie le plafond par $16/b$. Quatre générations se superposent (chapitre 5) :

- **les scalaires par groupes** : les K-quants de llama.cpp, GPTQ, EfficientQAT [25], HIGGS [26] ; simples à décoder, ils perdent beaucoup sous 3 bits ;
- **les codes vectoriels** : les i-quants de llama.cpp (grilles tirées du réseau E8), QuIP# [27], AQLM [28] et PV-tuning [29], VPTQ [30] ;
- **les codes en treillis** : QTIP [31] et sa version simplifiée EXL3, et depuis l'automne 2026 CurveTQ [32] (sans rotation), XOR-Trellis [33], Tetra [34] (réseau de Leech) ; à 2 bits sur la source gaussienne, ils ferment 90 % de l'écart entre le scalaire optimal et la borne de Shannon ;
- **les modèles nés en bas débit** : BitNet [35], ParetoQ [36], et les formats MXFP4 et NVFP4 [37, 38] dans lesquels gpt-oss, Kimi K2.6 et K3, DeepSeek V4 publient leurs poids.

Les rotations (QuaRot [39], SpinQuant [40]) rendent la source gaussienne et servent de brique aux codes des deux générations suivantes. Le constat qui pèse pour ce mémoire : **les meilleurs codes ne se décodent qu'en CUDA**, et le meilleur 2 bits décodable sous Vulkan (IQ2_XXS) vaut, sur la source gaussienne, exactement le scalaire optimal.

2.5 La compression sans perte

Après quantification, il reste de la redondance : les codes ne sont pas uniformes et les échelles des blocs sont très prévisibles. Les travaux récents l'exploitent par codage entropique. DFloat11 [41] code par Huffman l'exposant des poids BF16 et réduit le modèle à environ 70 % de sa taille sans perte ; ZipNN [42], Huff-LLM [43], ECF8 [44], ZipServ [45] et les travaux qui approchent la borne de Shannon [46] suivent la même voie sur d'autres formats ; EntroLLM [47], EntroPack [48], EntQuant [49], WaterSIC [50] et vram-compression [51] combinent quantification et codage entropique, ce dernier avec un noyau CUDA qui fond le décodage rANS au produit matrice-vecteur.

Côté décodeurs, le codage par systèmes numériques asymétriques (ANS) [52] et son entrelacement [53] ont donné des décodeurs massivement parallèles sur carte graphique [54, 55], des bibliothèques (nvCOMP [56], DietGPU [57]) et, sous Windows, la décompression GDeflate et Zstandard [58] sur la carte par DirectStorage [5]. L'étude d'entropie du projet (chapitre 6) mesure sur les modèles de la machine un plafond sans perte de 5,3 % pour les K-quants et un taux réel de 1,1026 pour les experts MXFP4 de gpt-oss-120b (9,3 % d'octets en moins, décodés exactement), dont l'essentiel tient aux échelles, que personne ne code ; et un décodeur rANS à 70 Go/s de codes produits sous Vulkan, 82 sous CUDA. Elle montre aussi, par un calcul exact, pourquoi les entropies de « 0,6 à 1 bit » publiées pour int4 ne s'appliquent pas aux formats par petits groupes. La leçon d'ensemble est sûre : **après une bonne quantification, la compression sans perte vaut quelques pour cent** ; le levier de compression est la représentation, et le sans-perte se joue sur les métadonnées.

2.6 La spéculation

Le décodage spéculatif [59, 60] fait proposer plusieurs jetons par un brouillon bon marché et les vérifie en une passe de la cible, sans perte. Ses variantes changent de brouillon (petit modèle, têtes Medusa [61], couche EAGLE [62, 63], module MTP natif [64, 65], diffusion par blocs DFlash [66], n-grammes [67]) ou de forme (arbres de SpecInfer [68], Sequoia [69], SpecExec [70]). Deux travaux visent le régime déchargé : SpecExec, qui vérifie des arbres de milliers de nœuds quand les poids passent par le lien, et SubSpec [71], qui loge en VRAM une copie basse précision de la cible comme brouillon. Pour les MoE, Cascade [72] montre que la spéculation peut ralentir, et MoE-Spec [73], AcceptMoE [74], EcoSpec [75] bornent l'ensemble d'experts de la vérification.

2.7 Ce que chaque famille change dans le toit

Toutes ces familles se lisent dans la même inégalité, jetons/s $\leq \min_i W_i/O_i$ (théorème 3.2), multipliée par $\mathscr{E}$, le nombre de jetons produits par lecture des poids. Le tableau 2.2 dit quel terme chacune déplace, et à quelle condition ; c'est cette lecture commune qui permet, au chapitre 4, de les combiner au lieu de les juxtaposer.

2.8 Positionnement

Le tableau 2.3 dit la forme du trou. Chaque brique existe quelque part : la lecture des experts sur le disque (Linux), la décompression sur la carte (pour des textures, ou depuis la VRAM), les codes proches de la borne (en CUDA), la spéculation en régime déchargé (avec 128 Go de RAM). **Personne ne les a réunies sur un portable Windows de 16 Go sans boîte à outils CUDA**, et les chapitres qui suivent montrent, chiffres mesurés à l'appui, que cet assemblage n'est pas une juxtaposition : les leviers se multiplient (la quantification fabrique le brouillon de la spéculation, la compression agit

Table 2.2 : Ce que chaque famille de techniques change dans $\mathscr{E} \cdot \min_i W_i/O_i$, et la condition sous laquelle le gain est réel. b : bits par poids ; r : taux sans perte ; h : succès du cache ; α : taux d'acceptation.

famille	terme déplacé	condition, et ce qui la mesure ici
quantification	tous les O_i, d'un facteur $16/b$	la qualité tient (perplexité, divergence KL, chapitre 5) et le noyau suit le débit de la VRAM
codage sans perte	les O_i des étages en amont du décodeur, d'un facteur r	le décodeur dépasse r fois le débit de l'étage soulagé (corollaire 6.2)
décompression sur la carte	l'étage qui porte les octets compressés	au-delà de $r^\star$, seule la carte peut décompresser sans devenir le goulot (section 3.8)
cache et préchargement	déplace hB_e vers un étage plus rapide	la localité du routage existe (h mesuré, chapitre 7)
placement	f_V, f_R, f_D	la VRAM et la RAM libres, mesurées après le système
sparsité des activations	O_i, d'un facteur de densité	le modèle a des activations creuses (ReLU) ; faible sous SwiGLU (chapitre 8)
spéculation	multiplie par $\mathscr{E}(\alpha, k)$	la passe de vérification lit à peine plus que celle d'un jeton ; pour un MoE, $\alpha > 1 - \rho$ (proposition 4.8)

deux fois sur un cache d'experts) ou s'annulent (la spéculation et les experts fins, la sparsité et la vérification).

Table 2.3 : Ce que les systèmes existants couvrent du problème de troncons, d'après leur publication et leur code [litt.]. « Disque » : poids lus sur le disque à chaque jeton ; « 16 Go » : publié ou mesuré avec moins de RAM que le modèle ; « carte décode » : décompression sur la carte des octets venus du lien.

Système	Windows	sans CUDA	disque	16 Go	carte décode	treillis
llama.cpp (`mmap`, `-ncmoe`)	oui	oui (Vulkan)	par mmap	oui	non	non
llama.cpp #25294	non	non	oui	non (128 Go)	non	non
moe-stream	oui	processeur seul	oui	oui	non	non
SSD-LLaMA	non	non	oui	16–32 Go	après transfert	non
KTransformers	non	non	non	non	non	non
PowerInfer, LLM in a flash	partiel	non	oui (flash)	oui	non	non
SpecExec, SubSpec	non	non	non (RAM)	non	non	non
EXL3, QTIP	non	non	non	—	oui (en VRAM)	oui
DirectStorage 1.4	oui	oui	oui	—	oui (textures)	non
troncons (visé)	**oui**	**oui**	**oui**	**oui**	**oui**	**oui**

Chapitre 3

La machine et sa physique

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, on ne disposait que de plafonds calculés d'après les fiches techniques (lien NVMe 7,877 Go/s, PCIe 15,754 Go/s, VRAM 256 Go/s, RAM 41,6 Go/s) et d'un seul chiffre mesuré par une autre étude, sous charge (PCIe 6,4 Go/s). **Ce chapitre établit** le débit réel de chaque étage sur la machine de Stone, avec témoins et séries ABBA, dans deux régimes (secteur et batterie), puis il démontre le toit qui en découle : une borne inférieure du temps par jeton, son régime permanent, et deux corollaires qui décident de l'architecture. **Il reste** la RAM et llama.cpp mesurés au calme sur secteur, la part de BitLocker dans le plafond du disque, et un noyau 4 bits sous Vulkan au niveau de la VRAM. Source : `fondations/01-machine.md` et `fondations/journaux/01-machine/`.

Ce chapitre est le juge de tout le mémoire. Une idée de compression, de cache ou de spéculation ne vaut que ce qu'elle fait gagner au plus lent des étages qu'un jeton traverse ; il faut donc savoir, à 1 % près si possible, ce que chaque étage livre réellement. On décrit d'abord la machine, puis la méthode de mesure, puis chaque étage du plus lent au plus rapide, et l'on termine par le théorème du toit.

3.1 La machine

Table 3.1 : La machine de Stone, relevé du 2026-10-15 (WMI, nvidia-smi, FSCTL_MANAGE_BYPASS_IO).

Organe	Relevé
Processeur	Intel i7-13620H : 6 cœurs performants, 4 efficaces, 16 fils ; L2 9,5 Mio, L3 24 Mio
RAM	**un seul module** Micron DDR5 de 16 Gio annoncé à 5 600, configuré à 5 200 MT/s (emplacement Controller0-ChannelA-DIMM0) ; plafond 5 200 × 8 = 41,6 Go/s [calc.]
Carte graphique	NVIDIA RTX 4060 Laptop (AD107), 8 188 Mio de GDDR6 sur 128 bits à 16 Gbit/s (horloge mémoire relevée 8 001 MHz), pilote 591.91 ; lien PCIe 4.0 ×8 au plus
Disque	NVMe Samsung MZVMA1T0HCLD (1 To), micrologiciel MXE7001Q, **derrière le contrôleur Intel RST VMD** (pilote `iaStor`) ; volume C : **chiffré par BitLocker**
Système	Windows 11 Pro 10.0.26300 ; sécurité par virtualisation (VBS) et intégrité de la mémoire (HVCI) actives
Outils	CUDA 13.1.2 dépaqueté sans installation ; wgpu 30.0.1 (Vulkan, DirectX 12) ; Rust 1.98.1 ; llama.cpp b11515

Deux traits de cette machine pèsent sur tout le reste. Le premier est le **module de RAM unique** : un seul canal de 64 bits, soit la moitié du débit d'un portable à deux modules, et une RAM qui n'est que 5,3 fois plus rapide que le lien du disque. Le second est le **chemin du disque** : chaque octet lu

traverse le filtre de chiffrement de BitLocker et le contrôleur VMD d'Intel, deux réglages par défaut d'un portable Windows grand public, dont on verra qu'ils coûtent plus de la moitié du débit annoncé.

3.2 La méthode de mesure

Chaque chiffre de ce chapitre suit les mêmes règles (annexe A) :

- **un témoin**, c'est-à-dire un cas dont la réponse est connue d'avance, et que l'on force à faux une fois pour vérifier que l'instrument tombe (`--fausser`) ;
- **un seuil déclaré avant** la mesure, dérivé d'une grandeur nommée (`00-seuils-declares.md`) ;
- **des séries ABBA** : la liste des configurations est mesurée dans un sens puis dans l'autre ; chaque cellule est vue deux fois, en ordre miroir, la dérive lente s'annule et l'écart $|A - B|$ rapporté à la moyenne sert de bruit de référence ;
- **un second chemin** indépendant pour tout chiffre dont on dépend ;
- **la source d'alimentation** relevée à chaque passe (`Win32_Battery`, `BatteryStatus` = 2 sur secteur), depuis la découverte du section 3.6.1.

Un biais permanent, dit d'avance. D'autres calculs du projet tournaient sur la machine pendant toutes ces mesures (quantifications llama.cpp jusqu'à 8,7 cœurs, calculs de perplexité sur la carte jusqu'à 7,7 Gio de VRAM, RAM libre entre 1,0 et 4,4 Gio). Chaque passe journalise cette charge. Les chiffres « au mieux » de ce chapitre sont donc des **bornes inférieures** de ce que la machine donne au repos ; les médianes disent ce qu'elle donne partagée.

3.3 Le disque

3.3.1 L'instrument

Le banc `disque.exe` (Rust, `windows-sys`) lit un fichier de test de 32 Gio, deux fois la RAM, ouvert avec `FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`, dans des tampons alignés par `VirtualAlloc`, avec q lectures en vol (la profondeur de file, QD) par port de complétion (IOCP) ou par IoRing. Le fichier porte un motif vérifiable : chaque page de 4 Kio commence par son numéro combiné par un ou exclusif à une constante, la suite est un flux xorshift64* dont la graine est ce numéro. Chaque lecture vérifie l'en-tête de chaque page reçue, et une page complète sur 64. Le balayage croise les blocs de 4 Kio à 16 Mio avec les profondeurs 1 à 64, en accès aléatoire et séquentiel, en ordre ABBA : 188 passes, puis 40 passes IOCP contre IoRing.

Les témoins. Forcé à faux (l'attendu décalé d'une page), l'instrument compte 978 688 pages fausses sur 978 688 ; sans cassure, 0 page fausse sur toutes les passes. Aucune lecture sans tampon n'a dépassé le lien (7,877 Go/s). Le second chemin est le compteur cumulé du système (`Win32_PerfRawData_PerfDisk_Physical`) relevé toutes les 0,5 s : relu par le cache de Windows, 1 Gio est livré à 1,65 Go/s pendant que le disque ne lit que 3,4 % de ces octets ; le cache est donc visible quand il joue, et absent des passes sans tampon, où le disque voit au moins 100 % des octets.

Une règle barrée. La règle déclarée « le moteur à tampon relu sur 1 Gio doit dépasser 7,877 Go/s » était fausse : ~~le cache relu dépasse le lien~~. L'essai a livré le cache à 3,1 Go/s avec 2,98 s de processeur pour 3 s de fenêtre : la copie depuis le cache de Windows est bornée par un fil de processeur, pas par la RAM. Elle a été barrée avant le balayage et remplacée par le témoin au compteur, plus fort.

3.3.2 Les résultats

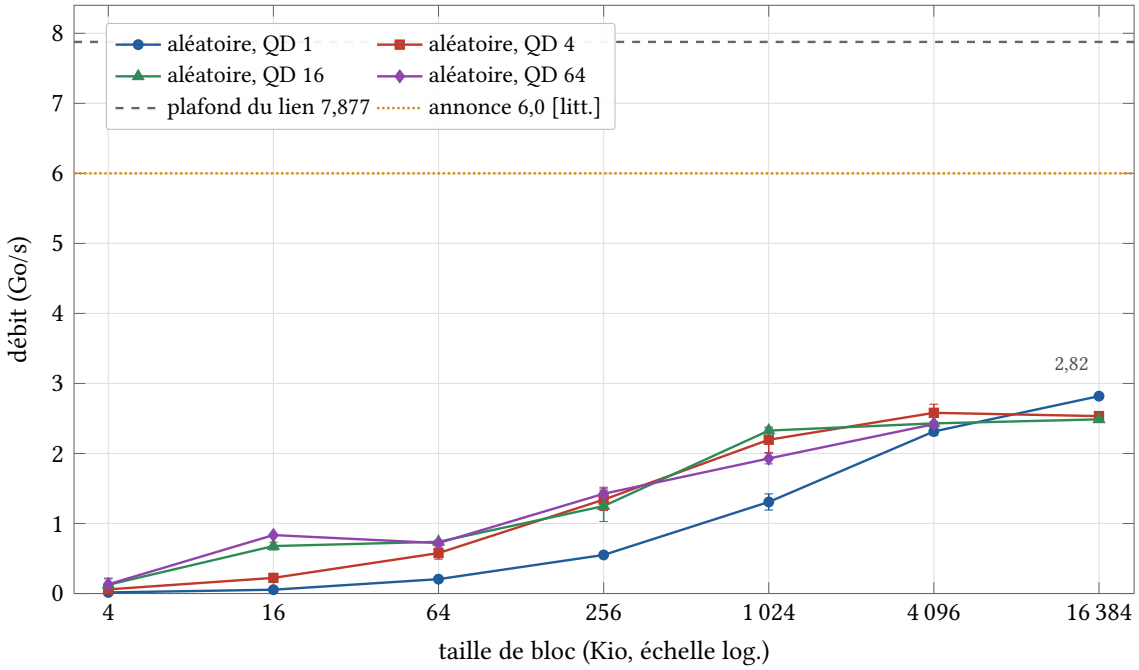


Figure 3.1 : Débit du disque en lectures aléatoires sans tampon, selon la taille de bloc et la profondeur de file [mes.]. Chaque point est la moyenne des passages A et B ; la barre d'erreur est le demi-écart $|A - B|/2$. Le plateau des gros blocs reste entre 2,4 et 2,9 Go/s quelle que soit la profondeur, à 36 % du lien et 47 % de l'annonce.

Données : `fondations/journaux/01-machine/disque-balayage.csv` (94 cellules), via `memoire/outils/donnees_figures.py`.

Table 3.2 : Débit du disque en Go/s, moyenne des passages A et B ; entre crochets, le bruit $|A - B|$/moyenne [mes.].

bloc	aléa QD 1	aléa QD 4	aléa QD 16	aléa QD 64	séq QD 1	séq QD 16
4 Kio	0,015 [53 %]	0,059 [44 %]	0,125 [136 %]	0,132 [134 %]	0,046 [11 %]	0,188 [3 %]
64 Kio	0,205 [11 %]	0,578 [30 %]	0,739 [6 %]	0,721 [10 %]	0,390 [16 %]	2,309 [0 %]
1 Mio	1,307 [18 %]	2,195 [17 %]	2,327 [3 %]	1,928 [8 %]	1,599 [8 %]	1,900 [40 %]
4 Mio	2,314 [5 %]	2,581 [10 %]	2,430 [0 %]	2,418 [2 %]	1,041 [13 %]	1,631 [92 %]
16 Mio	**2,819 [1 %]**	2,533 [0 %]	2,487 [2 %]	—	2,340 [43 %]	2,206 [17 %]

Ce qu'on en tire (figure 3.1 et tableau 3.2) :

- **Le plateau des gros blocs est à 2,4–2,9 Go/s**, quelle que soit la profondeur. Le meilleur point reproductible est la lecture aléatoire de 16 Mio à QD 1 : **2,82 Go/s, bruit 1 %**. Deux cellules ont touché 3,84 Go/s au passage A sans se reproduire au passage B ; elles ne sont pas retenues comme plafond.
- Les petits blocs sont bornés par le processeur : 4 Kio aléatoire plafonne vers 54 000 entrées-sorties par seconde au mieux, la latence médiane à QD 1 est de 200 à 230 µs, et le banc consomme environ 4 s de processeur par Go lu en 4 Kio (16 µs par entrée-sortie), contre 0,07 à 0,11 s/Go au-delà de 1 Mio.

– **IOCP et IoRing ne se distinguent pas** : sur cinq cas en ABBA double, l'écart (0 à 5 %) reste sous la règle déclarée (au moins deux fois le bruit et 3 %). IoRing coûte un peu moins de processeur sur les gros blocs, sans gain de débit.

3.3.3 Pourquoi si loin de l'annonce

Samsung annonce pour le PM9C1a 6 000 Mo/s en lecture séquentielle et 900 000 IOPS [litt.] [76, 77] ; l'identification de la référence MZVMA1T0HCLD à ce modèle est probable (préfixe MZVMA), pas prouvée. Ce que l'on sait, mesuré :

1. **BitLocker est actif et son filtre `fvevol` refuse BypassIO** (statut `0xC00004C9`). La documentation de Microsoft décrit exactement ce cas [78] : chaque octet lu traverse le déchiffrement logiciel du volume.
2. **Le pilote de stockage est `iaStor` (Intel RST VMD)**, pas `stornvme` ; la même page réserve le chemin BypassIO complet à StorNVMe.
3. Le processeur était chargé par d'autres calculs du projet ; les petits blocs, liés au processeur, en souffrent le plus.

La part de BitLocker, celle de VMD et celle du disque lui-même ne se séparent pas sans lire le disque sans chiffrement, ce qui demande un geste de Stone sur un réglage de sécurité. Réponse par délégation du 2026-10-15, réversible : on garde BitLocker, protection de ses données si le portable est volé, et l'on conçoit pour 2,82 Go/s.

3.3.4 Le coût de BitLocker, mesuré par un second chemin

On peut en revanche mesurer ce que coûte, sur ce processeur, le déchiffrement que `fvevol` applique à chaque octet. L'outil `aes_xts.py` appelle l'API CNG de Windows (`bcrypt.dll`, algorithme `XTS-AES` [79]) avec des unités de 4 Kio, comme un secteur de BitLocker, de 1 à 8 fils.

Le mécanisme. Le mode XTS [80] chiffre le bloc de 16 octets j de l'unité numéro i par

$$C_j = E_{K_1}(P_j \oplus T_j) \oplus T_j, \qquad T_j = E_{K_2}(i) \otimes \alpha^j \in \mathrm{GF}(2^{128}), \tag{3.1}$$

où E_K est l'AES de clé K et α l'élément primitif du corps. Chaque bloc coûte un AES (10 tours pour une clé de 128 bits, 14 pour 256), exécuté par les instructions AES-NI, plus une multiplication par α (un décalage et un ou exclusif conditionnel). Les blocs sont indépendants : le déchiffrement se parallélise parfaitement entre unités. Témoins : le vecteur 1 de l'annexe B de la norme IEEE 1619-2007 et l'aller-retour chiffrer-déchiffrer sur 1 Mio ; `--fausser` les fait tomber tous deux.

Sur un fil, le déchiffrement va à 3,2–4,2 Go/s, **le même ordre que le plateau du disque** ; sur huit fils, 8,7 Go/s (figure 3.2). Si `fvevol` déchiffre une requête sur un seul fil, une lecture à QD 1 est bornée par un cœur. Mais le plateau ne monte pas avec la profondeur alors que le processeur sait déchiffrer 8,7 Go/s : soit `fvevol` sérialise, soit la borne est ailleurs (VMD, le disque, sa gestion thermique). On ne peut pas trancher sans lire sans BitLocker ; c'est dit comme hypothèse, chiffrée. Ce que BitLocker coûte au projet en temps de processeur est, lui, mesuré : 1/4,2 à 1/3,2, soit **0,24 à 0,31 s de cœur par Go lu**. À 2,82 Go/s, c'est 0,7 à 0,9 cœur occupé en permanence, que notre propre décompression sur le processeur ne pourra pas utiliser.

3.3.5 IoRing, BypassIO, DirectStorage

IoRing est présent dans le noyau en version maximale 400, plus récente que les en-têtes de `windows-sys` ; utilisable, sans gain mesurable ici [81]. BypassIO est refusé sur C:. DirectStorage n'est pas livré

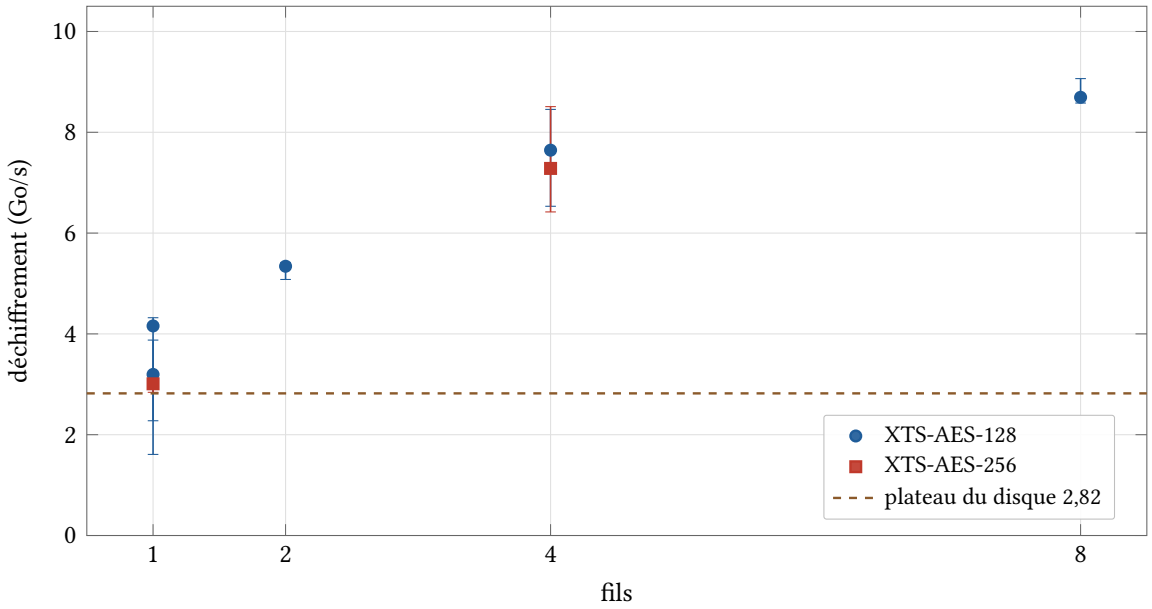


Figure 3.2 : Déchiffrement XTS-AES par l'API CNG de Windows, unités de 4 Kio, sur batterie [mes.]. Point : médiane ; barre : minimum et maximum de la série. Deux passages à un fil (A puis B) sont tracés. Sur un fil, le déchiffrement va au rythme du plateau du disque.

Données : `fondations/journaux/01-machine/aes-xts-batterie.log`.

avec Windows : son SDK est un paquet NuGet de 2,8 Mo (version 1.3.0) ou 4,0 Mo (préversion 1.4.0-preview2, qui ajoute Zstandard décompressé sur la carte) [82, 83]. Il n'est pas mesuré ici, et tournerait de toute façon sans BypassIO.

3.4 La RAM

Le banc `memoire.exe` mesure la lecture (somme de mots de 64 bits, quatre accumulateurs), l'écriture, la copie, de 1 à 16 fils, sur des tampons de 512 Mio (21 fois le L3), et la latence par chasse de pointeurs sur un cycle de Sattolo, un saut par ligne de 64 octets. Témoin : la somme du motif est connue, $n(n-1)/2$.

Ce que la première mesure a appris à l'instrument. La première mesure (machine très chargée) a livré 15,6 Go/s en lecture sur un fil, puis s'est effondrée à 2–4 Go/s pour tous les nombres de fils. Avec 1 à 2 Gio de RAM libre, Windows retire des pages de l'ensemble de travail du banc et les range dans son magasin de compression : une lecture « de RAM » paie alors une faute de page et une décompression. Le banc relève donc désormais son plancher d'ensemble de travail, **verrouille ses tampons** (`VirtualLock`) et compte les fautes de page. Témoin neuf, déclaré avant : moins de 1 % des pages lues ont fauté. Forcé à faux (un tampon neuf à chaque passe), il tombe : 197 033 fautes pour 196 608 pages lues. Verrouillé : 15 à 267 fautes, témoin tenu.

La mesure verrouillée, faite sur batterie, a un bruit ABBA qui dépasse 100 % en débit : rien ne s'en tire comme plafond. Deux faits seulement (figure 3.3) : la latence d'un accès dépendant est de 0,75 à 0,96 µs au-delà du L3, sans faute de page ; et 16 fils lisent moins vite qu'un seul. Une DRAM DDR5 répond en une centaine de nanosecondes ; un échec du TLB en coûte quelques centaines de plus quand la traduction est *imbriquée*, ce qui est le cas ici puisque VBS et HVCI tournent (chaque échec parcourt deux tables de pages). Ajoutée au processeur à 1,5 GHz sur batterie, c'est l'explication la

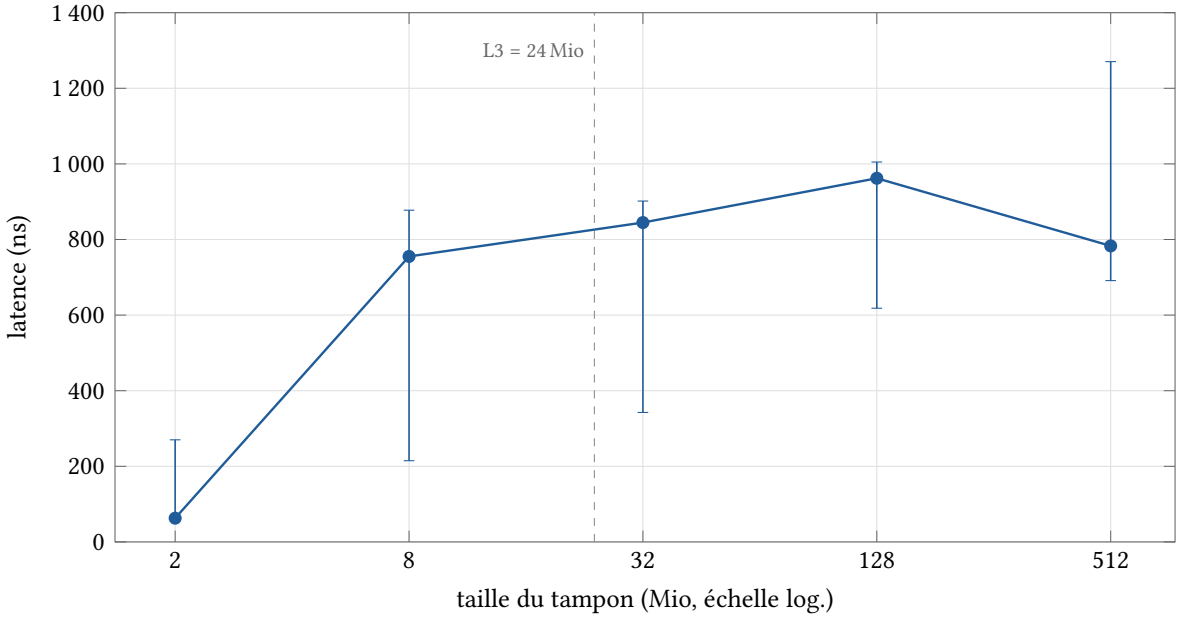


Figure 3.3 : Latence d'un accès dépendant (chasse de pointeurs), tampons verrouillés, sans faute de page, sur batterie [mes.]. Point : médiane ; barre : minimum et maximum. Au-delà du L3, un accès coûte 0,75 à 0,96 µs, sept à dix fois la latence d'une DRAM.

Données : `fondations/journaux/01-machine/memoire-batterie-verrou.log`.

plus probable : **hypothèse, non prouvée** (il faudrait des grandes pages, qui demandent un privilège d'administrateur). Le meilleur débit du processeur vers la RAM reste 15,6 Go/s (un fil, première passe, témoin tenu), 37 % du plafond d'un module ; la mesure au calme sur secteur est en attente.

3.5 De l'hôte à la carte

3.5.1 L'instrument, et le piège que le témoin a levé

Le banc `carte.exe transfert` (wgpu 30.0.1 [84], Vulkan et DirectX 12) mesure en ordre ABBA, avec les horodatages de la carte : la vraie DMA depuis un tampon placé en RAM hôte vers la VRAM ; l'écriture du processeur dans un tampon projeté ; l'écriture naïve par `queue.write_buffer` ; les sens carte vers hôte et carte vers carte. Témoin : aller-retour octet pour octet.

Une découverte par le témoin de plafond. Le premier essai donnait « hôte vers carte » à 80–220 Go/s : le témoin « jamais au-dessus du lien » l'a marqué faux. Raison : **sous Vulkan, le pilote place les tampons projetés en écriture dans la VRAM visible du processeur (BAR redimensionnable)** ; la copie mesurée était une copie de VRAM à VRAM, et le vrai passage sur le lien avait eu lieu pendant l'écriture du processeur dans la projection. Sous DirectX 12, le même tampon vit en RAM hôte (copie à 12,2 Go/s). Une exception confirme : quand la VRAM était presque pleine, le tampon est retombé en RAM hôte et la copie a donné 7,9 Go/s, sous le plafond.

3.5.2 Les résultats

La DMA depuis la RAM hôte atteint **12,2 Go/s, 77 % du lien** PCIe 4.0 ×8 (tableau 3.3). Une phrase a été barrée en cours d'étude : ~~à 256 Mio, la DMA tombe à 4,4–6,3 Go/s, un effet de taille~~. Le balayage de

Table 3.3 : Transferts entre l'hôte et la carte, 64 Mio [mes.].

Opération	machine calme	sous charge	remarque
hôte → carte, DMA depuis la RAM hôte	**12,10 / 12,24 / 12,20**	3,2 à 6,3	77 % du lien
processeur → BAR (Vulkan)	9,1	0,45 à 4,5	au mur, projection comprise
`write_buffer` (Vulkan)	7,5 à 8,2	0,1 à 4,3	une écriture par soumission
carte → hôte	6,2 à 9,4	3,3 à 6,8	

taille a donné 3,2–4,3 Go/s à *toutes* les tailles, 64 Mio compris, puis trois séries à 64 Mio sont revenues à 12,1–12,2 Go/s un quart d'heure plus tard : la variable est le temps (la charge des autres sur la carte), pas la taille. Second chemin indépendant : l'étude du chargement a mesuré par le runtime CUDA 6,4 Go/s en mémoire épinglée, sous charge ; c'est dans la plage des séries chargées, et le chiffre de 6,4 n'était pas un plafond de CUDA mais un état de la machine.

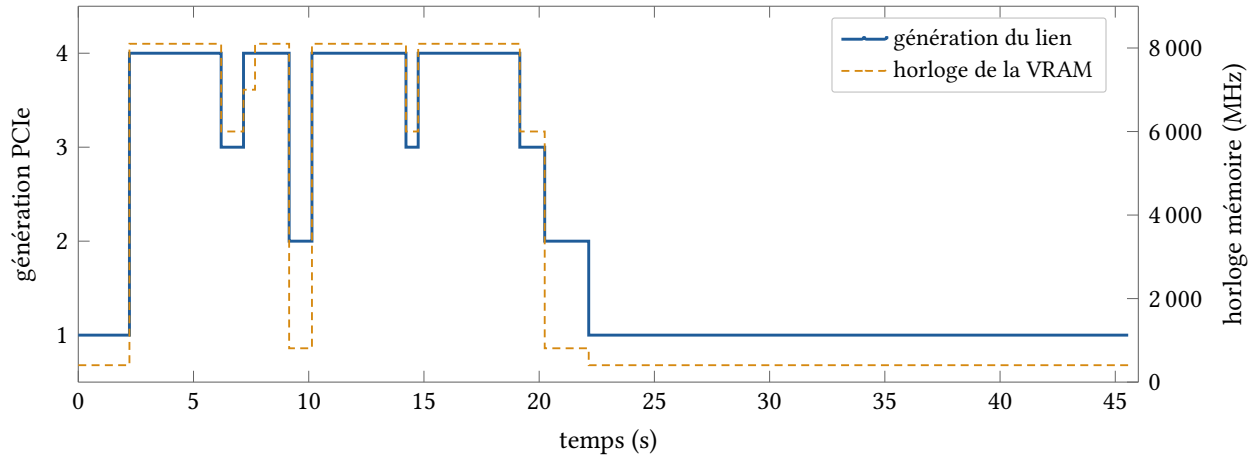


Figure 3.4 : Génération du lien PCIe et horloge de la VRAM relevées par nvidia-smi toutes les 0,1 s pendant trois séries de copies [mes.]. Le lien dort en génération 1 et ne remonte en génération 4 que sous charge ; 216 relevés en génération 1, 118 en génération 4.

Données : `fondations/journaux/01-machine/carte-lien-pcie.csv`.

Le lien change de génération. Relevé pendant les copies (figure 3.4), le lien est en génération 1 au repos, en génération 4 sous charge, en génération 3 sur batterie. Le niveau bas d'environ 3,8 Go/s des séries lentes ressemble au plafond d'un lien de génération 2 ×8 (4 Go/s) ; non prouvé. **Toute mesure de transfert doit relever la génération du lien pendant la mesure, pas avant.** L'étude du chargement l'a aussi vu (chapitre 7) : un pipeline qui ne fait que copier laisse le lien en génération 3 ; il faut que la carte calcule pendant qu'on copie.

3.6 La carte

VRAM. Un shader de lecture (somme de `vec4<u32>` sur 512 Mio, somme vérifiée contre le processeur) atteint **251,3 Go/s, 98,2 % du plafond** de 256 Go/s. En dessous de 32 Mio, les copies dépassent 256 Go/s : c'est le cache L2, attendu, et c'est le témoin positif du cache.

Calcul. En chaînes indépendantes, horodatées par la carte (Vulkan) : FMA f32 jusqu'à 12,6 Tflop/s, f16 13,4, `dot4I8Packed` 12,3 Top/s, soit 77 % du plafond calculé (24 SM × 128 voies × 2 × 2,67 GHz = 16,4 Tflop/s). Un décodage 4 bits (décalage, masque, conversion, FMA) depuis les registres va à **2,61 Top/s, soit 1,31 billion de poids par seconde** : 2,6 fois ce que la VRAM livre en 4 bits. Les témoins comparent à une référence du processeur exécutant la même suite (écart 0, 0 et $4 \cdot 10^{-7}$) ; forcés à faux, ils tombent.

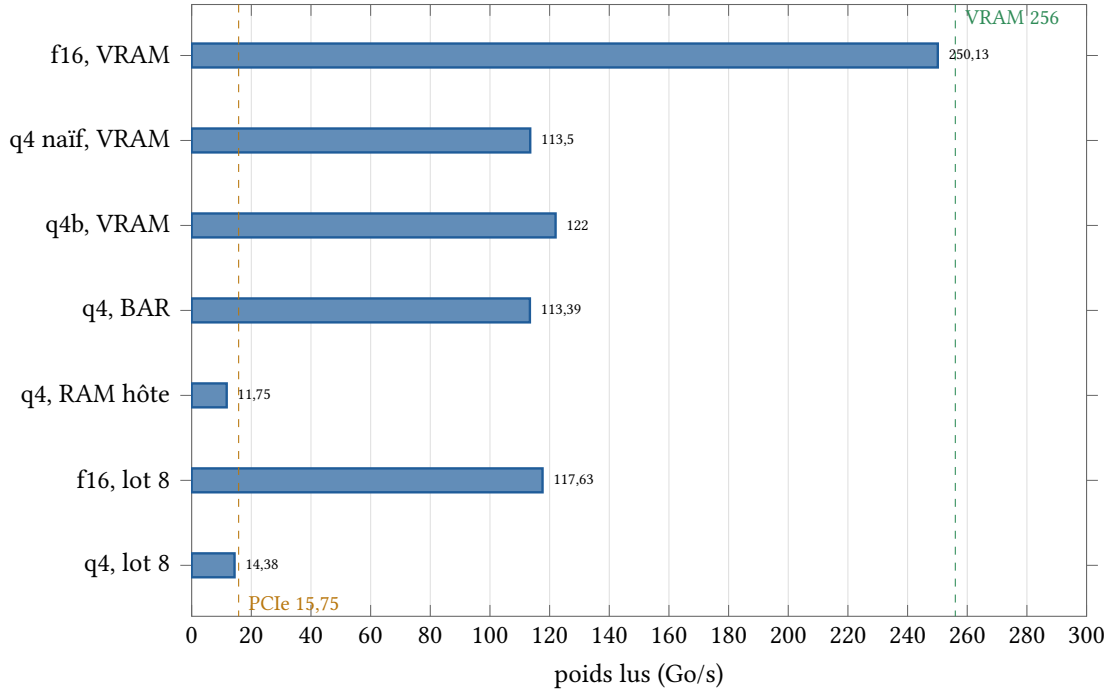


Figure 3.5 : Produits matrice-vecteur écrits ici (Vulkan, RTX 4060), meilleure médiane horodatée par la carte, témoin tenu [mes.]. Le f16 atteint la VRAM ; le 4 bits plafonne à 122 Go/s avec ces noyaux ; avec les poids laissés en RAM hôte (zéro copie), la carte calcule au débit du lien.

Données : `fondations/journaux/01-machine/carte.jsonl` et `carte-complement.jsonl`.

Le produit matrice-vecteur. C'est le cœur d'un jeton en décodage : chaque matrice de poids multiplie un seul vecteur d'activations, chaque poids n'est utilisé qu'une fois, et le noyau est borné par la mémoire. La figure 3.5 en tire quatre faits. Le f16 atteint la VRAM (250 Go/s) : la machine sait lire ses 251 Go/s dans un vrai calcul. Le 4 bits plafonne à 122 Go/s alors que le décodage depuis les registres coûte 2,6 fois moins que ce que la VRAM livre : le manque est dans le noyau (accès aux échelles et au vecteur, parallélisme mémoire), pas dans la machine. **En zéro copie**, la carte calcule une GEMV dont les poids restent en RAM hôte à 11,75 Go/s, le débit de la DMA : pour un modèle plus gros que la VRAM, on peut consommer les poids directement depuis la RAM épinglée, sans tampon de transit. Enfin, les lots de 8 vecteurs s'effondrent avec ces noyaux, qui relisent le vecteur pour chaque bloc : noyau à refaire avant tout usage en décodage spéculatif.

3.6.1 Thermique, et la découverte de la batterie

La règle déclarée avant la passe thermique dit « bridage si les 30 dernières secondes tombent sous 95 % des 30 premières ». Sur secteur, elle est tenue (rapport 1,001, bruit 6,3 %). Ce test n'est pas une charge maximale : la boucle soumet 20 GEMV puis attend, la carte n'est occupée qu'à 70 % environ ;

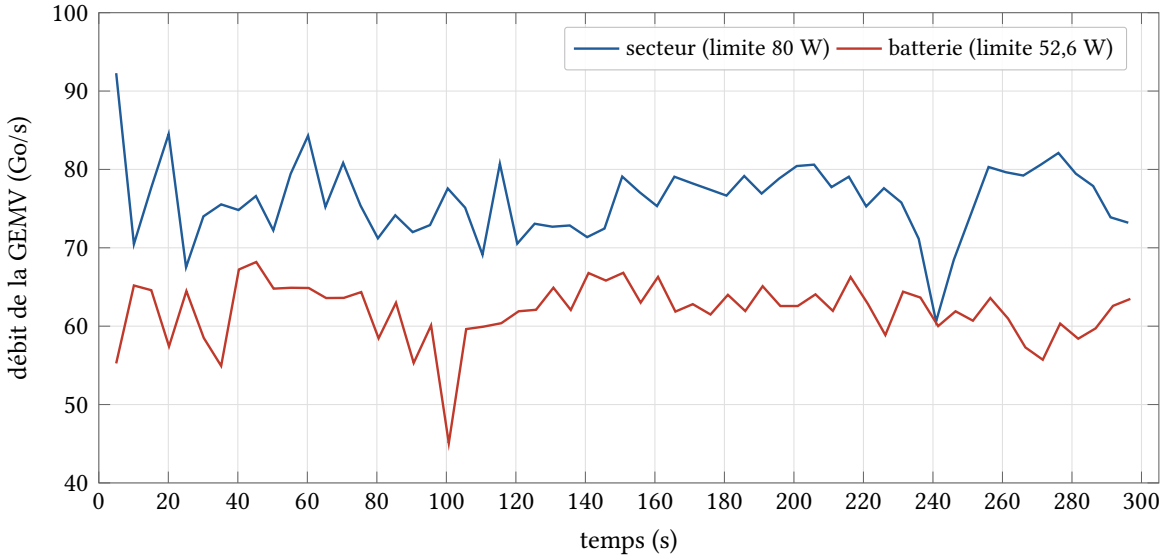


Figure 3.6 : Cinq minutes de GEMV q4b en boucle, débit par tranche de 5 s [mes.]. Sur secteur : rapport fin/début 1,001, aucune raison de bridage, 2 685 MHz, 75 °C, 72 W. Sur batterie : la carte est bridée dès la première seconde (plafond imposé 52,6 W, mémoire à 6 001 MHz).

Données : `fondations/journaux/01-machine/thermique-secteur-charge.csv`, `thermique-batterie.csv`.

c'est une charge de décodage, celle du projet.

La deuxième passe, faite à 07:25, a révélé que **la machine était sur batterie**, et que **sur batterie la machine est une autre machine** : le pilote abaisse la limite de puissance de la carte de 80 W à 52,6 W, la mémoire de la carte reste à 6 001 MHz (25 % de débit de VRAM en moins, 192 Go/s au plus [calc.]), le lien redescend en génération 3, le processeur tourne vers 1,5 GHz. Sur des mesures courtes, la DMA hôte vers carte reste à 11,8 Go/s, mais l'écriture du processeur dans le BAR tombe de 9,1 à 1,7 Go/s et `write_buffer` de 7,5 à 1,25 Go/s : **tout ce qui passe par le processeur s'effondre**. Pour le projet, un chargeur de poids qui compte sur le processeur perd un facteur 4 à 6 sur batterie ; un chargeur qui ne fait que de la DMA et décode sur la carte perd peu. C'est un argument mesuré pour l'architecture B (section 3.8).

3.7 La base de comparaison : llama.cpp

Le pilote `pilote_llama.py` lance `llama-bench` [85] (b11515, moteur CUDA 12.4) sur Qwen3.5-9B Q4_K_M (5 680 522 464 octets, 8 953 803 264 paramètres), génération seule, une configuration par processus, en ordre ABBA, et relève pendant la vie du processus la mémoire de pointe, les fautes de page et la VRAM.

Lecture prudente (tableau 3.4) : 34,1 jetons/s tout en VRAM font 69 % du toit de 49,3 (section 3.8), soit $34{,}1 \times 5{,}10 = 174$ Go/s de poids lus [calc.], mieux que nos 122 Go/s. L'écart entre mmap et lecture directe (16,5 contre 34,1) ne vient pas du chargement (les poids sont en VRAM dans les deux cas) : un autre processus a pris de la VRAM pendant la mesure (la passe `-ngl 24` ajoute plus de VRAM que la passe tout en VRAM). On ne retient que l'ordre de grandeur ; l'ABBA complet attend une machine calme sur secteur. Ce qui est sûr : avec huit couches sur 32 laissées au processeur, la génération tombe d'un facteur 12.

Table 3.4 : llama.cpp b11515 sur Qwen3.5-9B Q4_K_M, premier passage (3 configurations sur 12, machine partagée) [mes.].

`-ngl`	chargement	jetons/s	é.-t.	ens. de travail	VRAM ajoutée	fautes
99 (tout en VRAM)	mmap	16,5	1,2	5 279 Mio	5 118 Mio	1 376 607
99	lecture directe	**34,1**	5,4	**964 Mio**	5 083 Mio	270 240
24 (sur 32 couches)	mmap	**2,8**	2,1	5 296 Mio	7 114 Mio	1 563 456

3.8 Le toit

3.8.1 Le modèle

Un jeton de décodage, au lot de un, lit B octets de poids : tous les poids qu'il utilise, une fois chacun. Les poids résident sur trois étages : une fraction f_V en VRAM, f_R en RAM hôte, f_D sur le disque ($f_V + f_R + f_D = 1$), la part disque stockée compressée d'un facteur r (octets servis sur octets stockés). Chaque étage i a un débit effectif mesuré W_i : $W_{\text{VRAM}} = 251{,}3$, $W_{\text{PCIe}} = 12{,}2$, $W_{\text{disque}} = 2{,}82$ Go/s (`debits-mesures.json`, chaque valeur avec son journal). La figure 3.7 résume ce chemin, la figure 3.8 le rendement de chaque étage.

On distingue deux architectures (figure 3.9), selon l'endroit où se fait la décompression des octets venus du disque :

Architecture A (le processeur décompresse, la PCIe porte les octets servis) : $O_{\text{VRAM}} = f_V B$, $O_{\text{PCIe}} = (f_R + f_D)B$, $O_{\text{disque}} = f_D B/r$, et le décodeur du processeur doit sortir $f_D B$ octets.
Architecture B (la carte décompresse, la PCIe porte les octets compressés) : $O_{\text{VRAM}} = f_V B$, $O_{\text{PCIe}} = (f_R + f_D/r)B$, $O_{\text{disque}} = f_D B/r$, et le décodeur de la carte doit sortir $f_D B$ octets, de préférence dans les registres, sans les réécrire en VRAM.

3.8.2 Le théorème du toit

On modélise un étage par sa seule contrainte de débit.

Hypothèse 3.1 (Étage à débit borné)**.** Sur tout intervalle de temps de longueur τ, l'étage i fait passer au plus $W_i\tau$ octets. Un jeton est terminé quand chaque étage a fait passer ses O_i octets.

Théorème 3.2 (Borne de débit d'une chaîne d'étages)**.** *Sous l'hypothèse 3.1, quel que soit l'ordonnancement, le temps t d'un jeton vérifie*

$$t \;\geq\; \max_i \frac{O_i}{W_i}, \qquad \textit{donc} \qquad \textit{jetons/s} \;\leq\; \min_i \frac{W_i}{O_i}. \tag{3.2}$$

Si les étages travaillent l'un après l'autre sans se recouvrir, $t = \sum_i O_i/W_i$.

Démonstration. Soit $[0, t]$ l'intervalle pendant lequel le jeton est traité. Par l'hypothèse 3.1, l'étage i fait passer au plus $W_i t$ octets dans cet intervalle ; il doit en faire passer O_i, donc $W_i t \geq O_i$, soit $t \geq O_i/W_i$, pour chaque i. La borne vaut pour le maximum. Sans recouvrement, les intervalles d'activité des étages sont disjoints et chacun dure au moins O_i/W_i ; s'ils travaillent à leur débit, chacun dure exactement O_i/W_i, et les durées s'additionnent. ∎

Pour un modèle dense lu entièrement par un seul étage, $B = Nb/8$ et le théorème donne la forme qui sert dans tout le mémoire :

$$\text{jetons/s} \;\leq\; \frac{8\,W}{N\,b}. \tag{3.3}$$

Sa pente en b est $\partial \ln(\text{jetons/s})/\partial \ln b = -1$: diviser le nombre de bits par poids par deux double le plafond, à n'importe quel étage. Aucun autre paramètre de la chaîne n'a cette pente sans contrepartie.

La borne est-elle atteignable ? Oui, à une pénalité de remplissage près que l'on peut chiffrer. Découpons le jeton en L morceaux égaux (les couches) que chaque étage traite dans l'ordre ; le morceau ℓ ne peut entrer à l'étage $i+1$ qu'une fois sorti de l'étage i (double tampon entre étages). C'est un atelier à cheminement unique, à tâches identiques.

Lemme 3.3 (Atelier à tâches identiques)**.** *Soient m étages en série et L tâches identiques, la tâche demandant p_i secondes à l'étage i, chaque étage traitant les tâches dans l'ordre, une à la fois. La durée totale est*

$$T_L \;=\; \sum_{i=1}^{m} p_i \;+\; (L-1)\,\max_i p_i. \tag{3.4}$$

Démonstration. Notons $C_{i,\ell}$ l'instant où la tâche ℓ quitte l'étage i. Les contraintes donnent la récurrence $C_{i,\ell} = \max(C_{i-1,\ell}, C_{i,\ell-1}) + p_i$, avec $C_{0,\ell} = C_{i,0} = 0$. En la déroulant, $C_{m,L}$ est le maximum, sur tous les chemins de la case (1,1) à la case (m,L) qui avancent d'un étage ou d'une tâche à chaque pas, de la somme des p_i des cases visitées. Un tel chemin visite $m+L-1$ cases et passe au moins une fois par chaque étage ; sa somme vaut $\sum_i p_i$ plus les $L-1$ cases supplémentaires, chacune au plus $\max_i p_i$. Le chemin qui descend tous les étages en passant ses $L-1$ cases supplémentaires sur l'étage le plus lent atteint cette borne. ∎

Corollaire 3.4 (Régime permanent)**.** *Avec L morceaux égaux, $p_i = O_i/(LW_i)$ et le temps par jeton vaut*

$$t_L \;=\; \max_i \frac{O_i}{W_i} \;+\; \frac{1}{L}\Big(\sum_i \frac{O_i}{W_i} - \max_i \frac{O_i}{W_i}\Big). \tag{3.5}$$

Il tend vers la borne du théorème 3.2 quand L croît, et la pénalité de remplissage est au plus $1/L$ de l'écart entre la somme et le maximum. Si les poids du jeton suivant peuvent être lus pendant le calcul du jeton courant (modèle dense, fenêtre de préchargement suffisante), le remplissage ne se paie qu'une fois et le temps par jeton tend vers $\max_i O_i/W_i$.

Démonstration. Remplacer p_i dans l'équation (3.4) et diviser par le nombre de jetons (un). ∎

Pour gpt-oss-120b, $L = 36$ couches : même sans recouvrement entre jetons, le temps par jeton est à moins de 3 % de l'écart somme–maximum au-dessus de la borne. La figure 3.10 montre les deux régimes.

3.8.3 Les deux corollaires qui décident de l'architecture

Corollaire 3.5 (Le facteur de croisement)**.** *En architecture A, avec tous les poids hors VRAM sur le disque ($f_R = 0$), le disque cesse d'être l'étage limitant dès que*

$$r \;\geq\; r^\star = \frac{W_{\text{PCIe}}}{W_{\text{disque}}} = \frac{12{,}2}{2{,}82} = 4{,}33 \quad \textit{[calc.]}. \tag{3.6}$$

Au-delà, compresser davantage ne sert à rien en A. Avec une part f_R en RAM, le seuil devient $r^\star\, f_D/(f_R+f_D)$, plus petit.

Démonstration. En A, $O_{\text{disque}}/W_{\text{disque}} = f_D B/(rW_{\text{disque}})$ et $O_{\text{PCIe}}/W_{\text{PCIe}} = (f_R+f_D)B/W_{\text{PCIe}}$. Le disque n'est plus limitant quand le premier terme ne dépasse pas le second, soit $r \geq W_{\text{PCIe}} f_D/(W_{\text{disque}}(f_R+f_D))$, qui vaut $r^\star$ pour $f_R = 0$. Au-delà, le maximum du théorème 3.2 est le terme PCIe, qui ne dépend pas de r. ∎

Corollaire 3.6 (En architecture B, pas de croisement)**.** *En architecture B avec $f_R = 0$, le disque reste l'étage limitant pour tout r, et le plafond du flux disque croît comme $r\,W_{\text{disque}}/(f_D B)$ jusqu'à ce que le décodeur de la carte ou la VRAM le borne.*

Démonstration. En B, disque et PCIe portent tous deux $f_D B/r$; le rapport de leurs temps est $W_{\text{PCIe}}/W_{\text{disque}} = 4{,}33 > 1$, indépendant de r : le disque est toujours le plus lent des deux. Son temps $f_D B/(rW_{\text{disque}})$ décroît en $1/r$, ce qui donne le plafond annoncé, jusqu'à ce qu'un autre terme (le décodeur, la VRAM) le dépasse. ■

La borne haute du décodeur de la carte se lit dans la mesure du section 3.6 : 1,31 billion de poids par seconde décodés depuis les registres, soit $1{,}31 \cdot 10^{12} \times 5{,}08/8 = 832$ Go/s servis [calc.]. Ce n'est pas une borne de décodage entropique (le chapitre 10 y revient), mais elle dit que, sur cette carte, le calcul n'est pas l'étage limitant.

3.8.4 Les plafonds par étage

Table 3.5 : Plafonds de jetons par seconde si tous les poids lus passent par l'étage nommé [calc.] sur mesures. B : octets lus par jeton (pour les MoE, toujours-lus plus experts, routage uniforme).

Modèle	B (Go)	VRAM	PCIe	disque, $r = 1$	disque, $r = r^\star$
Qwen3.5-9B Q4_K_M	5,10	**49,3**	2,39	**0,55**	2,39
Qwen3.5-4B Q4_K_M	2,73	92,1	4,47	1,03	4,47
gpt-oss-120b MXFP4	3,60	69,8	3,39	0,78	3,39
Qwen3.8-Flash-Next UD-Q2_K_XL	4,44	56,6	2,75	0,64	2,75
Kimi-K2.6 UD-Q2_K_XL	15,42	16,3	0,79	0,18	0,79

Données : `fondations/journaux/01-machine/toit-modeles.csv`.

Le placement réel (figure 3.12) suppose, comme l'étude des modèles (chapitre 9), 6,5 Go de VRAM et 11 Go de RAM disponibles pour des poids, les toujours-lus d'abord en VRAM, les experts répartis dans le même ordre, un routage uniforme et aucun cache dynamique : c'est la borne pessimiste. Pour gpt-oss-120b, il donne **1,98 jeton/s** recouvert (1,53 en série) : 1,84 Go par jeton servis par la VRAM, 0,34 par la RAM, 1,42 par le disque.

Second chemin, et ce qu'il corrige. L'outil de l'étude des modèles (`modeles_physique.py`) donne 5,1 à 5,6 jetons/s pour gpt-oss-120b. Refait avec ses propres hypothèses, l'outil du toit retrouve le même ordre (1,42 Go par jeton à 7,877 Go/s, soit 5,5 jetons/s) : les deux modèles s'accordent. L'écart vient du débit du disque, le plafond du lien d'un côté, la mesure de l'autre : **le plafond réel de gpt-oss-120b sur cette machine, sans cache et sans compression, est d'environ 2 jetons/s, pas 5,5.** Le juge de l'étude du chargement, relancé avec le disque mesuré, retrouve 1,99 jeton/s par un troisième chemin (chapitre 7).

3.8.5 Le budget du décodeur, et pourquoi B

À un débit d'entrée compressé égal au lien (12,2 Go/s, architecture A au croisement), la carte dispose de $2{,}61 \cdot 10^{12}/12{,}2 \cdot 10^{9} \approx$ **214** opérations par octet compressé [calc.] ; au débit du disque (architecture B), **926**. Un décodeur entropique de la famille rANS fait de l'ordre de la dizaine d'opérations par symbole : le calcul n'est pas la borne, la question est le parallélisme et l'accès mémoire du décodeur.

Trois raisons mesurées de préférer l'architecture B : en A, tout facteur au-delà de 4,33 est perdu (corollaire 3.5) ; en A, le processeur paie déjà 0,24 à 0,31 s de cœur par Go pour BitLocker avant toute décompression (section 3.3.4) ; sur batterie, tout ce qui passe par le processeur perd un facteur 4 à 6 alors que la DMA reste à 11,8 Go/s (section 3.6.1). Et la carte sait consommer des poids sans les copier en VRAM (zéro copie, 11,75 Go/s).

Ce que $r^\star$ veut dire en bits. Le Q4_K_M sert en moyenne 5,08 bits par poids ; stocké à $r^\star = 4{,}33$, il tiendrait en $5{,}08/4{,}33 = \mathbf{1{,}17}$ bit par poids sur le disque. ~~soit environ 1,05 bit par poids stocké pour 4,5 bits servis~~ (barré dans l'étude d'origine : le Q4_K_M n'est pas à 4,5 bits, il mêle des tenseurs Q6_K). Aucun code sans perte n'y arrive sur des codes de 4 bits, dont l'entropie mesurée laisse quelques pour cent : $r^\star$ ne s'atteint qu'en changeant de représentation (chapitre 5). Sur un gros MoE, la compression n'est d'ailleurs pas le seul levier : le taux de succès du cache d'experts multiplie l'effet de r (chapitres 7 et 9).

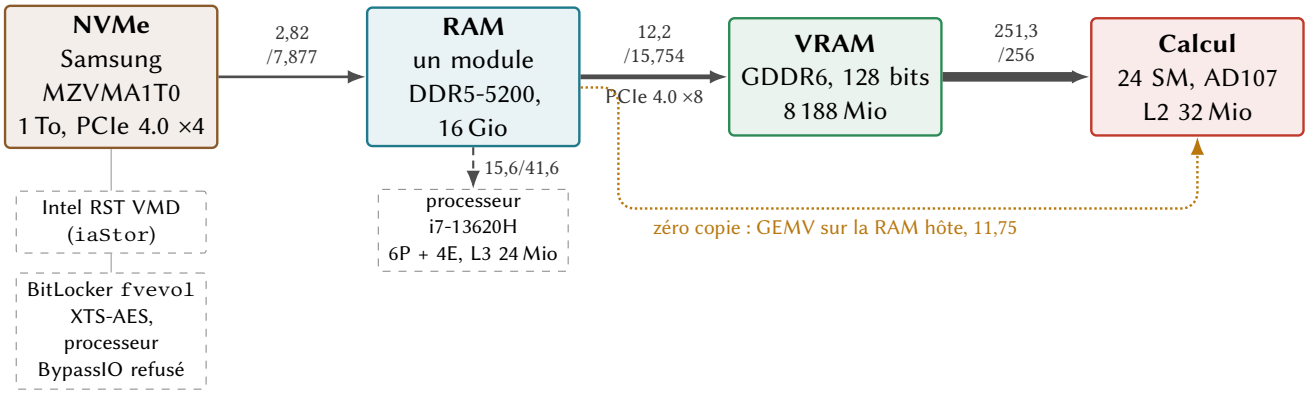
3.9 Ce qui manque dans le monde

1. **Un toit mesuré, pas annoncé, pour l'inférence hors mémoire sur portable.** Les travaux d'inférence depuis le stockage raisonnent sur des débits de fiche technique ou de station de travail. Ici, le disque livre 47 % de son annonce à cause de deux réglages par défaut ; un plafond calculé au lien est faux d'un facteur 2,8.
2. **Le régime batterie.** Aucun banc d'inférence lu ne note la source d'alimentation ; elle change ici la limite de puissance de la carte, l'horloge de sa mémoire, la génération du lien, et divise par 4 à 6 tout ce qui passe par le processeur.
3. **Le coût processeur de BitLocker pour un flux de poids**, 0,24 à 0,31 s de cœur par Go, et la question ouverte de savoir si `fvevol` parallélise.
4. **La décompression sur la carte, alimentée par DMA depuis la RAM épinglée ou en zéro copie, sous Vulkan.** DirectStorage décompresse sur la carte, mais vers des ressources Direct3D 12 et pour des textures ; les moteurs d'inférence décodent leurs formats sur la carte, mais depuis des poids déjà en VRAM. Le chemin « disque → RAM épinglée → PCIe (octets compressés) → décodage sur la carte » n'existe, à notre connaissance, nulle part sous Vulkan.
5. **Un noyau 4 bits Vulkan au niveau de la VRAM** : nos noyaux plafonnent à 48 %, llama.cpp sous CUDA atteint 69 % ; la machine offre 98 %.

Ce qui pourrait être faux

- **La machine n'a jamais été au calme.** Les chiffres « au mieux » sont des bornes inférieures du repos ; le plafond du disque est le plus exposé (jusqu'à 0,6 Go/s de lectures étrangères au passage A).
- **Le toit suppose des débits indépendants.** La RAM est partagée entre la DMA vers la carte, la lecture du disque et le processeur, sur un seul module. Le recouvrement parfait peut être inatteignable ; la colonne en série encadre par le bas.
- $B = 5{,}10$ **Go par jeton** ignore le cache KV et les activations : juste pour un contexte court, trop bas pour un long contexte.
- **L'identification du disque au PM9C1a** est probable, pas prouvée.
- **La latence RAM attribuée à la traduction imbriquée** est une hypothèse.
- **Les mesures de llama.cpp sont contaminées** par une autre charge sur la carte ; l'écart mmap / lecture directe n'est pas un fait.

- **Le test thermique n'est pas une charge maximale.**
- **La borne du décodeur (832 Go/s servis)** mesure un décodage de champs de 4 bits ; un décodeur entropique réel a d'autres goulets.
- **La clé XTS de BitLocker** (128 ou 256 bits) n'est pas connue ; l'écart est d'environ 10 %.



En gras : meilleur débit reproductible mesuré ici (Go/s). Après la barre : plafond calculé de l'étage.

Figure 3.7 : Le chemin des données sur la machine de Stone, du disque aux unités de calcul. Chaque flèche porte le meilleur débit reproductible mesuré ici et le plafond calculé de l'étage ; l'épaisseur croît avec le débit. Le chemin pointillé est le « zéro copie » : la carte calcule directement sur des poids en RAM hôte, à travers le lien.

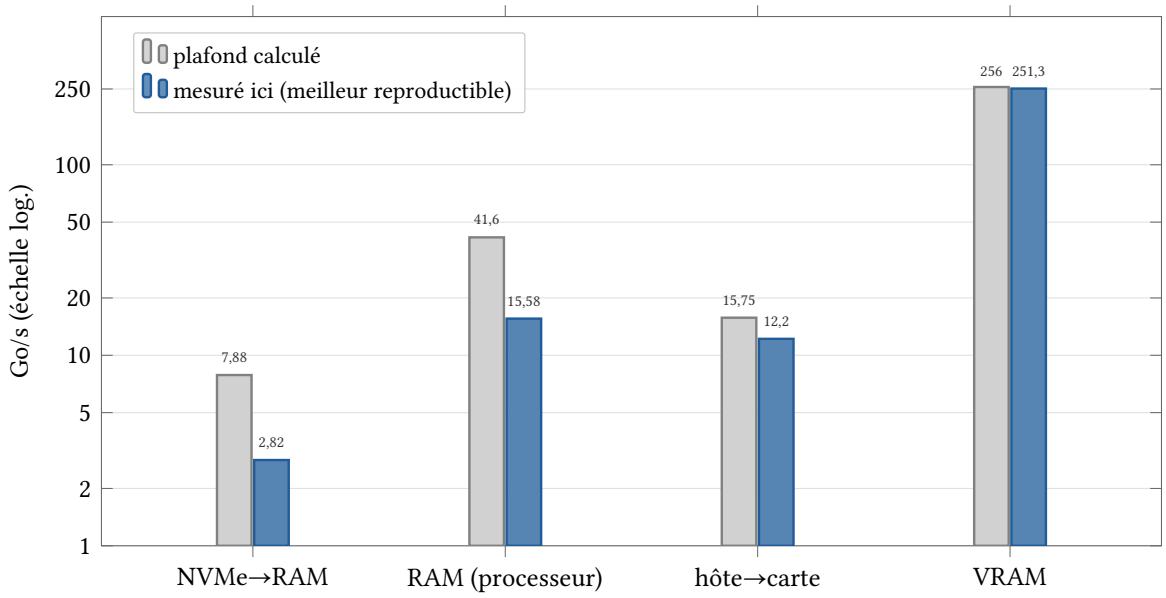
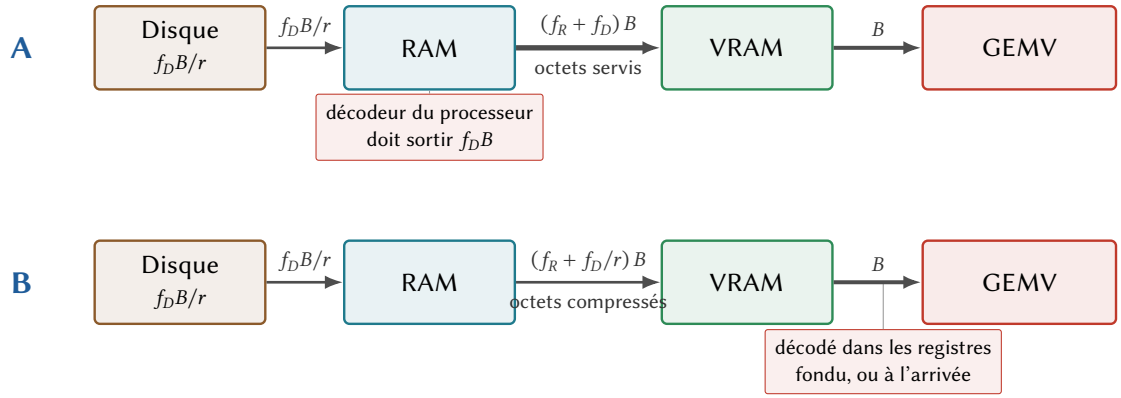


Figure 3.8 : Plafond calculé contre débit mesuré, par étage (échelle logarithmique) [mes.] [calc.]. Le disque rend 36 % de son lien, la RAM 37 % d'un module, la DMA 77 % du lien PCIe, la VRAM 98 %.

Données : `fondations/journaux/01-machine/debits-mesures.json`, `memoire.log`.



En A, tout facteur r au-delà de $r^\star = W_{\text{PCIe}}/W_{\text{disque}} = 4{,}33$ est perdu : le lien porte les octets servis. En B, disque et lien portent tous deux $f_D B/r$; le toit croît avec r jusqu'à ce que le décodeur de la carte ou la VRAM le borne (corollaire 3.6).

Figure 3.9 : Les deux architectures. En A, le processeur décompresse et le lien porte les octets servis ; en B, la carte décompresse et le lien porte les octets compressés. Les quantités sur les flèches sont les octets par jeton.

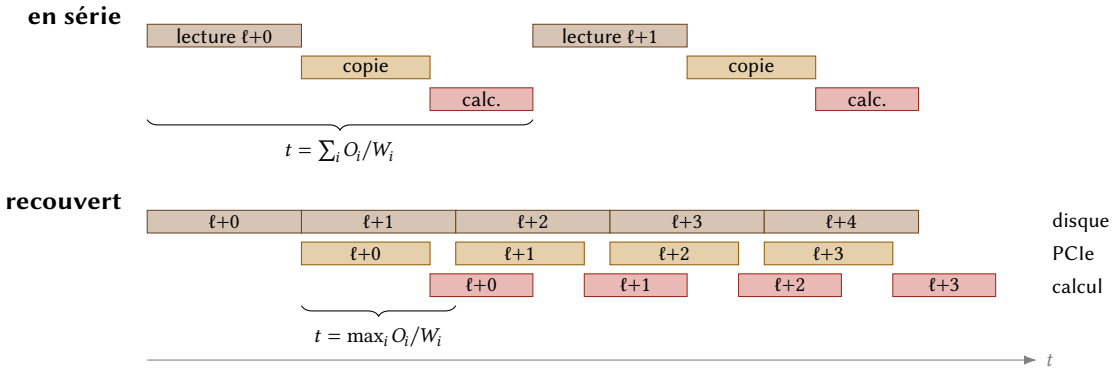


Figure 3.10 : Chronogramme des étages pour une suite de couches. En série, le temps est la somme des étages ; avec un double tampon, chaque étage travaille sur une couche différente et le plus lent fixe le pas (lemme 3.3 et corollaire 3.4).

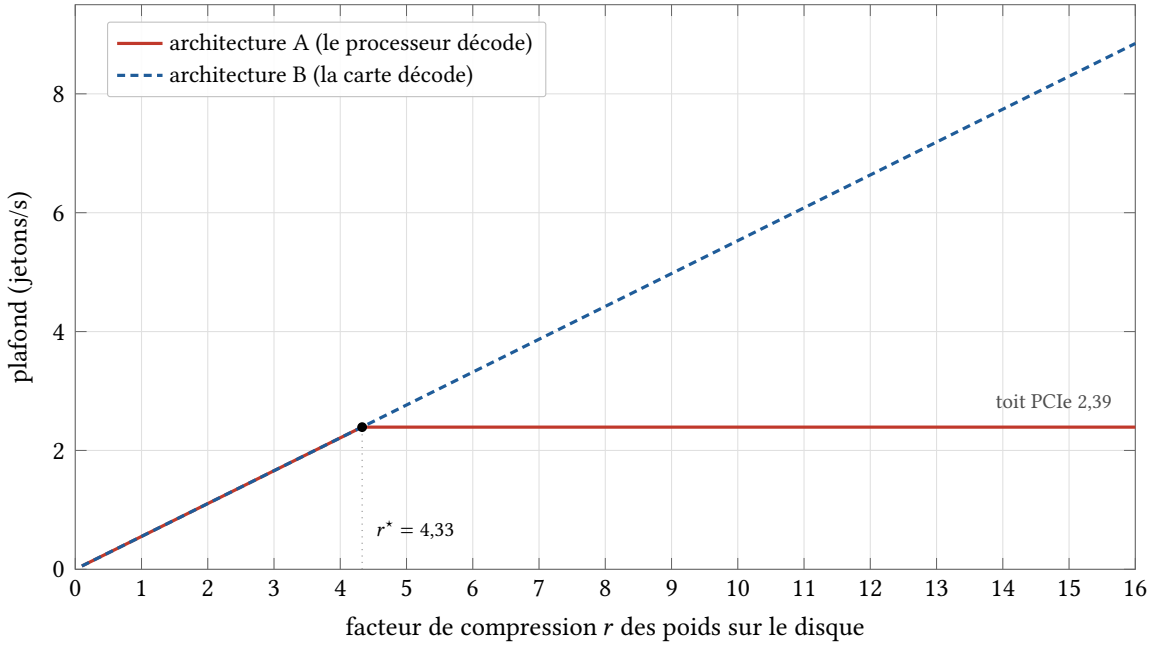


Figure 3.11 : Plafond de jetons par seconde du Qwen3.5-9B Q4_K_M ($B = 5{,}10$ Go par jeton) entièrement sur disque, selon le facteur de compression r [calc.]. En architecture A, tout facteur au-delà de $r^\star = 4{,}33$ est perdu ; en architecture B, le plafond continue de croître.

Données : `fondations/journaux/01-machine/toit-9b-compression.csv` (outil `outils/banc/toit.py`).

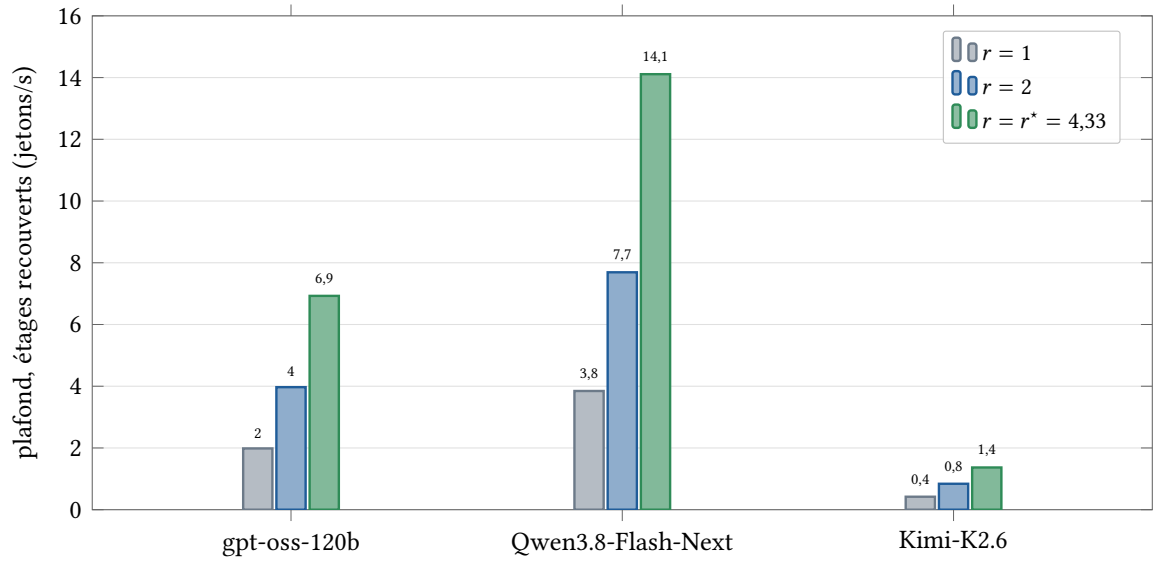


Figure 3.12 : Plafond (étages recouverts) des trois MoE visés sur la machine, architecture A, routage uniforme, sans cache, pour trois facteurs de compression de la part disque [calc.]. Le plancher sans recouvrement est dans le fichier source.

Données : `fondations/journaux/01-machine/toit-placements.csv`.

Chapitre 4

Formulation du problème

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, un théorème et quatre leviers mesurés chacun de son côté : la borne de débit d'une chaîne d'étages (théorème 3.2), puis la quantification, le chargement, la spéculation et les modèles visés (chapitres 5 et 7 à 9). **Ce chapitre établit** une seule équation qui les contient tous, le temps d'un jeton d'un mélange d'experts servi par la carte, terme par terme ; un modèle de localité à une constante, avec ses trois scénarios calés sur la littérature ; le problème d'optimisation que troncons doit résoudre, avec ses contraintes de mémoire et de qualité ; l'élasticité du plafond à chaque grandeur, démontrée, qui dit lesquelles comptent et lesquelles sont de l'ornement sur cette machine ; un exemple posé pas à pas ; le terme qui manquait à la condition de la spéculation ; et les témoins qui font tomber l'équation quand on la fausse. **Il reste** à remplacer le modèle de localité par la courbe mesurée de gpt-oss-120b, et la base estimée par la base mesurée (chapitre 10). Source : `fondations/07-ce-qui-manque.md` (§ 1 et annexe A), outil `outils/07-synthese/plafonds.py`, journal `fondations/journaux/07-synthese/plafonds.json`.

4.1 Le jeton comme chaîne d'étages

Un jeton de décodage, au lot de un, applique chaque poids actif une fois. Ses octets traversent la chaîne de la figure 3.7 : disque, RAM, lien PCIe, VRAM, unités de calcul. Le théorème 3.2 borne son temps par le maximum des temps d'étage, et le corollaire 3.4 dit que ce maximum s'atteint à une pénalité de remplissage près. Trois termes manquent encore pour décrire un vrai moteur ; les études les ont rencontrés un par un.

Définition 4.1 (Équation du jeton)**.** Le temps d'un jeton produit est

$$t \;=\; \frac{1}{\mathscr{E}}\Big[\max\big(T_{\mathrm{D}}, T_{\mathrm{P}}, T_{\mathrm{G}}\big) + t_{\mathrm{lat}}\Big] \;+\; t_0, \tag{4.1}$$

où T_{D}, T_{P}, T_{G} sont les temps d'occupation du disque, du lien et de la carte (section 4.2), et :

- t_0 est le coût fixe d'un jeton (lancements de noyaux, synchronisations, couches récurrentes) ;
- $t_{\mathrm{lat}} \geq 0$ est la latence de lecture que le préchargement n'a pas cachée ; elle vaut zéro si la file du disque ne se vide jamais ;
- $\mathscr{E} \geq 1$ est le nombre moyen de jetons rendus par une lecture des poids (théorème 8.3) ; $\mathscr{E} = 1$ sans spéculation.

D'où vient chaque terme. $t_0 \approx 3{,}5\,\mathrm{ms}$ [calc.] sur une mesure de l'étude de la quantification : Qwen3.5-4B Q4_K_M tout en VRAM rend de 68,9 à 69,9 jetons/s selon la série ABBA, soit 14,3 à 14,5 ms par jeton, alors que ses 2,73 Go lus à 251 Go/s n'en demandent que 10,9 (`fondations/journaux/02-quantification/vitesse-4B-3.json`, champ `reference_jetons_s`). Le terme t_{lat}

est l'écart entre le plafond (le maximum) et le plancher (la somme) du théorème 3.2 ; il peut dominer : chez Edge0, le temps bloqué sur les lectures va de 101,9 à 575 ms par pas alors que le disque travaille au plus 12 % du pas [litt.] (chapitre 7). Le facteur $\mathscr{E}$ est celui du chapitre 8 ; pour un mélange d'experts, la lecture d'une passe de vérification grossit avec le nombre de jetons vérifiés, ce que traite la section 4.9.

4.2 Les termes pour un mélange d'experts servi par la carte

On reprend les grandeurs du chapitre 7 : B_t octets toujours lus (attention, routeurs, normes, tête), résidents en VRAM ; B_e octets d'experts routés lus par jeton ; E_{tot} octets de tous les experts. Une part h_v des octets d'experts est servie par la VRAM, h_r par la RAM, $h_s = 1 - h_v - h_r$ par le disque. Les octets qui traversent le disque et le lien sont compressés des facteurs r_d et r_p (octets servis sur octets transportés). La carte décode à W_{dec} (octets servis produits par seconde) et calcule le produit matrice-vecteur à W_g sur des poids décodés. Enfin $\delta = \max(0, B_t - V)$ est la part des toujours-lus qui ne tient pas dans la VRAM utile V.

Proposition 4.2 (Temps d'étage d'un MoE dont la carte décode tout)**.** *Si la carte calcule toutes les couches et décode à l'arrivée tout ce qui ne vient pas de sa propre mémoire, les temps d'étage de l'équation (4.1) sont*

$$T_{\text{D}} = \frac{h_s\, B_e}{r_d\, W_{\text{D}}}, \tag{4.2}$$

$$T_{\text{P}} = \frac{1}{W_{\text{P}}}\Big(\frac{h_r\, B_e}{r_p} + \frac{h_s\, B_e}{r_d} + \delta\Big), \tag{4.3}$$

$$T_{\text{G}} = \frac{(1 - h_v)\, B_e}{W_{\text{dec}}} + \frac{B_t + B_e}{W_g}. \tag{4.4}$$

Démonstration. Le disque ne lit que les octets d'experts absents de la VRAM et de la RAM, soit $h_s B_e$ octets servis, transportés sous $h_s B_e / r_d$ octets ; d'où l'équation (4.2) par l'hypothèse 3.1. Le lien porte tout ce qui entre dans la carte : les experts venus de la RAM (compressés de r_p), ceux venus du disque (compressés de r_d, puisqu'ils ne sont décodés qu'à l'arrivée) et le débord des toujours-lus. La carte décode tout ce qui n'était pas déjà décodé dans sa mémoire, $(1 - h_v)B_e$ octets servis, puis lit une fois chaque poids actif, $B_t + B_e$ octets, pour le produit. Décodeur et produit occupent les mêmes multiprocesseurs : leurs temps s'additionnent au lieu de se recouvrir. ■

Remarque 4.3 (Trois sens du mot « architecture »). Au section 3.8, l'architecture A est celle où le processeur décompresse, l'architecture B celle où la carte décompresse ; le corollaire 3.6 y a tranché pour B. La proposition 4.2 est écrite pour cette architecture B. Les trois *candidates* A, B et C du chapitre 10 sont toutes bâties sur elle et s'opposent sur un autre axe : la précision servie à chaque étage. On garde les noms des études pour ne pas casser leurs renvois.

Le disque et le lien se recouvrent avec la carte : leurs moteurs de copie sont libres pendant le calcul, et la part cachée mesurée (équation (7.1)) vaut 0,98 à 1,01 [mes.] (`fondations/journaux/04-chargement/`). C'est ce qui autorise le maximum de l'équation (4.1).

4.3 La localité, en une constante

Les parts h_v et h_r dépendent de la capacité de chaque étage et de la façon dont les jetons successifs réutilisent leurs experts. Faute de mesure sur le modèle visé, on pose un modèle à une seule constante.

Hypothèse 4.4 (Succès en loi de puissance). Si un étage contient une part c des octets d'experts, il sert une part $h(c) = c^{\beta}$ des octets demandés, avec $0 < \beta \leq 1$. Pour deux étages emboîtés (VRAM de capacité c_v, RAM de capacité c_r), $h_v = h(c_v)$ et $h_v + h_r = h(c_v + c_r)$.

$\beta = 1$ est le routage uniforme, sans aucune réutilisation : le pire cas, celui de l'étude des modèles. Les deux autres valeurs se calent sur le seul point publié exploitable, un cache sur une couche de Qwen3-30B-A3B (128 experts, 8 actifs) : à 13,4 % du budget, un LRU sert 65,9 % des demandes et l'oracle de Belady 79,1 % [litt.] (chapitre 7). Une seule constante fait passer une seule courbe par un point :

$$\beta_{\text{LRU}} = \frac{\ln 0{,}659}{\ln 0{,}134} = \frac{-0{,}4170}{-2{,}0099} = 0{,}2075, \tag{4.5}$$

$$\beta_{\text{Belady}} = \frac{\ln 0{,}791}{\ln 0{,}134} = \frac{-0{,}2345}{-2{,}0099} = 0{,}1167 \quad \text{[calc.]}. \tag{4.6}$$

Second point, pour vérifier : moe-stream implique au moins 53 % de succès à 31 % du budget (lemme 7.1) ; le scénario LRU y donne $0{,}31^{0{,}2075} = 0{,}784 \geq 0{,}53$: compatible. La figure 4.1 montre les trois courbes.

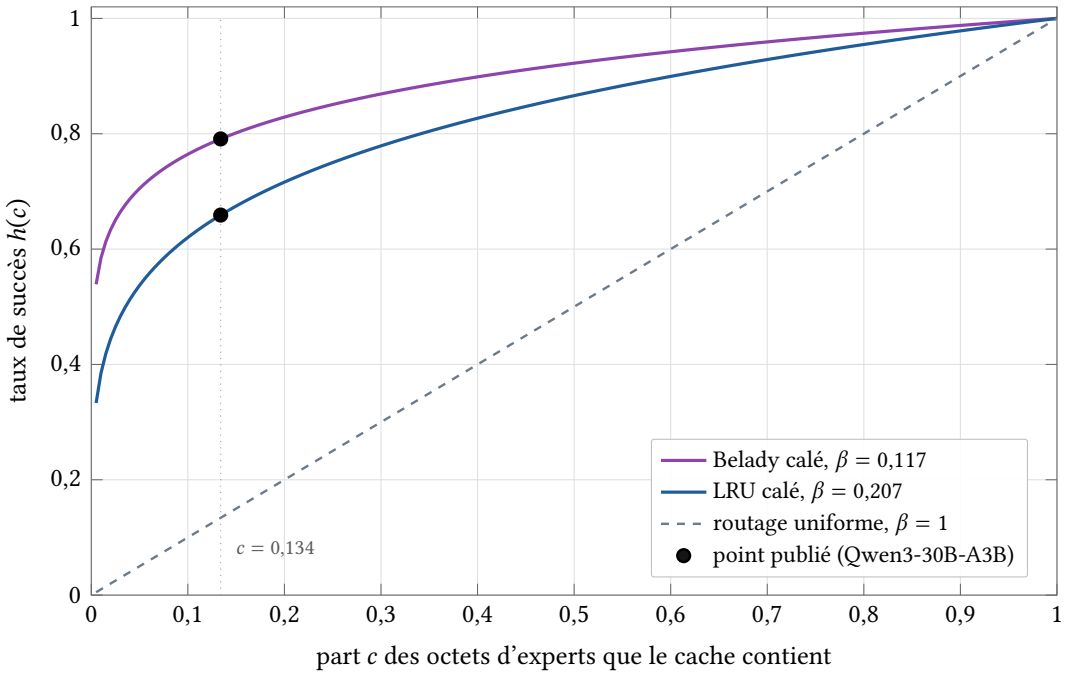


Figure 4.1 : Les trois scénarios de localité $h(c) = c^{\beta}$ [calc.]. Chaque β fait passer sa courbe par le point publié correspondant (LRU, Belady ; Qwen3-30B-A3B, une couche) ; le routage uniforme ($\beta = 1$) est le pire cas. Ce sont des hypothèses, pas des mesures du modèle visé : la première mesure sur gpt-oss est au chapitre 10.

Données : `fondations/journaux/07-synthese/plafonds.json` (champ `betas`), tiré par `memoire/outils/donnees_figures.py`.

Ce modèle est une hypothèse dite, pas une loi. Il suppose que la forme de la courbe de Qwen3-30B (8 actifs sur 128) vaut pour gpt-oss-120b (4 actifs sur 128), et il extrapole un point unique. Tous les plafonds des architectures candidates en dépendent d'un facteur 5 ; c'est pourquoi les trois scénarios sont toujours donnés ensemble.

Table 4.1 : Les grandeurs de l'équation du jeton sur la machine de Stone, sur secteur. Les débits sont ceux des chapitres de mesure ; chaque ligne nomme son journal.

Grandeur	Valeur		Journal
W_{D}, disque, lectures de 4 Mio en QD 4 (taille d'une matrice d'expert)	2,58 Go/s	[mes.]	`01-machine/disque-analyse.md`
W_{D}, disque, lectures de 16 Mio en QD 1	2,82 Go/s	[mes.]	`01-machine/debits-mesures.json`
disque, 4 Kio aléatoires (fautes de page)	0,22 Go/s	[mes.]	`01-machine/disque-balayage.csv`
W_{P}, DMA hôte → carte depuis la RAM épinglée	12,2 Go/s	[mes.]	`01-machine/carte-complement.jsonl`
lecture de la VRAM par un shader	251,3 Go/s	[mes.]	`01-machine/carte.jsonl`
W_g, produit matrice-vecteur MXFP4 brut, Vulkan	209,4 Go/s	[mes.]	`03-entropie/`
W_{dec}, décodeur rANS v4, Vulkan	70 Go/s	[mes.]	`03-entropie/`
décodeur et réassemblage des trois flux de classe	41,6 Go/s	[mes.]	`03-entropie/`
r, codage sans perte de gpt-oss-120b (trois classes)	1,1026	[mes.]	`03-entropie/`
V, M : VRAM et RAM utiles pour des poids	6,5 et 11 Go	hyp.	étude 06
t_0, coût fixe d'un jeton	3,5 ms	[calc.]	`02-quantification/vitesse-4B-3.json`
B_t, B_e, E_{tot} de gpt-oss-120b	1,686 ; 1,909 ; 61,07 Go	[mes.]	`07-synthese/inventaire-gptoss.json`

Données : Chemins relatifs à `fondations/journaux/`.

4.4 Les chiffres de la machine

Deux mises en garde valent pour tout ce qui suit. **La machine était partagée** : toutes les mesures ont tourné avec d'autres agents, et les débits « au mieux » sont des bornes inférieures du repos (chapitre 3). **Sur batterie**, c'est une autre machine : la carte passe à 52,6 W, le lien en Gen 3, tout ce qui passe par le processeur perd un facteur 4 à 6 ; la DMA tient à 11,8 Go/s, le décodeur tombe à 26,8 Go/s (section 3.6.1). Les chiffres de ce chapitre sont ceux du secteur.

4.5 Le problème, posé

Ce que Stone demande se lit maintenant comme un problème d'optimisation sous contraintes. On note θ l'ensemble des décisions d'un moteur : le format des poids (bits par poids, codage, donc B_t, B_e, r_d, r_p), le placement et la politique de cache (donc c_v, c_r et, par l'hypothèse 4.4, h_v, h_r, h_s), l'ordonnancement des lectures (donc t_{lat}), la spéculation (donc $\mathscr{E}$), et la politique de défaut (servir exact, attendre, servir une version moins précise, re-router).

Définition 4.5 (Le problème de troncons)**.** Pour un modèle donné et la machine de Stone, trouver

$$\theta^\star \in \arg\max_{\theta} \frac{1}{t(\theta)} \quad \text{sous} \quad \begin{cases} \mathrm{VRAM}_{\max}(\theta) \le 8\,188\ \mathrm{MiB}, \\ \mathrm{RAM}_{\max}(\theta) \le M_{\text{fenêtre}} \quad \text{(indépendant de la taille du modèle, définition 1.1)}, \\ \mathrm{KL}\big(p_\theta \,\|\, p_{\text{exact}}\big) \le \kappa, \quad \text{premier choix identique} \ge \pi_0, \end{cases} \tag{4.7}$$

où $t(\theta)$ est donné par l'équation (4.1), p_θ est la loi du jeton suivant que rend le moteur, p_{exact} celle que rendent les poids d'origine, et (κ, π_0) un budget de qualité déclaré avant la mesure.

Le budget retenu est dérivé d'une grandeur mesurée, pas posé au jugé : $\kappa = 0{,}0055$ et $\pi_0 = 95{,}96\,\%$, l'écart mesuré entre deux moteurs justes lisant les mêmes poids (section 9.6). La raison : une perte plus petite que l'écart entre deux implémentations correctes ne se distingue pas d'un changement de moteur. Une solution exacte (KL $= 0$ par construction) satisfait la troisième contrainte quel que soit κ.

Ce qui rend le problème difficile. Les contraintes couplent ce que les études mesuraient séparément. Comprimer avec perte réduit B_e mais consomme le budget κ. Comprimer sans perte augmente r_d mais charge le décodeur (T_{G}). Un grand cache en RAM augmente h_r mais viole la fenêtre si $M_{\text{fenêtre}}$ est petit. La spéculation augmente $\mathscr{E}$ mais fait lire l'union des experts de plusieurs jetons. La section suivante dit, sur cette machine, lequel de ces couplages compte.

4.6 Ce qui tient le maximum : les élasticités

Proposition 4.6 (Élasticités du plafond)**.** *Soit $d = 1/t$ le plafond donné par l'équation (4.1) avec $\mathscr{E} = 1$, $t_{\mathrm{lat}} = 0$, et supposons que le disque tienne strictement le maximum : $T_{\mathrm{D}} > \max(T_{\mathrm{P}}, T_{\mathrm{G}})$. Notons $\eta = T_{\mathrm{D}}/(T_{\mathrm{D}} + t_0)$. Alors*

$$\frac{\partial \ln d}{\partial \ln W_{\mathrm{D}}} = \frac{\partial \ln d}{\partial \ln r_d} = +\eta, \qquad \frac{\partial \ln d}{\partial \ln h_s} = \frac{\partial \ln d}{\partial \ln B_e} = -\eta, \qquad \frac{\partial \ln d}{\partial \ln x} = 0 \ \ \textit{pour } x \in \{W_{\mathrm{P}}, W_{\mathrm{dec}}, W_g, r_p\}, \tag{4.8}$$

et $\partial \ln d / \partial \ln t_0 = -(1-\eta)$.

Démonstration. Sous l'hypothèse stricte, le maximum vaut T_{D} dans un voisinage du point (les trois fonctions sont continues), donc $t = T_{\mathrm{D}} + t_0$ y est dérivable et ne dépend ni de W_{P}, ni de W_{dec}, ni de W_g, ni de r_p : leurs élasticités sont nulles. Pour $x \in \{W_{\mathrm{D}}, r_d, h_s, B_e\}$, l'équation (4.2) donne $\partial \ln T_{\mathrm{D}} / \partial \ln x = \mp 1$, et

$$\frac{\partial \ln d}{\partial \ln x} = -\frac{\partial \ln t}{\partial \ln x} = -\frac{T_{\mathrm{D}}}{T_{\mathrm{D}} + t_0} \frac{\partial \ln T_{\mathrm{D}}}{\partial \ln x} = \pm \eta.$$

B_e figure aussi dans T_{P} et T_{G}, qui ne sont pas le maximum : sa dérivée ne passe que par T_{D}. Enfin $\partial \ln t / \partial \ln t_0 = t_0/(T_{\mathrm{D}} + t_0) = 1 - \eta$. ∎

Pour gpt-oss-120b dans le scénario LRU calé (section 4.7), $T_{\mathrm{D}} = 157$ ms, $T_{\mathrm{P}} = 58$ ms, $T_{\mathrm{G}} = 36$ ms : le disque tient le maximum avec une marge de 2,7 sur le lien, et $\eta = 157/160{,}5 = 0{,}978$ [calc.]. Le tableau 4.2 en tire ce que vaut chaque gain.

Corollaire 4.7 (Les noyaux de la carte sont de l'ornement en régime disque)**.** *Tant que le disque tient le maximum, rendre les noyaux de la carte plus rapides (décodeur, produit matrice-vecteur, coût fixe) rapporte moins de 2 % sur gpt-oss-120b.*

Démonstration. Par la proposition 4.6, seules t_0 (élasticité $-0{,}022$) et les grandeurs d'élasticité nulle dépendent des noyaux. Supprimer entièrement t_0 ferait passer t de 160,5 à 157 ms, soit +2,2 % au plus ; tout gain partiel rapporte moins. (Le chiffre de 2 % est arrondi depuis $1 - \eta = 0{,}022$.) ∎

Table 4.2 : Élasticité du plafond de gpt-oss-120b, architecture candidate A, scénario LRU calé [calc.] (proposition 4.6, $\eta = 0{,}978$).

Grandeur	Élasticité	10 % gagnés valent	Remarque
W_{D} (débit du disque)	+0,98	+9,8 %	BitLocker et VMD en prennent 53 % (chapitre 3)
r_d (compression des octets lus)	+0,98	+9,8 %	sans perte : 1,10 atteint ; avec perte : la quantification
B_e (bits par poids actif)	−0,98	+10,8 %	la pente de la quantification (chapitre 5)
h_s (part lue du disque)	−0,98	+10,8 %	un point de succès gagné vaut +4,4 % à $h_s = 0{,}234$ (de 160,5 à 153,8 ms)
W_{P}, W_{dec}, W_g	0	0	tant qu'ils ne tiennent pas le maximum
t_0	−0,02	+0,2 %	3,5 ms sur 160

C'est une idée tuée par le nombre, pour ce régime : le travail sur la vitesse des noyaux en VRAM ne redevient un levier que lorsque le disque cesse de limiter, par exemple sur un volume non chiffré où le lien reprend la main (chapitre 10). La figure 4.2 montre les deux régimes sur une même courbe.

4.7 Un exemple posé pas à pas

gpt-oss-120b MXFP4, codage sans perte à trois classes ($r_d = r_p = 1{,}1026$), architecture candidate A, scénario LRU calé, $V = 6{,}5$ Go, cache tiède de 11 Go en RAM (`fondations/07-ce-qui-manque.md`, annexe A.3) [calc.].

1. Capacité de la VRAM pour les experts : $c_v = (6{,}5 - 1{,}686)/61{,}07 = 0{,}0788$, d'où $h_v = 0{,}0788^{0{,}2075} = 0{,}590$.
2. Capacité de la RAM : le cache tiède garde des experts codés, donc 11 Go en contiennent $11 \times 1{,}1026 = 12{,}13$ Go servis : $c_r = 12{,}13/61{,}07 = 0{,}1986$. Alors $h_v + h_r = 0{,}2774^{0{,}2075} = 0{,}766$, d'où $h_r = 0{,}176$ et $h_s = 0{,}234$.
3. Disque : $T_{\mathrm{D}} = 0{,}234 \times 1{,}909/1{,}1026/2{,}58 = 0{,}157$ s.
4. Lien : $T_{\mathrm{P}} = (0{,}176 + 0{,}234) \times 1{,}909/1{,}1026/12{,}2 = 0{,}058$ s (les toujours-lus tiennent en VRAM : $\delta = 0$).
5. Carte : $T_{\mathrm{G}} = 0{,}410 \times 1{,}909/41{,}6 + (1{,}686 + 1{,}909)/209{,}4 = 0{,}0188 + 0{,}0172 = 0{,}036$ s (décodeur à trois flux de classe, 41,6 Go/s).
6. Plafond : $t = 0{,}157 + 0{,}0035 = 0{,}161$ s, soit **6,24 jetons/s** ; plancher en série : $t = 0{,}157 + 0{,}058 + 0{,}036 + 0{,}0035 = 0{,}254$ s, soit **3,93 jetons/s**.

Les mêmes six lignes, refaites par l'outil, donnent 6,2414 et 3,9322 (`fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, ligne `lru_cale, A_sans_perte`).

4.8 La fenêtre de travail, chiffrée

La définition 1.1 demande une mémoire de pointe qui ne dépende pas de la taille du modèle. Dans l'équation (4.1), la RAM n'apparaît que par c_r, la capacité du cache tiède : c'est un réglage, pas une conséquence du modèle. Le reste de la RAM de pointe tient aux tampons d'arrivée (les défauts d'un jeton, en double tampon) et aux structures du moteur.

C'est la réponse chiffrée à « sans RAM ni rien » (figure 4.3) : la RAM ne disparaît pas, elle cesse de dépendre du modèle. Le partisan de A a resserré la fenêtre depuis : avec des tampons de quelques

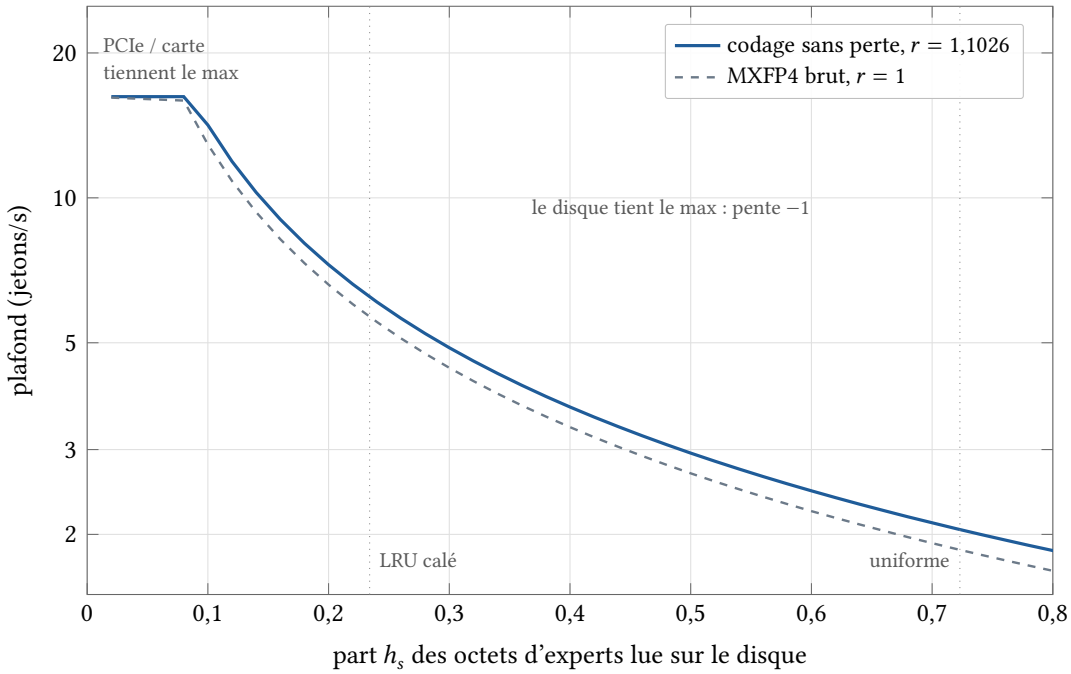


Figure 4.2 : Plafond de gpt-oss-120b selon la part h_s des octets d'experts lue sur le disque, architecture candidate A [calc.]. À droite, le disque tient le maximum et la courbe a la pente -1 en échelle logarithmique (proposition 4.6) ; à gauche, sous $h_s \approx 0{,}1$, le lien et la carte reprennent le maximum et la courbe s'aplatit. Le codage sans perte décale tout le régime disque d'un facteur $r = 1{,}1026$.

Données : `fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-hs.csv` (outil `outils/07-synthese/plafonds.py`).

couches au lieu de deux jetons entiers, A0 descend à 0,64 Go de RAM de pointe (1,26 Go si la table d'enchâssement reste en RAM) [calc.] (`decisions/partisan-A.md`, § 0, point 7). Il en tire le témoin direct de la définition 1.1 : gpt-oss-20b et gpt-oss-120b ont la même taille d'expert (13,25 Mo), le même nombre d'experts actifs et la même table d'enchâssement, donc leur RAM de pointe en A0 doit être la même à 0,5 Mo près, alors que leurs fichiers font 12,1 et 63,4 Go.

4.9 Le terme qui manquait à la spéculation

Le corollaire 8.8 conclut que la spéculation ne paie jamais sur des experts fins *indépendants*. Mais deux jetons voisins réutilisent une partie de leurs experts ; un seul nombre suffit à en tenir compte.

Proposition 4.8 (Spéculation sous réutilisation des experts)**.** *Soit $\rho \in [0,1]$ la part moyenne des experts d'un jeton qui figuraient déjà parmi ceux du jeton précédent, dans la même couche. Une passe de vérification d'un jeton proposé ($k = 1$) réduit les octets d'experts lus par jeton produit si et seulement si*

$$\alpha \;>\; 1 - \rho. \tag{4.9}$$

Démonstration. Sans spéculation, chaque jeton lit k_e experts par couche. La passe de vérification porte sur deux positions ; le second jeton apporte $k_e(1-\rho)$ experts nouveaux en moyenne, donc la passe lit $U(2) = k_e(2-\rho)$ experts par couche. Elle rend $\mathscr{E}(\alpha,1) = 1+\alpha$ jetons (théorème 8.3). Les octets par jeton produit baissent si et seulement si $k_e(2-\rho)/(1+\alpha) < k_e$, soit $\alpha > 1-\rho$. ∎

Sous l'indépendance, $\rho = k_e/E$ et l'on retrouve le corollaire 8.8 : $\alpha > 1 - 4/128 = 0{,}969$ pour gpt-oss-120b. Avec les réutilisations publiées, la condition tombe à $\alpha > 0{,}556$ pour Qwen3-30B-A3B

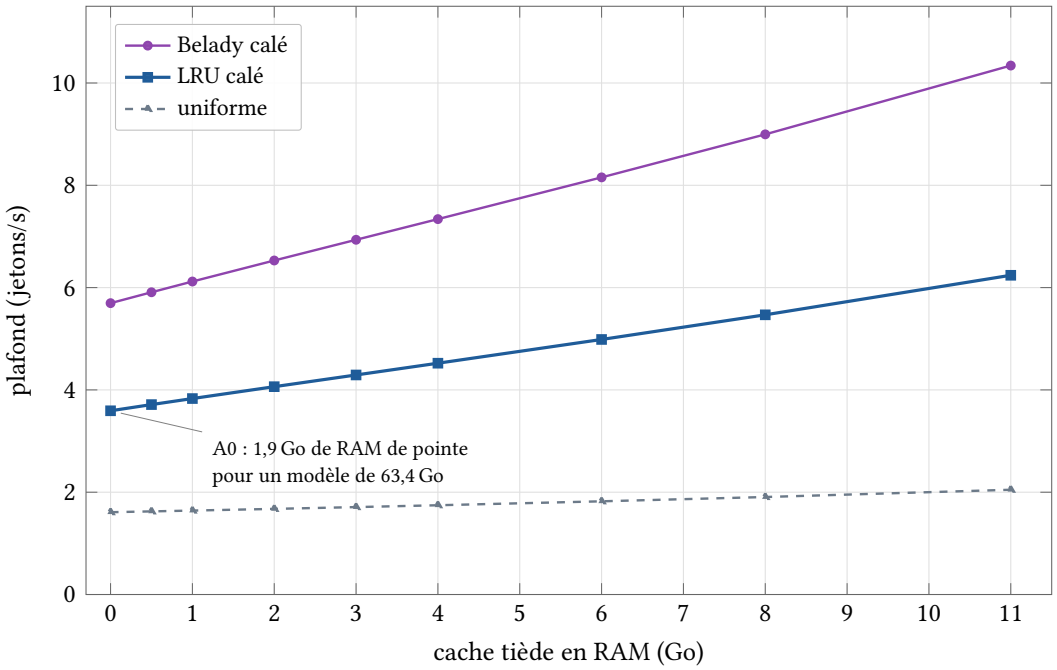


Figure 4.3 : Plafond de gpt-oss-120b (63,4 Go) selon le cache tiède en RAM, architecture candidate A [calc.]. La première colonne est A0 : sans aucun cache en RAM, 1,9 Go de RAM de pointe et 3,6 jetons/s de plafond dans le scénario LRU calé. Chaque Go de cache ajoute environ 0,24 jeton/s dans ce scénario ; la RAM de pointe va de 1,9 Go (sans cache) à 12,3 Go (11 Go de cache).

Données : `fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-ram.csv`.

($\rho = 0{,}444$) et $\alpha > 0{,}75$ pour Qwen3.6-35B-A3B ($\rho \approx 0{,}25$) [calc.] sur [litt.] (`fondations/journaux/07-synthese/plafonds.json`, champ `moe_speculation_condition_k1`). Si ρ valait 0,44 sur le modèle visé, un brouillon à $\alpha = 0{,}8$ rendrait 1,8 jeton pour 1,56 fois les experts d'un jeton : +16 % de débit, là où le chapitre 8 concluait à une perte. **La conclusion « k optimal = 0 » dépend donc d'un nombre que personne ne publie pour gpt-oss-120b** ; la trace de routage qui donne $h(c)$ donne aussi ρ.

4.10 Les témoins de l'équation

Une équation qu'on ne peut pas faire tomber ne prouve rien. L'outil `plafonds.py` refait trois chiffres publiés par d'autres études, le seuil étant déclaré avant le lancement : 1 % d'écart sur chaque valeur.

Un écart corrigé en passant. L'étude de l'entropie chiffrait à +14,4 % l'effet du taux sans perte (1,1026) sur la capacité du cache, en supposant tout le cache codé, VRAM comprise. Or sa propre mesure dit que le produit fondu au décodeur est 3,7 fois plus lent que le produit brut : la carte doit garder ses experts décodés, et seule la RAM grandit. L'effet est alors de +13,1 % [calc.] (champ `gain_taux_sur_cache_uniforme` de `plafonds.json`).

Ce qui pourrait être faux

– **Le modèle $h(c) = c^{\beta}$ extrapole un point.** Il vient d'un LRU sur une couche de Qwen3-30B-

Table 4.3 : Les témoins de l'équation du jeton [calc.]. La cassure remplace le débit mesuré du disque par le plafond du lien : le témoin qui en dépend tombe, les deux autres tiennent.

Témoin	Attendu	Obtenu	Tient	Cassé
T1 : gpt-oss-120b, placement du section 3.8, $r = 1$, recouvert	1,98	1,994	oui	**tombe** (5,569)
T2 : Qwen3.5-9B tout sur le disque, $r = 16$, la carte décode	8,85	8,847	oui	tient
T3 : le placement de T1 au débit du lien (7,877 Go/s), chiffre de l'étude 06	5,6	5,569	oui	tient

Données : `fondations/journaux/07-synthese/plafonds.json` et `plafonds-casse.json`.

A3B, sur un domaine ; la courbe de gpt-oss-120b peut être plus plate ou plus raide. Les trois scénarios encadrent une croyance, ils ne bornent pas la réalité.

- **Le plafond suppose un recouvrement parfait** et une file de disque jamais vide ($t_{\text{lat}} = 0$). Le plancher en série est plus proche de ce qu'obtiennent les moteurs publiés, qui tombent entre 18 et 45 % de leur borne (chapitre 7).
- $t_0 = 3{,}5$ **ms vient d'un autre modèle**, sous CUDA ; un moteur Vulkan qui synchronise après chaque routeur (36 fois par jeton) peut avoir un coût fixe plus grand, et l'élasticité de t_0 grandirait avec lui.
- **11 Go de RAM épinglée** : le pilote d'affichage de Windows peut limiter la mémoire hôte visible de la carte ; si le cache tiède doit vivre en mémoire paginable, chaque succès en RAM passe par une copie du processeur. Non vérifié.
- **La condition** $\alpha > 1 - \rho$ suppose α constant et ρ indépendant de la position ; elle ne dit rien de $k > 1$, où l'union de n jetons décroît moins vite qu'un modèle à un paramètre.
- **Le budget de qualité** κ vient d'un modèle hybride (Qwen3.5-4B) ; l'écart entre deux moteurs justes sur gpt-oss peut être autre, et doit être remesuré sur lui.

Deuxième partie

Les leviers, mesurés

Chapitre 5

Quantification extrême

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la littérature annonçait des quantifications à 2 bits « presque sans perte » sur des familles anciennes (Llama 2), avec des noyaux CUDA, et aucune mesure n'existait sur les modèles de la machine ni sous Vulkan. **Ce chapitre établit** quatre choses : la borne de Shannon et la distance de chaque codeur à cette borne, refaites par un programme écrit ici ; le premier décodeur de treillis portable (Vulkan et DirectX 12), mesuré sur la RTX 4060 ; l'échelle réelle des quantifications de llama.cpp sur Qwen3.5 4B et 9B, à octets lus égaux ; et une règle de partage des bits selon l'étage de mémoire, démontrée, puis corrigée ici par les débits mesurés au chapitre 3. **Il reste** un codeur en treillis sans rotation décodé en Vulkan, un code emboîté lié au placement, et une base en BF16 au lieu de Q4_K_M. Source : `fondations/02-quantification.md`, `02-quantification-derivations.md`, `fondations/journaux/02-quantification/`.

5.1 Le levier

Pour un modèle dense de N paramètres stockés à b bits par poids, métadonnées comprises, un jeton en décodage lit chaque poids une fois : $B = Nb/8$ octets, plus le cache KV et les activations, négligeables pour un contexte court. Le théorème 3.2 donne alors l'équation (3.3), jetons/s $\leq 8W/(Nb)$: **diviser b par deux double le plafond, à n'importe quel étage**. Pour un mélange d'experts, N devient le nombre de paramètres actifs par jeton. Le tableau 5.1 chiffre ce levier avec les débits mesurés sur la machine.

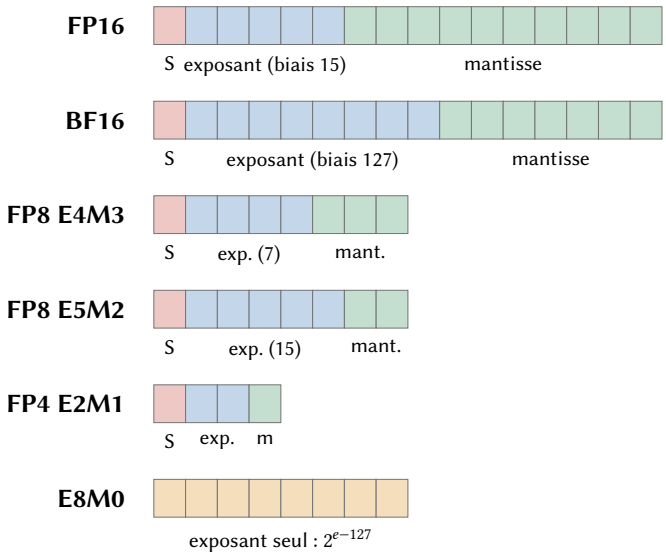
Table 5.1 : Plafond de jetons par seconde, poids lus intégralement depuis l'étage nommé, sans cache, jetons/s $= 8W/(Nb)$ [calc.] sur les débits mesurés (VRAM 251,3 ; PCIe 12,2 ; disque 2,82 Go/s).

Modèle (paramètres lus), étage	$b = 16$	$b = 4{,}5$ (Q4_K)	$b = 2{,}06$ (IQ2_XXS)	$b = 1{,}56$ (IQ1_S)
9 G dense, VRAM	14,0	49,9	108,9	143,8
9 G dense, PCIe	0,68	2,42	5,29	6,98
9 G dense, disque	0,16	0,56	1,22	1,61
70 G dense, disque	0,020	0,072	0,156	0,206
MoE à 3 G actifs, PCIe	2,03	7,23	15,8	20,8
MoE à 3 G actifs, disque	0,47	1,67	3,65	4,81

Lecture : hors VRAM, seuls les mélanges d'experts deviennent interactifs, et la quantification multiplie le plafond par $16/b$. Aucun autre levier de ce mémoire n'a cette pente sans contrepartie de débit ; c'est sa raison d'être. La contrepartie est ailleurs : la qualité.

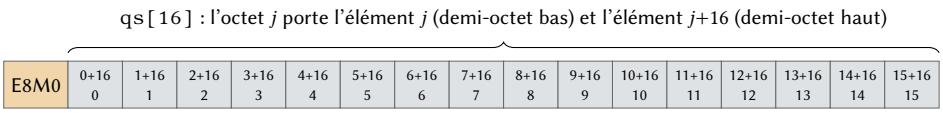
5.2 Les formats, au bit près

Un format de quantification est une disposition de bits et une règle de reconstruction. Les schémas qui suivent sont lus dans la norme IEEE 754 [86], la spécification MX de l'OCP [37, 87] et le code de llama.cpp [9] (`ggml/src/ggml-common.h` et `ggml-quants.c`, commit `843e7dc`) ; les `static_assert` du code donnent la taille de chaque bloc en octets et servent de témoin à chaque ligne du tableau 5.2.



Valeurs E2M1 : $\pm\{0, \frac{1}{2}, 1, \frac{3}{2}, 2, 3, 4, 6\}$. llama.cpp les range doublées ($\{0,1,2,3,4,6,8,12\}$) et divise l'échelle E8M0 par deux.

Figure 5.1 : Les formats flottants, au bit près : FP16 et BF16 (16 bits), FP8 en E4M3 et E5M2 [88], FP4 en E2M1, et l'échelle E8M0 des formats MX (exposant seul). BF16 garde l'exposant du FP32 et sacrifie la mantisse [89] ; E8M0 ne code qu'une puissance de deux.



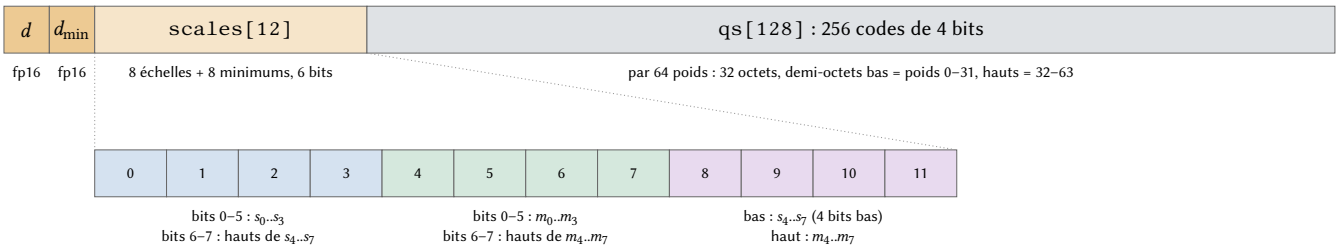
$w_j = 2^{e-127} \cdot \mathrm{E2M1}(c_j)$; $8 + 32 \times 4 = 136$ bits pour 32 poids = **4,25** bits/poids.

Figure 5.2 : Le bloc MXFP4 : 32 poids partagent un exposant E8M0 ; chaque poids est un code E2M1 de 4 bits. C'est le format natif des experts de gpt-oss, la première cible de ce mémoire.

Le bit lu par jeton n'est pas le bit du type. Un fichier « IQ2_XXS » mélange les types : on le mesure tenseur par tenseur (`quant-gguf-bits.py`, témoin : la fin du dernier tenseur égale la fin du fichier). Le « IQ2_XXS » de Qwen3.5-9B lit 2,757 Go par jeton, soit 2,76 bits par poids : la sortie reste en Q5_K (699 Mo, 25 % des octets lus), 32 matrices en Q4_K (472 Mo), et les matrices vraiment en IQ2_XXS ne font que 1 516 Mo [mes.]. **À 2 bits, la tête de sortie devient le premier poste** d'un petit modèle à grand vocabulaire : 29 % des octets lus par jeton dans le « IQ2_XXS » du 4B, dont la table d'enchâssement liée à la sortie (248 320 × 2 560) est relue en entier à chaque jeton.

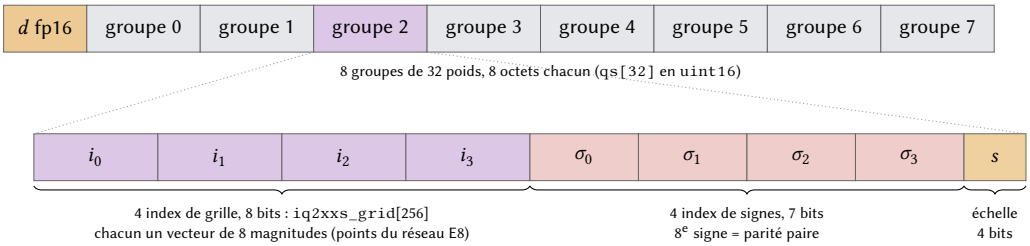
5.3 La source gaussienne et sa borne

Pourquoi la gaussienne. Les méthodes qui approchent la borne aujourd'hui (QuIP# [27], QTIP [31], HIGGS [26], QuaRot [39]) multiplient chaque matrice de poids, à gauche et à droite, par une trans-



$w = d\, s_j\, c - d_{\min}\, m_j$ pour un poids du sous-bloc j (32 poids) de code $c \in \{0..15\}$; $(16+16+96+1024)/256 = \mathbf{4{,}5}$ bits/poids.

Figure 5.3 : Le super-bloc Q4_K de llama.cpp : 256 poids en 144 octets. Deux échelles fp16 (d, $d_{\min}$) multiplient huit échelles et huit minimums de 6 bits, empaquetés dans 12 octets selon le schéma du bas (fonction `get_scale_min_k4`).



$w = d\,(\tfrac{1}{2} + s)\,\tfrac{1}{4}\, g_{i_l}[k]\,(-1)^{\sigma_l[k]}$; $(16 + 8\times 64)/256 = \mathbf{2{,}0625}$ bits/poids.

Figure 5.4 : Le super-bloc IQ2_XXS de llama.cpp : 256 poids en 66 octets. Un groupe de 32 poids tient en 64 bits : quatre index dans une grille de 256 vecteurs de dimension 8 tirés du réseau E8, quatre index de signes de 7 bits (le 8e signe est fixé par la parité), une sous-échelle de 4 bits.

formée de Hadamard à signes aléatoires, $W' = H_g\,\mathrm{diag}(s) W\,\mathrm{diag}(s') H_d$: après cette rotation, les poids d'une ligne sont presque gaussiens indépendants et les valeurs aberrantes disparaissent. La quantification devient un problème de codage de source gaussienne. HIGGS relie de plus cette erreur à la qualité : pour de petites erreurs, l'augmentation de perplexité est approximativement une somme pondérée, couche par couche, des erreurs quadratiques relatives (« théorème de linéarité ») [litt.]. Chaque gain d'erreur quadratique se traduit donc à peu près linéairement en gain de qualité.

Théorème 5.1 (Shannon, source gaussienne sans mémoire [90, 91])**.** *Pour une source $X \sim \mathcal{N}(0, \sigma^2)$ indépendante et identiquement distribuée et la distorsion quadratique, la fonction débit-distorsion vaut $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$ pour $0 < D \le \sigma^2$; de façon équivalente, la plus petite erreur quadratique moyenne atteignable au débit de R bits par échantillon est*

$$D(R) = \sigma^2\, 2^{-2R}. \tag{5.1}$$

Démonstration. Le théorème du codage de source avec critère de fidélité [90] dit que le plus petit débit atteignable par des codes longs à distorsion moyenne D est $R(D) = \min I(X;\hat{X})$, le minimum portant sur les lois conditionnelles $p(\hat{x} \mid x)$ telles que $\mathbb{E}(X-\hat{X})^2 \le D$. Il reste à calculer ce minimum.

Borne inférieure. Pour toute loi admissible,

$$\begin{aligned} I(X;\hat{X}) &= h(X) - h(X \mid \hat{X}) = h(X) - h(X - \hat{X} \mid \hat{X}) \\ &\ge h(X) - h(X-\hat{X}) \ge \tfrac{1}{2}\log_2(2\pi e\sigma^2) - \tfrac{1}{2}\log_2\!\big(2\pi e\, \mathbb{E}(X-\hat{X})^2\big) \ge \tfrac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

Table 5.2 : La comptabilité des bits, format par format [calc.] (dispositions lues dans `ggml-common.h` ; témoin : les `static_assert`).

Format	Bloc	bits du bloc	bits/poids
Q2_K	256 poids : codes 2 bits (512), 16 octets d'échelles et minimums 4+4 bits (128), d et $d_{\min}$ fp16 (32)	672	**2,625**
Q4_K	256 : codes 4 bits (1 024), 12 octets d'échelles 6 bits (96), d, $d_{\min}$ (32)	1 152	**4,5**
Q6_K	256 : 4 bits bas (1 024) + 2 bits hauts (512), 16 échelles int8 (128), d (16)	1 680	**6,5625**
IQ2_XXS	8 groupes de 32 : 4 index 8 bits, 4 × 7 bits de signe, échelle 4 bits ; d fp16	528	**2,0625**
IQ3_XXS	256 : index 8 bits par 4 poids (512) ; par 32 poids, signes et échelle (256) ; d (16)	784	**3,0625**
IQ1_S	256 : 8 bits bas d'index par 8 poids (256) ; par 32 poids, bits hauts, échelle, décalage (128) ; d (16)	400	**1,5625**
TQ1_0	256 : 240 trits à 5 par octet, 16 trits à 4 par octet ; d (16)	432	**1,6875**
MXFP4	32 : codes E2M1 (128) ; exposant E8M0 (8)	136	**4,25**
NVFP4	64 : codes E2M1 (256) ; 4 échelles UE4M3, une par 16 (32)	288	**4,5**
Treillis, état libre	suite de T poids, k bits par poids, L bits d'état	$Tk + L - k$	$k + \frac{L-k}{T}$
Treillis tail-biting	tuile de 256 poids lue circulairement	$256k$	k

La deuxième égalité vient de ce que, $\hat{X}$ connu, X et $X - \hat{X}$ se déduisent l'un de l'autre par translation ; la première inégalité de ce que le conditionnement réduit l'entropie ; la deuxième de ce que, à second moment donné, la loi normale maximise l'entropie différentielle.

Atteinte. Soient $\hat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ et $Z \sim \mathcal{N}(0, D)$ indépendants, et $X = \hat{X} + Z$ (canal test « à rebours »). Alors $X \sim \mathcal{N}(0,\sigma^2)$, $\mathrm{E}(X - \hat{X})^2 = D$ et $I(X;\hat{X}) = h(X) - h(Z) = \frac{1}{2}\log_2(\sigma^2/D)$: toutes les inégalités sont des égalités. Donc $R(D) = \frac{1}{2}\log_2(\sigma^2/D)$, et en inversant, $D(R) = \sigma^2 2^{-2R}$. ∎

À $R = 2$, $D = 0{,}0625\,\sigma^2$. La borne dit ce qu'aucun codeur ne peut battre ; elle ne dit pas comment l'atteindre. Pour comparer des codeurs, on mesure leur distance à elle en bits.

Définition 5.2 (Distance en bits à la borne)**.** Un quantificateur q qui atteint l'erreur D_q au débit R est à

$$\Delta(q) \;=\; R - R(D_q) \;=\; R + \tfrac{1}{2}\log_2 \frac{D_q}{\sigma^2}$$

bits de la borne : c'est le débit qu'un code optimal économiserait à qualité égale.

Lemme 5.3 (Un rapport d'erreurs en bits)**.** *Deux quantificateurs de même pente $D \propto 2^{-2R}$ autour du point de fonctionnement, d'erreurs $D_1 > D_2$ au même débit, sont séparés de $\frac{1}{2}\log_2(D_1/D_2)$ bit à qualité égale.*

Démonstration. Le premier doit gagner δ bits pour atteindre D_2 : $D_1 2^{-2\delta} = D_2$, soit $\delta = \frac{1}{2}\log_2(D_1/D_2)$. ∎

5.4 Le quantificateur scalaire optimal

Le quantificateur de Lloyd-Max [92, 93] est le meilleur quantificateur scalaire à $M = 2^R$ niveaux $y_1 < \cdots < y_M$ et seuils t_j. Ses deux conditions nécessaires d'optimalité sont :

1. le plus proche voisin : $t_j = (y_j + y_{j+1})/2$;
2. le centroïde : $y_j = \mathbb{E}[X \mid t_{j-1} < X \le t_j] = \dfrac{\varphi(t_{j-1}) - \varphi(t_j)}{\Phi(t_j) - \Phi(t_{j-1})}$ pour $X \sim \mathcal{N}(0,1)$, où φ et Φ sont la densité et la répartition normales.

On les itère jusqu'au point fixe sur la densité exacte (`quant-mse-gaussien.py`). L'erreur s'écrit alors en forme fermée,

$$D = 1 - \sum_j y_j^2 \left[\Phi(t_j) - \Phi(t_{j-1})\right], \tag{5.2}$$

parce que $\mathbb{E}[X^2] = 1$ et que l'erreur est orthogonale à la reconstruction au centroïde. On trouve 0,36338 à 1 bit (niveaux $\pm\sqrt{2/\pi} = \pm 0{,}79788$), **0,117482 à 2 bits** (niveaux ±0,45278 et ±1,51042), 0,034548 à 3 bits : les valeurs de Max (1960) [calc.].

Remarque 5.4 (Pourquoi le banc a été refait). La formule de Panter-Dite [94] pour le scalaire optimal en haute résolution, $D \approx \frac{\sqrt{3}\pi}{2}\sigma^2 2^{-2R} = 2{,}72\,\sigma^2 2^{-2R}$, donnerait 0,170 à $R = 2$ au lieu de 0,1175 : à 2 bits on n'est pas en haute résolution, et seules les erreurs calculées ou mesurées valent (figure 5.5).

5.5 Le banc théorique, refait ici

Le témoin. Lloyd-Max calculé analytiquement contre Monte-Carlo sur 2^{20} échantillons, seuil déclaré avant le lancement : 4 erreurs-types. Le témoin tient à 1, 2 et 3 bits. **Il a d'abord été trop faible** : la première cassure (un seul niveau décalé de 0,05) ne le faisait pas tomber (+0,0008 d'erreur à 1 bit, sous le seuil), parce que le plus proche voisin réaffecte les échantillons et absorbe le décalage. Cassure renforcée (tous les niveaux décalés de 0,1) : il tombe à 1 et 2 bits. Les deux journaux sont gardés.

Table 5.3 : Erreur quadratique moyenne à environ 2 bits sur $\mathcal{N}(0,1)$, erreur-type de Monte-Carlo entre parenthèses ; distance Δ à la borne (définition 5.2).

Quantificateur	bits/poids	MSE [mes.]	publiée [litt.]	Δ [calc.]
Scalaire Lloyd-Max	2	0,1175 (0,0002)	0,118	0,455
VQ dim. 8, livre gaussien aléatoire de 2^{16} mots	2	0,0935 (0,0004)	—	0,291
VQ dim. 8, livre calculé façon IQn_KT	2	0,0928 (0,0004)	—	0,285
QuIP# E8P (réseau E8)	2	non refait	0,089	—
IQ2_XXS de llama.cpp	2,0625	**0,1070 (0,0005)**	—	0,450
Treillis à décalage $L = 16$, table aléatoire	2,0034	0,0680 (0,0007)	0,068	0,064
Treillis $L = 16$, code « 3INST » (QTIP)	2,0034	0,0687 (0,0007)	0,069	0,070
Treillis $L = 16$, code « mul1 » (EXL3)	2,0034	0,0682 (0,0007)	—	0,066
Borne de Shannon	2	0,0625 [calc.]	0,063	0

Trois résultats (tableau 5.3 et figure 5.5) :

- **Les trois codes du treillis reproduisent le tableau 1 de QTIP à l'erreur-type près**, par un programme écrit depuis la définition mathématique, sans leur code. Le treillis ferme $(0{,}1175 - 0{,}068)/(0{,}1175 - 0{,}0625) = 90\,\%$ de l'écart entre le scalaire et la borne ; la VQ de dimension 8 en ferme 44 % [calc.]. À 1 bit (24 suites de 4 096 poids), le treillis atteint 0,2631 contre 0,3634 pour Lloyd-Max et 0,25 pour la borne : il ferme 88 % de l'écart.
- **IQ2_XXS, mesuré sur la même source, vaut exactement le scalaire optimal à son débit** : 0,107 à 2,0625 bits, quand Lloyd-Max ramené à ce débit donne $0{,}1175 \times 2^{-0{,}125} = 0{,}108$ [calc.]. Le

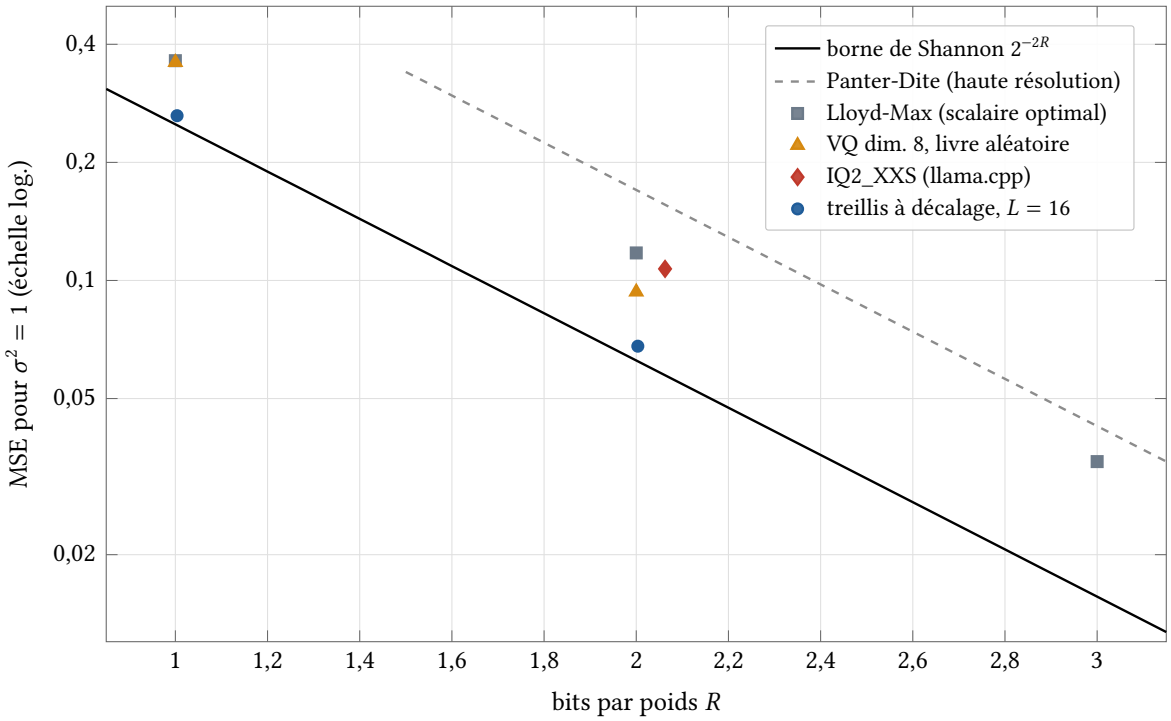


Figure 5.5 : Distorsion contre débit sur la source $\mathcal{N}(0,1)$ [mes.], avec la borne de Shannon (théorème 5.1) et la formule de haute résolution de Panter-Dite. Le treillis à décalage ferme 90 % de l'écart entre le scalaire optimal et la borne à 2 bits, la VQ de dimension 8 en ferme 44 % ; IQ2_XXS vaut exactement le scalaire optimal à son débit.

Données : `fondations/journaux/02-quantification/csv/fig-q1-*.csv` (journaux `mse-gaussien-passe1-partielle.log`, `mse-treillis-2b.json`, `mse-iq2xxs.json`).

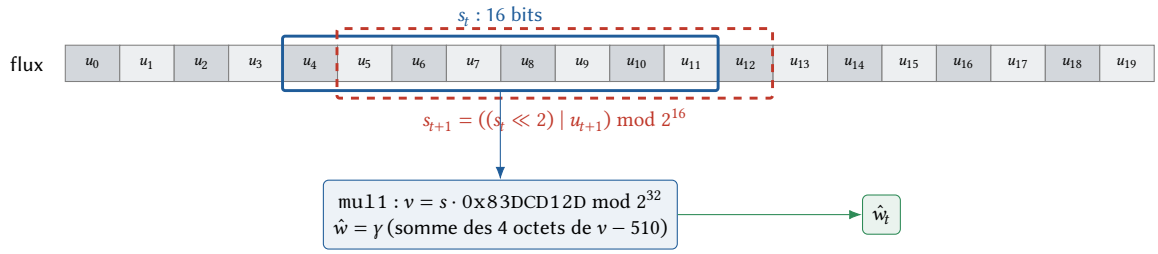
codeur a été écrit d'après le format (grille de 256 vecteurs lue dans `ggml-common.h`, parité des signes, sous-échelles de 4 bits), optimal à échelle donnée ; son témoin, une grille $\{0{,}45278; 1{,}51042\}^8$ sans parité, redonne Lloyd-Max (0,1173 ± 0,0009) et tombe quand on la casse. Sur une source gaussienne, c'est-à-dire après rotation, la grille E8 d'IQ2_XXS n'apporte rien ; elle a été choisie sur des poids réels non tournés, où elle gagne. La contrainte de parité coûte autant que le bit qu'elle économise.

– **Le « treillis » d'ik_llama.cpp n'en est pas un** : lu dans son code, chaque groupe de 8 poids est décodé d'une graine de 16 bits indépendante, par un plus proche voisin par grappes. Mesuré : la même erreur qu'une VQ de dimension 8 à livre aléatoire (0,0928 contre 0,0935). Un vrai treillis à décalage ferait 27 % d'erreur en moins au même débit.

Les distances du tableau 5.3 sont calculées avec le débit réel, amorce comprise (2,0034 bits pour le treillis) ; avec le débit nominal de 2 bits, celle du treillis à table aléatoire vaut 0,061 bit, le chiffre de l'étude d'origine.

5.6 Le treillis à décalage

Le code. L'état à l'instant t est une fenêtre de L bits s_t du flux codé ; chaque poids fait entrer k nouveaux bits : $s_{t+1} = ((s_t \ll k) \mid u_t) \bmod 2^L$, avec $u_t \in \{0, \dots, 2^k - 1\}$. La reconstruction est $\hat{w}_t = f(s_{t+1})$, où $f : \{0,1\}^L \to \mathbb{R}$ est une table (aléatoire gaussienne) ou une fonction calculée. C'est une quantification codée en treillis [95] dont le treillis est « à décalage » : comme f ne dépend que de



Chaque poids fait entrer $k = 2$ bits neufs dans la fenêtre ; la reconstruction ne dépend que de la fenêtre de 16 bits, d'où un livre de 2^{16} valeurs calculé, pas lu. Le codeur choisit le flux par Viterbi (erreur quadratique totale minimale) ; le décodeur coûte une multiplication et un `dot4` par poids. Tail-biting : la tuile de 256 poids est lue circulairement, et le coût vaut exactement k bits par poids.

Figure 5.6 : Le treillis à décalage ($L = 16, k = 2$) et son décodeur calculé « mul1 ». L'état est une fenêtre de 16 bits du flux codé ; chaque poids y fait entrer 2 bits ; la valeur reconstruite est une fonction calculée de la fenêtre.

L bits, la table de 2^{16} valeurs peut être calculée au lieu d'être lue, par exemple par le code « mul1 » d'EXL3 (figure 5.6). Le codeur cherche le flux de distorsion minimale par l'algorithme de Viterbi [96].

Lemme 5.5 (Les prédécesseurs d'un état). *Les prédécesseurs d'un état s' sont exactement les 2^k états $s = (s' \gg k) \mid (h \ll (L-k))$, $h \in \{0, \dots, 2^k - 1\}$.*

Démonstration. La transition $s' = ((s \ll k) \mid u) \bmod 2^L$ fixe les $L - k$ bits bas de s, qui deviennent les $L - k$ bits hauts de s', et laisse libres ses k bits hauts, perdus par le décalage. Réciproquement, tout s de cette forme mène à s' avec $u = s' \bmod 2^k$. ∎

Corollaire 5.6 (Viterbi vectorisé). *Le coût cumulé $J_{t+1}(s') = \min_h J_t\big(s(h, s')\big) + (w_{t+1} - f(s'))^2$ se calcule en remodelant le vecteur J_t de taille 2^L en une matrice $(2^k, 2^{L-k})$ et en prenant le minimum sur l'axe des h : $O(2^L)$ opérations par poids.*

Démonstration. Par le lemme 5.5, les prédécesseurs de s' ne dépendent que de $s' \gg k$, et ils occupent, dans l'ordre des indices, la colonne $s' \gg k$ de la matrice $(2^k, 2^{L-k})$ dont la ligne est h. Le minimum de chaque colonne sert aux 2^k états s' qui partagent la même valeur de $s' \gg k$, d'où la répétition. ∎

Le listing 5.1 est l'implantation de ce corollaire dans le banc, en numpy ; elle tient en six lignes.

Listing 5.1 : Le code « mul1 » et le Viterbi vectorisé du treillis à décalage, tels qu'ils tournent dans le banc (corollaire 5.6).

```
# Extrait de outils/quant-mse-gaussien.py (commit 3adf9c3), lignes 119-136, recopié tel quel.
def code_mul1(etats):
    """EXL3 « mul1 » : x <- s * 0x83DCD12D mod 2^32, somme des 4 octets, centrée sur 510."""
    x = (etats.astype(np.uint64) * 0x83DCD12D) & 0xFFFFFFFF
    somme = (x & 0xFF) + ((x >> 8) & 0xFF) + ((x >> 16) & 0xFF) + ((x >> 24) & 0xFF)
    return somme.astype(np.float64) - 510.0


def viterbi_treillis(suite, table, L, k):
    """Coût minimal (somme des erreurs quadratiques) d'une suite sur le treillis à
    décalage : état suivant = ((état << k) | b) mod 2^L ; valeur émise = table[état].
```

```
    État initial libre. Les prédécesseurs de s' sont (s' >> k) | (h << (L - k))."
    """
    n_etats = 1 << L
    cout = (suite[0] - table) ** 2
    for t in range(1, len(suite)):
        # min sur les 2^k bits sortants h, pour chaque valeur de (s' >> k)
        m = cout.reshape(1 << k, n_etats >> k).min(axis=0)
        cout = np.repeat(m, 1 << k) + (suite[t] - table) ** 2
    return float(cout.min())
```

Le débit exact. Avec un état initial libre, une suite de T poids coûte $(Tk + L - k)/T$ bits par poids : 2,0034 bits pour $T = 4\,096$, $k = 2$, $L = 16$. Avec le « tail-biting » de QTIP et d'EXL3, on impose $s_T = s_0$ (la tuile est lue circulairement) et le coût vaut exactement k bits. CurveTQ [32] mesure que le tail-biting sur des blocs de 64 coûte, au même débit utile, +4,10 de perplexité contre un état initial stocké (0,22 bit par poids) [litt.].

5.7 Le coût du décodage sur la carte

5.7.1 En CUDA

Le banc `quant-gemv-treillis.cu` (CUDA 13.1.2, `-arch=sm_89`), écrit de zéro, fait un produit matrice-vecteur 16 384 × 16 384 (268 millions de poids, bien au-delà des 32 Mio de cache L2) en quatre formats. Les deux noyaux 2 bits ont la même disposition et le même parcours : leur écart est le coût du décodage du treillis. Témoin : 64 lignes refaites sur le processeur, erreur relative déclarée inférieure à 10^{-3} ; il tient sur les quatre formats et, cassé (constante du code faussée côté processeur seul), il tombe sur le treillis et seulement sur lui.

Table 5.4 : Produit 16 384 × 16 384 en CUDA sur la RTX 4060, paires ABBA à référence propre [mes.].

Format	octets de poids	temps (ms)	Go/s équivalents	accélération / FP16
FP16	512 Mio	2,17 – 2,25	238 – 247	1
scalaire 4 bits	128 Mio	0,567 – 0,580	231 – 237	3,8
scalaire 2 bits	64 Mio	0,317 – 0,319	211 – 212	6,9
treillis mul1 2 bits	64 Mio	0,430 – 0,433	155 – 156	5,1

Le treillis est limité par le calcul, pas par la mémoire (tableau 5.4) : une dizaine d'opérations entières par poids le plafonnent à 624 milliards de poids par seconde dans ce noyau naïf ; il coûte 36 % de temps de plus que le scalaire 2 bits et reste 1,33 fois plus rapide que le 4 bits.

5.7.2 En Vulkan et en DirectX 12 : le premier treillis portable mesuré

Le banc `quant-vulkan-gemv` (Rust, wgpu 30.0.1, shaders WGSL écrits de zéro, traduits par Naga en SPIR-V pour Vulkan et en HLSL pour DirectX 12) refait la même mesure, même matrice, même tuile, même témoin. Le treillis y est décodé de deux façons : somme d'octets écrite à la main, ou `dot4U8Packed`, l'équivalent WGSL de `dp4a`. Trois passes ont été nécessaires et sont gardées : la première indexait un tableau local par une variable de boucle (le 2 bits tombait sous le 4 bits) ; la deuxième relisait tout le vecteur d'activations pour chaque ligne (1 Gio de trafic L2 pour 64 Mio de poids) ; la troisième fait quatre lignes par groupe de 32 voies et lit les activations une fois pour les quatre. Une quatrième passe ajoute une extraction de la fenêtre de 16 bits sans `select` (listing 5.2).

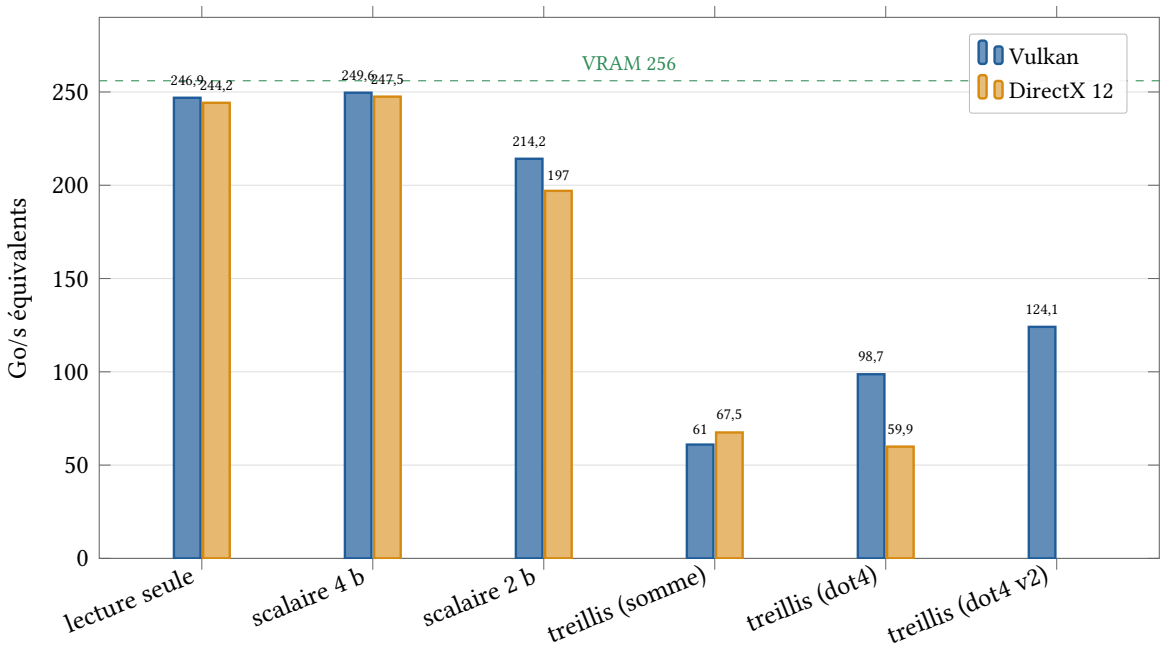


Figure 5.7 : Produit matrice-vecteur 16 384 × 16 384 sur la RTX 4060, Go/s de poids équivalents, médianes des passes 3 et 4 [mes.]. Avec `dot4U8Packed` et l'extraction sans sélection (v2), le treillis 2 bits va aussi vite que le scalaire 4 bits en lisant deux fois moins d'octets.

Données : `fondations/journaux/02-quantification/csv/fig-q2-*.csv` (journaux `vulkan-gemv-3.json`, `vulkan-gemv-4.json`, `dx12-gemv-3.json`).

Listing 5.2 : Le décodeur du treillis à décalage dans le shader WGSL de la GEMV : extraction de quatre fenêtres de 16 bits à cheval sur deux mots, puis le code « mul1 » par `dot4U8Packed`.

```
// Extrait de outils/quant-vulkan-gemv/src/main.rs (commit 89b509c), lignes 50-61 :
// le décodeur du treillis à décalage, code « mul1 », dans le shader WGSL de la GEMV.
// Le gabarit SOMMEV du dépôt est remplacé ici par sa valeur dans la variante retenue
// (dot4U8Packed, ligne 509 du même fichier) ; rien d'autre n'est changé.
fn fenetre4(w0: u32, w1: u32, off: vec4<u32>) -> vec4<u32> {
    let c = min(off, vec4<u32>(31u));
    let bas = ((vec4<u32>(w0) >> c) | (vec4<u32>(w1) << (vec4<u32>(32u) - c))) & vec4<u32>(0xffffu);
    let d = select(vec4<u32>(0u), off - vec4<u32>(32u), off >= vec4<u32>(32u));
    let haut = (vec4<u32>(w1) >> d) & vec4<u32>(0xffffu);
    return select(bas, haut, off >= vec4<u32>(32u));
}
fn mul1v(e: vec4<u32>) -> vec4<f32> {
    let v = e * vec4<u32>(0x83DCD12Du);
    let s = vec4<u32>(dot4U8Packed(v.x, 0x01010101u), dot4U8Packed(v.y, 0x01010101u),
                      dot4U8Packed(v.z, 0x01010101u), dot4U8Packed(v.w, 0x01010101u));
    return (vec4<f32>(s) - vec4<f32>(510.0)) * (1.0 / 147.7);
}
```

Ce que disent ces mesures (figure 5.7) :

– **Un treillis à décalage se décode en shader portable**, dans le produit, sans matrice intermédiaire :

124 Go/s équivalents en Vulkan, 68 en DirectX 12. À notre connaissance, aucune mesure de ce genre n'est publiée.
- `dot4U8Packed` vaut un facteur 1,6 en Vulkan (instruction entière native) et rien en DirectX 12 (où Naga semble l'émuler) : **Vulkan est l'API à viser**.
- Le décodage coûte quand les poids sont en VRAM : le treillis 2 bits reste 1,7 fois plus lent que le scalaire 2 bits et aussi lent que le scalaire 4 bits. **Hors VRAM, ce coût disparaît** : à 124 Go/s, le décodeur va dix fois plus vite que le lien PCIe mesuré (12,2 Go/s) et 44 fois plus vite que le disque. Dans le régime de troncons, le décodage est gratuit et seuls comptent les bits.

5.7.3 Un treillis sans multiplication n'est pas plus rapide

XOR-Trellis [33] annonce un décodeur à environ 14 portes XOR par valeur, sans publier ses masques. Question posée ici : un code calculé fait seulement de parités, écrit de zéro, garde-t-il l'erreur des codes de QTIP et d'EXL3, et se décode-t-il plus vite ? Le code : état de 16 bits s, index $j = \sum_{q=0}^{3} \text{parité}(s\&m_q)\, 2^q$, fonction linéaire de s sur GF(2), valeur prise dans la palette des 16 quantiles gaussiens. Si les quatre masques sont de rang 4, chaque index est atteint par 2^{12} états et la loi des valeurs émises est uniforme sur la palette.

Résultat [mes.] : **erreur 0,0712 ± 0,0009** à 2,0034 bits (masques `0xdfea`, `0x30b1`, `0xb566`, `0x753c`), contre 0,0682 pour mul1 sur les mêmes suites, soit $\frac{1}{2}\log_2(0{,}0712/0{,}0682) = 0{,}031$ bit de perte (lemme 5.3). Le témoin repasse mul1 par la même machinerie et retrouve *exactement* 0,068236 (même générateur consommé dans le même ordre). Mais en Vulkan, ce décodeur va à 52 Go/s équivalents contre 119 à 121 pour mul1 : quatre comptages de bits par valeur coûtent plus qu'une multiplication entière et un `dot4`, qui sont des instructions natives de la carte. La variante sans table (palette uniforme) va à la même vitesse : ce n'est pas la table qui coûte, ce sont les comptages. **Sur une RTX 4060, mul1 reste le meilleur code à décoder** ; l'avantage de XOR-Trellis est pour un circuit dédié.

5.8 L'échelle de llama.cpp, mesurée sur Qwen3.5

Protocole. llama.cpp b11515, base Q4_K_M de la machine (biais déclaré en fin de chapitre) ; matrice d'importance sur 128 morceaux de 512 jetons de wikitext-2 « train » ; quantification vers chaque type avec le mélange par défaut de llama.cpp ; bits réels lus dans le fichier ; perplexité et divergence de Kullback-Leibler contre les logits de la base sur 40 morceaux de 512 jetons de wikitext-2 « test ». La matrice d'importance $I_j = \sum_t x_{t,j}^2$ accumule le carré de chaque colonne d'entrée sur le texte d'étalonnage ; la quantification minimise alors $\sum_j I_j (w_j - q_j)^2$, l'approximation diagonale du hessien $X^\top X$ de GPTQ et QuIP#.

Ce que dit l'échelle (tableau 5.5 et figure 5.8) :

- **À octets égaux, l'i-quant bat la K-quant de 11 %** : IQ3_XXS 9,777 contre Q2_K 10,957 pour 1,89–1,91 Go lus par jeton (le « Q2_K » de llama.cpp est en fait à 3,6 bits par poids : ses matrices sensibles montent en Q3_K et Q4_K).
- **IQ1_S est inutilisable sur un 4B ordinaire** (perplexité multipliée par 5,5) ; le 1,56 bit ne vaut que pour des modèles entraînés pour lui.
- **Le 4B en 4 bits bat le 9B en 2 bits de 22 % de perplexité, à 1 % d'octets près** (2,730 contre 2,757 Go ; 8,723 contre 10,605) ; les intervalles à deux erreurs-types, [8,27; 9,18] et [10,06; 11,15], sont disjoints. Avec le meilleur 2 bits portable d'aujourd'hui, rapetisser vaut mieux que comprimer pour Qwen3.5.
- **Le croisement a lieu vers 3,2 bits par poids, pas à 2**. Par interpolation linéaire du front du 9B entre IQ2_XXS et IQ2_M, il atteint 8,723 à $2{,}757 + \frac{10{,}605-8{,}723}{10{,}605-8{,}612} \times 0{,}402 = 3{,}137$ Go, soit

Table 5.5 : L'échelle de llama.cpp sur Qwen3.5-4B et 9B ; ± : erreur-type de llama-perplexity sur les 40 morceaux [mes.].

Modèle	type	bits/poids	Go lus/jeton	PPL	KLD	même 1er jeton
4B	Q4_K_M (base)	5,193	2,730	8,723 ± 0,229	0	100 %
4B	IQ4_XS	4,762	2,503	8,908 ± 0,236	0,030	91,9 %
4B	IQ3_XXS	3,602	1,894	9,777 ± 0,259	0,148	82,1 %
4B	Q2_K	3,623	1,905	10,957 ± 0,300	0,254	77,4 %
4B	IQ2_M	3,292	1,731	10,956 ± 0,293	0,269	76,4 %
4B	IQ2_XXS	2,905	1,527	15,493 ± 0,437	0,608	65,8 %
4B	IQ1_S	2,565	1,348	48,027 ± 1,467	1,780	44,7 %
9B	Q4_K_M (base)	5,066	5,097	7,174 ± 0,178	0	100 %
9B	IQ3_XXS	3,509	3,490	7,913 ± 0,198	0,113	85,0 %
9B	IQ2_M	3,213	3,159	8,612 ± 0,217	0,206	80,2 %
9B	IQ2_XXS	2,761	2,757	10,605 ± 0,271	0,425	72,3 %

Données : `fondations/journaux/02-quantification/csv/echelle.csv`, journaux `echelle-4B/`, `echelle-9B/`.

$3{,}137 \times 8/7{,}94 = 3{,}16$ bits par poids lu [calc.]. Au-delà, le 9B comprimé est meilleur que tout le 4B mesuré.

En VRAM, les octets économisés ne deviennent qu'à moitié de la vitesse. La passe de vitesse retenue (machine calme, `-1m none`, 128 jetons × 5, ABBA, bruit de la référence 0,1 à 2,8 % pour un seuil de 5 %) donne la figure 5.9. Du Q4_K_M à l'IQ2_XXS, les octets lus sont divisés par 1,79 (4B) et 1,85 (9B), la vitesse multipliée par 1,34 et 1,49 seulement : la part du plafond de 256 Go/s tombe de 74–84 % à 55–68 %. Le décodage des i-quants (grilles en mémoire partagée) et le coût fixe par jeton prennent le reste. Hors VRAM, le rapport s'inverse : au débit du lien, le décodage des i-quants (au moins 142 Go/s effectifs mesurés) va dix fois plus vite que le lien, et seuls les octets comptent.

5.9 À octets lus égaux : le meilleur rapport d'aujourd'hui

On compare à octets lus égaux, car c'est ce que paie chaque jeton. Le seul ensemble publié où toutes les méthodes et toutes les tailles sortent du même programme est le tableau de QTIP sur Llama 2 (figure 5.10).

- **Avec un treillis et un ajustement fin, un modèle deux fois plus grand en 2 bits égale ou bat le modèle deux fois plus petit en 4 bits**, sur Llama 2 [litt.]. Avec une quantification scalaire (EfficientQAT [25]), c'est l'inverse : le croisement dépend de la méthode autant que de la taille.
- **Sur une famille très entraînée (Qwen3), avec les codes de septembre 2026, le croisement a lieu dès 8B** au MMLU [34] (figure 5.11). Les auteurs écrivent que cette comparaison a été choisie après la mesure.
- **Avec les i-quants de llama.cpp, le croisement n'a pas lieu à 9B** (section 5.8).

À partir de quelle taille les 2 bits deviennent-ils acceptables ? Réponse par délégation du 2026-10-15, réversible : « acceptable » veut dire que le modèle de taille $2N$ à $b/2$ bits est au moins aussi bon que celui de taille N à $b \approx 4$ bits, à octets lus égaux ; c'est le seul critère qui dit s'il vaut

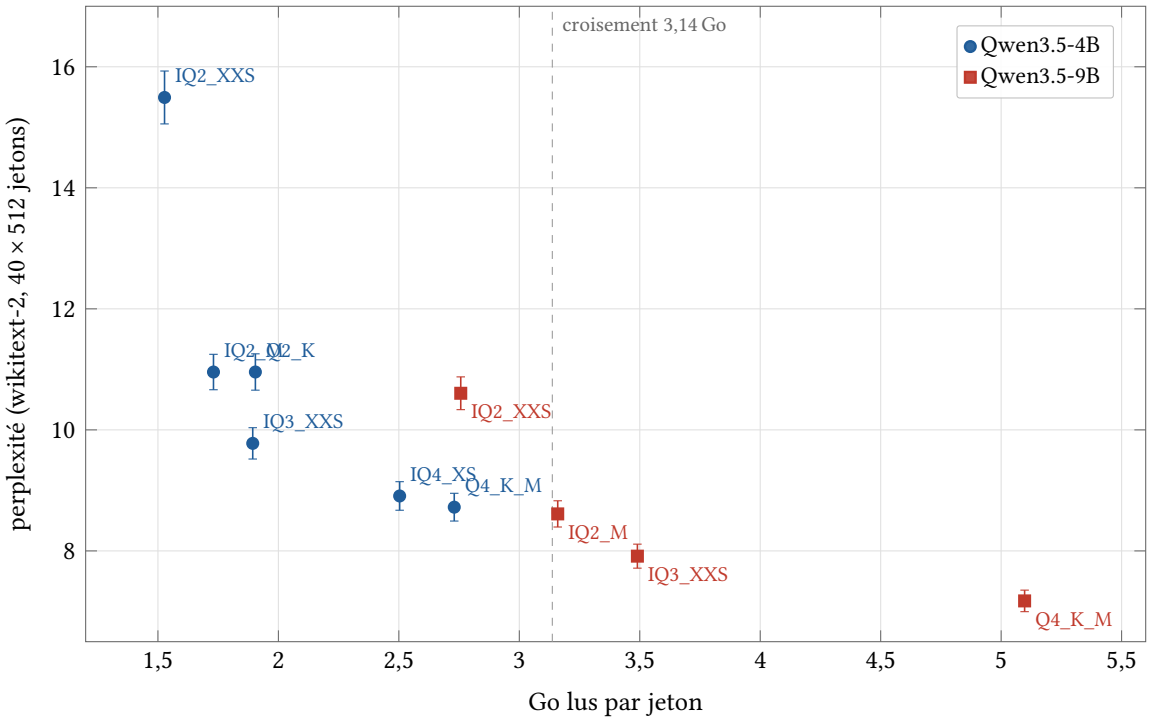


Figure 5.8 : Perplexité contre octets lus par jeton, Qwen3.5-4B et 9B sous llama.cpp [mes.]. Barres : ± une erreur-type. En dessous d'environ 3,14 Go lus par jeton, le 4B gagne ; au-delà, le 9B comprimé domine tout le front du 4B.

Données : `fondations/journaux/02-quantification/csv/fig-q3-4B.csv`, `fig-q3-9B.csv`.

mieux comprimer ou rapetisser. La perte relative du « IQ2_XXS » contre la base de même taille vaut $\ell(4{,}21\,\mathrm{G}) = 0{,}777$ et $\ell(8{,}95\,\mathrm{G}) = 0{,}479$ [mes.]. Une loi de puissance par ces deux points donne $\ell(N) = 0{,}479\,(N/8{,}95)^{\alpha}$ avec $\alpha = \ln(0{,}479/0{,}777)/\ln(8{,}95/4{,}21) = -0{,}640$ [calc.]. Elle tombe sous l'écart de qualité d'un doublement de taille, 22 % entre 4B et 9B, vers $N = 8{,}95 \cdot (0{,}22/0{,}479)^{1/\alpha} \approx 30$ milliards de paramètres, et sous 10 % vers 100 milliards. **Avec IQ2_XXS, sur Qwen3.5, le seuil est au-delà de 30B ; avec un code de deuxième génération, Qwen3 le franchit à 8B : le codeur déplace le seuil d'un facteur 4 à 10.** Deux points seulement portent cette loi : c'est un ordre de grandeur.

5.10 Le partage des bits selon l'étage

Les outils publiés (HIGGS dynamique, les recettes d'EXL3, `--tensor-type` de llama.cpp) minimisent l'erreur sous un budget de *stockage*. Dans troncons, un octet lu en VRAM et un octet amené par le lien n'ont pas le même prix.

Hypothèse 5.7 (Linéarité, pente de Shannon, placement donné)**.** (H1) La perte de qualité est $\sum_i c_i\,\varepsilon_i$, où ε_i est l'erreur relative de la matrice i et $c_i \geq 0$ sa sensibilité (théorème de linéarité de HIGGS). (H2) $\varepsilon_i = \kappa\, 2^{-2b_i}$ (théorème 5.1, à une constante près par codeur). (H3) La matrice i, de n_i poids, est servie par un étage de débit W_i fixé. (H4) Le temps de transfert par jeton $\sum_i n_i b_i/(8W_i)$ ne dépasse pas un budget T.

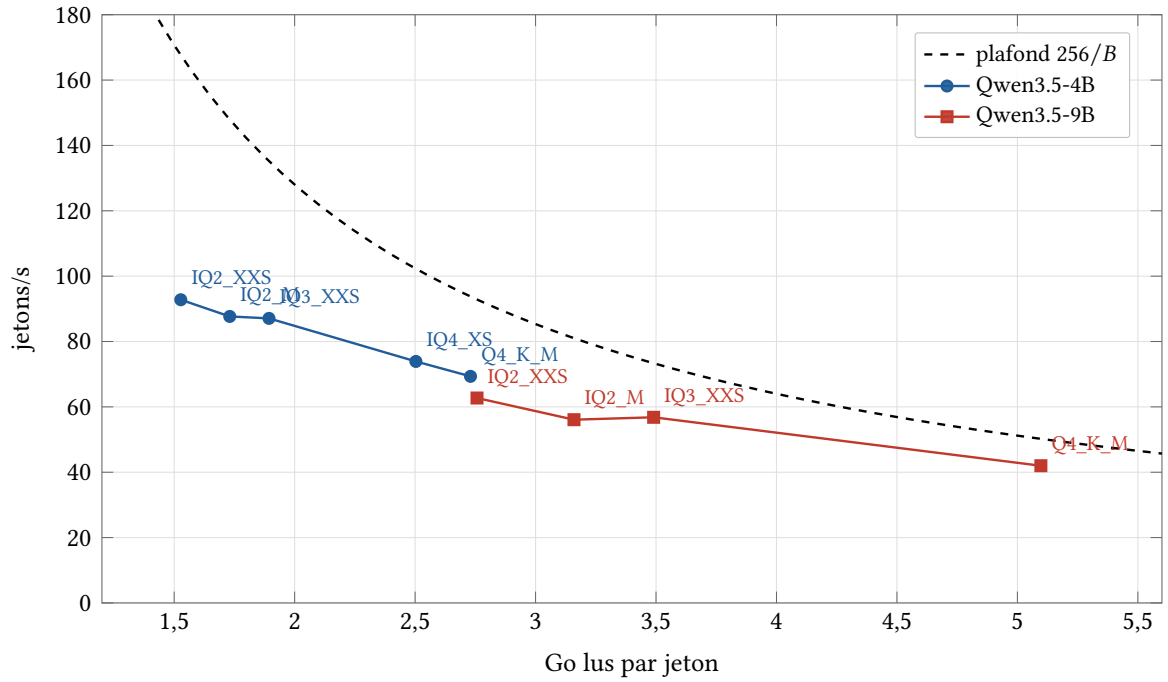


Figure 5.9 : Vitesse de décodage sous llama.cpp (CUDA, lot de 1, tout en VRAM) contre octets lus par jeton [mes.]. La courbe pointillée est le plafond du théorème 3.2 à 256 Go/s : les formats les plus compacts s'en éloignent, car leur décodage se paie en VRAM.

Données : `fondations/journaux/02-quantification/csv/fig-q6-vitesse-*.csv` (journaux `vitesse-4B-3.json`, `vitesse-9B-3.json`).

Proposition 5.8 (Bits optimaux par matrice). *Sous l'hypothèse 5.7, le problème* $\min_b \sum_i c_i \kappa 2^{-2b_i}$ *sous* $\sum_i n_i b_i / W_i \le 8T$ *a pour solution*

$$b_i^\star = \tfrac{1}{2}\log_2\!\left(\frac{2\ln 2\,\kappa\, c_i\, W_i}{\lambda\, n_i}\right), \tag{5.3}$$

où $\lambda > 0$ *est fixé par la contrainte active.*

Démonstration. Le lagrangien $\mathscr{L} = \sum_i c_i\kappa 2^{-2b_i} + \lambda(\sum_i n_i b_i/W_i - 8T)$ est convexe en b : somme d'exponentielles décroissantes et d'un terme linéaire. La condition du premier ordre $\partial\mathscr{L}/\partial b_i = -2\ln 2\, c_i\kappa\, 2^{-2b_i} + \lambda n_i/W_i = 0$ donne $2^{-2b_i} = \lambda n_i/(2\ln 2\, c_i\kappa W_i)$, d'où la formule. La convexité en fait le minimum global ; la contrainte est active à l'optimum, puisque la perte décroît strictement en chaque b_i. ∎

Corollaire 5.9 (La règle des bits par étage). *Deux matrices de même densité de sensibilité* c_i/n_i*, l'une servie par l'étage de débit* W_1*, l'autre par l'étage* W_2*, reçoivent*

$$b_1^\star - b_2^\star = \tfrac{1}{2}\log_2\frac{W_1}{W_2} \tag{5.4}$$

bits d'écart. Avec les débits mesurés au chapitre 3 : ***2,18 bits*** *entre la VRAM et le lien PCIe* ($\frac{1}{2}\log_2(251{,}3/12{,}2)$)*,* ***3,24 bits*** *entre la VRAM et le disque* ($\frac{1}{2}\log_2(251{,}3/2{,}82)$)*, 1,06 bit entre le lien et le disque [calc.].*

Démonstration. Faire le rapport des deux équation (5.3) : λ, κ et c_i/n_i se simplifient. ∎

L'étude d'origine avait chiffré cette règle avec un lien à 7 Go/s : ~~une matrice résidente en VRAM reçoit $\frac{1}{2}\log_2(256/7) \approx 2{,}6$ bits de plus qu'une matrice lue par le PCIe~~. Ce débit était un état chargé de la

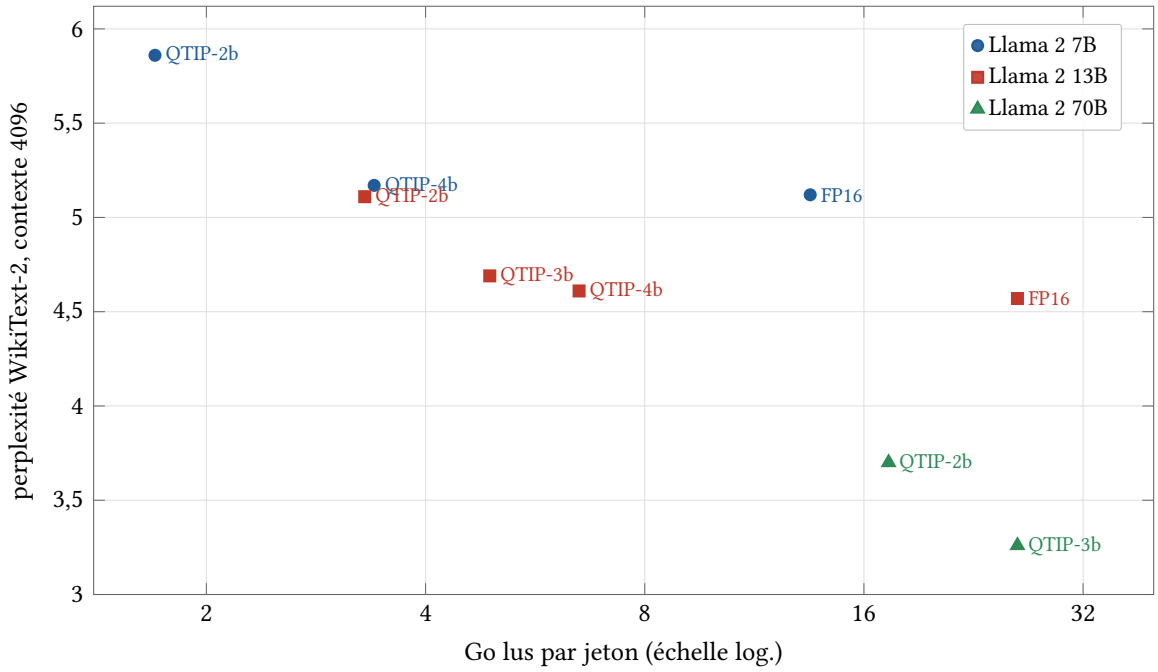


Figure 5.10 : À octets lus égaux, Llama 2 quantifié en treillis QTIP [litt.] [calc.]. Un 13B en 2 bits (3,3 Go, 5,11) égale un 7B en 4 bits (3,4 Go, 5,17) ; un 70B en 2 bits (17 Go, 3,70) bat un 13B en FP16 (26 Go, 4,57).

Données : `fondations/journaux/02-quantification/csv/fig-q4-llama2-*.csv` (QTIP, tableau 5).

machine ; au calme, le lien livre 12,2 Go/s (section 3.5), et l'écart tombe à 2,18 bits. Les poids lus du disque, eux, méritent 3,24 bits de moins que les résidents : résidents en 4 à 5 bits, flux disque autour de 1 à 2 bits.

Proposition 5.10 (La fréquence décide du placement, la sensibilité des bits)**.** *Ajoutons une probabilité d'usage p_i par jeton (experts d'un MoE ; $p_i = 1$ pour un modèle dense) : le temps devient $\sum_i p_i n_i b_i / W_i$ et la perte $\sum_i p_i c_i \kappa 2^{-2b_i}$. Alors (a) $b_i^\star$ est donné par l'équation (5.3) et ne dépend pas de p_i ; (b) le gain à rendre la matrice i résidente est proportionnel à p_i et vaut, quand $W_V \gg W_P$, environ $\lambda\, p_i\, n_i \left(\frac{1}{2 \ln 2} + b^\star_{i,P}\right)/W_P$.*

Démonstration. (a) Dans la condition du premier ordre, p_i multiplie les deux termes et se simplifie. (b) À l'optimum, $c_i \kappa 2^{-2b^\star} = \lambda n_i / (2 \ln 2\, W)$; la contribution de la matrice au lagrangien vaut donc $\Lambda_i(W) = p_i\left(c_i \kappa 2^{-2b^\star} + \lambda n_i b^\star / W\right) = \lambda\, p_i\, n_i \left(\frac{1}{2 \ln 2} + b_i^\star(W)\right)/W$, et le gain de résidence est $\Lambda_i(W_P) - \Lambda_i(W_V)$, dominé par le premier terme quand $W_V \gg W_P$. ∎

D'où la règle de remplissage de la VRAM : **trier par fréquence d'usage d'abord, sensibilité ensuite**. À notre connaissance, aucun outil publié ne pose ce problème conjoint (bits, étage) ; Q-Palette [97] le pose à placement fixe avec un coût de latence profilée.

Le code emboîté. ApexQuant [98] requantifie le résidu après une nouvelle rotation aléatoire, passe après passe ; chaque préfixe des passes est un modèle valide, et le « prix de l'élasticité » est une distance constante à la borne par passe : 0,45 bit pour un scalaire, 0,10 bit pour un treillis [litt.]. Dans troncons, cela donne un seul fichier pour les deux étages : une matrice lue en flux ne lit que sa passe 1 (environ 2 bits), une matrice résidente lit les passes 1 et 2 (environ 4 bits), et quand la fenêtre de résidence bouge, rien ne se requantifie. Coût [calc.] : environ 0,10 bit sur les résidents, soit 2,5 % d'octets en VRAM ; gain contre deux copies séparées (2 + 4 bits) : 1,9 bit par poids de disque, 32 % du

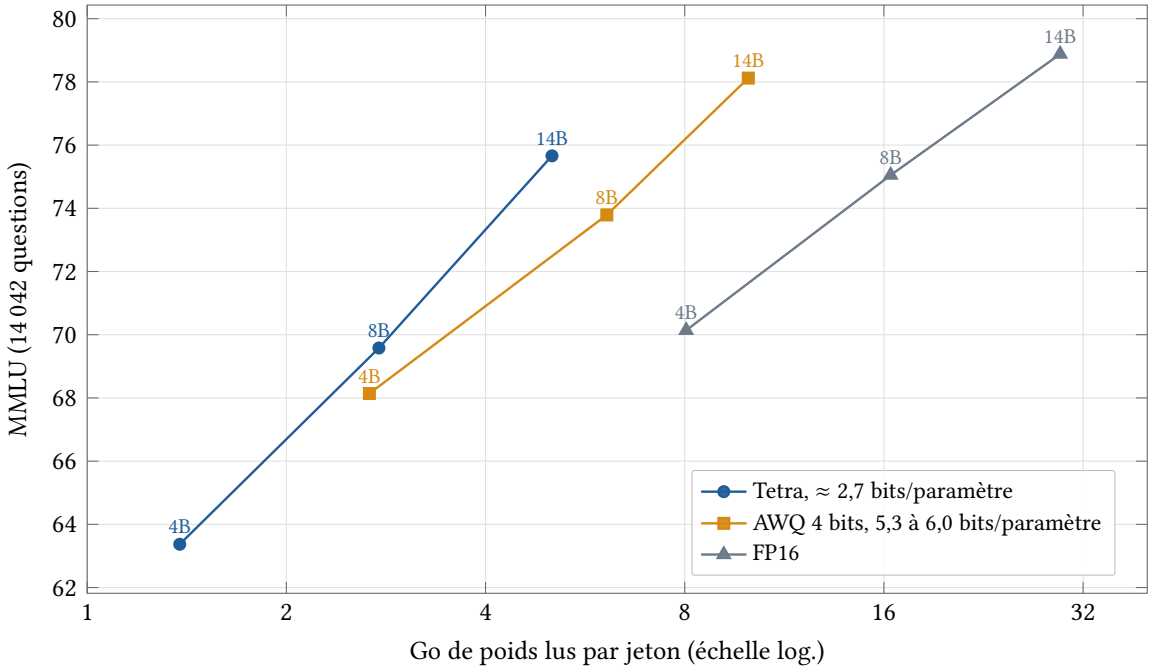


Figure 5.11 : À octets lus égaux, Qwen3 au MMLU : le code sur réseau de Leech de Tetra à environ 2,7 bits par paramètre contre AWQ 4 bits et FP16 [litt.] [calc.]. Le Qwen3-8B Tetra (2,76 Go) bat le Qwen3-4B AWQ (2,67 Go), 69,6 contre 68,1.

Données : `fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-*.csv` (Tetra, tableau 3).

stockage. C'est la règle du corollaire 5.9 rendue dynamique ; aucun moteur publié ne lit des préfixes de précision selon l'étage de mémoire.

5.11 Les techniques, en une table

5.12 Ce qui manque dans le monde

1. **Aucun décodeur de treillis (classe QTIP, EXL3) en Vulkan ou DirectX 12**, hors celui de ce chapitre. Enjeu [calc.] : à 2 bits, treillis 0,068 contre 0,093 pour une VQ de dimension 8 et 0,107 pour IQ2_XXS, soit 0,23 à 0,33 bit par poids à qualité égale (lemme 5.3) ; sans ajustement fin, le gain publié en perplexité est plus petit (QTIP 3INST 6,82 contre IQ2_XXS 7,07 sur Llama 2 7B). CurveTQ montre qu'un treillis sans rotation, avec la courbure dans la métrique du Viterbi et un état initial stocké par bloc, fait mieux que QTIP : le décodeur à écrire est donc plus simple que celui mesuré ici, et le code à porter est mul1 avec `dot4U8Packed`.
2. **Personne n'alloue les bits selon le coût du transfert** (proposition 5.8) ; c'est une contribution directe pour troncons.
3. **Aucun code emboîté n'est lié au placement.**
4. **Pas de frontière iso-octets publiée pour les familles très entraînées sous 3 bits avec des noyaux portables** ; la figure 5.8 en donne un premier point pour Qwen3.5.
5. **Quantifier un grand modèle sur la machine** : QTIP et PV-tuning demandent des jours de cartes de centre de calcul ; un quantificateur en treillis qui travaille matrice par matrice depuis le disque, sur la carte, en Vulkan, n'existe pas.

Table 5.6 : Les techniques de quantification extrême, état au 2026-10-15. Pertes : perplexité WikiText-2, FP16 → quantifié [litt.].

Technique	bits/poids	Perte publiée (exemples)	Vulkan	Licence
K-quants (llama.cpp)	2,625–6,5625	Q2_K : +3,52 sur Llama 3 8B	**oui**	MIT
i-quants + imatrix	1,5625–4,25	IQ2_XXS : 7B 5,12→7,07 ; 70B 3,12→4,08	**oui**	MIT
EfficientQAT [25]	≈ 2,25	L2-7B 5,47→6,86 ; L3-70B 2,85→6,08	non	MIT
HIGGS [26]	2–4,25	L3.1-8B 5,607→6,388 à 3,25 bits	non	Apache-2.0
QuIP# [27]	2, 3, 4	70B 3,12→3,91 à 2 bits	non	GPL-3.0
AQLM [28] + PV-tuning [29]	1,0–2,3	L2-7B 5,12→5,84 ; L3-70B 2,59→4,57	non	Apache-2.0
VPTQ [30]	1,375–4	L2-70B 2,07 bits : 3,93	non	MIT
QTIP [31]	2, 3, 4	2 bits : 7B 5,86 ; 70B 3,70	non	GPL-3.0
CurveTQ [32]	2,29 tout compris	Qwen3-4B 9,72→11,50 ; Qwen3-32B 6,17→6,90	non (CUDA)	—
XOR-Trellis [33]	≈ 2	L3.1-8B 5,84→8,00 (QTIP 8,48)	non publié	—
Tetra [34]	2,148 lus	MMLU Qwen3-8B 69,6 (FP16 75,1)	non (CUDA)	Apache-2.0
GSQ [99]	2,13 / 3,13	projetable en Q2_K / Q3_K_M	**oui si projeté**	Apache-2.0
ApexQuant [98]	$P \times b$, préfixes	chaque préfixe valide	non publié	non déclarée
EXL3	1–8	courbes du dépôt	**non** (CUDA)	MIT
IQn_KT (ik_llama.cpp)	1,75–4,0	erreur ≈ VQ 8 aléatoire [mes.]	non	MIT
BitNet b1.58 [35]	1,58	natif, au niveau des modèles pleins (2B4T)	via TQ1_0	MIT
MXFP4 / NVFP4 [38]	4,25 / 4,5	NVFP4 natif « comparable à FP8 »	stockage oui	—

6. **Le meilleur 2 bits décodable en Vulkan est encore une K-quant mal remplie** : GSQ améliore un Q2_K de Qwen3-8B de 50,03 à 56,28 de moyenne sans changer de format [litt.].

Ce qui pourrait être faux

- **La carte et le processeur étaient partagés** : les temps de quantification sont des majorants ; les mesures de qualité n'en dépendent pas.
- **Les bases sont des Q4_K_M, pas des BF16** : la requantification ajoute l'erreur du Q4_K_M ; estimée à environ 10 % de la perte à 2 bits, ce qui rend les résultats bas débit pessimistes d'autant. La divergence KL est mesurée contre le Q4_K_M.
- **La matrice d'importance est calculée sur le même domaine que le test** (biais optimiste pour les i-quants).
- **20 480 jetons d'évaluation** ; la perplexité absolue n'est pas comparable aux chiffres publiés en contexte 4096.
- **Les chiffres de la littérature viennent de programmes différents** ; seules les comparaisons internes à un article ou à notre échelle sont propres.
- **L'erreur sur gaussienne n'est pas la qualité du modèle** ; le passage à la perplexité repose sur un théorème de linéarité valable pour de petites erreurs, ce que 2 bits ne sont plus.
- **La règle des bits par étage** suppose des erreurs additives et la pente de Shannon : elle donne la direction et l'ordre de grandeur, pas la valeur.
- **Le front d'octobre 2026 n'est pas reproduit** ; CurveTQ, Tetra, XOR-Trellis et ApexQuant ont moins de six semaines.
- **L'extrapolation à 30B** tient sur deux points.

Chapitre 6

Compression sans perte, à la vitesse du calcul

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la littérature promettait deux choses contradictoires : des codeurs sans perte qui rendent 30 % sur des poids BF16 (DFloat11, ZipNN), et l'affirmation que les fichiers GGUF quantifiés « ne se compressent pas du tout ». Un article de l'ISCA 2026 annonçait en plus « 0,6 à 1,0 bit d'entropie réelle » pour des poids de 4 bits, ce qui aurait voulu dire un facteur 4 caché dans chaque modèle quantifié. Aucun décodeur entropique de poids n'avait de débit publié sous Vulkan, ni sur une carte de portable. **Ce chapitre établit** ce que contient vraiment un code quantifié de la machine (3,86 bits sur 4, l'ordre 0 dit tout) ; pourquoi, par un calcul exact qui retrouve les mesures au centième ; que le format cible, MXFP4, cache son gain dans ses échelles, et qu'un codage par champs le prend exactement (taux 1,1026 sur gpt-oss-120b, décodé au bit) ; un codeur rANS entrelacé dont l'exactitude est démontrée, décodé sur la carte à 70 Go/s par Vulkan et à 82 Go/s par CUDA ; et la réponse mesurée à la question clé : décoder *dans* le produit est 3,7 fois plus lent, décoder *à l'arrivée* ne coûte que 3,6 % du temps de la carte et rend tout le taux au disque. **Il reste** la quantification taillée pour le codeur (0,41 à 0,46 bit par poids à prendre, mesuré sur des substituts), l'écriture du décodeur directement à la place finale, et la reproduction de DietGPU et Recoil sur la machine. Source : `fondations/03-entropie.md`, `fondations/03-entropie-derivations.md` (version du commit `f75d2cc`), `fondations/journaux/03-entropie/`.

6.1 Le juge : quand le sans-perte paie

Compresser sans perte d'un taux r divise par r les octets qui traversent le disque, la RAM et le lien. Cela ne gagne que si le décodeur suit : son débit de sortie doit dépasser r fois celui de l'étage qu'il soulage, et son temps de calcul ne doit pas manger le gain. Dans ce chapitre, W_{dec} est le débit de sortie du décodeur sur la carte (octets décodés par seconde) et W_g le débit auquel le produit matrice-vecteur consomme des poids décodés, comme au chapitre 4.

Proposition 6.1 (Temps d'un jeton, chaîne recouverte et décodage sur la carte)**.** *Soit un jeton qui lit B octets de poids (non compressés) par le disque et le lien, stockés au taux r. Si les étages sont recouverts et si le décodeur et le produit partagent la carte, alors*

$$t(r) = \max\Big(\frac{B}{r\,W_{\text{disque}}},\ \frac{B}{r\,W_{\text{PCIe}}},\ B\Big(\frac{1}{W_{\text{dec}}} + \frac{1}{W_g}\Big)\Big). \tag{6.1}$$

Démonstration. C'est le théorème 3.2 appliqué à trois étages : le disque et le lien voient B/r octets ; la carte exécute en série le décodage (B/W_{dec} secondes) puis le produit (B/W_g), ce qui en fait un

seul étage de débit $(1/W_{\text{dec}} + 1/W_g)^{-1}$. Le recouvrement rend la borne atteignable à la pénalité de remplissage du lemme 3.3 près. ■

Corollaire 6.2 (Le gain vaut exactement r, puis plafonne). *Si $W_{\text{disque}} \le W_{\text{PCIe}}$, le gain $t(1)/t(r)$ vaut exactement r tant que $r\,W_{\text{disque}} \le (1/W_{\text{dec}} + 1/W_g)^{-1}$, et plafonne au-delà à $W_{\text{disque}}\,(1/W_{\text{dec}} + 1/W_g)$.*

Démonstration. Sous la condition, le premier terme de (6.1) domine pour tout taux entre 1 et r, donc $t(1)/t(r) = (B/W_{\text{disque}})/(B/(rW_{\text{disque}})) = r$. Au-delà, le troisième terme domine $t(r)$ et ne dépend plus de r. ■

Le calcul, avec les débits de la machine. $W_{\text{disque}} = 2{,}82$ Go/s (chapitre 3), $W_{\text{dec}} = 70$ Go/s (décodeur v4 sous Vulkan, sur secteur, section 6.6) et $W_g = 209$ Go/s (produit MXFP4 brut, section 6.7) donnent $(1/70 + 1/209)^{-1} = 52{,}4$ Go/s [calc.], contre $r\,W_{\text{disque}} = 1{,}10 \times 2{,}82 = 3{,}1$ Go/s. La condition tient avec un facteur 17 : **chaque pour cent de compression devient un pour cent de jetons par seconde**, et la carte n'est occupée qu'à 6 % par le décodage et le produit réunis. Sur batterie, où la carte est plafonnée à 43 W ($W_{\text{dec}} = 26{,}8$, $W_g = 66{,}4$), le terme de la carte vaut encore 19,1 Go/s, six fois $r\,W_{\text{disque}}$ [calc.].

Le taux agit deux fois sur un mélange d'experts. Une mémoire qui garde une part h des experts en clair en garde $h\,r$ s'ils y sont codés ; les octets lus au disque par jeton passent de $(1-h)B_e$ à $(1-hr)B_e/r$. L'étude chiffrait ainsi +14,4 % de jetons par seconde pour gpt-oss-120b à $r = 1{,}1026$ et $h = 26$ % (routage uniforme). ~~+14,4 % de jetons par seconde~~ Le chapitre 4 a corrigé ce chiffre : la carte doit garder ses experts *décodés* (section 6.7), et seule la RAM grandit ; l'effet est de +13,1 % [calc.].

6.2 Ce que contient un code quantifié

6.2.1 L'estimateur, et son biais

L'entropie d'un champ (les codes de 4 bits, les échelles d'un bloc) s'estime par les fréquences observées : $\hat{H} = -\sum_s \hat{p}_s \log_2 \hat{p}_s$, $\hat{p}_s = n_s/N$, sur tous les blocs d'un modèle. Un estimateur par fréquences sous-estime l'entropie ; il faut savoir de combien avant de lire un écart de quelques millièmes de bit.

Lemme 6.3 (Biais au premier ordre). *Pour N tirages indépendants d'une loi à K symboles de probabilités non nulles,*

$$\mathbb{E}[\hat{H}] = H - \frac{K-1}{2N \ln 2} + O(N^{-2}). \tag{6.2}$$

Démonstration. Posons $\phi(p) = -p \ln p$, donc $\phi''(p) = -1/p$. Un développement de Taylor à l'ordre 2 autour de p_s donne $\mathbb{E}[\phi(\hat{p}_s)] = \phi(p_s) + \frac{1}{2}\phi''(p_s)\,\mathrm{Var}(\hat{p}_s) + O(N^{-2})$, le terme d'ordre 1 s'annulant puisque $\mathbb{E}[\hat{p}_s] = p_s$. Avec $\mathrm{Var}(\hat{p}_s) = p_s(1-p_s)/N$, chaque symbole contribue $-(1-p_s)/(2N)$, et la somme sur les K symboles vaut $-(K - \sum_s p_s)/(2N) = -(K-1)/(2N)$ nats. On divise par $\ln 2$ pour des bits. ■

Pour les codes Q4_K de Qwen3.5-9B, $K = 16$ et $N \approx 7{,}4 \cdot 10^9$: le biais vaut $15/(2 \times 7{,}4 \cdot 10^9 \times 0{,}693) = 1{,}5 \cdot 10^{-9}$ bit [calc.], négligeable. Pour le champ d en fp16 d'un petit tenseur ($K \approx 5\,000$ valeurs vues, $N = 40\,000$ blocs), il atteint 0,09 bit [calc.] : les gains sur les métadonnées en fp16 sont donc optimistes, et on leur préfère le taux d'un codage réel.

6.2.2 Les codes : presque uniformes, et l'ordre 0 dit tout

L'étude a écrit son propre lecteur de GGUF et découpé chaque bloc en ses champs (figures 5.2 et 5.3) ; la déquantification refaite depuis ces champs égale celle de la bibliothèque `gguf` de llama.cpp à écart nul, et le témoin (deux demi-octets inversés dans notre seul chemin) fait tomber l'égalité [mes.] (`temoin-champs.log`). Le tableau 6.1 donne ce qui reste dans chaque type de bloc de Qwen3.5-9B.

Table 6.1 : Entropie des blocs K-quant de Qwen3.5-9B Q4_K_M, par type [mes.]. « Idéal » : codage d'ordre 0 de chaque champ (codes et métadonnées) ; taux = bits stockés / idéal. zstd au niveau 3 et lz4 sur les tenseurs entiers. Le 4B donne 5,187 → 4,930 bits par poids (taux 1,0521).

type	part	stockés	H_0 codes	H | préc.	idéal	taux	zstd 3	lz4
Q4_K	58,5 %	4,500	3,864 / 4	3,858	4,256	1,0574	1,0215	1,000
Q5_K	14,7 %	5,500	4,861 / 5	4,853	5,253	1,0470	1,0000	1,000
Q6_K	26,7 %	6,562	5,789 / 6	5,785	6,274	1,0460	1,0000	1,000
Q8_0	0,1 %	8,500	7,556 / 8	7,185	7,807	1,0888	1,0546	1,000
modèle	100 %	**5,062**			**4,808**	**1,0529**	**1,0125**	**1,000**

Données : `fondations/journaux/03-entropie/entropie-9B-resume.txt`, `entropie-4B-resume.txt`.

Trois faits s'en dégagent. **Les codes sont presque uniformes** : 3,864 bits sur 4 pour Q4_K, 4,86 sur 5, 5,79 sur 6. **Le contexte n'apporte rien** : le code précédent retire 0,005 bit, l'échelle du sous-bloc pas davantage ; tout est dans l'ordre 0. **Une seule table suffit** : une table de fréquences par tenseur contre une table pour tout le modèle, 3,86519 contre 3,86521 bits par code [mes.] (`rans-4B-*.json`) ; les rôles (`ffn_gate`, `attn_*`, …) diffèrent de moins de 0,01 bit. La figure 6.1 montre la forme de la loi : une cloche décalée vers les codes 7 et 8, avec des queues relevées aux deux bouts, signature d'un ajustement échelle-minimum qui écrête pour remplir les 16 niveaux.

6.2.3 Les métadonnées, et ce que voit un codeur d'octets

Les métadonnées d'un bloc Q4_K pèsent 0,5 bit par poids : d et $d_{\min}$ en fp16 portent 9,46 à 9,53 et 10,11 à 10,35 bits sur 16, les échelles et minimums de 6 bits portent 4,89 et 5,19 bits [mes.]. Leur idéal d'ordre 0, par poids, est

$$\frac{H(d) + H(d_{\min})}{256} + \frac{8\left(H(\text{éch}_6) + H(\min_6)\right)}{256} = \frac{9{,}46 + 10{,}35}{256} + \frac{8\,(4{,}89 + 5{,}19)}{256}$$
$$= 0{,}0774 + 0{,}315 = 0{,}392 \text{ bit} \quad \text{[calc.]},$$

soit un gain de 0,108 bit par poids, contre $4 - 3{,}864 = 0{,}135$ pour les codes. **Coder les codes seuls laisse près de la moitié du gain sur la table.**

Le tableau 6.2 réconcilie les faits publiés. ZipNN écrit que les GGUF « ne se compressent pas » [42] : c'est vrai pour un LZ et pour un codeur d'octets sur le bloc entier (1,8 % au mieux). Ce ne l'est plus pour un codeur qui connaît les champs : pour Q4_K, un codeur d'octets par champ prend presque tout (1,052 sur 1,057). Pour Q5_K et Q6_K, le 5e et le 6e bit sont rangés à part (champ qh), et un codeur d'octets les voit comme du bruit (7,999 bits par octet sur le qh de Q5_K [mes.]) : il faut des alphabets de 32 et 64 symboles.

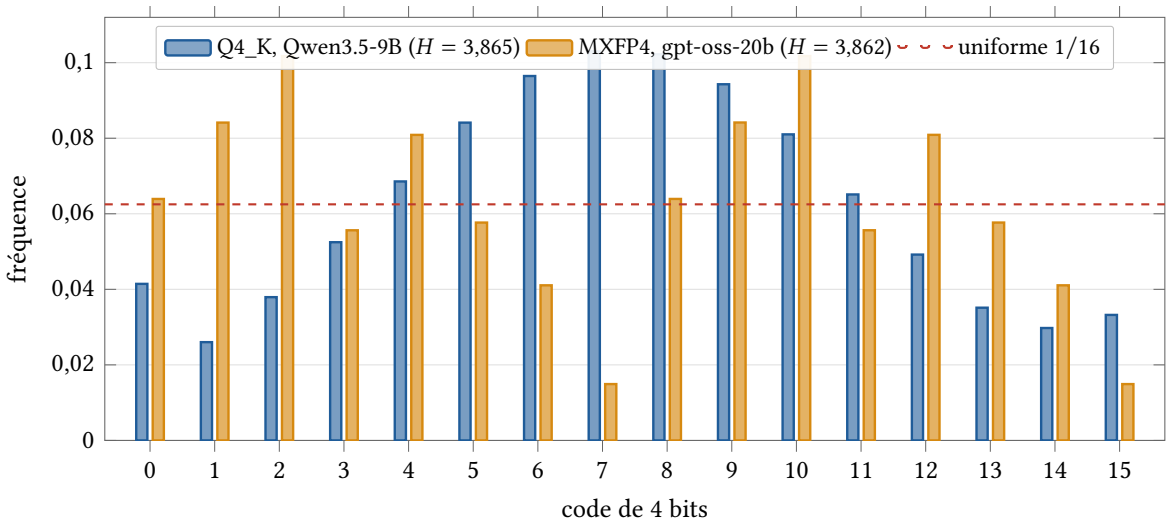


Figure 6.1 : Fréquence de chacun des 16 codes, tous les tenseurs Q4_K de Qwen3.5-9B et tous les experts MXFP4 de gpt-oss-20b [mes.]. Les deux lois sont proches de l'uniforme (3,865 et 3,862 bits sur 4) ; celle de MXFP4 est celle d'un code signe-grandeur E2M1 (les codes 0 à 7 sont positifs, 8 à 15 négatifs).

Données : `fondations/journaux/03-entropie/csv/fig-histogramme-codes.csv` (depuis `entropie-9B.json` et `entropie-gptoss20b.json`).

Table 6.2 : Taux d'un codage d'ordre 0, selon ce que le codeur voit [mes.]. La première ligne borne DEFLATE, GDeflate et les littéraux de zstd ; la deuxième est le « byte grouping » de ZipNN transposé aux champs des blocs K ; la troisième code des symboles entiers (5 et 6 bits pour Q5_K et Q6_K).

codage d'ordre 0	Q4_K	Q5_K	Q6_K	modèle 9B
octets, bloc entier, une table	1,0267	1,0031	1,0085	1,0183
octets, un flux par champ	1,0519	1,0179	1,0325	1,0416 [calc.]
symboles entiers par champ	1,0574	1,0470	1,0460	1,0529

Données : `fondations/journaux/03-entropie/flux-octets-9B.json`, `csv/flux-octets.csv`.

6.3 Pourquoi si peu : l'entropie d'un min-max

Les codes d'un K-quant sont presque uniformes parce que la quantification est faite pour cela : un min-max par groupe (ou sa version optimisée, qui écrête) étale les poids d'un groupe sur toute la grille. Combien d'entropie reste-t-il alors ? La question a une réponse exacte, que ce mémoire calcule par un chemin indépendant de la mesure (`memoire/outils/calculs_entropie.py`).

Définition 6.4 (Deux grilles de 4 bits)**.** Soit une source X de variance σ^2, quantifiée sur une plage d'étendue R. La *grille min-max* (à point zéro) a 16 niveaux $-R/2 + k\,R/15$, $k = 0, \dots, 15$, et code chaque poids par le niveau le plus proche, les deux niveaux extrêmes prenant les queues. La *grille symétrique* a 15 niveaux $k\,R/14$, $k = -7, \dots, 7$ (le maximum de $|X|$ vaut $R/2$, le niveau 0 au centre).

Proposition 6.5 (Entropie des codes à haute résolution)**.** *Si la densité p de X est assez régulière pour être presque constante sur chaque cellule de pas Δ, et si la plage couvre presque toute la masse, alors*

$$H(\mathit{codes}) = h(X) - \log_2 \Delta + o(1), \qquad h(X) = -\int p \log_2 p, \tag{6.3}$$

avec $h = \frac{1}{2}\log_2(2\pi e\,\sigma^2)$ *pour une gaussienne et* $h = \log_2(\sqrt{2}\,e\,\sigma)$ *pour une laplacienne.*

Démonstration. La cellule k a une probabilité $p_k = \int_{\text{cellule}} p \approx p(x_k)\,\Delta$. Alors

$$-\sum_k p_k \log_2 p_k \;\approx\; -\sum_k p(x_k)\,\Delta \log_2 p(x_k) \;-\; \log_2 \Delta \sum_k p(x_k)\,\Delta ;$$

la première somme est une somme de Riemann de $-\int p \log_2 p$, la seconde tend vers 1. Pour la gaussienne et pour la laplacienne de paramètre $b = \sigma/\sqrt{2}$,

$$-\mathbb{E}[\log_2 p(X)] = \tfrac{1}{2}\log_2(2\pi\sigma^2) + \frac{\mathbb{E}[X^2]}{2\sigma^2 \ln 2} = \tfrac{1}{2}\log_2(2\pi e\sigma^2), \qquad -\mathbb{E}[\log_2 p(X)] = \log_2(2b) + \frac{\mathbb{E}|X|}{b \ln 2} = \log_2(2eb).$$

■

Le calcul pour un min-max par groupes de 32. Pour $\sigma = 1$, $h = \frac{1}{2}\log_2(2\pi e) = 2{,}047$ bits. L'étendue moyenne de 32 tirages gaussiens vaut environ 4,1, donc $\Delta \approx 4{,}1/15 = 0{,}273$ et $H \approx 2{,}047 + 1{,}87 = 3{,}92$ bits [calc.]. Le calcul exact (Monte-Carlo à graine fixe, 2^{18} poids par point, figure 6.2) donne 3,865 ; l'approximation surestime de 0,05 bit parce que les deux poids extrêmes de chaque groupe tombent toujours sur les codes 0 et 15. **Les mesures de l'étude sur des poids réels** (min-max sur trois tenseurs, kurtosis 3,8 à 6,6) donnent 3,855 à 3,864 bits à $g = 32$ et 3,728 à 3,748 à $g = 64$, **contre** 3,865 **et** 3,753 **calculés pour une gaussienne** : l'accord est au centième. L'entropie de 3,86 bits n'est pas une propriété mystérieuse des réseaux : c'est ce que donne n'importe quelle source en cloche sous un min-max par petits groupes.

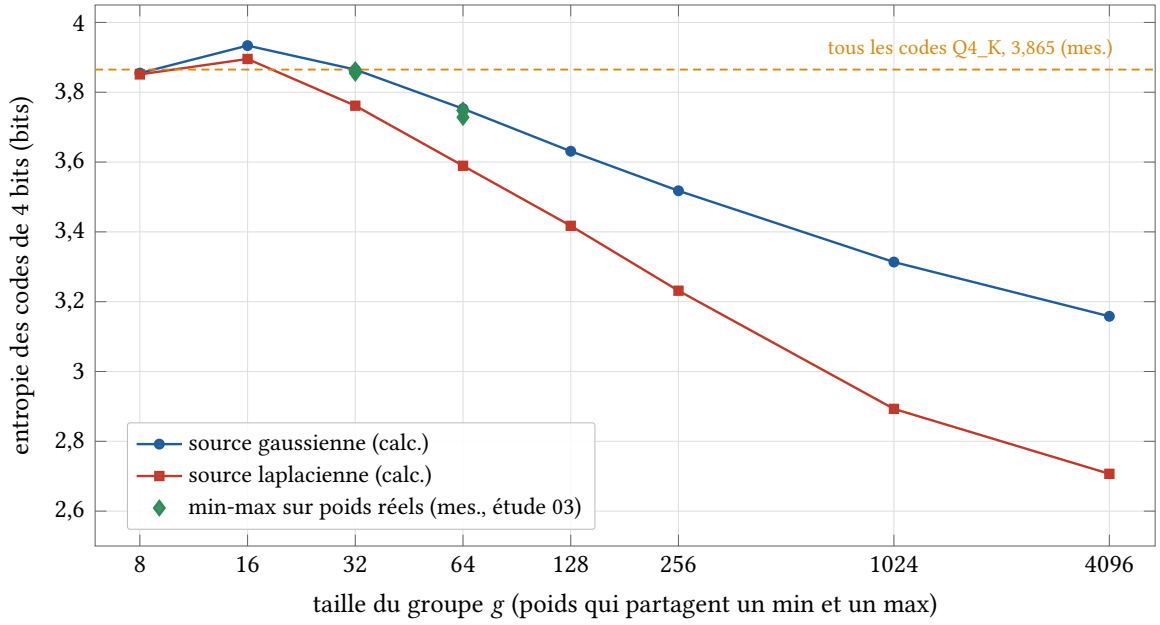


Figure 6.2 : Entropie des codes de 4 bits d'un min-max par groupes de g poids [calc.], pour une source gaussienne et une source laplacienne (kurtosis 6), et les min-max mesurés par l'étude sur des poids réels [mes.]. L'entropie ne descend que lentement avec la taille du groupe : encore 3,16 bits (gaussienne) et 2,71 (laplacienne) à $g = 4\,096$.

Données : `memoire/figures/donnees/entropie-groupes.csv` et `entropie-groupes-mesures.csv` (`calculs_entropie.py`, depuis `rd-proxy-4B.json` pour les mesures).

6.3.1 Une contradiction publiée, et une phrase corrigée

Un article de l'ISCA 2026 écrit que des poids int4 « ne gardent que 0,6 à 1,0 bit d'entropie réelle » [46] [litt.] ; une discussion autour de MLX avance environ 2,17 bits [100] [litt.], EntroLLM 1,39 bit pour un uint4 par tenseur [47] [litt.]. L'étude mesure 3,864 à 3,865 bits sur tous les codes Q4_K de deux modèles et 3,862 sur MXFP4.

Pour quelle plage tombe-t-on à 1 bit ? La figure 6.3 répond exactement. **Sur la grille min-max, jamais** : zéro y tombe sur la frontière des codes 7 et 8, la moitié d'une source symétrique part de chaque côté, et l'entropie décroît vers 1 bit sans l'atteindre (encore 1,01 bit à $R = 50\sigma$ pour une gaussienne [calc.]). **Sur la grille symétrique**, il faut $R = 33{,}9\sigma$ (gaussienne) ou $31{,}1\sigma$ (laplacienne) pour 1 bit, et 45σ pour 0,6 bit [calc.] : un maximum de $|X|$ à 17 à 22 écarts-types, ce que seule donne une échelle par tenseur ou par canal dominée par des valeurs aberrantes, le cas d'EntroLLM, payé en perplexité (10,11 contre 9,03 en fp16 sur phi3 [litt.]).

L'étude écrivait : « ~~Pour tomber à 1 bit, il faudrait que la plage d'un groupe dépasse environ dix fois l'écart-type de ses poids.~~ » Le calcul exact la contredit : à $R = 10\sigma$, l'entropie vaut encore 2,47 à 2,66 bits selon la grille et la loi ; 1 bit demande 31 à 34σ sur une grille symétrique et n'est jamais atteint sur une grille min-max [calc.] (`entropie-seuils.csv`). La conclusion de l'étude en sort renforcée, pas affaiblie : aucune de ces entropies publiées ne s'applique aux K-quants ni à MXFP4, et un gain annoncé de « 1,8 à 6 fois en plus de la quantification » est faux pour eux d'un facteur 30 à 100 sur le gain (0,24 bit au lieu de 2 à 3). Le chiffre publié n'est pas barré : il n'a pas été reproduit sur leurs poids.

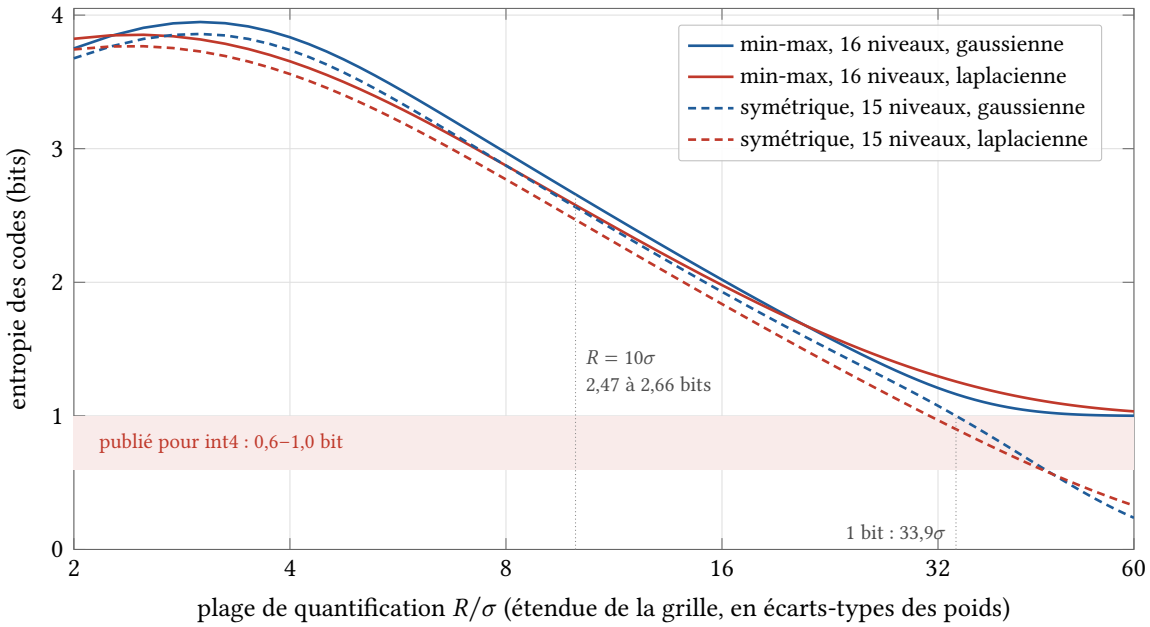


Figure 6.3 : Entropie exacte des codes d'une source gaussienne ou laplacienne de variance unité, selon l'étendue R de la grille de 4 bits [calc.]. Trait plein : grille min-max à 16 niveaux, bornée en dessous par 1 bit. Tirets : grille symétrique à 15 niveaux, qui passe sous 1 bit à $R \approx 31$ à 34σ. La bande rouge est l'intervalle publié pour int4 [46].

Données : `memoire/figures/donnees/entropie-plage.csv`, `entropie-seuils.csv` (`calculs_entropie.py`).

6.4 MXFP4 : le gain est dans les échelles

Les experts de gpt-oss sont en MXFP4 natif : blocs de 32 poids, codes E2M1 de 4 bits, une échelle E8M0 d'un octet par bloc, 4,25 bits par poids (figure 5.2). Ses codes portent 3,862 bits, comme ceux des K-quants. Mais **ses échelles portent l'essentiel du gain**, et c'est là que ce chapitre trouve ce que personne ne code (tableau 6.3).

Table 6.3 : Entropie des échelles E8M0 des 72 tenseurs d'experts de gpt-oss-20b, moyennes pondérées par le nombre de blocs [mes.]. Le contexte « médiane de ligne » est le meilleur et n'introduit aucune dépendance d'un bloc à l'autre. Sur gpt-oss-120b (36 premiers tenseurs) : 1,232 à l'ordre 0, 1,004 sachant le bloc de gauche, 0,910 sachant la médiane (plus 0,010 pour la médiane).

quantité	bits	lecture
$H(e)$, ordre 0	1,554 / 8	de 0,89 (`ffn_up`) à 2,28 (`ffn_down` de la couche 0)
$H(e \mid$ bloc de gauche$)$	1,152	
$H(e \mid$ bloc du dessus$)$	1,538	les lignes voisines ne se ressemblent presque pas
$H(e \mid$ gauche et dessus$)$	1,146	le dessus n'ajoute rien à la gauche
$H(e \mid$ médiane de sa ligne$)$	**1,011**	plus 0,015 pour la médiane ; sans dépendance entre blocs
$H(\text{code})$	3,862 / 4	presque uniforme
$H(\text{code} \mid e)$	3,823	0,039 bit d'information mutuelle code-échelle

Données : `fondations/journaux/03-entropie/mxfp4-contexte-gptoss20b.json`, `mxfp4-contexte-gptoss120b.json`.

Le témoin de l'estimateur : les échelles du premier tenseur, permutées au hasard, gardent $H(e) = 2{,}283$ et perdent tout le gain de contexte ($H(e \mid \text{gauche}) = 2{,}283$) [mes.]. L'estimateur n'invente pas de structure.

Pourquoi la médiane de ligne. Une ligne de matrice (un canal de sortie) a sa propre norme ; l'échelle d'un bloc la suit à ±1 près, parce qu'une échelle E8M0 n'est qu'un exposant, $\lceil \log_2 \max_{\text{bloc}} |w| \rceil$ à un décalage près. Connaître la médiane $\tilde{e}$ de la ligne dit presque tout, et la reconstruction ne lit aucun autre bloc (figure 6.4).

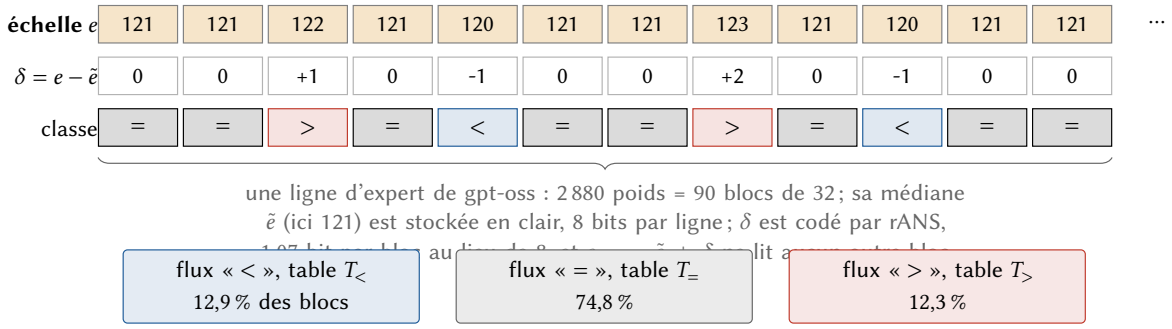


Figure 6.4 : Le codage des échelles E8M0 d'une ligne (valeurs d'exemple). La médiane de la ligne est stockée en clair, l'écart $\delta = e - \tilde{e}$ est codé par rANS. La classe du bloc (sous, à, au-dessus de la médiane) choisit la table du flux de codes qui le porte ; les proportions sont celles mesurées sur gpt-oss-20b [mes.].

Données : `fondations/journaux/03-entropie/proportions-classes-gptoss20b.txt`, `dispersion-classes-secteur.json`.

Proposition 6.6 (Reconstruction locale des échelles). *Avec le codage $e = \tilde{e} + \delta$, où $\tilde{e}$ est la médiane de la ligne stockée en clair et δ un symbole du flux rANS, chaque échelle se reconstruit à partir de son seul symbole et d'un octet par ligne. Le codage par différence avec le voisin de gauche, $e_j = e_{j-1} + \delta'_j$, impose une somme préfixe le long de la ligne et coûte* 0,126 *bit de plus par échelle sur gpt-oss-20b.*

Démonstration. La première affirmation est immédiate : e ne dépend que de $\tilde{e}$ et de δ. Pour la seconde, $e_j = e_0 + \sum_{i \le j} \delta'_i$: chaque échelle dépend de tous les symboles qui la précèdent sur la ligne, ce qui est la définition d'une somme préfixe. Le coût vient du tableau 6.3 : $H(e \mid \text{gauche}) - (H(e \mid \tilde{e}) + 0{,}015) = 1{,}152 - 1{,}026 = 0{,}126$ bit [calc.]. ■

Le bilan en bits par poids, pas à pas. Sur les 72 tenseurs de gpt-oss-20b (10,15 Go), avec un rANS à alphabet d'octet et une table par tenseur [mes.] (`taux-mxfp4-gptoss20b.json`) :

- codes : 3,866 bits par poids (pour une entropie de 3,862) ;
- écarts δ : $H(\delta) = 1{,}071$ bit par bloc, soit $1{,}071/32 = 0{,}0335$ bit par poids ;
- médianes : 8 bits par ligne de 2 880 poids, soit $8/2\,880 = 0{,}0028$ bit par poids ;
- deux tables de 384 octets par tenseur, comptées ;
- échelles, total réel : 0,037 bit par poids, au lieu de 0,25 ;
- **total :** 3,903 **bits par poids pour** 4,25 **stockés, taux** $4{,}25/3{,}903 = 1{,}0888$.

Les échelles rendent $0{,}25 - 0{,}037 = 0{,}213$ bit par poids, les codes $4 - 3{,}866 = 0{,}134$ [calc.] : **les échelles valent une fois et demie les codes**. Chacun des 72 tenseurs est décodé et comparé octet à octet, codes et échelles reconstruites : tous exacts ; le témoin (une médiane faussée) rend la reconstruction inexacte [mes.]. Sur gpt-oss-120b (108 tenseurs, 60,9 Go), le taux vaut 1,0887, à 0,0001 du 20b : il ne dépend pas de la taille du modèle.

Trois classes d'échelle. L'information mutuelle code-échelle (0,039 bit) s'encaisse sans changer le décodeur : on répartit les blocs en trois flux selon la place de leur échelle par rapport à la médiane (sous, à, au-dessus), chaque flux ayant sa table ; chaque groupe de travail de la carte ne charge que la table de son flux, et le réassemblage, guidé par δ que le décodeur a déjà, est vérifié octet à octet (tableau 6.4).

Table 6.4 : Taux sans perte réel des experts MXFP4, codes et échelles, décodage exact de chaque tenseur [mes.]. Trois classes prennent 0,053 bit, plus que l'information mutuelle mesurée (0,039) ; cinq classes n'ajoutent que 0,0008 bit.

modèle	variante	codes (b/p)	total (b/p)	taux	exact
gpt-oss-20b, 72 tenseurs	une table par tenseur	3,8662	3,9032	1,0888	oui
	trois classes d'échelle	**3,8135**	**3,8505**	**1,1037**	oui
	cinq classes ($\lvert\delta\rvert \le 2$)	3,8127	3,8498	1,1040	oui
gpt-oss-120b, 108 tenseurs	une table par tenseur	3,8695	3,9039	1,0887	oui
	trois classes d'échelle	**3,8201**	**3,8545**	**1,1026**	oui

Données : `fondations/journaux/03-entropie/taux-mxfp4-classes-gptoss{20b,120b}.json`, `taux-mxfp4-classes5-gptoss20b.json`.

La raison physique du gain des classes : une échelle au-dessus de la médiane de sa ligne signale un bloc dont un poids est grand ; ses autres codes sont alors petits, et leur loi se resserre vers zéro. La classe relative à la ligne en dit plus que la valeur absolue de l'échelle, et l'alphabet d'octet ajoute la

dépendance entre les deux codes d'un même octet, qui partagent leur bloc. La figure 6.5 montre que le gain est uniforme d'un tenseur à l'autre : de 1,0846 à 1,1041 avec une table, de 1,0987 à 1,1177 avec trois classes, sur les 72 tenseurs du 20b (valeurs extrêmes recalculées ici depuis le journal, égales à celles de l'étude [calc.]).

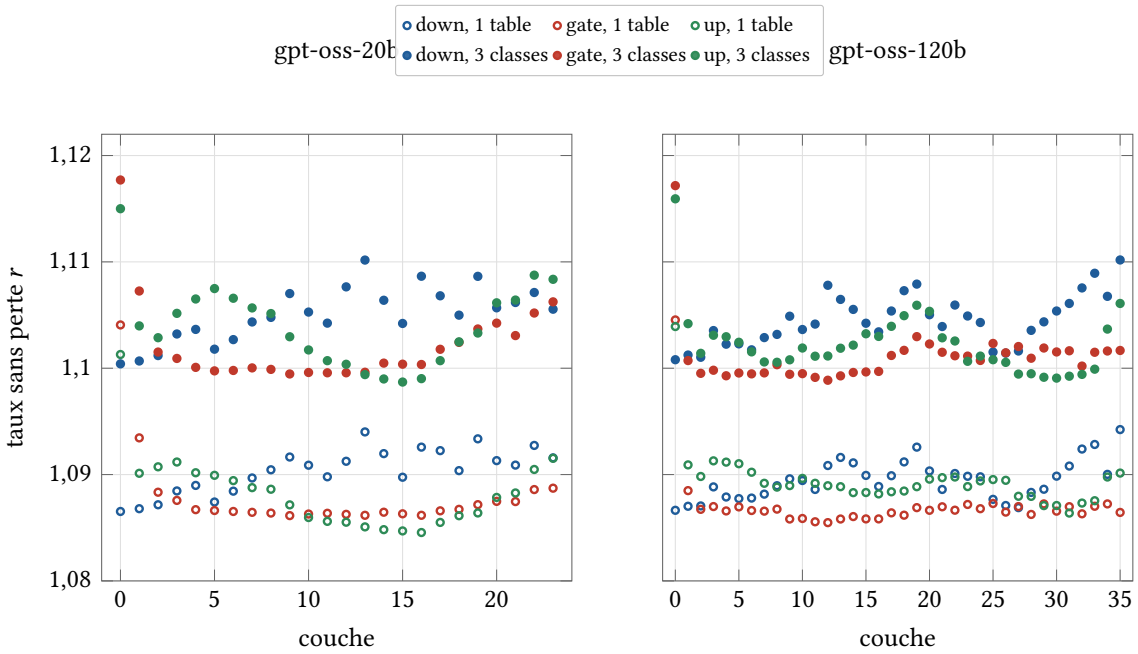


Figure 6.5 : Taux sans perte réel de chaque tenseur d'experts, une table par tenseur (cercles) et trois classes d'échelle (disques), par couche et par rôle [mes.]. Les tenseurs down, dont les échelles sont les plus désordonnées à l'ordre 0, gagnent le plus (1,1051 en moyenne à trois classes sur le 20b, contre 1,1022 pour `gate`) : la médiane de ligne absorbe ce désordre (figure 6.6).

Données : `memoire/figures/donnees/entropie-taux-{20b,120b}-{down,gate,up}.csv` (`donnees_figures.py`, depuis `taux-mxfp4-classes-gptoss{20b,120b}.json`).

Tout gpt-oss-20b, champ par champ. Sur les 11,5 Go de tenseurs quantifiés [mes.] (`entropie-gptoss20b-resume.txt`) : les experts MXFP4 (9 682 Mio) ont un idéal d'ordre 0 de 1,0869 (zstd 3 : 1,0405 ; lz4 : 1,000) ; l'attention en Q8_0 (645 Mio), 1,069 à 1,095 ; la tête de sortie, 1,0758 ; les plongements (`token_embd`, 587 Mio), **1,4371**. Les plongements se compressent de 44 %, mais un jeton n'en lit qu'une ligne : ce gain sert au disque et à la RAM, pas au débit. L'attention, toujours lue et résidente en VRAM dans le plan du chapitre 9, ne gagne rien en vitesse par le sans-perte (section 6.7). **Ce qui compte pour le débit, ce sont les experts lus par le disque** : 1,1026 réel sur le 120b.

6.5 Le codeur rANS entrelacé

Le codage par systèmes numériques asymétriques (ANS) de Duda [52] code un symbole de probabilité q en $\log_2(1/q)$ bits comme un codeur arithmétique, mais avec un état entier unique et des opérations entières ; sa variante par intervalles (rANS) se décode par une table. Giesen [53] a montré qu'on peut entrelacer plusieurs codeurs indépendants dans un seul flux. L'étude a écrit son propre format et ses propres décodeurs, sans reprendre une ligne des dépôts lus (`outils/03-entropie/rans/`).

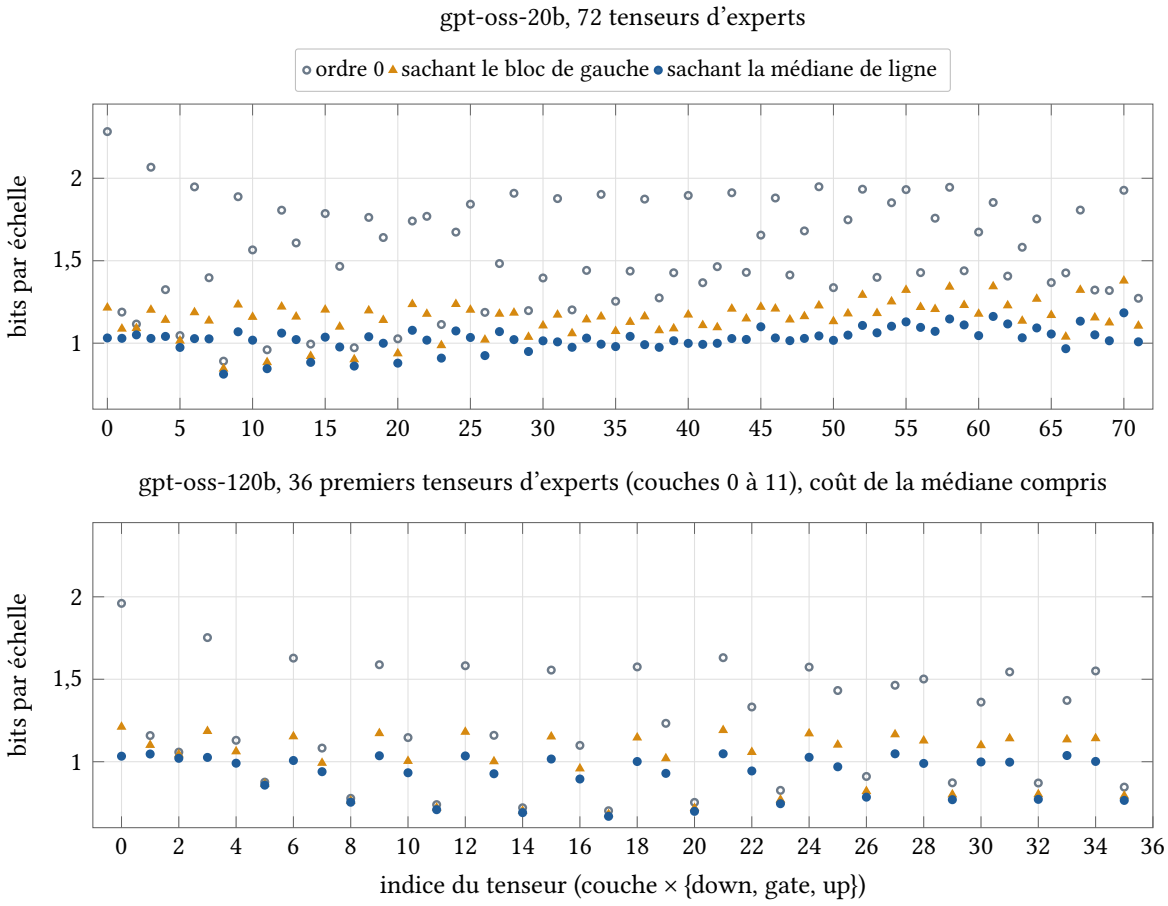


Figure 6.6 : Entropie des échelles E8M0 de chaque tenseur d'experts selon le contexte [mes.]. Sachant la médiane de la ligne, toutes les échelles tombent près de 1 bit, quel que soit leur désordre à l'ordre 0 ; les `ffn_down` (indices 0, 3, 6, …) sont les plus désordonnés à l'ordre 0 et les plus aidés par le contexte.

Données : `memoire/figures/donnees/entropie-echelles-{20b,120b}.csv` (depuis `mxfp4-contexte-gptoss{20b,120b}.json`).

6.5.1 Le format

Les probabilités sont quantifiées sur $M = 2^{12}$; l'état x tient sur 32 bits ; sa borne basse est $L = 2^{16}$ (dans ce chapitre, L désigne cette borne) ; la renormalisation se fait par mots de 16 bits. Un symbole s a une fréquence $f_s \geq 1$ et un cumul $c_s = \sum_{t<s} f_t$, avec $\sum_s f_s = M$.

Définition 6.7 (Pas de codage et de décodage)**.**

$$C(s,x) = \left\lfloor \frac{x}{f_s} \right\rfloor M + (x \bmod f_s) + c_s, \qquad D(x) = \big(s,\, f_s\lfloor x/M \rfloor + (x \bmod M) - c_s\big),$$

où, dans D, s est l'unique symbole tel que $c_s \leq x \bmod M < c_s + f_s$ (la *fente* $x \bmod M$ désigne le symbole).

Les figures 6.7 et 6.8 donnent la disposition au bit près : une tuile est un sous-groupe de 32 voies qui décode N macro-pas ; la voie ℓ du macro-pas m écrit le mot de 32 bits $32m + \ell$, de sorte que **les 32 voies écrivent 128 octets contigus à chaque macro-pas**. La table de décodage a 4 096 fentes de 32 bits ; la v4 (alphabet de 256 symboles, l'octet entier de deux demi-octets) range dans chaque entrée le symbole (8 bits), $f - 1$ (12 bits) et le biais = fente − cumul (12 bits). Le biais est $< f \leq 4\,096$ et tient donc sur 12 bits ; $f - 1$ aussi.

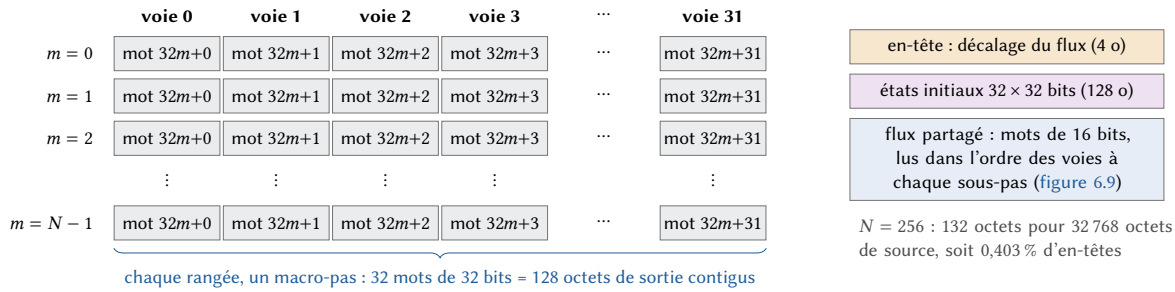


Figure 6.7 : La tuile du codeur : 32 voies × N macro-pas. Chaque macro-pas écrit 128 octets contigus (une écriture coalescée parfaite) ; la tuile porte un décalage de flux, 32 états initiaux et son flux partagé.

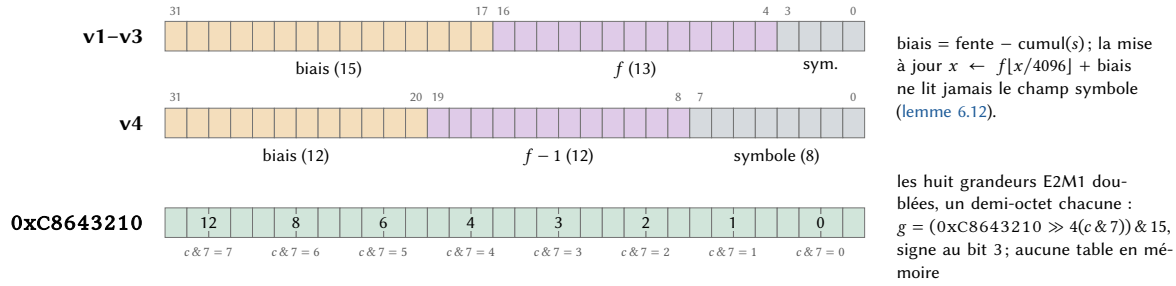


Figure 6.8 : L'entrée de 32 bits de la table de décodage (4 096 fentes, 16 Kio en mémoire partagée), pour l'alphabet de 16 symboles (v1 à v3) et pour l'alphabet d'octet (v4) ; en dessous, le mot de 32 bits qui porte les huit grandeurs E2M1 doublées et remplace toute table de déquantification MXFP4.

6.5.2 Exactitude

Lemme 6.8 (Inversion). *Pour tout symbole s et tout $x \geq 0$, $D(C(s,x)) = (s,x)$.*

Démonstration. Posons $q = \lfloor x/f_s \rfloor$ et $\rho = x \bmod f_s$, donc $0 \leq \rho < f_s$ et $x = qf_s + \rho$. Alors $y = C(s,x) = qM + \rho + c_s$ avec $0 \leq \rho + c_s < c_s + f_s \leq M$. Donc $\lfloor y/M \rfloor = q$ et $y \bmod M = \rho + c_s \in [c_s, c_s + f_s)$: la fente désigne bien s. Enfin $f_s\lfloor y/M \rfloor + (y \bmod M) - c_s = f_s q + \rho = x$. ∎

Lemme 6.9 (Intervalle, et une seule lecture par pas). *Si le décodeur maintient $x \in [L, 2^{32})$ et applique $x \leftarrow D(x)_2$, puis, si $x < L$, $x \leftarrow 2^{16}x + w$ avec w le mot suivant du flux, alors un seul mot suffit toujours, et l'état revient dans $[L, 2^{32})$.*

Démonstration. Après D, $x' = f_s\lfloor x/M \rfloor + (x \bmod M) - c_s \geq f_s\lfloor L/M \rfloor = 16 f_s \geq 16$. Si $x' < L$, alors $2^{16}x' + w \geq 2^{20} > L$ et $2^{16}x' + w \leq 2^{16}(L-1) + 2^{16} - 1 < 2^{32}$. Si $x' \geq L$, on a aussi $x' \leq x < 2^{32}$, car $x' \leq M\lfloor x/M \rfloor + (x \bmod M) = x$ dès que $f_s \leq M$. ∎

La boucle de la carte n'a donc qu'une branche de lecture, remplacée par un `select` : aucune divergence de voies.

Lemme 6.10 (Le codeur, symétrique). *Le codeur maintient $x \in [L, 2^{32})$, part de $x = L$ et, pour coder s : si $x \geq f_s 2^{20}$, il émet $x \bmod 2^{16}$ et pose $x \leftarrow \lfloor x/2^{16} \rfloor$; puis $x \leftarrow C(s,x)$. Une émission suffit toujours, et $C(s,x) \in [L, 2^{32})$.*

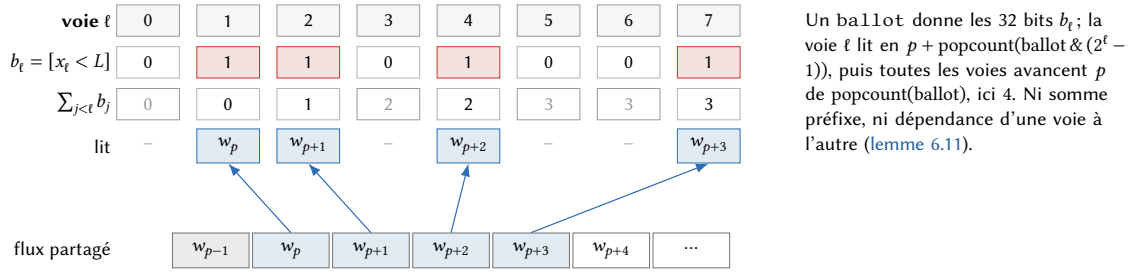
Démonstration. Montrons qu'avant C, $x \in [16 f_s, 2^{20} f_s)$. Sans émission, $x < 2^{20} f_s$ par la condition, et $x \geq L = 16M \geq 16 f_s$. Avec émission, $x \geq 2^{20} f_s$ donne $\lfloor x/2^{16} \rfloor \geq 16 f_s$, et $x < 2^{32}$ donne $\lfloor x/2^{16} \rfloor < 2^{16} \leq 2^{20} f_s$. Ensuite, avec $q = \lfloor x/f_s \rfloor \in [16, 2^{20} - 1]$: $C(s,x) = qM + \rho + c_s \geq 16M = L$ et $C(s,x) \leq (2^{20} - 1)M + (M - 1) = 2^{32} - 1$. ∎

La condition d'émission avant C et la condition de lecture après D sont les deux faces d'une même transition : par le lemme 6.8, l'état que lit le décodeur juste après D est l'état du codeur juste avant C, dans $[16 f_s, 2^{20} f_s)$; il est sous L exactement quand le codeur avait émis.

Lemme 6.11 (Flux partagé entre 32 voies)**.** *Au sous-pas k, soit $b_\ell \in \{0,1\}$ l'indicateur « la voie ℓ renormalise ». Si le décodeur donne à la voie ℓ la position $p + \sum_{j<\ell} b_j$ puis avance p de $\sum_j b_j$, et si le codeur parcourt les symboles dans l'ordre exactement inverse (macro-pas, sous-pas et voies décroissants), en émettant avant de coder, puis renverse sa pile, alors chaque voie lit exactement le mot que son codeur a émis.*

Démonstration. Le codage d'une voie est une pile : dernier émis, premier lu. Les voies étant entrelacées, la pile globale reçoit, pour le sous-pas k parcouru à rebours, les mots des voies $31, 30, \dots, 0$ qui émettent. Renversée, elle présente pour le sous-pas k, dans l'ordre direct, les mots des voies $0, 1, \dots, 31$; et chaque voie qui émettait au codage est exactement une voie qui lit au décodage (lemme 6.8 voie par voie, et lemme 6.10). La position de la voie ℓ est donc le nombre de mots lus avant elle dans ce sous-pas, $\sum_{j<\ell} b_j$. ∎

Sur la carte, $\sum_{j<\ell} b_j = \mathrm{popcount}(\mathrm{ballot}(b) \,\&\, (2^\ell - 1))$: un vote du sous-groupe et un comptage de bits, au lieu d'une somme préfixe (figure 6.9). Ce principe d'un flux unique lu dans l'ordre des voies est celui que Microsoft a posé en 2022 pour GDeflate [5] ; DietGPU le fait aussi par `__ballot_sync` et `__popc` [57]. La v2 du décodeur de l'étude l'a retrouvé indépendamment.



Un `ballot` donne les 32 bits b_ℓ ; la voie ℓ lit en $p + \mathrm{popcount}(\mathrm{ballot} \,\&\, (2^\ell - 1))$, puis toutes les voies avancent p de popcount(ballot), ici 4. Ni somme préfixe, ni dépendance d'une voie à l'autre (lemme 6.11).

Figure 6.9 : Un sous-pas du flux partagé, 8 voies montrées sur 32. Les voies dont l'état est passé sous L lisent des mots consécutifs, dans l'ordre croissant des voies.

Lemme 6.12 (Ce que le contrôle d'état garde, et ce qu'il ne garde pas)**.** *Chaque voie part de L au codage, donc le décodeur exact finit sur L : c'est un contrôle gratuit. Mais la mise à jour de l'état ne dépend que de f et du biais lus à la fente, et des mots du flux ; elle ne dépend pas du champ symbole de l'entrée. Une corruption confinée au champ symbole change la sortie sans changer aucun état : le contrôle $x_{\text{fin}} = L$ ne la voit pas.*

Démonstration. Dans D, la mise à jour s'écrit $x' = f\lfloor x/M \rfloor + \text{biais}$, avec $\text{biais} = (x \bmod M) - c_s$: deux champs de l'entrée, aucun n'est le symbole. ∎

C'est exactement ce qu'a montré le témoin cassé : un bit retourné dans la table en mémoire partagée fait tomber les v1 à v3 deux fois (sortie fausse, et 129 278 à 129 280 voies sur 129 280 hors de L), mais la v4 seulement sur la sortie, **toutes ses voies finissant sur** L [mes.] : la corruption touchait le champ symbole. En production, il faut une empreinte de la table ou de la sortie, en plus du contrôle d'état ; le plan de l'architecture A compare chaque tronçon décodé à son empreinte (chapitre 10).

Proposition 6.13 (Longueur réelle du code)**.** *Pour une source sans mémoire de loi p codée avec la table quantifiée $q_s = f_s/M$, la longueur moyenne par symbole tend vers $H(p) + D(p \,\|\, q)$, plus les en-têtes de tuile et les fins de flux.*

Démonstration. Le pas C multiplie l'état par M/f_s à un arrondi près ; sur n symboles, l'état a crû d'un facteur $\prod_i M/f_{s_i}$, et chaque mot de 16 bits émis divise l'état par 2^{16}, l'état restant dans $[L, 2^{32})$. Le nombre de bits émis vaut donc $\sum_i \log_2(M/f_{s_i}) + O(1)$ par voie, dont l'espérance par symbole est $\sum_s p_s \log_2(1/q_s) = H(p) + D(p \,\|\, q)$. Les en-têtes et les états finaux s'ajoutent. ■

Les chiffres se tiennent. Les en-têtes coûtent $4+32\times4 = 132$ octets par tuile de $32\times4\times256 = 32\,768$ octets de source, soit 0,403 % [calc.]. Sur tous les demi-octets Q4_K du 4B (1 209 794 560 octets) [mes.] : $H(p) = 3{,}8652$ bits par code, la table de 12 bits rend 3,8652 (donc $D(p \,\|\, q) < 10^{-4}$), le codage réel 3,8774, soit un taux de 1,0316 sur les demi-octets et de 1,0280 sur le Q4_K entier, métadonnées brutes. L'écart à l'entropie, 0,0122 bit par code, vaut $0{,}0122 \times 65\,536/8 = 100$ octets par tuile, moins que les 132 octets d'en-tête et d'états ($132 \times 8/65\,536 = 0{,}0161$ bit par code, une tuile portant 65 536 codes) [calc.]. La différence n'est pas une erreur : les 32 états stockés portent eux-mêmes une part de la charge. L'état final du codeur, qui sert d'état initial au décodeur, est un entier de $[2^{16}, 2^{32})$, porteur de jusqu'à 16 bits d'information au-dessus de L par voie ; le compte dit qu'ils en portent 32 octets par tuile, 8 bits par voie en moyenne [calc.]. (L'étude attribuait l'écart à « l'arrondi des fins de tuiles », ce qui en est la même cause, dite autrement.)

6.5.3 Le cœur de la v4

Le listing 6.1 montre la boucle intérieure du décodeur v4, telle qu'elle est commitée, sans autre changement que des commentaires. Un pas coûte une lecture en mémoire partagée, une multiplication, un vote, deux comptages de bits, deux échanges et quelques masques : environ 25 opérations entières pour un octet produit.

Listing 6.1 : Boucle intérieure du décodeur rANS v4, extraite telle quelle de `outils/03-entropie/rans/src/decodeur.wgsl` (fonction `decoder_v4`) ; seuls les commentaires de fin de ligne sont ajoutés.

```
// Extrait de outils/03-entropie/rans/src/decodeur.wgsl (fonction decoder_v4), boucle intérieure,
// code inchangé ; seuls les commentaires de fin de ligne sont ajoutés ici.
for (var m = 0u; m < t.y; m++) {                // un macro-pas
    var mot = 0u;
    for (var k = 0u; k < 4u; k++) {             // 4 sous-pas, un octet chacun
        let e = lut[x & 4095u];                 // fente = x mod M
        x = (((e >> 8u) & 0xFFFu) + 1u) * (x >> 12u) + (e >> 20u);
        mot |= (e & 0xFFu) << (8u * k);         // symbole : hors de l'état
        let besoin = x < 65536u;                // x < L : lire un mot
        let vote = subgroupBallot(besoin).x;
        let pos = ptr + countOneBits(vote & masque_inf);
        let wi = (pos >> 1u) - base;
        let a = subgroupShuffle(cour, wi & 31u);  // fenêtre en registres
        let b = subgroupShuffle(suiv, wi & 31u);
        let w32 = select(a, b, wi >= 32u);
        let w = (w32 >> ((pos & 1u) * 16u)) & 0xFFFFu;
        x = select(x, (x << 16u) | w, besoin);  // une seule lecture
        ptr += countOneBits(vote);
        if ((ptr >> 1u) - base >= 32u) {        // fenêtre suivante
            base += 32u;
            cour = suiv;
            suiv = mots[base + 32u + voie];
        }
    }
    sortie[dest] = mot;                         // 128 octets contigus
```

```
    dest += 32u;
}
```

6.6 Quatre décodeurs, et ce que chacun a appris

v1 Somme préfixe du sous-groupe, puis lecture du mot en mémoire globale au milieu de la chaîne de dépendance. Limitée par la latence : à chaque sous-pas, au moins une voie sur 32 renormalise (la probabilité qu'aucune ne le fasse vaut environ $0{,}76^{32} \approx 1{,}5 \cdot 10^{-4}$), donc chaque sous-pas attend une lecture globale de plusieurs centaines de cycles.
v2 Un vote et un comptage de bits remplacent la somme préfixe ; le flux se lit par fenêtres de 32 mots de 32 bits chargées d'avance, une par voie, distribuées par échange. Un sous-pas consomme au plus 32 mots de 16 bits, donc la fenêtre suivante a toujours été demandée au moins un sous-pas avant d'être lue : la latence est cachée. Environ deux fois la v1.
v3 La v2 avec une table compacte de 4 Kio, pour loger plus de groupes de travail par multiprocesseur. **Aucun gain** (42,1 contre 42,1 Go/s sur secteur) : l'occupation n'était pas le plafond. Idée morte, tuée par le nombre.
v4 L'alphabet d'octet : deux fois moins de pas par octet produit, pour la même chaîne par pas. 1,67 fois la v2 sur secteur (70,2 contre 42,1 Go/s), au même taux (1,0316).

Deux références bornent ces débits : la *recopie* de même géométrie de lecture et d'écriture, sans décodage (c'est le chemin non compressé), et une recopie par `vec4` à pas de grille, mesure de la mémoire. Toutes les passes sont vérifiées : chaque décodeur rend exactement les octets d'origine, comparés mot à mot sur tout le lot, et aucune voie ne finit hors de L [mes.].

Le second chemin : le même décodeur en CUDA. Pour que le débit ne repose pas sur un seul outil, l'algorithme a été réécrit en CUDA (`__ballot_sync`, `__popc`, `__shfl_sync`), compilé par nvcc 13.1 pour sm_89 et nourri du *même flux* écrit sur disque (256 Mio de demi-octets Q4_K, 8 192 tuiles). La sortie CUDA égale la source octet pour octet ; le témoin cassé la fait tomber [mes.]. Deux décodeurs écrits dans deux langages et compilés par deux chaînes rendent les mêmes octets depuis le même flux : le format est confirmé indépendamment du code qui l'a écrit.

Table 6.5 : Le décodeur v4 sous CUDA (A) et sous Vulkan (B), passes dans l'ordre A B B A, toutes sur secteur, médianes [mes.].

passe	décodeur v4 (Go/s écrits)	recopie de même géométrie
A1, CUDA	80,8	112,8
B1, Vulkan	70,2	107,5
B2, Vulkan	71,6	110,4
A2, CUDA	82,1	115,7

Données : `fondations/journaux/03-entropie/cuda-4B-A{1,2}-secteur.json`, `rans-4B-B{1,2}-secteur.json`.

Le compilateur de NVIDIA tire $(80{,}8 + 82{,}1)/(70{,}2 + 71{,}6) = 1{,}149$ fois plus du même décodeur [calc.] ; la recopie ne diffère que de 4 %. L'écart est dans le code généré pour la boucle (naga puis le compilateur du pilote Vulkan, contre nvcc), pas dans la mémoire. C'est l'une des raisons chiffrées du choix de CUDA pour le jalon J1 (chapitre 10).

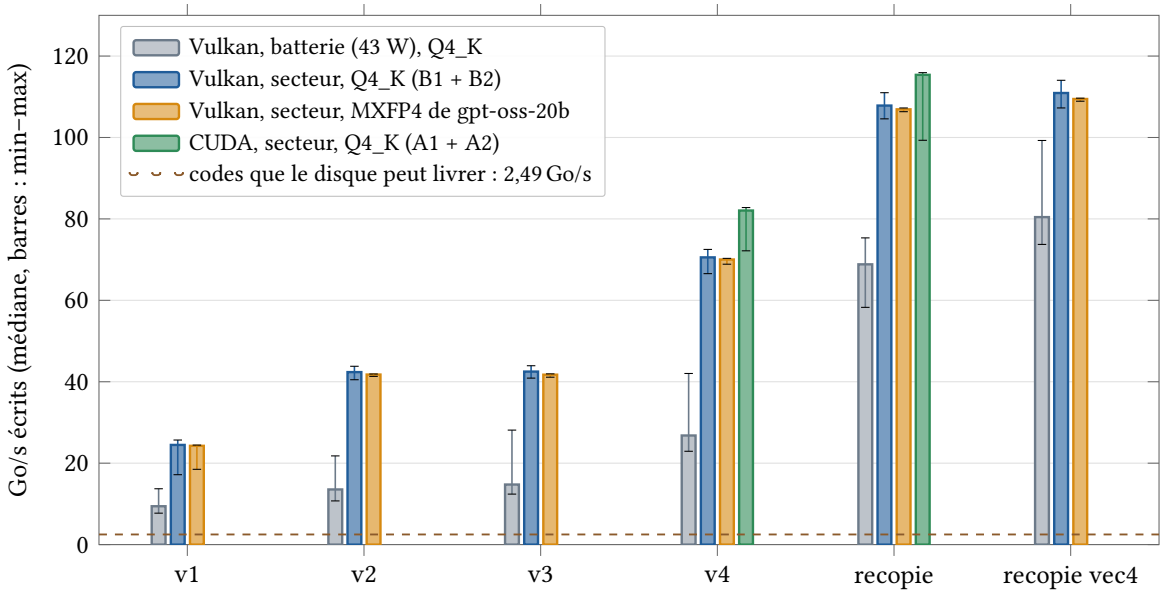


Figure 6.10 : Débit écrit des quatre décodeurs et des deux recopies de référence, médiane et étendue de chaque série [mes.]. Sur secteur, la v4 produit 70 à 72 Go/s par Vulkan, le même débit sur les codes MXFP4 que sur les demi-octets Q4_K (le débit ne dépend presque pas de la loi des symboles), et 82 Go/s par CUDA. Sur batterie (carte plafonnée à 43 W), elle perd un facteur 2,6. La ligne du bas est ce que le disque peut livrer.

Données : `memoire/figures/donnees/entropie-debits-{batterie,secteur,mxfp4,cuda}.csv` (`donnees_figures.py`, depuis `csv/debits-carte-series.csv` et `csv/debits-cuda-series.csv`).

Ce que le décodeur coûte en calcul. La v4 lit environ 63 Go/s de flux et écrit 65 Go/s à l'essai : ses 128 Go/s de trafic restent sous les 189 Go/s (lus et écrits) de la recopie, **elle n'est pas limitée par la mémoire**. La signature batterie dit qu'elle l'est par l'horloge des multiprocesseurs : sous le plafond de puissance, le décodeur perd 59 % et la recopie 27 % [mes.]. Reste à savoir si c'est le débit arithmétique ou la chaîne de dépendance de chaque voie. L'étude écrivait : « ~~À 130 milliards de pas par seconde, cela fait environ 3,3 billions d'opérations par voie et par seconde [...] Le décodeur est donc à environ la moitié du plafond arithmétique.~~ » Or la v4 fait un pas par *octet* : 65 Go/s écrits sont 65 milliards de pas par seconde (130 milliards de *codes*), soit, à environ 25 opérations par pas, $1{,}6 \cdot 10^{12}$ opérations par seconde [calc.], pour une carte qui en offre de l'ordre de $24 \times 128 \times (2{,}0 \text{ à } 2{,}5)\,\text{GHz} = 6 \text{ à } 7{,}7 \cdot 10^{12}$ (ordre de grandeur, non mesuré). Le décodeur est donc au quart environ du plafond arithmétique, non à la moitié : ce qui le borne est plus probablement la latence de la chaîne d'un pas (lecture de table, multiplication, vote, échange, dans cet ordre) que le débit des unités. La conséquence pour la suite est favorable : il peut rester plus qu'un facteur 2 à gagner, par plus d'états indépendants par voie (deux flux entrelacés dans la même voie recouvrent leurs chaînes). La proposition 6.14 n'en dépend pas : elle compte des opérations par octet, que le pas soit borné par le débit ou par la latence.

Le processeur ne suit pas. Le décodeur rANS scalaire coûte environ 16 cycles par code (la chaîne du pointeur partagé est sérielle) ; il fait 0,075 Go/s par fil sur un micro-banc [mes.]. Même en AVX-512, à environ 0,7 Go/s par cœur comme l'annonce Recoil [55] [litt.], six cœurs performants feraient 4 Go/s, à peine plus que le disque, quand la carte en offre vingt-cinq fois plus. **Le décodage appartient à la carte** ; le processeur lit le disque et remplit les tampons de transfert.

6.7 La question clé : décoder dans le produit, ou juste avant ?

Proposition 6.14 (Régime résident : quand fondre paierait). *Les poids résident en VRAM, de débit W_{VRAM}. Un produit fondu au décodeur doit, par octet de poids, lire $1/r$ octet de flux et exécuter $a_{\text{dec}} + a_{\text{mv}}$ opérations ; le chemin brut lit 1 octet et exécute a_{mv} opérations. Avec A le débit arithmétique de la carte,*

$$\frac{t_{\text{fondu}}}{t_{\text{brut}}} = \frac{\max\big(1/(r\,W_{\text{VRAM}}),\,(a_{\text{dec}} + a_{\text{mv}})/A\big)}{\max\big(1/W_{\text{VRAM}},\,a_{\text{mv}}/A\big)}, \tag{6.4}$$

et le fondu ne gagne que si $(a_{\text{dec}} + a_{\text{mv}})/A < 1/W_{\text{VRAM}}$, c'est-à-dire si la carte offre au moins $a_{\text{dec}} + a_{\text{mv}}$ opérations par octet de VRAM.

Démonstration. Chaque chemin est borné par le plus lent de ses deux ressources (théorème 3.2 appliqué à la mémoire et au calcul) ; le rapport s'en déduit. Si $(a_{\text{dec}} + a_{\text{mv}})/A \geq 1/W_{\text{VRAM}}$, le numérateur vaut au moins $1/W_{\text{VRAM}}$ et le dénominateur, si le brut est limité par la mémoire, vaut exactement $1/W_{\text{VRAM}}$: le rapport est ≥ 1. ■

Le calcul. Un pas v4 coûte environ 25 opérations et produit un octet : $a_{\text{dec}} \approx 25$. La carte offre $A/W_{\text{VRAM}} \approx 6$ à $7{,}7 \cdot 10^{12}/251 \cdot 10^{9} \approx 24$ à 31 opérations par octet [calc.]. Le décodage seul mange déjà presque tout le budget avant la moindre multiplication : le fondu sera limité par le calcul, à un débit proche de celui du décodeur seul, quand le brut l'est par la mémoire. La prédiction : un rapport d'environ $446/130 = 3{,}4$ (poids Q4_K livrables par la VRAM contre codes produits par le décodeur, en milliards par seconde [calc.]).

La mesure. Un banc écrit pour cela (`outils/03-entropie/rans/src/bin/banc_gemv.rs`) fait un produit matrice-vecteur de lot 1 sur des **poids MXFP4 réels de gpt-oss-20b** (trois tenseurs d'experts, 256 Mio de codes, 131 072 lignes de 4 096 poids), par quatre chemins qui partagent exactement la même déquantification (le mot `0xC8643210` de la figure 6.8) : *brut* (codes bruts et échelles) ; *fondu* (décodeur v4, chaque mot décodé consommé dans les registres) ; *décodage seul* ; *deux passes* (décodage puis brut dans la même soumission).

Lemme 6.15 (Une vérification exacte d'un produit flottant). *Si chaque terme $w_i x_i$ d'une somme de n termes est un entier de valeur absolue au plus a, et si $n\,a < 2^{24}$, alors la somme calculée en f32 est exacte, dans n'importe quel ordre.*

Démonstration. Toute somme partielle est un entier de valeur absolue au plus $na < 2^{24}$. Tout entier de valeur absolue inférieure à 2^{24} est représentable exactement en f32 (24 bits de significande) ; l'addition de deux tels nombres dont la somme reste sous 2^{24} est donc exacte, et aucun arrondi ne se produit à aucune étape, quel que soit l'ordre. ■

Avec des échelles toutes à 2^0 et des x entiers dans $[-2, 2]$, chaque terme vaut au plus $12 \times 2 = 24$ (grandeur doublée maximale de E2M1 fois $|x|$), et $4\,096 \times 24 = 98\,304 < 2^{17}$: la carte doit égaler la référence du processeur *au bit*. Résultat [mes.] : **0 ligne fausse sur 131 072** pour le brut, le fondu et les deux passes. Les témoins : une seule fente de la table corrompue rend 38 329 lignes fausses, toutes les fentes 130 564. Une tolérance flottante aurait laissé passer la fente unique. Sur les vraies entrées, l'écart relatif au maximum de $|y|$ vaut $9{,}5 \cdot 10^{-8}$ (brut) et $1{,}3 \cdot 10^{-7}$ (fondu) contre une référence en f64.

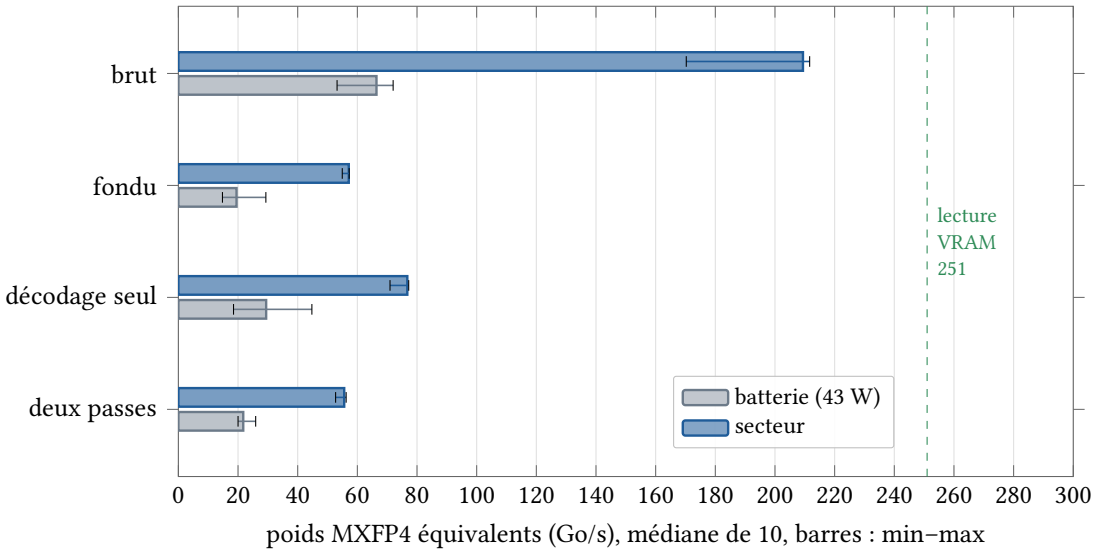


Figure 6.11 : Produit matrice-vecteur de lot 1 sur des poids MXFP4 réels de gpt-oss-20b, quatre chemins, en Go/s de poids MXFP4 équivalents [mes.]. Le brut atteint 83 % de la lecture de la VRAM ; fondu au décodeur, il est 3,7 fois plus lent, et pas plus rapide que deux passes.

Données : `memoire/figures/donnees/entropie-gemv-{secteur,batterie}.csv` (depuis `gemv-gptoss20b-{secteur,calme}.json`).

Table 6.6 : La question clé mesurée, cinq séries palindromiques, carte seule, sur secteur [mes.] ; en dernière colonne, la même mesure sur batterie.

chemin	temps médian	G poids/s	Go/s équiv.	rapport au brut	batterie (Go/s)
brut	1,36 ms	394,1	**209,4**	1	66,4
fondu	4,99 ms	107,5	**57,1**	**0,27**	19,5
décodage seul	3,72 ms	144,5	76,7	—	29,4
deux passes	5,13 ms	104,6	**55,6**	**0,27**	21,7

Données : `fondations/journaux/03-entropie/gemv-gptoss20b-secteur.json`, `gemv-gptoss20b-calme.json`.

Lecture. (1) Le brut fait 394 milliards de poids par seconde pour un plafond de 472 : 83 % de la VRAM, limité par la mémoire, et c'est le produit 4 bits le plus rapide mesuré sur la machine (l'étude 01 avait 122 Go/s en Q4_0) ; sa recette tient en une ligne : un sous-groupe traite 4 lignes, chaque voie charge les 32 x d'un bloc une fois pour les 4, et le mot `0xC8643210` remplace toute table. (2) **Le fondu est 3,7 fois plus lent que le brut** ; le calcul prédisait 3,4. (3) **Fondre ne gagne rien sur deux passes** : 57,1 contre 55,6 Go/s, dans la dispersion. Les temps s'additionnent : 3,72 + 1,36 = 5,08 ms pour 4,99 (fondu) et 5,13 (deux passes) mesurés ; la fusion n'économise que l'aller-retour des codes en VRAM (environ 2 ms de trafic), que le décodeur, limité par le calcul, recouvre déjà. (4) Sur batterie, les rapports restent à 0,27–0,33.

Réponse à la question clé. Décoder *dans* le noyau n'apporte rien sur cette carte, et décoder *juste avant* coûte autant : dans les deux cas, 3,7 fois le temps du produit brut quand les poids résident en VRAM. Le décodage entropique ne paie que là où il soulage un étage plus lent que lui, **le disque et le lien** (figure 6.12). Dans ce schéma, décoder un jeton qui lit tout par le disque coûte 2,49/70 ≈ 3,6 % du temps de la carte sur secteur (9 % sur batterie) [calc.], où 2,49 Go/s est le volume de codes que le disque

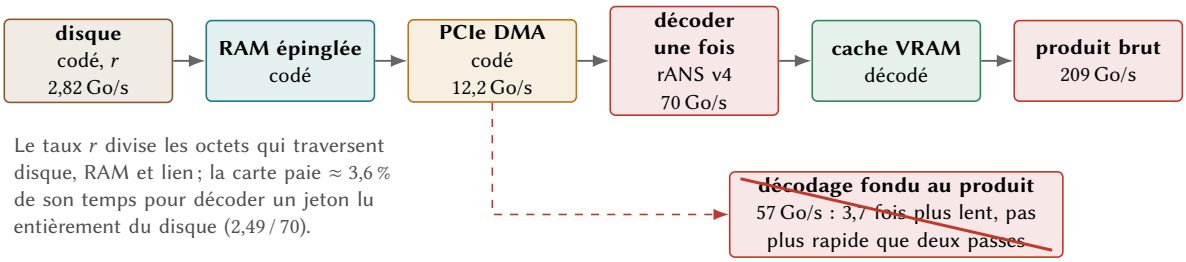


Figure 6.12 : La réponse à la question clé. Le flux codé traverse le disque, la RAM et le lien ; il est décodé une fois, à l'arrivée en VRAM, avant d'entrer dans le cache de la carte, où il sert ensuite au débit du produit brut. Le chemin barré, décoder dans le produit, est 3,7 fois plus lent [mes.].

livre à 2,8 Go/s de blocs Q4_K (2,8 × 256/144/2) ; et il rend au disque tout le taux r (corollaire 6.2). Le réassemblage des trois flux de classe, mesuré à part, écrit à 102,4 Go/s contre 113,5 pour une recopie contiguë, sortie exacte et témoin qui tombe [mes.] ; décodage puis réassemblage font ensemble 41,6 Go/s, encore 17 fois les 2,49 Go/s de codes que livre le disque [calc.].

La seule conclusion qui dépende de la qualité du noyau fondu tient d'elle-même : aucune écriture du fondu ne peut descendre sous le temps du décodeur seul, qui vaut déjà $3{,}72/1{,}36 = 2{,}7$ fois celui du brut ; avec le décodeur CUDA, 15 % plus rapide, le rapport resterait au-dessus de 2,3 [calc.]. Il faudrait un décodeur deux à trois fois plus rapide pour renverser la conclusion, ce que ni la mesure ni la littérature ne donnent sur une carte de cette classe. vram-compression [51], qui fond un rANS dans un produit de lot 1 sur une RTX 3050 Laptop, rapporte d'ailleurs un noyau « 2 à 3 fois plus lent que bf16 cuBLAS » [litt.] : la même conclusion, par un autre chemin.

6.8 Au-delà du sans-perte : une quantification taillée pour le codeur

Pourquoi les K-quants laissent-ils si peu ? Parce que leur recherche d'échelle et de minimum écrête pour remplir les 16 niveaux : des codes presque uniformes sont l'optimum d'un code *à débit fixe*. La théorie classique dit ce que coûte le débit fixe.

Théorème 6.16 (Gish et Pierce [101], haute résolution)**.** *Une quantification scalaire uniforme de pas Δ suivie d'un codage entropique idéal (ECSQ) atteint, quand $\Delta \to 0$,*

$$D = \frac{\Delta^2}{12}, \qquad R = H(Q) = h(X) - \log_2 \Delta + o(1), \qquad \textit{donc} \qquad R_{\mathrm{ECSQ}}(D) = h(X) - \tfrac{1}{2}\log_2(12D). \quad (6.5)$$

Démonstration (haute résolution). Le débit vient de la proposition 6.5. Pour la distorsion : sur une cellule où la densité est presque constante, l'erreur $X - Q(X)$ est presque uniforme sur $[-\Delta/2, \Delta/2]$, de variance $\int_{-\Delta/2}^{\Delta/2} u^2 \, \mathrm{d}u/\Delta = \Delta^2/12$; la moyenne sur les cellules garde cette valeur. On élimine Δ : $\log_2 \Delta = \frac{1}{2}\log_2(12D)$. ∎

Corollaire 6.17 (L'écart de l'ECSQ à la borne de Shannon)**.** *La borne inférieure de Shannon pour l'erreur quadratique est $R(D) \geq h(X) - \frac{1}{2}\log_2(2\pi e D)$ [91]. L'ECSQ en est à*

$$R_{\mathrm{ECSQ}}(D) - R_{\mathrm{SLB}}(D) = \tfrac{1}{2}\log_2 \frac{2\pi e}{12} = \tfrac{1}{2}\log_2 \frac{\pi e}{6} = \tfrac{1}{2}\log_2 1{,}4233 = 0{,}2546 \textit{ bit, soit } 1{,}53 \textit{ dB.}$$

Ce que coûte le débit fixe, pour une gaussienne. La meilleure quantification scalaire à 2^R niveaux atteint à haute résolution $D \approx \frac{1}{12}\big(\int p^{1/3}\big)^3 2^{-2R}$ (Panter et Dite [94]). Pour $\mathcal{N}(0,\sigma^2)$:

$$\int p^{1/3} = (2\pi\sigma^2)^{-1/6}\int e^{-x^2/(6\sigma^2)}\,\mathrm{d}x = (2\pi\sigma^2)^{-1/6}\sqrt{6\pi}\,\sigma,$$
$$\Big(\int p^{1/3}\Big)^3 = (2\pi)^{-1/2}\sigma^{-1}(6\pi)^{3/2}\sigma^3 = 6\sqrt{3}\,\pi\,\sigma^2,$$

d'où $D_{\mathrm{PD}} = \frac{\sqrt{3}\,\pi}{2}\,\sigma^2 2^{-2R}$. Comparée à la borne exacte $D(R) = \sigma^2 2^{-2R}$ (théorème 5.1), la perte en débit vaut $\frac{1}{2}\log_2(\sqrt{3}\pi/2) = \frac{1}{2}\log_2 2{,}7207 = 0{,}722$ bit ; l'ECSQ ne perd que 0,2546 (corollaire 6.17, la gaussienne atteignant la borne de Shannon). **Différence :** $0{,}722 - 0{,}255 = 0{,}467$ **bit par poids** [calc.], ce que gagne en théorie, à haut débit, un codage entropique sur le meilleur débit fixe.

La mesure, sur des substituts. Aucun poids BF16 n'étant sur la machine, l'étude a pris des substituts déquantifiés (deux tenseurs Q8_0 et un Q6_K de Qwen3.5-4B) et comparé, à erreur quadratique égale, une quantification façon Q4_K réécrite de zéro (KQ4, 4,5 bits par poids, dont les codes portent 3,874 à 3,883 bits contre 3,865 pour le vrai Q4_K : même forme) à une ECSQ (pas $\Delta = c\,\sigma$ par bloc de 256, σ en fp16, codes entiers non bornés, une table par tenseur) [mes.] (`rd-proxy-4B.json`). **À erreur égale, l'ECSQ demande 0,41 à 0,46 bit de moins par poids** (figure 6.13), en accord avec les 0,467 de la théorie malgré un régime à 4 bits qui n'est pas asymptotique et des sources non gaussiennes. C'est deux fois le plafond sans perte de Q4_K (0,24 bit), et le décodeur est la v4, avec un alphabet de 38 à 96 symboles.

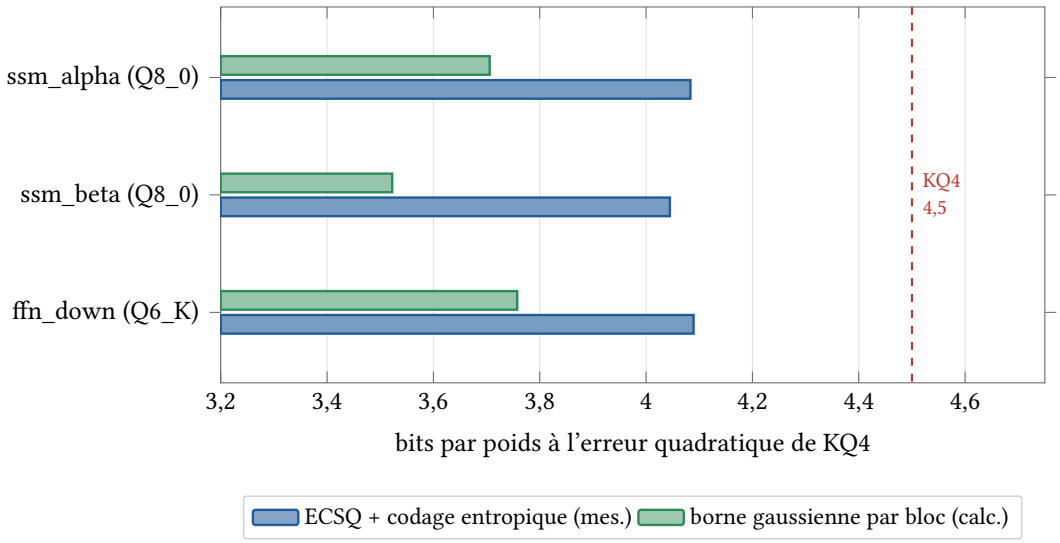


Figure 6.13 : Bits par poids d'une ECSQ suivie d'un codage entropique, à l'erreur quadratique de KQ4 (4,5 bits par poids), sur trois substituts [mes.], et la borne gaussienne par bloc [calc.]. Le gain va de 0,411 à 0,455 bit par poids ; il reste 0,3 à 0,5 bit jusqu'à la borne.

Données : `fondations/journaux/03-entropie/csv/rd-proxy.csv` (depuis `rd-proxy-4B.json`).

Ce chiffre n'est pas une qualité : l'erreur quadratique n'est pas la perplexité, et l'erreur pondérée par l'importance des poids peut déplacer le gain. vram-compression publie, sur Qwen2.5-1.5B, une perplexité d'ECSQ à pas par colonne à 3,25 bits (9,81) presque égale à celle de GPTQ à 4,25 bits (9,78) [litt.] : environ un bit gagné à qualité presque égale, deux fois notre mesure à erreur quadratique égale ; leur pas par colonne pondéré par la hessienne, que notre mesure n'a pas, peut expliquer l'écart.

EntQuant [49] (ICML 2026) fait la même famille de choses en Float8 codé par ANS, décodé par bloc avant le produit, et WaterSIC [50] se place à 0,255 bit de la limite de la théorie de l'information par une allocation de débit par colonne. Ce qui manque encore à ces travaux, au regard de la mission : tourner sans CUDA, viser le régime du flux depuis le disque, et s'appliquer aux formats que la machine fait tourner.

6.9 L'état de l'art, technique par technique

Table 6.7 : Les codeurs sans perte de poids et leurs voisins, au regard de la mission. Gains et débits des éditeurs [litt.], sauf ceux de troncons [mes.]. « Sans CUDA » : tourne nativement sous Windows sur la carte de la machine sans boîte à outils CUDA.

technique	mécanisme	sur nos formats	sans CUDA	licence
DFloat11 [41]	Huffman des exposants BF16, tables de 256 entrées en cascade	sans objet (BF16)	non	Apache-2.0
ZipNN [42]	flux d'octets de même rang, Huffman ou zstd	1,042 au mieux par champ [calc.]	oui (processeur)	MIT
Huff-LLM [43]	décodeur Huffman matériel devant un réseau systolique	sans objet	sans objet	aucun code
Ecco [102]	motifs k-means et tables de Huffman par groupe, matériel	sans objet	sans objet	aucun code
ZipServ [45]	code de longueur fixe de 3 bits sur les exposants, plans de bits, fondu	sans objet (BF16)	non	Apache-2.0
ISCA 2026 [46]	rANS de DietGPU, tuiles du produit, fondu	int4 annoncé, contredit ici	non	aucun code
EntroPack [48]	réseau E8, modèle conditionnel, tuiles indépendantes	sans objet (diffusion)	inconnu	non vérifiée
nvCOMP [56]	ANS par morceaux, GDeflate, Bitcomp	$\leq 1{,}018$ (octets)	non	propriétaire
DirectStorage [5]	GDeflate et zstd sur la carte (DX12)	$\leq 1{,}018$	**oui**	MIT, Apache-2.0
DietGPU [57]	rANS, renormalisation coopérative du warp	idem, plus par champ	non	MIT
Recoil [55]	un flux rANS, points de coupe	idem	non	GPL-3.0
zstd, lz4 [58]	LZ et entropie, processeur	1,012 et 1,000 [mes.]	oui	BSD
EntQuant [49]	Float8 sous lagrangien ℓ_1, ANS par nvCOMP	sans objet (Float8)	non	Apache-2.0
vram-compression [51]	ECSQ et GPTQ, rANS fondu au produit	ECSQ, pas un GGUF	non	aucune
ECF8 [44]	FP8 codé par entropie des exposants	sans objet (FP8)	inconnu	aucun code
troncons rANS v4	rANS entrelacé par sous-groupe, flux partagé, échelles par médiane de ligne	**1,0316** (Q4_K) ; **1,1026** (MXFP4, 120b) [mes.]	**oui** (Vulkan, 70 Go/s)	à nous

Le tableau 6.7 résume les seize lignes de l'étude. Deux leçons de structure en sortent. **Dans le régime résident, seul un code à décodage en temps constant passe dans la multiplication** : ZipServ ne gagne contre cuBLAS qu'avec un code de longueur fixe de 3 bits sur des exposants BF16 très creux ; nos codes de 4 bits sont presque uniformes et n'ont pas cette forme. **Dans le régime du flux, un code de longueur variable paie tout son taux**, pourvu que son décodeur soit sur la carte et suive le disque d'un facteur large : c'est le cas ici, d'un facteur 28.

6.10 Ce qui manque dans le monde

1. **Personne ne code les échelles des formats MX.** Aucun des travaux lus ne mesure ni ne code les échelles partagées E8M0, alors qu'elles portent le gain : 0,213 bit par poids contre 0,134 pour les

codes sur gpt-oss-20b [calc.]. Le trou est chiffré par un taux réel, décodé exactement : 1,1026 sur gpt-oss-120b, quand zstd prend 1,0405 sur les mêmes tenseurs et lz4 rien. NVFP4 (échelle FP8 par bloc de 16) a la même structure ; aucun fichier de ce format n'était sur la machine pour la mesurer.

2. **Aucun outil ne code par entropie les champs des K-quants** : le plafond est de 5,3 %, zstd en prend 1,2 %, tout codeur d'octets sur le bloc entier au plus 1,8 % ; sur le 9B, 180 Mo par jeton relu ne traverseraient pas le disque.
3. **Aucun décodeur entropique de poids sans CUDA n'avait de débit publié** sur une carte de portable ; celui-ci fait 70 à 72 Go/s par Vulkan, 28 fois le disque.
4. **La quantification taillée pour le codeur existe en CUDA** (EntQuant, vram-compression), ~~mais n'existe nulle part pour les LLM grand public~~ (phrase de la première version de l'étude, barrée à sa reprise) ; aucune ne tourne sans CUDA, ne vise le flux depuis le disque, ni ne s'applique aux formats GGUF.
5. **Le régime résident n'a pas de code à longueur variable sur cette classe de carte** : il manque un facteur 3,4 au décodeur, retrouvé par la mesure (3,7). Un code à décodage en temps constant qui garde une part du gain de l'ECSQ reste à inventer.
6. **Les entropies publiées des int4 divergent d'un facteur 4** de la mesure, et le calcul exact de la section 6.3.1 montre pourquoi : il faudrait, pour les atteindre, une grille symétrique de plus de 30 écarts-types d'étendue. Il manque une mesure publique, outillée, par schéma de quantification.
7. **Aucune mesure publiée ne compare deux chemins de compilation du même décodeur** sur la même carte : 15 % ici entre nvcc et Vulkan. Une décision « CUDA ou Vulkan » prise sur un débit publié dans un autre langage se trompe d'environ ce facteur.

Ce qui pourrait être faux

- **La contention.** D'autres agents faisaient tourner llama.cpp et des scripts Python pendant les mesures ; les débits du processeur n'en sont que des ordres de grandeur. Les débits de la carte cités viennent de passes jugées au calme sur la puissance tirée (repos 5,5 W, seuil déclaré 12 W), avec un échantillonneur toutes les 500 ms qui dit après coup si la passe est restée seule.
- **La batterie.** La première moitié de la reprise s'est faite sur batterie (carte plafonnée à 43 W, horloge mémoire à 6 001 MHz au lieu de 8 001). Seuls les chiffres du secteur sont cités comme débits ; ceux de la batterie restent, étiquetés.
- **Les gains sur les métadonnées fp16 sont optimistes** (lemme 6.3) : le chiffre réaliste est celui du codage par octet du champ (1,052 pour Q4_K), pas l'idéal 1,057.
- **Les débits viennent de lots de 126 à 256 Mio** ; un lancement par matrice d'expert (4,4 Mo dans gpt-oss-120b) perdra en démarrage et en fin de vague. Ce n'est pas mesuré ici.
- **La section ECSQ repose sur des substituts** déquantifiés, posés sur une grille, ce qui peut abaisser l'entropie de l'ECSQ et gonfler le gain ; KQ4 n'a ni l'imatrix ni la recherche exacte de llama.cpp ; l'erreur quadratique n'est pas la qualité.
- **Le budget arithmétique de la carte** (6 à 7,7 · 10^{12} opérations par seconde) est un ordre de grandeur ; la proposition 6.14 et le facteur 3,4 prédit en dépendent, la mesure (3,7) non.
- **Le modèle du sous-groupe** exige des sous-groupes de 32 invocations consécutives : vrai ici (vérifié par l'exactitude), faux sur AMD (64). Le décodeur s'arrête proprement par un compteur si la taille diffère.
- **Le second chemin CUDA est indépendant par le code et le compilateur, pas par l'algorithme** : il suit le même format. Ce qui le rend probant : l'exactitude est jugée contre la source, non contre l'autre décodeur.
- **Le calcul de la section 6.3** suppose des poids indépendants de loi gaussienne ou laplacienne ;

il retrouve les mesures au centième, mais une loi réelle plus lourde aux queues abaisserait l'entropie d'un min-max à grand groupe.
– **Les débits de la littérature** (DietGPU, Recoil, ZipNN) sont ceux des éditeurs, sur d'autres machines ; aucun n'est reproduit, bien que CUDA soit désormais sur la machine.

Chapitre 7

Chargement à la demande

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, une question : quand un modèle ne tient pas en mémoire, où sont ses poids à l'instant où un calcul en a besoin, combien d'octets faut-il déplacer par jeton, par quel étage, et quelle part de ce déplacement se cache derrière le calcul ? **Ce chapitre établit** la revue de tout ce qui fait tourner un modèle plus gros que la mémoire, du « couche par couche » de 2022 aux moteurs de 2026 qui lisent les experts d'un MoE sur le disque, jugée par une seule règle (les octets par jeton divisés par le débit de chaque étage) ; la grandeur qui décide de tout pour un MoE, le taux de succès du cache ; le succès qu'implique toute mesure publiée ; et le juge refait avec le disque mesuré. **Il reste** une trace de routage réelle sur un MoE de 2026, une passe PCIe propre, et la politique de défaut qui réunit les quatre réponses possibles. Source : `fondations/04-chargement.md`, `fondations/journaux/04-chargement/`.

7.1 La loi du chargement

Au décodage, chaque jeton lit chaque poids actif une fois. Le théorème 3.2 borne le débit entre deux régimes : étages parfaitement recouverts, $d \le 1/\max_i(O_i/W_i)$, et étages en série, $d = 1/\sum_i(O_i/W_i)$. Un système réel tombe entre les deux, et la place où il tombe se mesure :

$$\text{part cachée} = \frac{\sum_i t_i - t_{\text{mesuré}}}{\sum_i t_i - \max_i t_i}, \qquad t_i = O_i/W_i. \tag{7.1}$$

Tous les systèmes lus disent que le décodage hors mémoire est un problème d'octets et non de calcul, et leurs chiffres le confirment : FlexGen passe plus de 99,5 % de son temps à transférer des poids ; HOBBIT mesure 80 ms pour charger une couche de Mixtral contre 3 ms pour la calculer sur une RTX 4090 [litt.] [8, 16]. Le juge « octets sur débit » (`chargement_juge.py`) borne chaque technique ; il a été validé sur cinq mesures publiées, dont aucune ne dépasse sa borne, et, débits divisés par 10, il tombe sur quatre des cinq : le témoin peut échouer.

7.2 Le mélange d'experts en trois grandeurs

Pour un MoE de L couches à experts, E experts routés par couche, k_e experts choisis par jeton, stocké à b bits par poids, on pose (figure 7.1) :

$$B_t = \text{octets toujours lus (attention, experts partagés, routeurs, normes, tête)},$$
$$e = \frac{P_{\text{experts}}}{E\,L} \cdot \frac{b}{8} \quad \text{(octets d'un expert routé)}, \qquad B_e = k_e\,L\,e \quad \text{(octets routés par jeton)}. \tag{7.2}$$

Une part h_v des octets routés vient de la VRAM, h_r de la RAM, $h_s = 1 - h_v - h_r$ du disque, qui doit livrer $h_s B_e$ octets par jeton. C'est presque toujours l'étage limitant, et **le taux de succès du cache $h = h_v + h_r$ est la grandeur qui décide de tout**.

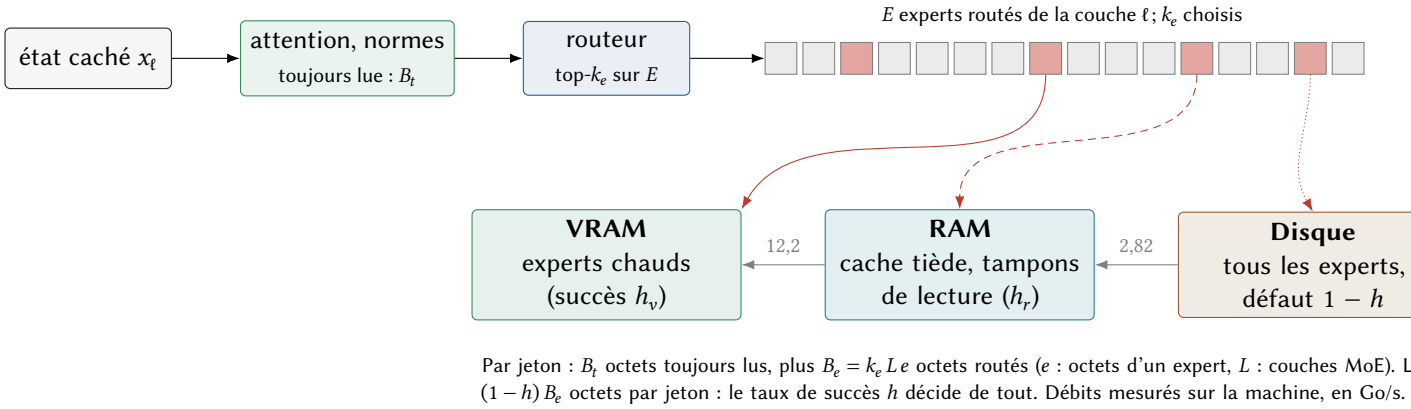


Figure 7.1 : Une couche de mélange d'experts et la hiérarchie de cache qui sert ses experts. Les toujours-lus résident en VRAM ; parmi les E experts de la couche, le routeur en choisit k_e ; chacun est servi par la VRAM, la RAM ou le disque.

À l'inverse, une mesure publiée de débit dit quelque chose de la localité qu'elle ne publie pas.

Lemme 7.1 (Le succès impliqué par une mesure)**.** *Si un système atteint d jetons par seconde avec un disque de débit W_{disque} et B_e octets routés par jeton, son cache sert au moins une part*

$$h_{\min} = 1 - \frac{W_{\text{disque}}}{d\, B_e} \tag{7.3}$$

des octets routés.

Démonstration. Par le théorème 3.2 appliqué à l'étage disque, $1/d \geq (1-h)B_e/W_{\text{disque}}$, soit $1 - h \leq W_{\text{disque}}/(dB_e)$. ∎

Table 7.1 : Le succès du cache impliqué par trois mesures publiées (lemme 7.1) [calc.] sur [litt.].

Mesure publiée	part du modèle en mémoire rapide	succès impliqué au moins
Qwen3-235B à chaud, 0,441 jeton/s, disque 2,4 Go/s [103]	environ 25–30 %	33 %
GLM-5.2, 2,20 jetons/s, disque ≤ 7,877 Go/s [22]	79 Go sur 254 (31 %)	53 % (publié : 79 %)
moe-stream, Qwen3.6-35B-A3B, 5,05 jetons/s, 6 Go [23]	6 sur 19,5 Go d'experts (31 %)	53 %

Le cas GLM sert de témoin : la borne (53 %) est sous le succès publié (79 %). Le cas moe-stream montre, sans trace de routage, qu'un cache tenant 31 % des experts sert au moins 53 % des octets : **la localité vaut au moins un facteur 1,7 sur la capacité** pour ce modèle.

7.3 La granularité

Un étage n'atteint son débit qu'au-delà d'une taille de requête et d'une profondeur de file. Le disque de la machine en est l'exemple extrême : 0,22 Go/s à 4 Kio contre 2,4 à 2,8 Go/s à 1–16 Mio (figure 3.1), un facteur 11 à 13. La littérature dit la même chose ailleurs : le cache de pages du système livre

148 Mo/s là où le même disque donne 1 496 Mo/s en lectures explicites à la taille d'un expert, avec 25 000 à 50 000 défauts de page par jeton [23] ; dans un fichier GGUF, les trois matrices d'un expert sont rangées dans trois tenseurs différents, d'où 6 912 pages touchées par expert contre 192 après regroupement [104] [litt.]. **Un expert doit être une seule lecture, contiguë, alignée sur 4 Kio.** Sur la carte, un transfert synchrone coûte une latence fixe d'environ 33 µs (64 Kio passent à 1,5 Go/s) [mes.].

7.4 Les modèles denses

Couche par couche (AirLLM [6]). Le modèle est découpé en un fichier par couche ; à chaque passe, la couche ℓ est lue, copiée, calculée, libérée, et un fil prépare $\ell + 1$. Ce système publie de la mémoire, pas de la vitesse (« 70B sur une carte de 4 Go »). Le juge le chiffre : un 70B dense en FP16 lit 140 Go par jeton ; au débit mesuré du disque, **49,6 s par jeton** [calc.]. C'est exactement la lecture littérale de « sans RAM ni rien », et c'est pourquoi elle ne suffit pas : la mémoire de pointe est résolue, le débit ne l'est pas.

FlexGen [8] et ZeRO-Inference [7]. FlexGen amortit la lecture des poids sur un grand bloc de lots, avec un modèle de coût par couche où cinq flux se recouvrent, $T = \max(\text{ctog}, \text{gtoc}, \text{dtoc}, \text{ctod}, \text{comp})$, et une politique de onze variables choisie par programmation linéaire. Il obtient 0,69 jeton/s sur OPT-175B, mais avec un lot effectif de 144 et des milliers de secondes de latence [litt.] : rien pour un usage interactif au lot de un.

llama.cpp [9]. Lu au commit `843e7dc` : projection du fichier en mémoire (`mmap`, avec `PrefetchVirtualMemor` sous Windows), le chargement à la demande le plus simple qui soit et le plus mal granulé ; `-ngl N` pour placer N couches sur la carte, le reste étant calculé par le processeur, sans recouvrement entre couches ; `-ncmoe N` pour laisser les experts de N couches au processeur ; `--load-mode dio` pour une lecture directe sans cache système ; lecture paresseuse des tenseurs de plus de 4 Gio ; et `--moe-cache-mib`, expérimental, un cache en VRAM des experts gardés en RAM. La lecture des experts sur le disque avec un cache explicite est une demande de fusion ouverte [22].

LLM in a flash [13] et PowerInfer [11, 12]. Le premier prédit les neurones actifs du FFN, garde en DRAM ceux des derniers jetons (fenêtrage), et regroupe ligne et colonne d'un même neurone pour le lire d'un tenant : 20 à 25 fois mieux que le chargement naïf sur la carte. Le second place sur la carte les neurones « chauds » et calcule les « froids » sur le processeur. Tous deux reposent sur la parcimonie d'activation des fonctions ReLU ; sur les SwiGLU modernes elle est faible, et le chapitre 8 la mesure.

7.5 Les mélanges d'experts

Déchargement, prédiction, cache. Mixtral-offloading [14] garde un cache LRU de quelques experts par couche en VRAM et applique le routeur de la couche $\ell + 1$ à l'état caché de la couche ℓ (le résidu change peu) pour précharger : ×1,47 sur une RTX 3060. Les systèmes qui suivent prédisent mieux (tableau 7.2) ; deux enseignements pèsent pour la suite. HOBBIT [16] remplace un expert absent par sa copie basse précision pour au plus un point de perte. Edge0 [19] mesure que la prédiction ne supprime pas d'octets mais retire la latence du chemin critique, et que seule une avance d'un jeton entier paie dans un moteur réel.

Table 7.2 : Prédire et précharger les experts : justesse et gain publiés [litt.].

Système	Prédiction et résultat publié	Machine
Mixtral-offloading [14]	routeur ℓ+1 sur l'état ℓ ; ×1,47 avec LRU	RTX 3060
ProMoE [15]	prédicteur appris, 84,7 % ; décodage ×2,07	RTX 4090
HOBBIT [16]	routeurs ℓ+1..ℓ+3, 96 % à ℓ+1 ; jusqu'à ×9,93	RTX 4090, Orin
AdapMoE [105]	porte adaptative (−25 % d'experts) ; ×1,35	bords
ExpertFlow [17]	prédicteur du chemin entier ; jusqu'à ×10	une carte
SeqMoE [18]	multi-pas, Belady probabiliste ; 96,97 % à 45 % de résidence	—
Edge0 [19]	prérouteur, un jeton d'avance ; +80 à +84 % au décodage	Mac mini M4 Pro

Calculer l'expert là où il est. Fiddler [20] calcule sur le processeur un expert absent de la carte si c'est moins cher que de le transférer ; KTransformers [21] calcule les experts en RAM avec des noyaux AVX-512 et AMX et *reporte* certains experts : leur sortie est ajoutée à la couche $\ell + 2$ au lieu de $\ell + 1$, ce qui laisse le processeur calculer pendant l'attention suivante, pour au plus deux points de perte. Sur la machine de Stone, l'étude avait chiffré qu'un expert de Qwen3-30B (2,86 Mo en Q4_K_M) coûte 82 µs à calculer sur le processeur (RAM supposée à 35 Go/s) contre 33 + 448 µs à transférer à 6,4 Go/s : ~~transférer coûte 5,9 fois plus que calculer sur place~~. Refait avec les débits mesurés au chapitre 3 [calc.] : le calcul sur le processeur coûte 2,86/15,6 = 183 µs, le transfert 33 + 2,86/12,2 = 267 µs, soit **1,5 fois** seulement ; le calcul sur la carte depuis la VRAM, 11 µs. Avec un seul canal de RAM, la règle de Fiddler reste vraie de justesse, et chaque expert gardé en VRAM vaut seize fois un expert gardé en RAM.

Lire les experts sur le disque (2026). La demande de fusion #25294 de llama.cpp [22] gère un cache d'emplacements en mémoire de la carte, lit les défauts par un pool de fils et obtient 2,20 jetons/s sur GLM-5.2 (254 Go) avec 79 % de succès, contre 0,87 en mmap. moe-stream [23] tourne sous Windows, sur le processeur seul : Qwen3.6-35B-A3B passe de 0,82 jeton/s en mmap à 5,05 avec un cache explicite de 6 Go. SSD-LLaMA [24] traite SSD, RAM et VRAM comme une seule hiérarchie et **compresse les experts** (trafic disque −33,2 %), sur une RTX 5090 avec PCIe 5.0. SmallThinker [106] prend le contre-pied : un modèle conçu pour le local, dont le routeur est placé avant l'attention pour que les experts se lisent pendant l'attention ; on peut acheter la localité à l'entraînement, pas sur un modèle existant [litt.].

Table 7.3 : Les politiques de cache d'experts : succès publiés [litt.].

Politique	Résultat publié	Source
LRU	65,9 % à 13,4 % du budget (Qwen3-30B, une couche)	[103]
LFU	60,1 % au même point	[103]
Épinglage statique	59,2 % au même point	[103]
Belady (optimum hors ligne)	79,1 % au même point	[103]
« Least-Stale »	88–92 % à 1 % de capacité	[107]
Appris (récence et fréquence)	+51 % de succès sur LRU et LFU	[108]
Routage conscient du cache	défauts de DeepSeek-V2 28 % → 7 %, perplexité +0,1 à 3 %	[109]

7.6 La localité des experts d'un jeton au suivant

Table 7.4 : La réutilisation des experts d'un jeton au suivant [litt.].

Modèle (E, k_e)	Réutilisation adjacente	« Au hasard »
Mixtral-8x7B (8, 2) [110]	premier choix répété 13,6–14,9 % (couche 0), 24,6–28,4 % (couche 15)	12,5 %
Qwen3-30B-A3B (128, 8) [103]	0,444	0,223
Qwen3.6-35B-A3B (256, 8) [19]	« environ un quart » de l'ensemble d'une couche	3,1 % [calc.]
Qwen1.5-MoE, DeepSeek-V2, Mixtral [109]	résidence moyenne en cache 26, 19, 5 jetons	—
Qwen3-30B (4 domaines) [103]	52,5 % des experts portent 95 % du trafic ; recouvrement code/autres 0,11–0,16	—

La localité existe et elle est faible (tableau 7.4) : un jeton reprend environ un quart à 44 % des experts du jeton précédent, deux à huit fois le hasard ; elle est plus forte au milieu du réseau qu'aux extrémités ; la popularité compte autant que la récence ; et le domaine change l'ensemble chaud. L'optimum contre lequel se jugent toutes les politiques est celui de Belady.

Théorème 7.2 (Belady [3])**.** *Pour une suite de demandes connue d'avance et un cache de C places, la politique qui, à chaque défaut, évince l'élément dont la prochaine demande est la plus lointaine minimise le nombre de défauts.*

Démonstration. Par échange. Soit une politique optimale $\mathcal{O}$ qui diffère de celle de Belady $\mathcal{B}$ pour la première fois à l'instant t : $\mathcal{B}$ évince x, dont la prochaine demande est la plus lointaine, et $\mathcal{O}$ évince $y \neq x$. Construisons $\mathcal{O}'$ qui évince x à l'instant t puis imite $\mathcal{O}$, avec y à la place de x, jusqu'à la prochaine demande de y (qui précède celle de x). À cette demande, $\mathcal{O}'$ a y en cache quand $\mathcal{O}$ ne l'a pas : $\mathcal{O}$ paie un défaut que $\mathcal{O}'$ ne paie pas, ou bien $\mathcal{O}$ a évincé x entre-temps, et les deux caches redeviennent égaux. Dans tous les cas $\mathcal{O}'$ ne paie pas plus de défauts que $\mathcal{O}$ et coïncide avec $\mathcal{B}$ un pas de plus ; par récurrence, $\mathcal{B}$ est optimale. ■

L'outil `chargement_localite.py` mesure, sur une trace de routage, la réutilisation adjacente, ses deux références de hasard, les distances de réutilisation et les succès de LRU, LFU, épinglage statique et Belady pour chaque budget. Ses témoins synthétiques tiennent : routage uniforme (réutilisation 0,0613 pour 0,0625 attendus, tolérance de 4 écarts-types, et LRU égal à C/E à chaque budget), routage figé (succès 0,9998), routage « collant » de paramètre $\rho = 0{,}6$ (0,604 mesuré), Belady jamais dépassé (théorème 7.2). L'ordre des jetons mélangé, la réutilisation du routage collant tombe à 0,0635 et le témoin échoue : l'instrument voit l'ordre. Deux erreurs de la première passe ont été trouvées par ces témoins et corrigées. La figure 7.2 montre déjà la forme du problème sur le routage collant : l'écart entre LRU et Belady, 9 à 13 points, est la marge qu'une prédiction parfaite pourrait reprendre. **L'instrument est prêt ; la trace réelle manque.**

7.7 Les pipelines à double tampon

Avec deux tampons, la lecture du morceau $\ell + 1$ se fait pendant le calcul du morceau ℓ ; le temps par morceau devient $\max(L_\ell, C_\ell)$ au lieu de $L_\ell + C_\ell$, plus un remplissage initial (lemme 3.3). Pour masquer une latence λ, il faut une profondeur de préchargement $d \geq \lceil \lambda / C_{\text{couche}} \rceil$ couches.

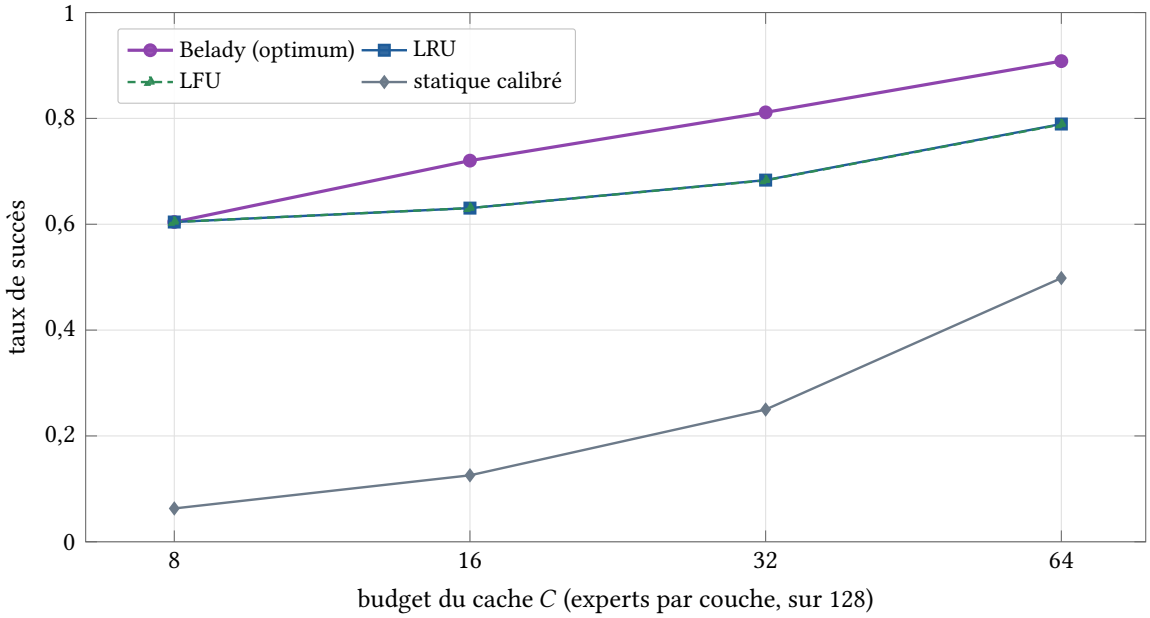


Figure 7.2 : Succès des politiques de cache sur le témoin synthétique de routage « collant » ($\rho = 0{,}6$, $E = 128$, $k_e = 8$) [calc.]. Belady borne toutes les politiques ; l'écart LRU–Belady est ce qu'une prédiction parfaite reprendrait.

Données : `fondations/journaux/04-chargement/localite-temoins.json`, témoin T3.

Proposition 7.3 (Le double tampon cache la latence, jamais le volume)**.** *Quel que soit le préchargement, le temps par jeton reste au moins* $\max_i O_i/W_i$ *(théorème 3.2) ; un préchargement de profondeur suffisante n'élimine que les attentes dues à la latence des requêtes.*

Démonstration. Le préchargement change l'instant des transferts, pas leur volume : chaque étage doit toujours faire passer O_i octets par jeton, et l'argument du théorème 3.2 ne dépend pas de l'ordre des transferts. ■

Exemple [calc.] : pour Qwen3-235B à environ 1 jeton/s, une couche dure 10,6 ms. L'étude avait chiffré la lecture d'un expert (10,76 Mo) à 1,6 ms en supposant un disque à 6,7 Go/s ; au débit mesuré (2,82 Go/s), elle dure 3,8 ms, et huit experts manquants 30,5 ms, trois fois la durée d'une couche. Une couche d'avance masque la latence d'un défaut ; aucune avance ne masque le volume de huit.

Ce que la machine donne au pipeline [mes.]. Une copie hôte vers carte se recouvre presque parfaitement avec un trafic VRAM sur un autre flux (part cachée 0,98 à 1,01, VRAM ralentie de 5,6 %) : les moteurs de copie sont libres pendant que la carte travaille. La mémoire épinglée vaut 21 % de débit de plus que la mémoire paginable. Et le lien n'atteint sa génération 4 que si la carte est en état de performance haut (figure 3.4) : **il faut que la carte calcule pendant qu'on copie**, ce qui tombe bien, c'est le but du recouvrement.

Les chemins de Windows. Lectures sans tampon par IOCP ou IoRing (l'équivalent d'io_uring avec `O_DIRECT`) ; DirectStorage, qui lit le NVMe vers une ressource Direct3D 12 et décompresse sur la carte (GDeflate depuis la version 1.1, Zstandard en préversion 1.4 [83]) ; CUDA, dépaqueté sur la machine, pour les copies asynchrones et la mémoire épinglée. Aucun moteur d'inférence lu ne se sert de DirectStorage.

7.8 Le juge appliqué à la machine

L'étude avait appliqué le juge avec des débits supposés (RAM 35 Go/s, disque 6,7 Go/s), puis avec la première lecture provisoire du disque (2,4 Go/s), en barrant ses conclusions à chaque correction. On le relance ici, même outil, avec le débit mesuré et retenu par le chapitre 3 (`--disque-go-s 2.82`); capacités supposées : 6,5 Go de VRAM et 10 Go de RAM pour des poids.

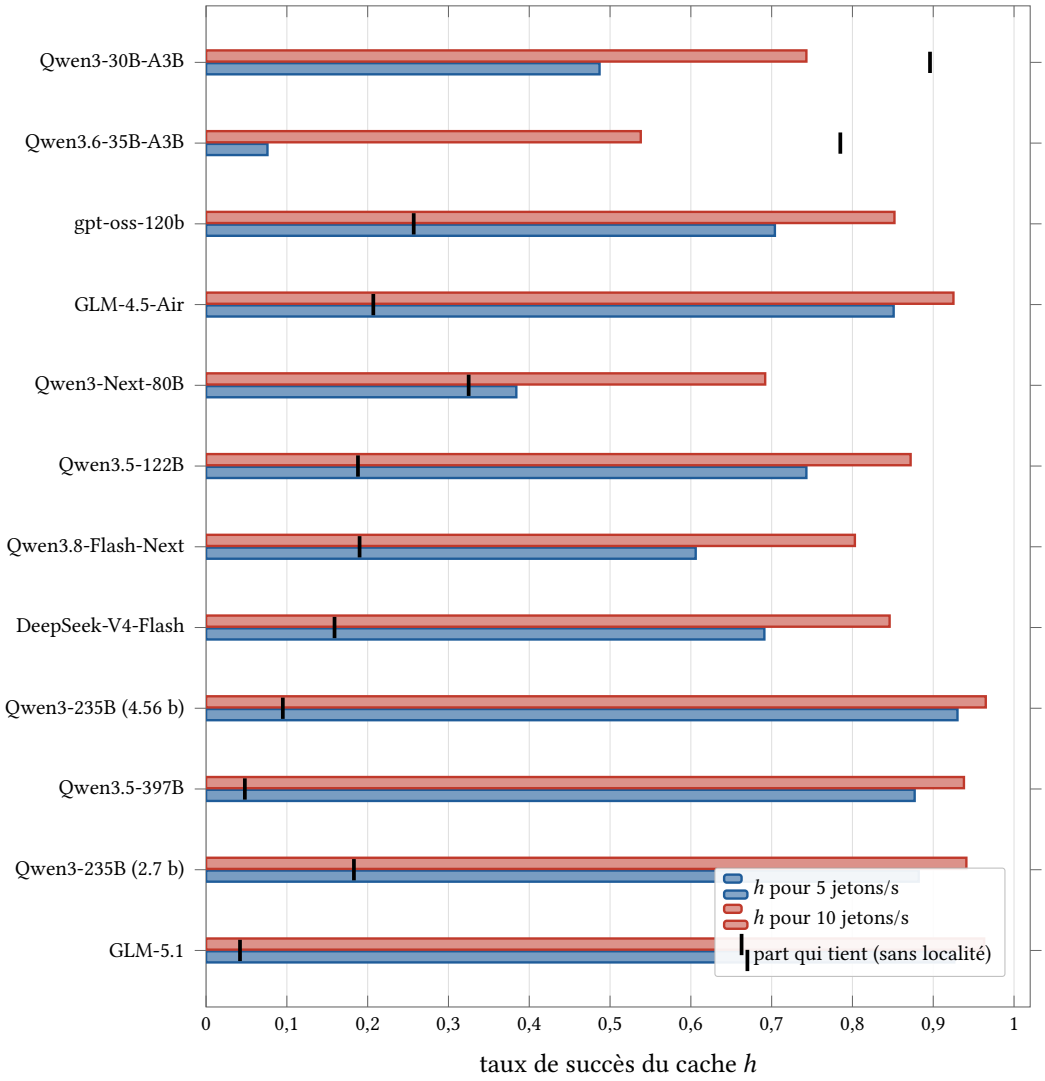


Figure 7.3 : Le succès du cache d'experts exigé pour 5 et 10 jetons/s, disque mesuré à 2,82 Go/s, et la part du modèle qui tient en mémoire rapide (trait noir) [calc.]. Partout où la barre dépasse le trait, il faut de la localité, de la prédiction ou moins d'octets.

Données : `memoire/figures/donnees/juge-stone-disque-2.82.json` (`outils/chargement_juge.py`) et `fondations/journaux/04-chargement/juge-stone-disque-2.4.json`.

Lecture (figure 7.3 et tableau 7.5) :

1. **Les MoE à faible B_e (au plus 1,5 Go routés par jeton), trois à cinq fois plus gros que la mémoire, sont à portée** : Qwen3-Next-80B (4,6 jetons/s sans localité) et Qwen3.8-Flash-Next (2,4) demandent 38 à 80 % de succès pour 5 à 10 jetons/s.
2. **Les gros MoE à experts épais ($B_e \geq 4{,}5$ Go) exigent 88 à 93 % de succès pour 5 jetons/s**,

Table 7.5 : Le juge sur la machine : borne sans localité et succès exigé, disque à 2,4 Go/s (étude 04) puis 2,82 Go/s (mesure retenue) [calc.].

Modèle	bits/poids	B_e (Go)	borne, 2,4	borne, 2,82	h pour 5	h pour 10	part qui tient
Qwen3-30B-A3B	4,85	1,10	21,08	24,77	0,49	0,74	0,90
Qwen3.6-35B-A3B	4,85	0,61	18,27	21,46	0,08	0,54	0,79
gpt-oss-120b	4,25	1,90	1,70	1,99	0,70	0,85	0,26
GLM-4.5-Air	4,85	3,78	0,80	0,94	0,85	0,93	0,21
Qwen3-Next-80B	4,85	0,92	3,89	4,57	0,38	0,69	0,33
Qwen3.5-122B	4,85	2,20	1,35	1,58	0,74	0,87	0,19
Qwen3.8-Flash-Next	4,85	1,43	2,07	2,43	0,61	0,80	0,19
DeepSeek-V4-Flash	4,50	1,83	1,56	1,84	0,69	0,85	0,16
Qwen3-235B (4.56 b)	4,56	8,09	0,33	0,39	0,93	0,97	0,10
Qwen3.5-397B	4,85	4,58	0,55	0,65	0,88	0,94	0,05
Qwen3-235B (2.7 b)	2,70	4,79	0,61	0,72	0,88	0,94	0,18
GLM-5.1	2,70	7,64	0,33	0,39	0,93	0,96	0,04

avec 4 à 18 % du modèle en mémoire rapide. Le seul point publié comparable (LRU 65,9 %, Belady 79,1 % à 13,4 % de budget) dit que les politiques de cache seules n'y arrivent pas : il faut réduire B_e (chapitre 5) ou changer le routage.

3. **gpt-oss-120b** : 1,99 jeton/s sans localité, 70 % de succès pour 5 jetons/s. C'est le troisième chemin indépendant qui retrouve le toit d'environ 2 jetons/s (section 3.8).

7.9 Ce qui manque dans le monde

1. **Aucun moteur publié ne lit les experts sur le disque sous Windows jusqu'à la VRAM.** SSD-LLaMA (io_uring), la demande #25294 (Linux), Edge0 (MLX) et KTransformers (Linux) n'y sont pas ; moe-stream tourne sous Windows mais sur le processeur seul. Le coût dominant de l'ignorer : les moteurs existants lisent par le cache de pages ou en petits morceaux, et ce disque donne 11 à 13 fois moins en petits blocs.
2. **L'état du lien PCIe d'un portable n'est dans aucune publication** (section 3.5).
3. **Pas de courbe « succès selon le budget » pour les MoE de 2026 aux budgets d'un portable** (5 à 20 % des experts) ; un seul point exact est publié.
4. **Le report d'expert n'a jamais servi à absorber un défaut de lecture.** KTransformers reporte des experts présents pour occuper le processeur ; Edge0 montre que la latence domine, pas le volume. Reporter d'une couche l'expert qui manque, au lieu d'attendre, retire cette latence du chemin critique sans prédiction et sans changer les octets.
5. **Personne n'a réuni les quatre réponses à un défaut sous un budget de qualité mesuré** (figure 7.5) : attendre, reporter, substituer une copie basse précision, re-router vers un expert présent.
6. **La compression dans la boucle de lecture n'est publiée que sur une machine à PCIe 5.0 et RTX 5090.** Sur la machine de Stone, où le disque borne, chaque point de compression des experts est un point de débit si la décompression tient dans le temps du calcul (corollaire 3.6).

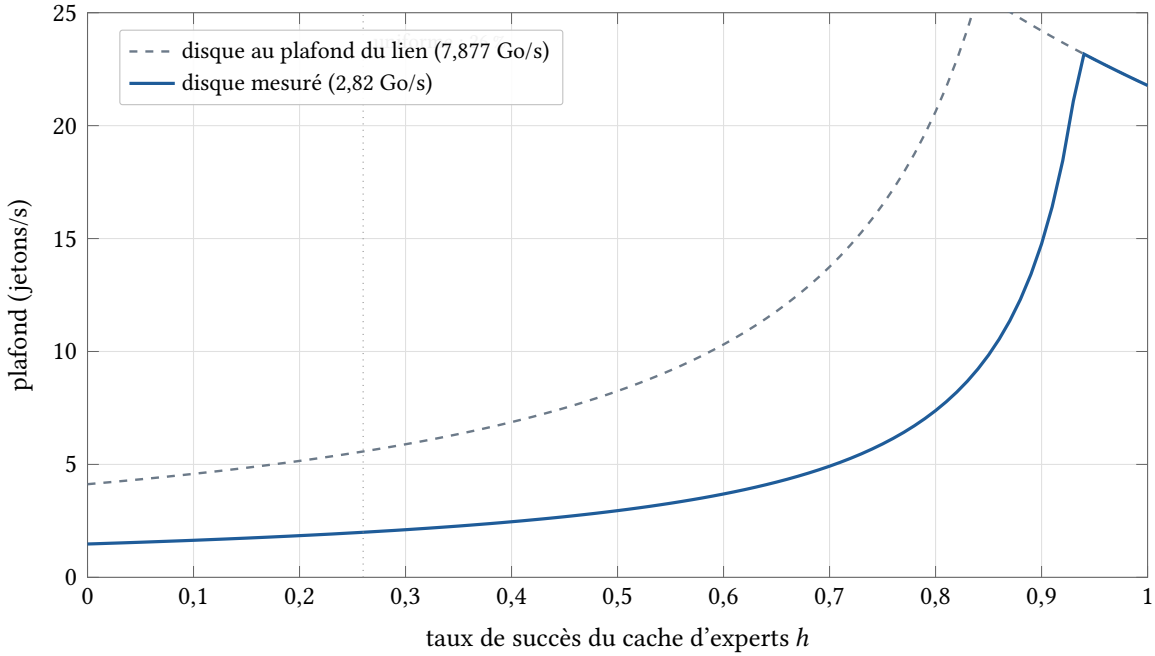


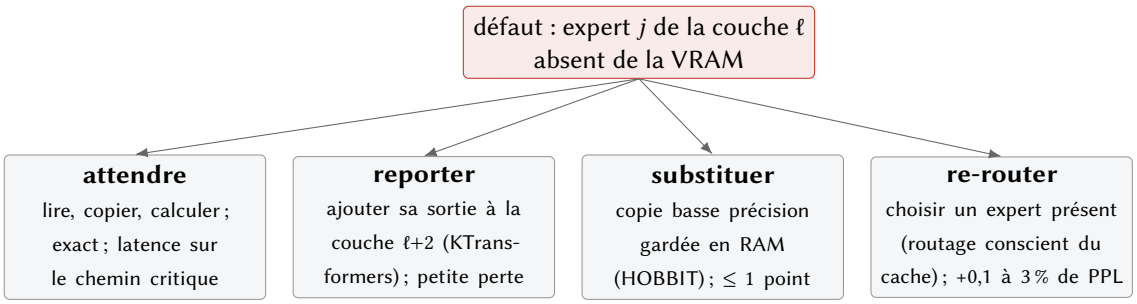
Figure 7.4 : Plafond de gpt-oss-120b selon le succès h du cache d'experts (modèle de coût de l'étude des modèles, $t = \max(B_t/256,\ hB_e/41{,}6,\ (1-h)B_e/W_{\text{disque}})$) [calc.]. Au débit mesuré du disque, passer de 26 % (routage uniforme) à 80 % de succès multiplie le plafond par 3,7.

Données : `memoire/figures/donnees/charge-sensibilite-h.csv`, d'après `fondations/journaux/01-machine/debits-mesures.json`.

Une proposition d'assemblage, pour l'arbitre. Ce que la revue et le juge suggèrent, sous forme d'hypothèses testables, chacune avec son témoin : attention, experts partagés, routeurs et cache d'experts chauds en VRAM ; RAM comme tampons de lecture épinglés et cache tiède ; disque contenant tous les experts, réécrits un expert par bloc contigu aligné, éventuellement codés sans perte ; lecture par IoRing sans tampon à profondeur de file d'au moins 16, ou par DirectStorage vers la VRAM ; prédiction d'un jeton d'avance ; politique de défaut choisie par le score de routage sous un budget de perplexité déclaré ; copies lancées pendant que la carte calcule ; mesure contre llama.cpp `-ncmoe` et `--moe-cache-mib` sur la même machine, en séries ABBA. Ce n'est pas une décision : c'est l'entrée du chapitre 10.

Ce qui pourrait être faux

- **Le juge suppose un recouvrement parfait** et ignore le cache KV, le préremplissage et la latence par requête : il borne, il ne prédit pas. Les mesures publiées tombent à 18 à 45 % de leur borne.
- **La capacité RAM du juge (10 Go) et sa PCIe (6,4 Go/s)** sont celles de l'étude d'origine ; seule la valeur du disque a été remplacée ici.
- **Les bits par poids moyens** appliquent la même précision aux experts et au reste, ce qui est faux en détail.
- **Plusieurs chiffres viennent de dépôts et de discussions communautaires**, non relus par des pairs ; deux articles se contredisent eux-mêmes. Ils sont étiquetés, pas écartés.
- **La réutilisation « 0,444 contre 0,223 »** dépend d'une définition du hasard que l'article ne donne pas.



Choisie défaut par défaut selon le score de routage de l'expert manquant, sous un budget de qualité déclaré avant la mesure : le terme que personne n'a réuni (section 7.9).

Figure 7.5 : Les quatre réponses possibles à un défaut de cache d'experts, chacune publiée séparément, jamais réunies.

- **Le report pour absorber un défaut n'a pas été mesuré**, et quelqu'un l'a peut-être fait sous un autre nom.
- **L'effet de l'état de performance sur le lien** est mesuré sur cette carte et ce pilote.

Chapitre 8

Sparsité et décodage spéculatif

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, deux familles de techniques promettaient de lire moins d'octets par jeton : la sparsité des activations (ne lire que les colonnes utiles) et le décodage spéculatif (lire les poids une fois pour plusieurs jetons), toutes deux publiées sur des machines où la RAM tient le modèle. **Ce chapitre établit** le théorème d'amortissement, $\mathscr{E}(\alpha,k) = (1-\alpha^{k+1})/(1-\alpha)$ jetons par lecture des poids, et ses corollaires ; la condition exacte sous laquelle la spéculation paie pour un mélange d'experts lu du disque, que le routage indépendant ne satisfait jamais ; la mesure, sur un modèle SwiGLU, que sparsité et spéculation ne se composent pas ; et les taux d'acceptation mesurés sur la famille Qwen3.5. **Il reste** la courbe d'union réelle des experts d'un MoE de 2026 sur des jetons de brouillon, l'accord du module MTP natif, et une vérification en arbre sous Vulkan. Source : `fondations/05-sparsite.md`, `fondations/journaux/05-sparsite/`.

8.1 Le juge, et le levier qui a la bonne pente

Une passe de la cible sur n jetons (un lot de vérification) coûte, étages parfaitement recouverts,

$$T(n) = \max\Big(\frac{O_{\text{disque}}(n)}{W_{\text{disque}}}, \frac{O_{\text{RAM}}(n)}{W_{\text{PCIe}}}, \frac{O_{\text{VRAM}}(n)}{W_{\text{VRAM}}}, \frac{C(n)}{F}\Big), \tag{8.1}$$

et, sans recouvrement, la somme des mêmes termes (théorème 3.2). $O_x(n)$ est le nombre d'octets de poids que la passe lit depuis l'étage x, $C(n) \approx 2nN_{\text{actifs}}$ le nombre d'opérations, F le débit de calcul effectif. Pour un modèle **dense**, $O_x(n)$ ne dépend pas de n : c'est tout l'intérêt. Pour un **MoE** ou un modèle **à activations creuses**, $O_x(n)$ croît avec n, parce que la passe lit l'union de ce que demandent ses n jetons.

Le débit de jetons est $\mathscr{E}/(T(n) + T_{\text{brouillon}})$, où $\mathscr{E}$ est le nombre de jetons acceptés par passe. Trois leviers : réduire $O(n)$ (sparsité, neurones chauds, prédiction des experts) ; augmenter $\mathscr{E}$ à $T(n)$ presque inchangé (décodage spéculatif) ; cacher $T_{\text{brouillon}}$ dans $T(n)$. Quand les poids viennent du disque, $T(n)$ se compte en secondes et $T_{\text{brouillon}}$ en millisecondes : **le deuxième levier écrase les deux autres, à une condition, que** $O(n)$ **reste plat en** n. Toute la suite chiffre cette condition.

Proposition 8.1 (Le coude du calcul)**.** *Pour un modèle dense de N poids actifs à b octets par poids, lus depuis un étage de débit W, la vérification de n jetons reste bornée par la lecture tant que*

$$n \;<\; n^\star = \frac{F\,b}{2\,W}. \tag{8.2}$$

Démonstration. La lecture coûte Nb/W secondes quel que soit n ; le calcul coûte $2nN/F$. Le second reste sous le premier tant que $2nN/F < Nb/W$, soit $n < Fb/(2W)$. ∎

Avec $F \approx 20$ Tflop/s (ordre de grandeur des cœurs tenseurs FP16 d'une RTX 4060 Laptop, non mesuré, et que la déquantification fait baisser) et $b = 0{,}6$ octet par poids (environ 4,8 bits) [calc.] : $n^\star \approx 24$ si les poids sont en VRAM (251,3 Go/s), ≈ 490 s'ils passent par le lien (12,2 Go/s ; l'étude, avec le lien chargé à 6,38 Go/s, donnait 940), $\approx 2\,100$ s'ils viennent du disque (2,82 Go/s). Ce tableau dit pourquoi SpecExec vérifie des arbres de 1 024 à 2 048 jetons en régime déchargé, et pourquoi EAGLE sur une carte qui tient le modèle s'arrête à quelques dizaines : ce n'est pas un choix, c'est $n^\star$. La mesure de la figure 8.1 le confirme malgré son bruit : jusqu'à 32 jetons, une passe tout en VRAM coûte au plus 1,35 fois une passe d'un jeton.

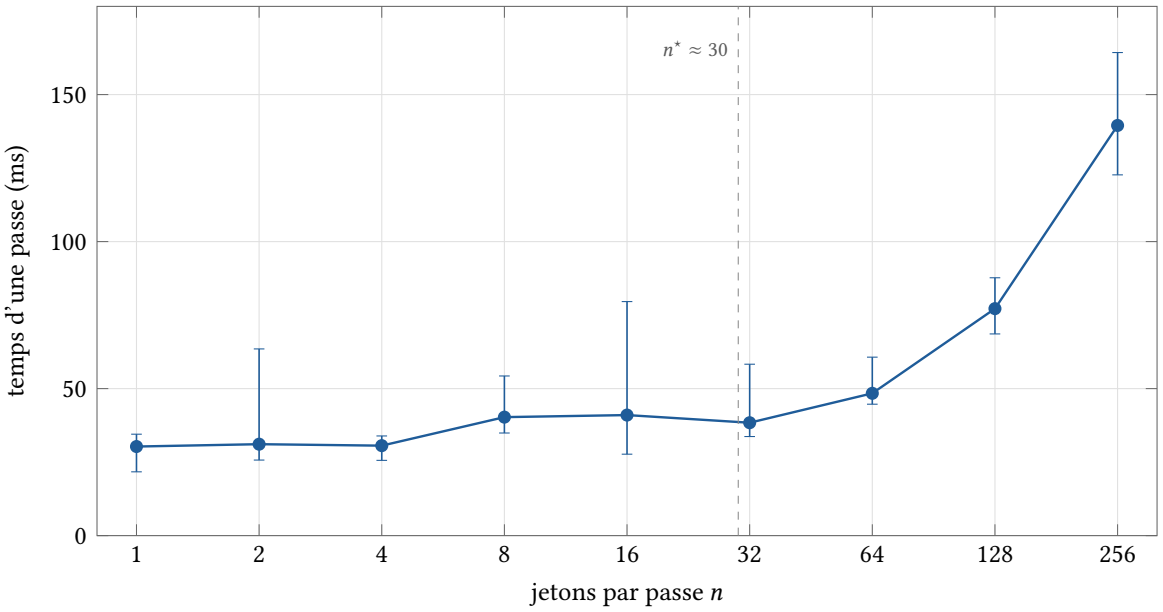


Figure 8.1 : Temps d'une passe de n jetons, Qwen3.5-4B Q4_K_M tout en VRAM, médiane et étendue de quatre séries ABBA [mes.]. Le seuil de bruit déclaré (10 %) n'est atteint pour aucun n (carte partagée) : chiffres indicatifs. Le coude est entre 32 et 128, là où la proposition 8.1 le place.

Données : `fondations/journaux/05-sparsite/cn/resume-4B-ngl99.txt`.

8.2 Le décodage spéculatif

8.2.1 La règle de vérification

Leviathan, Kalman et Matias [59] et Chen et al. [60] posent la même règle. Un brouillon de loi q propose $x_1, \dots, x_k$; la cible p calcule en une passe $p(\cdot \mid \text{préfixe}, x_1, \dots, x_i)$ pour $i = 0, \dots, k$. Chaque x_i est accepté avec la probabilité $\min(1, p(x_i)/q(x_i))$; au premier refus, on tire un jeton dans la loi résiduelle $r = (p - q)^+ / \|(p - q)^+\|_1$; si tout est accepté, on tire un jeton de plus dans p.

Proposition 8.2 (La spéculation est exacte)**.** *Le jeton émis à chaque position suit exactement la loi p de la cible, et la probabilité d'acceptation d'une position vaut $\alpha = \sum_x \min(p(x), q(x)) = 1 - \frac{1}{2}\|p - q\|_1$.*

Démonstration. La probabilité d'émettre x est celle de le proposer et l'accepter, plus celle de refuser puis de tirer x dans r :

$$q(x)\min\Big(1, \frac{p(x)}{q(x)}\Big) + (1-\alpha)\,\frac{(p(x)-q(x))^+}{1-\alpha} = \min(p(x), q(x)) + (p(x)-q(x))^+ = p(x),$$

où l'on a utilisé $\|(p-q)^+\|_1 = \sum_x (p(x) - \min(p(x),q(x))) = 1-\alpha$. La probabilité d'acceptation est $\sum_x q(x)\min(1, p(x)/q(x)) = \sum_x \min(p(x),q(x))$, et $\sum_x \min(p,q) = 1 - \frac{1}{2}\sum_x |p-q|$ puisque $\min(a,b) = \frac{1}{2}(a+b-|a-b|)$. ∎

En décodage glouton, un jeton est accepté s'il est l'argmax de la cible ; α est alors le taux d'accord des argmax, qu'on mesure au section 8.5.1.

8.2.2 Le théorème d'amortissement

Théorème 8.3 (Jetons par lecture des poids). *Si chacune des k positions proposées est acceptée avec la même probabilité $\alpha < 1$, indépendamment des autres et tant qu'aucune n'a été refusée, le nombre N de jetons produits par une passe de vérification (une lecture complète des poids actifs) a pour espérance*

$$\mathscr{E}(\alpha, k) = \mathbb{E}[N] = \sum_{i=0}^{k} \alpha^i = \frac{1-\alpha^{k+1}}{1-\alpha}. \tag{8.3}$$

Démonstration. Soit A le nombre de jetons proposés acceptés avant le premier refus, $0 \le A \le k$. La passe rend toujours un jeton de plus que A : la correction tirée dans r au premier refus, ou le jeton bonus tiré dans p si tout est accepté. Donc $N = A + 1$. Les i premières positions sont toutes acceptées avec la probabilité α^i, d'où $\mathbb{P}(A \ge i) = \alpha^i$ pour $0 \le i \le k$, et $\mathbb{P}(A \ge i) = 0$ au-delà. Pour une variable à valeurs entières positives, $\mathbb{E}[A] = \sum_{i \ge 1} \mathbb{P}(A \ge i)$; ainsi $\mathbb{E}[N] = 1 + \sum_{i=1}^{k} \alpha^i = \sum_{i=0}^{k} \alpha^i$, somme géométrique qui vaut $(1-\alpha^{k+1})/(1-\alpha)$. ∎

Corollaire 8.4 (La borne de la chaîne). *$\mathscr{E}(\alpha,k) < 1/(1-\alpha)$ pour tout k, et la chaîne atteint la fraction $1-\alpha^{k+1}$ de cette borne. À $\alpha = 0{,}8$, $k = 8$ donne déjà 87 % de la borne : au-delà, seul un meilleur brouillon ou un* arbre *fait monter $\mathscr{E}$.*

Démonstration. $\mathscr{E}(\alpha,k)(1-\alpha) = 1-\alpha^{k+1} < 1$. ∎

Corollaire 8.5 (Le k optimal quand la passe coûte cher). *Avec une passe de coût T et un brouillon de coût t_b par jeton proposé, le débit $\mathscr{E}(\alpha,k)/(T+kt_b)$ est maximal en un $k^\star$ qui croît comme $\log(T/t_b)/\log(1/\alpha)$; pour $T = 8$ s et $t_b = 5$ ms, $k^\star = 25$ à $\alpha = 0{,}8$ et 48 à $\alpha = 0{,}9$ [calc.].*

Démonstration. Le gain marginal du $(k+1)$-ième jeton proposé est α^{k+1} jeton, son coût marginal t_b pour un coût total $T+kt_b$; l'optimum vérifie à peu près $\alpha^{k^\star}\mathscr{E}^{-1} \approx t_b/(T+k^\star t_b)$, d'où $k^\star \approx \log(T/t_b)/\log(1/\alpha)$ à des termes logarithmiques près. Les valeurs numériques viennent de la maximisation exacte de $(1-\alpha^{k+1})/((1-\alpha)(T+kt_b))$ sur les entiers. ∎

Le témoin. L'outil `amortissement.py` simule 20 000 chaînes par cas et compare à l'équation (8.3) : écart inférieur à 4 erreurs-types dans les 9 cas. Avec une formule fausse (sans le jeton bonus), 7 cas sur 9 tombent [mes.]. La figure 8.2 et le tableau 8.1 donnent les valeurs.

8.2.3 Les arbres

Avec un arbre de K nœuds, la passe vérifie toutes les branches à la fois (masque d'attention en arbre) et garde le plus long chemin accepté (figure 8.3). SpecInfer [68] échantillonne les branches, Sequoia [69] construit l'arbre optimal par programmation dynamique sous un budget, SpecExec [70] garde les K continuations les plus probables du brouillon : c'est la variante faite pour le régime déchargé. Sa propriété décisive : $\mathscr{E}$ n'est plus borné par $1/(1-\alpha)$ mais par la couverture de l'arbre. SpecExec rapporte jusqu'à 20,6 jetons acceptés par passe (budget 2 048) sur Llama 2 70B déchargé, et, **sur une RTX 4060 de bureau, 70B en GPTQ : 9,7 jetons par passe, 3,28 jetons/s,** ×4,6 [litt.].

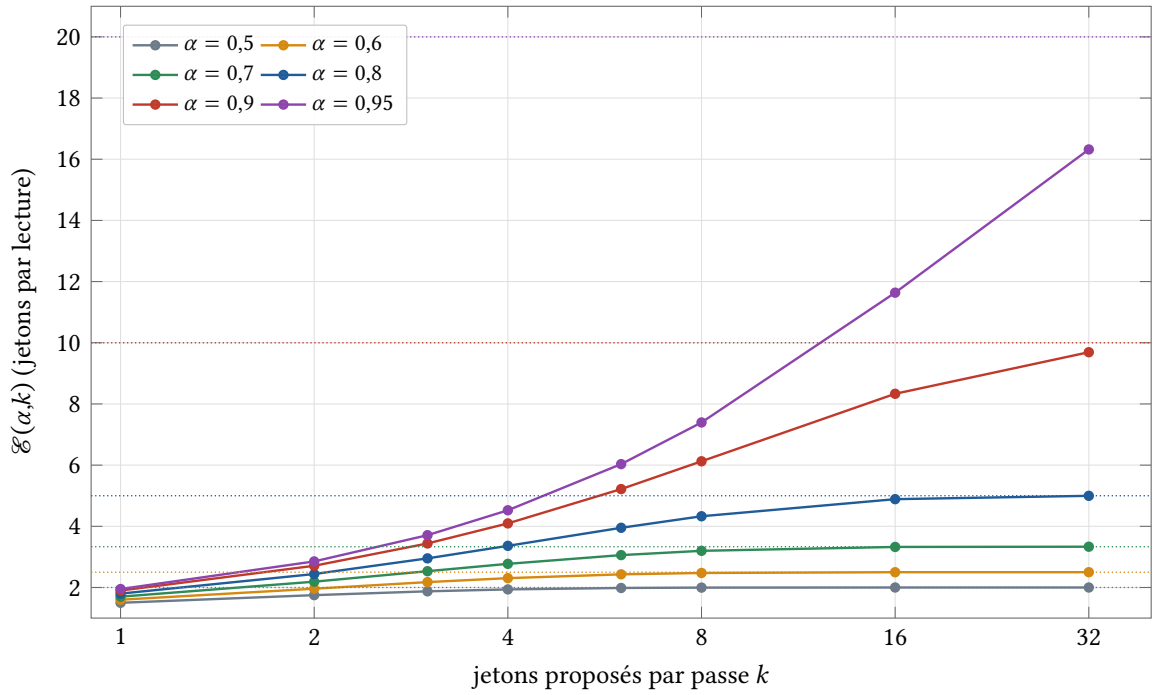


Figure 8.2 : Jetons produits par lecture complète des poids actifs, $\mathscr{E}(\alpha, k)$, et leurs asymptotes $1/(1-\alpha)$ en pointillé (théorème 8.3 et corollaire 8.4) [calc.]. La borne arrive vite : passé $k \approx 8$, une chaîne plus longue ne sert presque plus.

Données : `fondations/journaux/05-sparsite/amortissement-tables.json` (`outils/05-sparsite/amortissement.py`).

8.2.4 Les variantes

Le piège des modèles hybrides. Qwen3.5, 3.6, 3.8 et Qwen3-Next mêlent des couches d'attention complète et des couches d'attention linéaire à règle delta (Gated DeltaNet), dont l'état est une matrice par tête mise à jour à chaque jeton. Un jeton refusé ne se retire pas d'un état récurrent : il faut avoir gardé l'état d'avant. Pour Qwen3.5-9B (32 têtes de valeur, état 128×128 par tête, 24 couches linéaires sur 32) [calc.] :

état par couche $= 32{\times}128{\times}128{\times}4$ octets $= 2$ Mio, état du modèle $= 24{\times}2 = 48$ Mio par position gardée.

Un arbre de 256 nœuds en demande 12 Gio : impossible sur 8 Go. STree [113] vérifie des arbres pour les modèles à espace d'états en exploitant la structure de leurs matrices de transition ; la transition de la règle delta, $\alpha_t(I - \beta_t k_t k_t^\top)$, n'est pas diagonale, et aucune vérification en arbre n'est publiée pour elle.

8.3 Mélanges d'experts et spéculation

Une passe de vérification d'un MoE lit l'union des experts de ses n jetons.

Lemme 8.6 (L'union des experts sous routage indépendant)**.** *Si chaque jeton choisit k_e experts distincts parmi E, uniformément et indépendamment des autres jetons, le nombre moyen d'experts distincts demandés par n jetons dans une couche est*

$$U(n) = E\left(1 - \left(1 - \frac{k_e}{E}\right)^n\right). \tag{8.4}$$

Table 8.1 : Jetons produits par lecture complète des poids actifs, $\mathscr{E}(\alpha,k)$ [calc.].

$\alpha \setminus k$	1	2	3	4	6	8	16	∞
0,5	1,50	1,75	1,88	1,94	1,98	2,00	2,00	2
0,6	1,60	1,96	2,18	2,31	2,43	2,47	2,50	2,5
0,7	1,70	2,19	2,53	2,77	3,06	3,20	3,33	3,33
0,8	1,80	2,44	2,95	3,36	3,95	4,33	4,89	5
0,9	1,90	2,71	3,44	4,10	5,22	6,13	8,33	10
0,95	1,95	2,85	3,71	4,52	6,03	7,40	11,64	20

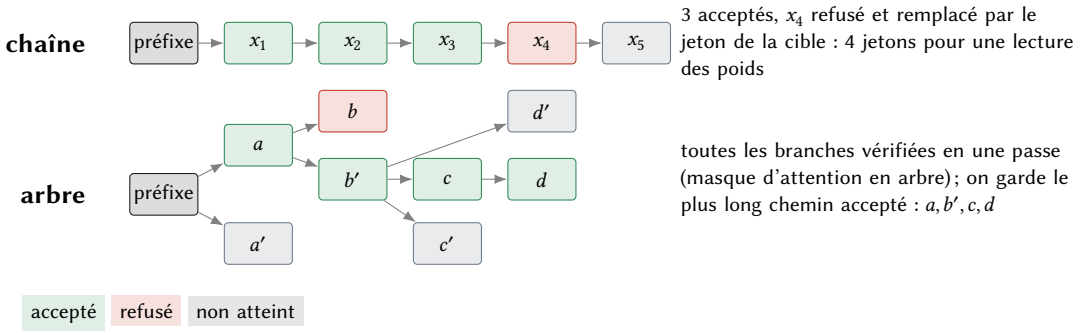


Figure 8.3 : Une passe de vérification en chaîne et en arbre. La chaîne s'arrête au premier refus ; l'arbre vérifie toutes ses branches dans la même passe et garde le plus long chemin accepté.

Démonstration. Un expert donné n'est pas choisi par un jeton avec la probabilité $1 - k_e/E$; par indépendance, il n'est choisi par aucun des n jetons avec la probabilité $(1 - k_e/E)^n$. Par linéarité de l'espérance, le nombre moyen d'experts choisis par au moins un jeton est E fois la probabilité complémentaire. ∎

Proposition 8.7 (Quand la spéculation paie pour un MoE lu du disque)**.** *Si les octets venus du disque sont dominés par les experts routés, une vérification de k jetons proposés réduit les octets lus par jeton produit si et seulement si*

$$\mathscr{E}(\alpha, k) \;>\; \frac{U(k+1)}{k_e}. \tag{8.5}$$

Démonstration. Sans spéculation, un jeton lit k_e experts par couche. La passe de vérification porte sur $k+1$ positions (le préfixe et les k jetons proposés) et lit $U(k+1)$ experts par couche ; elle produit $\mathscr{E}(\alpha,k)$ jetons en moyenne (théorème 8.3). Les octets par jeton produit baissent si et seulement si $U(k+1)/\mathscr{E}(\alpha,k) < k_e$. ∎

Corollaire 8.8 (La spéculation ne paie jamais sur des experts fins indépendants)**.** *Pour $k = 1$, la condition devient $1 + \alpha > 2 - k_e/E$, soit $\alpha > 1 - k_e/E$: $\alpha > 0{,}977$ pour DeepSeek-V4-Flash ($k_e/E = 6/256$), $\alpha > 0{,}980$ pour Qwen3.8-Flash-Next ($10/512$) [calc.]. À la limite des experts fins ($k_e(k+1) \ll E$), $U(k+1)/k_e \to k+1 > \mathscr{E}(\alpha,k)$ pour tout $\alpha < 1$.*

Démonstration. Par le lemme 8.6, $U(2)/k_e = 2 - k_e/E$ et $\mathscr{E}(\alpha,1) = 1 + \alpha$. À la limite, $(1 - k_e/E)^{k+1} \approx 1 - (k+1)k_e/E$, donc $U(k+1)/k_e \to k+1$; or $\mathscr{E}(\alpha,k) = \sum_{i=0}^{k} \alpha^i < k+1$ dès que $\alpha < 1$. ∎

Le régime de Stone (lot de un, poids sur le disque) est exactement celui où Cascade [72] observe que la spéculation ralentit les MoE. La spéculation ne paie donc, pour un MoE fin lu

Table 8.2 : Les variantes du décodage spéculatif : brouillon, gain publié, disponibilité sous Windows sans boîte à outils CUDA [litt.]. llama.cpp [9] (commit `e467e5b`) ne vérifie que des chaînes.

Variante	Brouillon et gain publié	Sous Vulkan
Petit modèle [59, 60]	modèle de la même famille, 10 à 50 fois plus petit ; sans perte	oui (chaîne)
Medusa [61]	têtes supplémentaires sur le dernier état ; ×2,2 à ×3,6	non
EAGLE-2, EAGLE-3 [62, 63]	une couche lisant les états de la cible, arbre dynamique ; jusqu'à ×6,5	chaîne seulement
MTP natif [64, 65]	module de prédiction multi-jetons entraîné avec le modèle ; brouillon gratuit s'il réside en VRAM	oui (`draft-mtp`)
DFlash [66]	diffusion par blocs, un bloc entier par passe ; plus de ×6	oui
N-grammes, recherche dans le contexte [67]	coût nul ; énorme sur la réécriture, presque nul sur la prose	oui
LayerSkip, QSpec [111, 112]	la cible elle-même, couches sautées ou activations 4 bits ; ×1,6 à ×2,2	non
SubSpec [71]	substituts 4 bits résidents des couches déchargées ; 27 à 35 jetons par passe ; Qwen2.5-7B sous 8 Go : 2,68 → 27,1 jetons/s	non (PyTorch)
SpecExec, Sequoia [69, 70]	arbres de centaines de nœuds en régime déchargé ; 10 à 20 jetons par passe	non

du disque, que si l'union réelle est très en dessous de l'indépendance (les jetons voisins partagent leurs experts), ou si l'ensemble d'experts de la vérification est borné : MoE-Spec [73] (budget fixe par couche), AcceptMoE [74] (éligibilité limitée aux experts déjà en cache : ×2,06, −0,27 point de précision), EcoSpec [75] et EVICT [114] (sans perte), DraftExpert [115] [litt.]. L'union réelle sur des jetons de brouillon n'est publiée pour aucun modèle que la machine peut lire.

Prédire les experts ne réduit pas les octets. Mixtral-offloading, ProMoE, HOBBIT (section 7.5) prédisent pour précharger : cela cache la latence du chargement derrière le calcul (proposition 7.3). En régime disque, où le disque travaille en permanence, le préchargement ne rend que le passage de la somme au maximum du théorème 3.2. Seuls le cache et la substitution avec perte réduisent les octets venus du disque.

8.4 La sparsité des activations

Ce qu'il faut pour qu'un zéro devienne un octet non lu. En VRAM : des poids rangés par colonnes et un noyau qui ne charge que les colonnes vivantes (le noyau de TEAL l'exige : `assert weight.stride(1) > 1`) ; aucun noyau de ce type n'existe dans llama.cpp, dont les types quantifiés sont rangés par lignes. Sur le disque : la granularité est la page, et un neurone du FFN de Qwen3.5-9B en 4,5 bits (trois vecteurs de 4 096) pèse environ 6,9 Kio [calc.], deux pages, très en dessous de la taille où ce disque donne son débit (figure 3.1).

Table 8.3 : Spéculation et MoE dont les experts viennent du disque, routage uniforme et indépendant (pessimiste) [calc.].

Modèle	B_t / B_e (Go)	E / k_e	$U(4)/U(1)$	k optimal, $\alpha = 0{,}9$
DeepSeek-V4-Flash Q2_K_XL	6,69 / 2,10	256 / 6	3,86	**0**
Qwen3.8-Flash-Next Q2_K_XL	3,54 / 0,90	512 / 10	3,88	**0**
gpt-oss-120b MXFP4	1,69 / 1,91	128 / 4	3,82	**0**
Kimi-K2.6 Q2_K_XL	8,52 / 6,90	384 / 8	3,88	**0**

Table 8.4 : Les méthodes de sparsité des activations [litt.].

Méthode	Ce qui est creux, gain publié	Condition
Deja Vu [10]	têtes et neurones par jeton, prédits une couche en avance ; ×2	ReLU (OPT)
PowerInfer [11]	neurones chauds sur la carte, froids sur le processeur ; jusqu'à ×11,69	ReLU
LLM in a flash [13]	fenêtre glissante des neurones récents ; regroupement ligne-colonne	ReLU
CATS [116]	seuil sur l'entrée de down ; +15 % de vitesse	—
TEAL [117]	seuil de magnitude sur l'entrée de toutes les matrices ; ×1,53 à 40 %	poids rangés par colonnes
R-Sparse [118]	canaux et composantes singulières ; +43 %	—
TurboSparse [119]	dReLU réentraîné ; ×2 à ×5	réentraînement

8.4.1 La mesure : unions, fenêtre, neurones chauds

L'instrument. `llama_numpy.py` est une passe avant numpy d'un modèle « llama » lu dans le GGUF, écrite de zéro, avec un crochet sur l'entrée de chaque projection. Modèle : SmolLM2-135M-Instruct F16 (30 couches, SwiGLU). Témoin : la perplexité de la passe numpy doit retrouver celle de `llama-perplexity` à moins de 0,5 % (seuil déclaré avant) ; elle donne 22,6768 contre 22,6739 sur 16 fenêtres (0,013 %, écart maximal 0,18 %), et **le témoin tombe quand on le casse** (rotation RoPE par moitiés au lieu des paires voisines : 232,8 contre 10,4) [mes.].

Les unions (figure 8.4) sont sous l'indépendance : la corrélation entre jetons voisins existe, mais elle est faible. Pondérée par les octets des quatre entrées d'un bloc, l'union vaut **0,895 à $n = 4$ et 0,976 à $n = 8$** pour 50 % de sparsité par jeton [calc.] : une vérification de quatre jetons ne lit que 10,5 % de moins qu'une passe dense. **Composée naïvement avec la spéculation, la sparsité par jeton s'évanouit.**

La fenêtre glissante de LLM in a flash réduit le trafic au prix d'une RAM qui tient presque toute la matrice : à RAM égale (72 % des colonnes), un cache statique dense laisse 28 % des octets au disque, la fenêtre $w = 1$ en laisse 21,8 %, un gain de 1,28 seulement. Et **il n'y a pas de loi de puissance des neurones chauds dans un modèle SwiGLU** : pour couvrir 80 % des sélections à 50 % de sparsité, il faut 74 % des canaux de down [mes.]. Le facteur 11,69 de PowerInfer suppose une sparsité de ReLU (plus de 90 % de neurones inactifs) et une loi de puissance que SwiGLU ne donne pas.

8.4.2 Un masque commun au bloc vérifié

La seule manière de garder les octets de la sparsité sous la vérification est que les n jetons d'un bloc **partagent un même masque**. Règle mesurée : pour chaque bloc de n jetons consécutifs, garder les

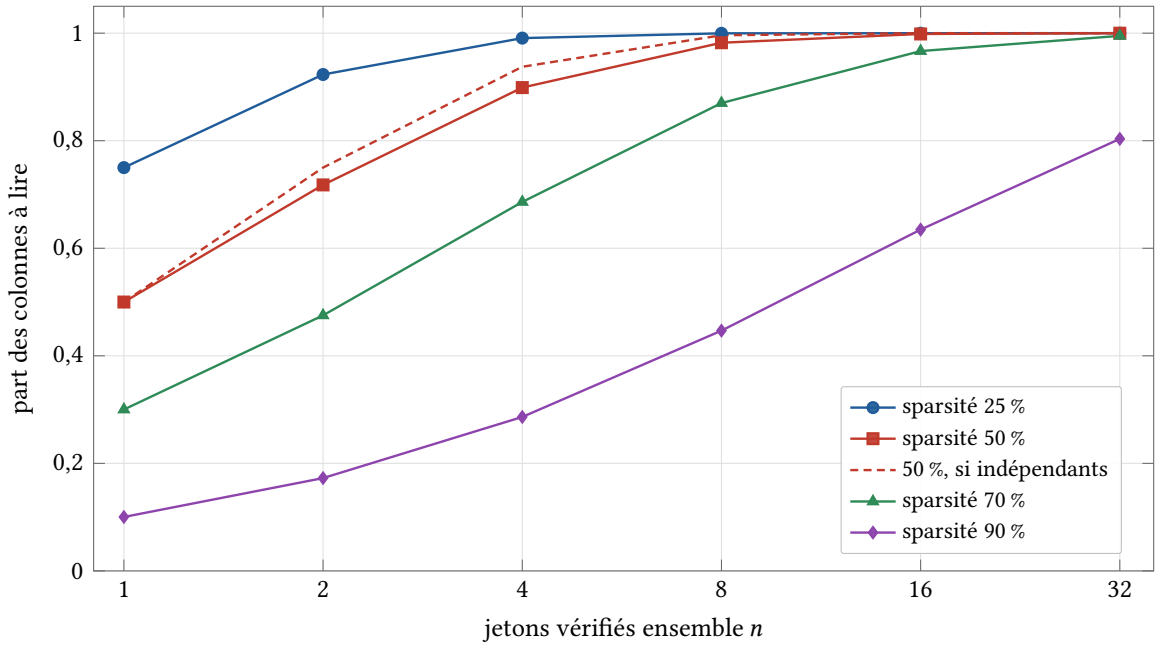


Figure 8.4 : Part des colonnes de down qu'une vérification de n jetons consécutifs doit lire si chacun garde son masque top-k, selon la sparsité par jeton, et la même union si les masques étaient indépendants (SmolLM2-135M, 2 048 jetons) [mes.]. À 50 % de sparsité, quatre jetons lisent déjà 90 % des colonnes.

Données : `fondations/journaux/05-sparsite/stats-smol.json`.

$(1-s)D$ canaux d'énergie $\sum_i v_{ij}^2$ la plus grande, éventuellement pondérée par $\|W_{:,j}\|^2$, la norme de la colonne qu'on renonce à lire.

Table 8.5 : Perplexité de SmolLM2-135M sous différentes règles de sparsité, WikiText-2, 1 020 positions appariées [mes.].

Règle	colonnes lues par passe de n jetons	perplexité	écart
aucune (dense)	100 %	17,86	—
top-k par jeton, $s = 25$ %	75 % à $n = 1$; ≈ 99 % à $n = 4$	18,24	+2,1 %
top-k par jeton, $s = 50$ %	50 % à $n = 1$; 89,5 % à $n = 4$	27,09	+51,6 %
masque commun, bloc de 2, pondéré, $s = 25$ %	75 % pour $n \leq 2$	19,84	+11,0 %
masque commun, bloc de 4, pondéré, $s = 25$ %	75 % pour $n \leq 4$	23,96	+34,1 %
masque commun, bloc de 2, pondéré, $s = 50$ %	50 % pour $n \leq 2$	62,87	×3,52
masque commun, bloc de 4, pondéré, $s = 50$ %	50 % pour $n \leq 4$	301,0	×16,9
masque commun, bloc de 16, pondéré, $s = 50$ %	50 % pour $n \leq 16$	4 276	×239

Données : `fondations/journaux/05-sparsite/ppl-smol.json`.

Sur un SwiGLU dense sans réentraînement, sparsité des activations et vérification spéculative ne se composent pas (tableau 8.5) : chaque jeton garde son masque et la passe lit presque tout, ou les jetons partagent un masque et la qualité s'effondre. Le diagnostic le confirme : la part de l'énergie d'un jeton gardée à densité 0,5 passe de 0,956–0,986 (masque par jeton) à 0,805–0,918 (bloc de 16), et l'énergie perdue, composée sur 30 couches et 4 entrées, fait exploser la perplexité ; un masque aléatoire, témoin, garde 0,499 à 0,502. Réserve : 135 millions de paramètres ; TEAL rapporte des pertes plus faibles à 7–70 milliards.

8.5 Le calcul pour la machine de Stone

8.5.1 Les taux d'acceptation, mesurés

Le « même premier jeton » de `llama-perplexity --kl-divergence` est la part des positions où l'argmax du brouillon égale celui de la cible, le texte servant de préfixe : c'est le taux d'acceptation glouton de la première position proposée, plutôt une borne basse (en génération réelle, le préfixe est le texte de la cible, en général plus prévisible).

Table 8.6 : Taux d'acceptation gloutons mesurés et jetons par passe qu'ils impliquent (théorème 8.3) [mes.] [calc.].

Brouillon → cible	α	KLD	$\mathscr{E}(\alpha,8)$	$1/(1-\alpha)$
témoin : 4B Q4_K_M contre lui-même	**100,000 %**	0	—	—
Qwen3.5-4B Q4_K_M → Qwen3.5-9B Q4_K_M	76,8 ± 0,9 %	0,279	3,91	4,31
4B IQ4_XS (4,76 bits) → 4B Q4_K_M	91,9 ± 0,3 %	0,029	6,57	12,35
4B IQ3_XXS (3,60 bits) → 4B Q4_K_M	82,1 ± 0,4 %	0,148	4,64	5,59
4B IQ2_XXS (2,91 bits) → 4B Q4_K_M	65,8 ± 0,5 %	0,608	2,86	2,92
9B Q2_K naïf (3,41 bits, sans imatrix) → 9B Q4_K_M	34,0 ± 1,0 %	2,60	1,52	1,52

Données : `fondations/journaux/05-sparsite/alpha-*.log`, `temoin-alpha-4B-contre-lui-meme.log`, étude 02 pour les copies quantifiées du 4B.

Le témoin tombe à 100 % exactement quand la cible est le brouillon : l'outil mesure bien l'accord (tableau 8.6). Un brouillon de la même famille, deux fois plus petit, rend au plus 4,3 jetons par passe en chaîne ; une copie quantifiée de la cible elle-même s'accorde bien mieux (91,9 % à environ 4,8 bits), d'où la longueur d'acceptation de SubSpec. **Le substitut naïf s'effondre** : le Q2_K du 9B requantifié sans matrice d'importance fait passer la perplexité de 8,47 à 105 et l'accord à 34 %. Un substitut se fabrique avec la meilleure quantification disponible, pas la plus simple.

8.5.2 Le substitut résident, transposé

SubSpec loge en VRAM une copie 4 bits de la cible et fait passer la cible 16 bits par le lien depuis 128 Go de RAM. Chez Stone, 16 Go de RAM imposent une cible déjà quantifiée (environ 4,8 bits) et 8 Go de VRAM un substitut très bas. Avec les accords mesurés, un budget de 6 Go de VRAM pour le substitut et une cible Q4_K_M en RAM, l'étude calcule pour une cible de 14B (9,1 Go, passe de 1,42 s, lien chargé à 6,38 Go/s) : 0,70 jeton/s sans spéculation, **1,75 jeton/s (**×2,5**)** avec le meilleur substitut qui loge (IQ2_XXS, 5,1 Go, $\alpha = 0{,}66$, $k^\star = 7$) [calc.]. Un substitut à 2 bits qui garderait 85 % d'accord porterait le 14B à environ 3,6 jetons/s. **C'est le point exact où la quantification et la spéculation se multiplient** : la quantification extrême ne sert plus seulement à lire moins, elle fabrique le brouillon.

8.5.3 Les modèles denses lus du disque

Dans les cas disque, la spéculation multiplie le débit par $\mathscr{E}$ presque exactement (tableau 8.7) : $k^\star$ est grand, le brouillon négligeable (×5 à $\alpha = 0{,}8$, ×10 à $\alpha = 0{,}9$). La quantification et la spéculation se multiplient : passer de 4,8 à 2,3 bits sort presque le modèle du disque, la spéculation multiplie ensuite par $\mathscr{E}$, et elle n'ajoute aucune perte de qualité.

Table 8.7 : Jetons/s d'un modèle dense en partie sur disque, sans et avec spéculation (brouillon résident, 5 ms par jeton) ; disque supposé entre 3 et 7 Go/s par l'étude, le débit mesuré (2,82 Go/s) étant sous la borne basse [calc.].

Cas	disque	$T(n)$	sans spéc.	$\alpha = 0{,}8$	$\alpha = 0{,}9$	arbre, 10/passe
70B Q4 (42 Go), 24 Go sur disque	3 Go/s, en série	9,76 s	0,10	0,50	0,99	1,02
idem	7 Go/s, recouvert	3,43 s	0,29	1,40	2,72	2,87
70B ≈2,3 bits (20 Go), 2 Go sur disque	3 Go/s, en série	2,43 s	0,41	1,96	3,76	4,04
idem	recouvert	1,72 s	0,58	2,72	5,15	5,64

Données : `fondations/journaux/05-sparsite/amortissement-tables.json`.

8.6 Ce qui manque dans le monde

1. **La spéculation avec des poids lus du NVMe quand la RAM est plus petite que le modèle.** SpecExec (140–192 Go de RAM), SubSpec (128 Go), Sequoia, TriForce lisent les poids depuis la RAM ; sur 16 Go et un disque à 2,82 Go/s, c'est un facteur 10 non mesuré.
2. **Des arbres sous Windows sans CUDA** : llama.cpp vérifie des chaînes, plafonnées à $1/(1-\alpha)$ (corollaire 8.4).
3. **Les courbes d'union d'experts réelles des MoE fins de 2026 sur des jetons de brouillon** : toute la rentabilité dépend de l'écart à l'équation (8.4).
4. **Le partage de 8 Go de VRAM entre brouillon et cible** : chaque Go donné au brouillon repart sur le lien ou le disque ; l'optimum n'est publié par personne.
5. **Des arbres de vérification pour les hybrides à règle delta** (48 Mio d'état par nœud).

Recommandations, pour l'arbitre. Le brouillon le moins cher et le plus juste est la cible elle-même, quantifiée très bas et résidente, ou son module MTP natif ; vérifier des arbres, pas des chaînes ; pour les MoE, ne vérifier que dans les experts déjà en mémoire et mesurer la perte ; ne pas investir dans la sparsité des activations par jeton pour le régime disque sur des modèles SwiGLU.

Ce qui pourrait être faux

- **Le modèle de mesure de la sparsité est petit** (135 M) ; les grands modèles sont plus creux, et la corrélation entre jetons voisins peut y être plus forte.
- **Le masque top-k par jeton n'est pas exactement TEAL** (seuils calibrés, densité variable) ; il a la même densité moyenne.
- **L'hypothèse α constant et indépendant** surestime $\mathscr{E}$ pour les longues chaînes.
- **Le routage indépendant** est pessimiste ; si les jetons voisins partagent beaucoup d'experts, « k optimal = 0 » tombe.
- $F = 20$ **Tflop/s** n'est pas mesuré ; $n^\star$ est un ordre de grandeur.
- **Le « même premier jeton » n'est pas exactement** α : forçage par un texte, première position seulement, glouton.
- **Les accords des substituts viennent du 4B** ; un 14B se quantifie mieux à bits égaux.
- **Les machines étaient partagées** : les perplexités n'en dépendent pas, les temps si.

Chapitre 9

Les modèles visés

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, la mission parle d'un « modèle ultra lourd quelconque » : il fallait dire lesquels existent, ce qu'ils lisent par jeton, sous quelle licence, et comment juger troncons contre une base. **Ce chapitre établit** un inventaire de 49 dépôts officiels lus dans leurs en-têtes, sans télécharger un seul poids, validé contre les chiffres des éditeurs par un témoin qui tombe ; les octets lus par jeton des fichiers exacts candidats ; leurs plafonds sur la machine, recalculés au débit mesuré du disque ; les tendances de 2026 qui jouent pour troncons ; et un protocole de jugement dont une mesure a déjà corrigé une hypothèse. **Il reste** la trace de routage d'un modèle de 2026 et le premier banc de base sur gpt-oss, téléchargé et vérifié. Source : `fondations/06-modeles.md`, `fondations/journaux/06-modeles/`.

9.1 Ce que « ultra lourd » veut dire ici

La machine offre environ 6,5 Go de VRAM et 11 Go de RAM pour des poids (hypothèses dites : 8 188 Mio moins l'écran, le contexte et un cache KV court ; 15,6 Go moins le système), soit environ 17,5 Go, et 404 Go de disque libre. Un modèle est « ultra lourd » quand son fichier dépasse nettement ces 17,5 Go ; il devient intéressant pour troncons quand il en est un multiple (trois fois et plus) **et** que ses octets lus par jeton restent petits. Un modèle dense lit tous ses poids à chaque jeton : à 4 bits, Llama-3.3-70B lit 34,8 Go par jeton [calc.] ; le terrain est le mélange d'experts.

9.2 L'inventaire, lu dans les en-têtes

La méthode. Un fichier safetensors commence par 8 octets de longueur puis un JSON qui donne le type et la forme de chaque tenseur. Deux requêtes HTTP à plage par fragment suffisent : on obtient la forme exacte de chaque tenseur d'un modèle de 2,8 billions de paramètres sans télécharger ses poids (`inventaire_modeles.py`). L'outil range chaque tenseur dans une catégorie : experts routés (un sur E lu par choix du routeur), toujours lus, plongement (une ligne par jeton), MTP (lus seulement en décodage spéculatif), recherche (tables Engram ou de n-grammes), hors texte. Paramètres actifs par jeton = toujours lus + $\sum_{\text{couches}}(k_e/E)\times$ experts routés de la couche. Pour les fichiers GGUF qu'on téléchargerait, l'outil lit l'en-tête du fichier exact (d'après la spécification publique [120]) ; son témoin intégré, la somme des tailles de tenseurs plus l'en-tête égale la taille du fichier, tient à l'octet pour les 8 fichiers lus.

Le témoin, et sa cassure. Second chemin : les chiffres publiés par l'éditeur. Seuils déclarés dans l'outil avant la première exécution : total à ±3 % (les éditeurs arrondissent au milliard ; 3 % ne laisse passer ni un facteur 2 ni un doublon de fichiers), actifs à ± max(0,6 G; 10 %).

– **Passe 3 (la bonne)** : 37 accords sur 38 ; le seul échec, Qwen3.8-Flash-Next (6,67 G actifs contre « 6B »), s'explique par la tête de sortie (0,636 G), que nous comptons : sans elle, 6,03 G. Le seuil reste ce qu'il était et l'échec est écrit.
– **Passe 2 (la cassure)** : le même témoin trouve 6 échecs, et ce sont 6 vrais bogues que la passe 3 corrige : Mistral compté deux fois (deux copies des mêmes poids à la racine), MiMo V2.6 et Kimi K3 comptés à moitié (FP4 empaqueté en octets non reconnu), Kimi K3 à 50 experts actifs au lieu de 16 (la clé `top_k` de sa configuration est un paramètre d'échantillonnage), Step-3.5-Flash à 196 G actifs, Qwen3.8 à 57,9 G actifs (la table de n-grammes comptée comme toujours lue).

Une première passe d'un prédécesseur donnait ~~DeepSeek-V4-Flash : 149,2 G au total, 10,6 G actifs~~ (FP4 empaqueté non reconnu, tête de sortie manquée) ; corrigé : 284,3 G hors MTP et 13,27 G actifs, contre 284 G et 13 G dans l'article [121].

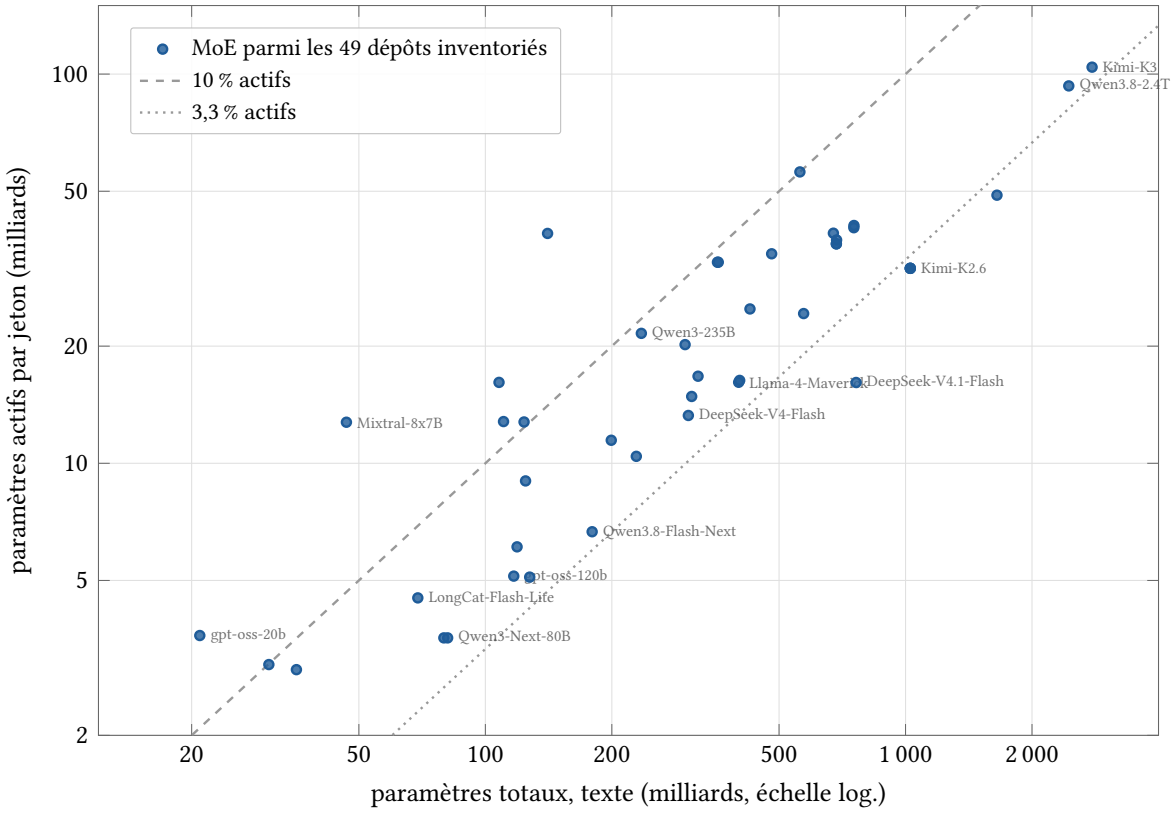


Figure 9.1 : Paramètres totaux (texte) contre paramètres actifs par jeton des mélanges d'experts inventoriés dans leurs en-têtes [mes.]. La plupart des modèles de 2025–2026 n'activent que 3 à 10 % de leurs paramètres par jeton : c'est la propriété qui rend un modèle « ultra lourd » jouable sur 8 Go.

Données : `fondations/journaux/06-modeles/octets-par-jeton.json` (`outils/inventaire_modeles.py`, passe 3).

9.3 Octets lus par jeton

Avec A les paramètres toujours lus, R les paramètres d'experts routés lus par jeton et b les bits par poids des experts, deux formules :

$$O_{\text{uniforme}} = (A + R)\,\frac{b}{8}, \qquad O_{\text{mixte}} = A\,\frac{8{,}5}{8} + R\,\frac{b}{8}, \tag{9.1}$$

Table 9.1 : Une sélection des mélanges d'experts inventoriés : total texte, actifs par jeton, architecture, licence [mes.] (littérature quand c'est dit). L couches, E experts routés, k_e actifs, P experts partagés.

Modèle	total	actifs	architecture, format publié	licence
gpt-oss-20b [122]	20,9 G	3,61 G	L 24, E 32, k_e 4 ; MXFP4	Apache-2.0
gpt-oss-120b [122]	116,8 G	5,13 G	L 36, E 128, k_e 4 ; GQA 64/8, fenêtre glissante une couche sur deux ; experts MXFP4 natifs	Apache-2.0
Qwen3-Next-80B-A3B	81,3 G	3,56 G	L 48, E 512, k_e 10 + P ; Gated DeltaNet 3 : 1	Apache-2.0
Qwen3.5-122B-A10B	124,6 G	9,01 G	L 48, E 256, k_e 8 + P, MTP	Apache-2.0
Qwen3.8-Flash-Next	179,6 G	6,67 G	L 48, E 512, k_e 10 + P ; table de n-grammes de 51,2 G	qwen-community
DeepSeek-V4-Flash-0731 [121]	284,3 G	13,27 G	L 43, E 256, k_e 6 + 1 ; 3 couches routées par hachage ; experts FP4	MIT
DeepSeek-V4.1-Flash	762,7 G	16,13 G	E 384, k_e 6 + 1 ; Engram de 196,6 G [123]	MIT
Kimi-K2.6 [124]	1 026 G	31,69 G	L 61, E 384, k_e 8 + 1, MLA ; INT4 natif	MIT modifiée
Kimi-K3	2 780 G	104,19 G	L 93, E 896, k_e 16 + 2 ; MXFP4 natif	kimi-k3
GLM-4.5-Air [125]	110,5 G	12,80 G	L 46, E 128, k_e 8 + 1, MTP	MIT
Qwen3-235B-A22B [126]	235,1 G	21,57 G	L 94, E 128, k_e 8	Apache-2.0
LongCat-Flash-Lite	69,1 G	4,51 G	L 14, E 256, k_e 12 ; n-grammes de 31,4 G	MIT

la seconde étant le choix réel des quantifications dynamiques (toujours-lus en Q8_0). Le plongement, les tables de recherche et la MTP sont exclus. Second chemin : les en-têtes des fichiers exacts (tableau 9.2).

Table 9.2 : Octets lus par jeton, en-têtes GGUF des fichiers exacts [mes.] contre la formule mixte [calc.].

Fichier	toujours lus + experts lus (exact)	formule mixte	écart
gpt-oss-120b MXFP4	1,69 + 1,91 = 3,60 Go	3,55 Go	−1,4 %
DeepSeek-V4-Flash UD-Q2_K_XL	6,69 + 2,10 = 8,79 Go	8,83 Go	+0,5 %
Qwen3.8-Flash-Next UD-Q2_K_XL	3,54 + 0,90 = 4,44 Go	5,17 Go	+16 %
Kimi-K2.6 UD-Q2_K_XL	8,52 + 6,90 = 15,42 Go	16,49 Go	+7 %

La formule est juste là où les toujours-lus sont vraiment en 8 bits ; elle surestime Qwen3.8 (une part de ses toujours-lus est en 4 à 6 bits) et se trompe de sens sur Kimi. **Pour décider, on prend l'en-tête du fichier, jamais la formule.** Deux lectures des tailles publiées : gpt-oss-120b pèse environ 63 Go quel que soit le niveau de quantification, parce que ses experts restent en MXFP4 et que seule l'attention change de type ; et pour les modèles nés en 4 bits (Kimi-K2.6, DeepSeek-V4-Flash), le niveau « 8 bits » pèse presque autant que le « 4 bits » : il n'y a rien au-dessus du 4 bits natif à garder.

9.4 Le juge physique

Le modèle de coût de l'étude (`modeles_physique.py`, déclaré avant tout usage) : les toujours-lus résident sur la carte s'ils tiennent dans 6,5 Go ; les experts routés se partagent un cache de 11 Go de RAM plus la VRAM restante, et le disque ; le routage est supposé uniforme et sans mémoire, donc le succès vaut $h = C/E_{\text{octets}}$, le pire cas du cache ; les experts trouvés en RAM sont calculés par le processeur à 41,6 Go/s, les manqués lus sur le disque ;

$$t_{\text{opt}} = \max\Big(\frac{A}{256}, \frac{hR}{41{,}6}, \frac{(1-h)R}{W_{\text{disque}}}\Big), \qquad t_{\text{pess}} = \text{la somme}. \tag{9.2}$$

L'étude avait pris pour W_{disque} le plafond du lien (7,877 Go/s). Le chapitre 3 a mesuré 2,82 Go/s : on recalcule ici la colonne qui en dépend.

Table 9.3 : Les fichiers candidats et leur plafond de jetons/s, routage uniforme, au plafond du lien (étude 06) et au débit mesuré du disque [calc.].

Fichier	taille	toujours lus	experts lus	h	plafond, 7,877	plafond, 2,82
gpt-oss-20b MXFP4	12,1 Go	—	1,27 Go	100 %	32,7	32,7
gpt-oss-120b MXFP4	63,4 Go	1,69 Go	1,91 Go	26 %	~~5,6~~	**2,0**
Qwen3.8-Flash-Next UD-Q2_K_XL	78,9 Go	3,54 Go	0,90 Go	30 %	~~12,6~~	**4,5**
Qwen3.8-Flash-Next UD-Q4_K_XL	111,3 Go	4,83 Go	1,50 Go	16 %	~~6,3~~	**2,2**
DeepSeek-V4-Flash UD-Q2_K_XL	96,8 Go	6,69 Go	2,10 Go	12 %	~~4,3~~	**1,5**
DeepSeek-V4-Flash UD-Q4_K_XL	155,1 Go	—	3,45 Go	7 %	~~2,5~~	**0,9**
Kimi-K2.6 UD-Q2_K_XL	340,5 Go	8,52 Go	6,90 Go	3 %	~~1,2~~	**0,4**

Données : `fondations/journaux/06-modeles/plafonds-fichiers.md`, recalculé par l'équation (9.2) avec $W_{\text{disque}} = 2{,}82$ Go/s.

Les plafonds barrés supposaient le disque au plafond de son lien ; au débit mesuré, ils sont 2,8 fois plus bas, car le disque borne partout où le cache ne sert pas tout (tableau 9.3). Le classement ne change pas : **le meilleur rapport utile sur physique est Qwen3.8-Flash-Next UD-Q2_K_XL, puis gpt-oss-120b**, qui a l'avantage décisif d'une référence sans perte (ses poids publiés sont déjà en 4 bits). DeepSeek-V4-Flash déborde la carte par ses toujours-lus (6,69 Go en UD-Q2_K_XL) ; Kimi-K2.6 tient sur le disque (340 Go sur 404) mais plafonne sous un demi-jeton par seconde : c'est la démonstration « un billion de paramètres sur un portable », pas un outil. Le levier dominant est le taux de succès du cache d'experts (figure 7.4).

9.5 Ce qui a changé en 2026 et sert troncons

1. **Les tables à recherche, faites pour l'extérieur de la carte.** L'article Engram [123] pose le principe : des lignes adressées par un hachage déterministe des n-grammes de jetons, donc connues avant le calcul et préchargeables ; il mesure une table de 100 G en DRAM hôte et propose, sans le mesurer, un étage NVMe. Trois modèles ouverts portent une telle table (26 à 45 % de leurs paramètres) : DeepSeek-V4.1-Flash, Qwen3.8-Flash-Next et LongCat-Flash-Lite. Un jeton de Qwen3.8 y lit 16 lignes, environ 1,4 Kio utiles dans un fichier de 28,8 Go [calc.] : moins d'un millième du budget disque d'un jeton. llama.cpp sait déjà laisser une telle table sur le disque (TENSOR_READ_LAZY, préchargement des seules lignes demandées).

2. **Les poids nés en 4 bits** : gpt-oss (MXFP4), Kimi K2-Thinking et K2.6 (INT4 par entraînement conscient de la quantification), Kimi K3 (MXFP4), DeepSeek V4 et MiMo V2.6 (experts FP4). Sur ces modèles, la seule compression sans perte qui reste est un codage entropique des codes de 4 bits et de leurs échelles ; les échelles E8M0 de V4-Flash pèsent 9,3 Go pour 148,2 Go de codes FP4 (6,3 %) [mes.].
3. **Le routage par hachage** : DeepSeek V4 route ses 3 premières couches par une table jeton → experts ; ces experts (environ 7 % des octets d'experts) sont connus dès que le jeton est tiré, une passe complète à l'avance. Personne ne s'en sert pour précharger.
4. **La prédiction multi-jetons intégrée** (DeepSeek, GLM, Qwen, MiMo) : un brouillon gratuit, sous la condition du proposition 8.7.
5. **L'attention linéaire hybride** (Gated DeltaNet, KDA) : le cache KV cesse d'être une contrainte à contexte court sur 8 Go ; le coût est dans les experts.

9.6 Le protocole de jugement

La base. llama.cpp b11515 tel quel, chargé en mmap (le seul mode qui démarre pour un fichier plus gros que la RAM), toutes les couches sur la carte et les experts en RAM ou en mmap (`--cpu-moe`, ou `--n-cpu-moe N` réglé au mieux) ; on publie les deux points.

Une passe. `modeles-base.py` lance une complétion sur un texte réel fixé (les 3 000 premiers octets de wikitext-2 test, empreinte SHA-256 notée), 256 jetons, glouton, et relève les jetons par seconde, la mémoire du processus (de pointe et au dernier relevé, toutes les 100 ms), la VRAM, les octets lus sur le disque (compteur système lu par l'API PDH sous son nom anglais, intégré, après 10 s de repos pour chiffrer la charge des autres) et l'empreinte du texte produit. Pourquoi pas `llama-bench` : sa génération part de jetons sans sens, alors que pour un MoE hors mémoire, le choix des experts, donc les octets lus, dépend du contenu.

Ce que les témoins ont déjà appris. L'hypothèse de départ, ~~le témoin de justesse d'un moteur sans perte est l'égalité des jetons gloutons avec llama.cpp~~, a été démentie par la mesure : llama.cpp lui-même, sur le même fichier, produit un texte glouton différent selon le moteur de calcul (bifurcation vers le 25e jeton entre la carte et le processeur). Trois niveaux, mesurés sur Qwen3.5-4B :

- **plancher de l'instrument** (même moteur, la carte contre ses propres logits) : divergence KL moyenne nulle, maximum $6{,}8 \cdot 10^{-5}$, premier choix identique à 100 % ;
- **écart entre deux moteurs justes** (processeur seul contre carte, mêmes poids) : KL moyenne 0,0055, 99,9^{e} centile 0,187, premier choix identique à 95,96 % : l'ordre de grandeur d'un pas de quantification ;
- **l'instrument tombe quand il doit** : Qwen3.5-9B contre la base du 4B, KL moyenne 0,294, premier choix identique à 76,9 %.

Règle qui en découle : **un moteur sans perte se juge contre une base du même étage de calcul**, et doit y retomber dans le plancher de l'instrument (KL au plus $7 \cdot 10^{-5}$, premier choix à 100 %). C'est pourquoi gpt-oss vient en premier : son MXFP4 est le poids d'origine, donc toute divergence au-dessus du plancher du même étage est une erreur de troncons, pas un effet de quantification.

Qualité et décision. Perplexité wikitext-2 (40 fenêtres de 512), divergence KL contre les logits de référence, HellaSwag, Winogrande, MMLU et ARC, identiques pour la base et pour troncons ; validés sur Qwen3.5-4B (perplexité 8,7259 ± 0,2295, déterministe ; HellaSwag 72,50 % ; Winogrande 67,50 ±

2,34). Séries ABBA répétées au moins deux fois ; une différence n'est réelle que si elle dépasse deux fois le plus grand bruit et 3 %. Pour une variante avec perte, pas de seuil arbitraire : on trace la KL moyenne contre les octets lus par jeton pour les quantifications standard du même modèle, et la variante gagne si elle passe sous cette frontière. Le rendement physique (jetons/s mesurés sur plafond calculé) se publie avec chaque résultat.

9.7 Les téléchargements

Avec l'accord de Stone (« au nom de la science tu as mon accord », 2026-10-15), l'orchestrateur a choisi les deux premiers fichiers recommandés par l'étude, et ils sont sur la machine, empreintes SHA-256 vérifiées contre Hugging Face :

- **gpt-oss-20b MXFP4** [127] : 12 109 566 624 octets, Apache-2.0 ; même architecture et même format que le 120b, mais tient en RAM : on y développe le moteur et on compare, jeton pour jeton, la version en mémoire et la version en flux ;
- **gpt-oss-120b MXFP4** [128] : 63 387 346 208 octets (45 reprises après coupures du serveur), Apache-2.0 ; 3,6 fois la mémoire utile, 5,1 G actifs, poids d'origine : **le premier ultra lourd**.

Suivent, dans l'ordre recommandé : Qwen3.8-Flash-Next UD-Q4_K_XL (111 Go, table de n-grammes) et DeepSeek-V4-Flash UD-Q2_K_XL (97 Go, routage par hachage, experts nés en FP4) ; Kimi-K2.6 seulement après une preuve sur les précédents.

Les licences. Apache-2.0 et MIT ne restreignent rien pour une recherche personnelle au Canada ; les licences modifiées (Kimi, Qwen3.8, GLM-5.3, Llama) n'imposent un affichage ou un accord séparé qu'au-delà de seuils de revenus ou d'utilisateurs ; aucune n'exclut le Canada (lecture d'un non-juriste, pas un avis juridique).

9.8 Ce qui manque dans le monde

1. **Aucune mesure publiée d'une table à recherche servie depuis un NVMe grand public**, alors que trois modèles ouverts y mettent 26 à 45 % de leurs paramètres.
2. **Aucune donnée de localité du routage, jeton par jeton, sur texte réel, pour les modèles de 2026** (512 experts chez Qwen3.5 et 3.8, 896 chez Kimi K3).
3. **Le routage par hachage de DeepSeek V4 n'est exploité par personne pour précharger.**
4. **DeepSeek V4.1-Flash n'a pas de base de comparaison locale** (pas d'architecture dans llama.cpp au 2026-10-15).
5. **Pas de banc standard « hors mémoire »** qui publie octets lus par jeton et rendement physique.
6. **La compression sans perte des modèles nés en 4 bits** est la seule voie sans perte qui reste sur gpt-oss, Kimi, DeepSeek V4 et MiMo.

> **Ce qui pourrait être faux**
>
> - **Les deux capacités supposées** (6,5 Go de VRAM, 11 Go de RAM pour des poids) gouvernent h et les plafonds ; elles se remplacent par des mesures au premier essai.
> - **Le calcul n'est pas gratuit** : le processeur ne déquantifie peut-être pas à 41,6 Go/s, et la RAM mesurée ne livre que 15,6 Go/s (section 3.4).
> - **Le routage uniforme** est pessimiste par construction ; ce n'est pas une prévision.
> - **La définition des actifs** inclut la tête de sortie et exclut le plongement et la MTP ; d'autres

comptes donnent ±0,5 à 1 G.
- **Les miroirs** (Llama 4 lu sur des copies, Llama 3.1 405B sur Hermes-3) et les cartes des éditeurs peuvent se tromper.
- **La validation du juge porte sur un seul modèle**, hybride ; sur gpt-oss (attention classique), l'écart entre moteurs peut être autre.
- **Les licences** : résumé d'un non-juriste.

Troisième partie

Vers le système

Chapitre 10

Architecture : les candidates, la décision, ce qui vient

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, l'équation du jeton (chapitre 4) et quatre leviers mesurés. Aucune architecture n'était choisie. **Ce chapitre établit** trois architectures candidates, opposées par leur angle et calculées sur les mêmes débits (`fondations/07-ce-qui-manque.md`, § 4) ; le mécanisme neuf de la deuxième, le raffinement exact d'un code 4 bits natif, avec sa proposition démontrée et sa mesure sur les poids réels ; la correction d'un chiffre qui l'avantageait à tort, barrée à sa place ; le débat des deux partisans et la décision de l'arbitre ; et les premières mesures du modèle visé, encore provisoires. **Il reste** les chapitres d'implémentation (le jalon J1) et de résultats, qui ne s'écrivent qu'avec un programme qui tourne. Sources : `fondations/07-ce-qui-manque.md`, `decisions/partisan-A.md`, `decisions/partisan-B.md`, `decisions/ARCHITECTURE.md` (commit `f0b0fea`), `mesures/` (agent de mesure, étude non rendue au moment où ce chapitre s'écrit).

10.1 Trois angles

Les trois candidates partagent la machine, le juge (section 9.6) et l'équation du jeton ; toutes font décoder la carte (corollaire 3.6). Elles s'opposent par l'angle :

A réduit les octets *sans rien perdre* et compte sur la carte seule ;
B échange de la précision, *là seulement où l'octet est cher* : au défaut de cache ;
C *lit moins souvent* au lieu de lire moins : une lecture des poids amortie sur plusieurs jetons par une vérification en arbre.

La figure 10.1 les place sur le même axe, avec la base estimée de llama.cpp.

10.2 A : tout par la carte, octets codés de bout en bout, sans perte

Le modèle. gpt-oss-120b MXFP4, déjà sur la machine, empreinte vérifiée ; jumeau de développement : gpt-oss-20b (12,1 Go, plus gros que la VRAM lui aussi). Aucun téléchargement.

Le chemin des octets (figure 10.2). Le disque livre les experts codés par le rANS de l'étude de l'entropie ($r = 1{,}1026$), à la taille d'une matrice d'expert : dans les fichiers de gpt-oss, l'expert est la dimension la plus lente de chaque tenseur, donc chaque matrice d'expert est un bloc contigu de 4 406 400 octets, lisible d'un seul tenant sans réécrire le fichier [mes.] (`fondations/journaux/07-synthese/inventaire-gptoss.json`). La RAM épinglée tient les tampons d'arrivée et un cache

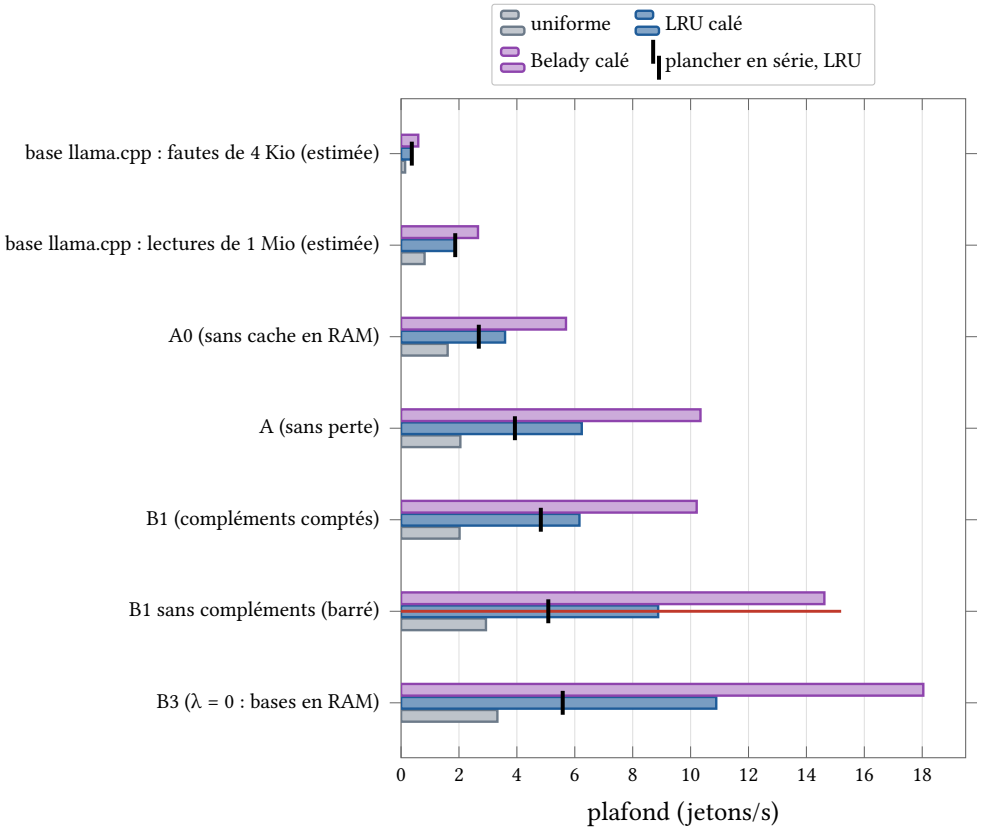


Figure 10.1 : Plafonds des architectures candidates sur gpt-oss-120b, trois scénarios de localité (hypothèse 4.4) ; le trait noir est le plancher en série du scénario LRU [calc.]. La rangée barrée en rouge est le chiffre de l'étude 07 pour B1, qui ne comptait pas les compléments (section 10.3.3). La base llama.cpp est estimée, pas mesurée.

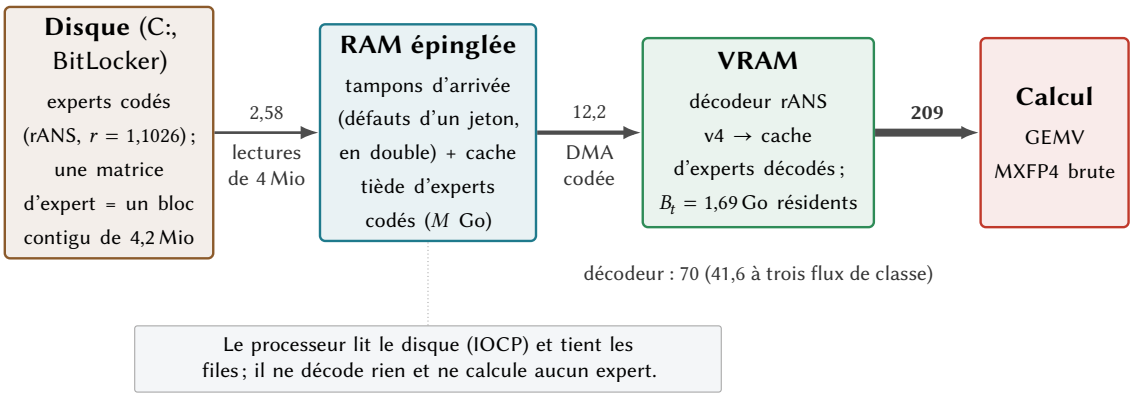
Données : `fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv` et `decisions/journaux/csv/b-complements-gptoss120b.csv`, tirés par `memoire/outils/donnees_figures.py`.

tiède d'experts *codés* ; la DMA porte les octets codés ; la carte décode à l'arrivée et garde un cache d'experts *décodés*, parce que le produit fondu au décodeur est 3,7 fois plus lent que le produit brut (chapitre 4, section 4.10).

Le chronogramme d'un jeton (figure 10.3). Pendant la couche ℓ, la file du disque reçoit les défauts prédits pour $\ell + 1$ par le routeur de $\ell + 1$ appliqué au résidu de ℓ (96 % juste à une couche d'avance [litt.], chapitre 7). Le disque est occupé en permanence et la carte l'attend : environ 1 ms de carte par couche contre 4,4 ms de disque en moyenne. Le plafond suppose que la file ne se vide jamais ; le plancher, qu'aucune lecture n'est lancée avant que le routeur de sa couche n'ait parlé.

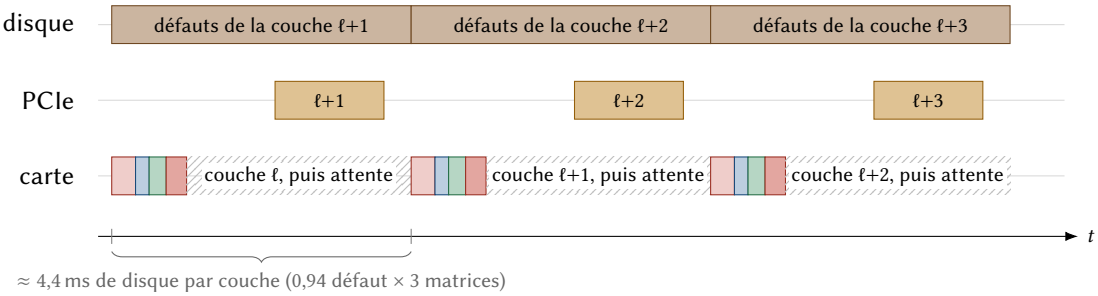
La mémoire de pointe. VRAM ≈ 7,5 Go : les toujours-lus (1,69 Go), un cache d'experts de 4,8 Go, le cache KV, les activations et les tampons de décodage (≈ 1 Go). L'écran passe par la carte intégrée : les 8 188 Mio de la carte sont disponibles. RAM : de 1,9 à 12,3 Go selon le cache tiède, et 0,64 Go dans la fenêtre minimale du partisan de A (section 4.8).

La qualité. Exacte par construction, et le témoin est double : chaque tronçon décodé est comparé à son empreinte (le décodeur rANS est exact au bit sur les 108 tenseurs de gpt-oss-120b [mes.]) ; et les



Débits mesurés sur la machine, en Go/s (sur secteur). Le disque tient le maximum de l'équation du jeton (proposition 4.6).

Figure 10.2 : Le chemin des octets de l'architecture candidate A. Le processeur ne fait que lire et tenir les files ; la carte décode et calcule. Débits mesurés (tableau 4.1).



Blocs de la carte : attention, routeur (bleu), décodage des défauts (vert), GEMV des experts. Environ 1 ms de carte par couche. La file du disque reçoit, pendant la couche ℓ, les défauts prédits pour ℓ+1 par le routeur de ℓ+1 appliqué au résidu de ℓ (96 % juste une couche d'avance [litt.]).

Figure 10.3 : Chronogramme d'un jeton dans l'architecture candidate A : trois couches successives. Le disque ne s'arrête jamais ; la carte calcule environ un quart du temps et attend le reste. C'est le régime de la proposition 4.6.

logits de A doivent être *identiques au bit* à ceux du même moteur lisant les experts non codés. Contre llama.cpp, on compare à l'écart mesuré entre deux moteurs justes (section 9.6).

Ce qu'elle sacrifie. Le processeur reste presque inactif (calculer les experts tièdes sur lui n'aiderait que le lien, qui ne tient pas le maximum : 58 ms contre 157) ; tout le passage avant de gpt-oss est à écrire (attention à puits et à fenêtre glissante une couche sur deux, RoPE YaRN, routeur, SwiGLU bornée) ; la vitesse reste bornée par la localité.

10.3 B : la précision selon l'étage

10.3.1 Le mécanisme

Un code E2M1 (le demi-octet du MXFP4) porte un signe s et une magnitude $m \in \{0; 0{,}5; 1; 1{,}5; 2; 3; 4; 6\}$, multipliée par l'échelle E8M0 de son bloc de 32 (chapitre 5). On le range en deux flux codés par rANS (figure 10.4) :

– la **base** $b = (s, \text{classe}(m))$, avec quatre classes de magnitude ;

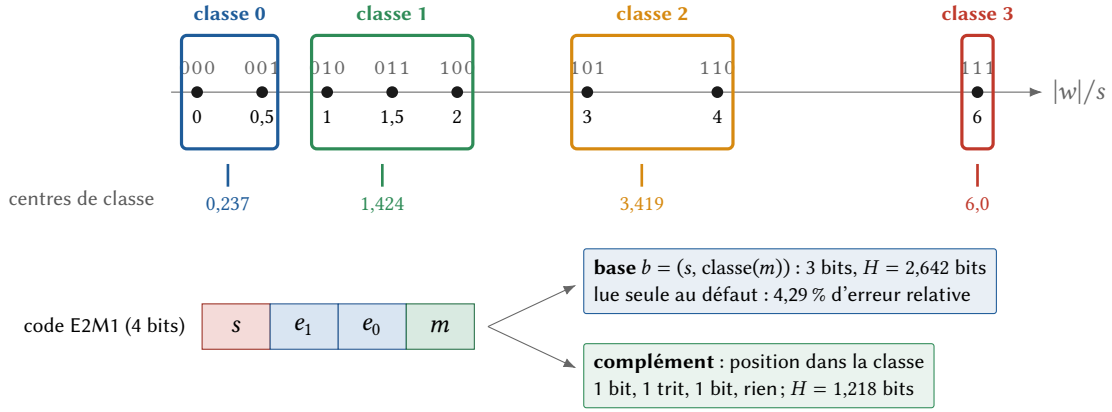
Table 10.1 : Plafonds de l'architecture candidate A sur gpt-oss-120b, disque à 2,58 Go/s, 11 Go de cache tiède [calc.]. Le même chemin sans codage ($r = 1$) donne 1,81 / 5,42 / 8,98.

Scénario	h_v / h_r / h_s	T_D / T_P / T_G (ms)	Plafond	Plancher en série
uniforme ($\beta = 1$)	0,079 / 0,199 / 0,723	485 / 131 / 59	2,05	1,47
LRU calé ($\beta = 0{,}207$)	0,590 / 0,176 / 0,234	157 / 58 / 36	**6,24**	3,93
Belady calé ($\beta = 0{,}117$)	0,744 / 0,118 / 0,139	93 / 36 / 29	10,34	6,17

Données : `fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, lignes `A_sans_perte` et `A_brut_r1`.

– le **complément**, la position de m dans sa classe, codé avec une table par classe.

Au défaut de cache, le disque ne lit que la base et la carte la sert, chaque classe remplacée par son centre ; quand l'expert entre au cache, la lecture du complément le rend exact.



Comme la base est une fonction du code, $H(\text{code}) = H(\text{base}) + H(\text{complément}) = 3{,}860$ bits (proposition 10.1) : découper ne coûte rien en entropie. Les centres sont pondérés par le carré de l'échelle E8M0, donc optimaux dans le domaine réel.

Figure 10.4 : La base et le complément d'un code E2M1, meilleure partition à quatre classes [mes.]. En haut, les huit magnitudes à l'échelle et leurs codes de 3 bits ; dessous, les centres de classe (unités E2M1). En bas, la découpe d'un code de 4 bits.

Données : `fondations/journaux/07-synthese/raffinement-mxfp4-gptoss20b.json` (outil `outils/07-synthese/raffinement_mxfp4.py`).

Proposition 10.1 (Le raffinement exact ne coûte rien en entropie)**.** *Soit X une variable discrète (le code) et $B = f(X)$ une fonction déterministe de X (la base). Alors*

$$H(X) \;=\; H(B) + H(X \mid B). \tag{10.1}$$

Un codeur entropique idéal qui code B, puis X sachant B, écrit donc autant de bits que s'il codait X d'un seul tenant.

Démonstration. Par la règle de chaîne, $H(X, B) = H(X) + H(B \mid X) = H(B) + H(X \mid B)$. Comme B est une fonction de X, connaître X fixe B : $H(B \mid X) = 0$. Donc $H(X) = H(B) + H(X \mid B)$. Le code optimal de X sachant B atteint $H(X \mid B)$ en moyenne, celui de B atteint $H(B)$, et la somme est $H(X)$. ∎

C'est le cas trivial, pour une source discrète et un sommet sans perte, du raffinement successif d'Equitz et Cover [129] ; le résultat continu est démontré par Drop-by-Drop [130] [litt.]. Ce qui est

neuf ici n'est pas l'identité : c'est son emploi sur un format 4 bits *natif* (le modèle a été entraîné dans ce format, la couche exacte est l'original lui-même), lié à l'étage, choisi pour sa distorsion et non par troncature, et sa mesure. Les voisins lus en entier par le partisan de B le confirment : SliceMoE tronque les bits de poids faible d'un entier [131], HOBBIT garde deux copies séparées [132] [litt.].

10.3.2 La mesure sur les poids réels

Table 10.2 : Couches de base d'un code E2M1, 12 tenseurs de gpt-oss-20b (couches 0, 8, 16, 23 ; gate, up, down ; 3 185 049 600 poids) [mes.]. $H(\text{code}) = 3{,}860$ bits ; la somme base + complément l'égale à 10^{-9} près sur chaque ligne (témoin de la proposition 10.1).

Base	Classes de magnitude	H(base)	H(compl.)	Erreur relative
signe + 2 bits, meilleure partition	{0 ; 0,5} {1 ; 1,5 ; 2} {3 ; 4} {6}	**2,642**	1,218	**4,29 %**
signe + 2 bits, troncature (« Matryoshka »)	{0 ; 0,5} {1 ; 1,5} {2 ; 3} {4 ; 6}	2,907	0,953	4,47 %
signe + 1 bit	{0 … 1,5} {2 … 6}	1,957	1,903	16,4 %
référence : MXFP4 sur une gaussienne				*1,33 %*

Données : `fondations/journaux/07-synthese/raffinement-mxfp4-gptoss20b.json` ; cassure : `raffinement-mxfp4-gptoss20b-casse.json`.

Trois lectures du tableau 10.2. **La troncature perd 0,27 bit par poids à erreur égale** : les niveaux du format ne sont pas uniformes, et couper les bits de poids faible regroupe mal les magnitudes. **La base à 1 bit est tuée par le nombre** : 16,4 % d'erreur, douze fois le bruit propre du format. **Les centres de classe** sont les centres de gravité pondérés par le carré de l'échelle E8M0, donc optimaux dans le domaine réel : 0,237 ; 1,424 ; 3,419 ; 6,0 ; le décodeur les tient dans quatre registres, et le produit sur la base coûte celui du MXFP4. Avec les échelles et les en-têtes, la base se lit à 2,684 bits par poids contre 3,855 pour le sans-perte : $3{,}855/2{,}684 = 1{,}436$ fois moins d'octets par défaut [calc.].

10.3.3 Le chiffre qu'il a fallu barrer

L'étude 07 définissait deux variantes : **B1** (la VRAM et la RAM gardent des experts exacts ; un défaut lit la base seule et la sert ; le complément ne se lit que si l'expert entre au cache) et **B2** (la RAM ne garde que des bases). Elle chiffrait B1 à ~~8,88 jetons/s, soit ×1,42 sur le plafond de A dans le scénario LRU calé~~.

Le partisan de B a trouvé l'erreur, contre son propre camp : l'outil ne comptait pas les octets des compléments. Or, sous LRU, tout défaut entre au cache, donc tout défaut finit par lire son complément. Comptés, **le plafond de B1 retombe à celui de A** : 6,16 jetons/s contre 6,24, parce que le disque lit base et complément, 3,906 bits contre 3,855 ; le gain de B1 ne vit plus que dans le plancher en série, 3,93 → **4,83** jetons/s (+23 %), parce que le complément quitte le chemin critique [calc.] (`decisions/partisan-B.md`, § 2). L'arbitre l'a refait par un chemin indépendant, sans importer aucun outil d'un autre : le chiffre barré se reproduit (8,879, témoin W4), le chiffre corrigé aussi (6,161 et 4,827, témoin W5), et la cassure qui remet les compléments à zéro fait tomber W5 seul [calc.] (`decisions/journaux/arbitre/verif-plafonds.json`, `verif-plafonds-casse.json`). Le ×1,42 est barré dans l'étude 07 elle-même.

10.3.4 Où B gagne vraiment : B3 et le bouton λ

Le vrai gain de débit de B est **B3** : la RAM garde des bases (1,44 fois plus d'experts tièdes), la VRAM un ensemble chaud exact, et une part λ des usages hors VRAM exige l'exact (le complément est alors lu). À $\lambda = 0$, le plafond vaut 10,89 jetons/s (×1,74 sur A, LRU calé), au prix de 48,5 % des usages d'experts servis par la base ; il rejoint A vers $\lambda \approx 0{,}67$ (figure 10.5). Le bouton λ est la politique de défaut selon le poids du routeur : servir la base aux experts qui pèsent peu, l'exact aux autres.

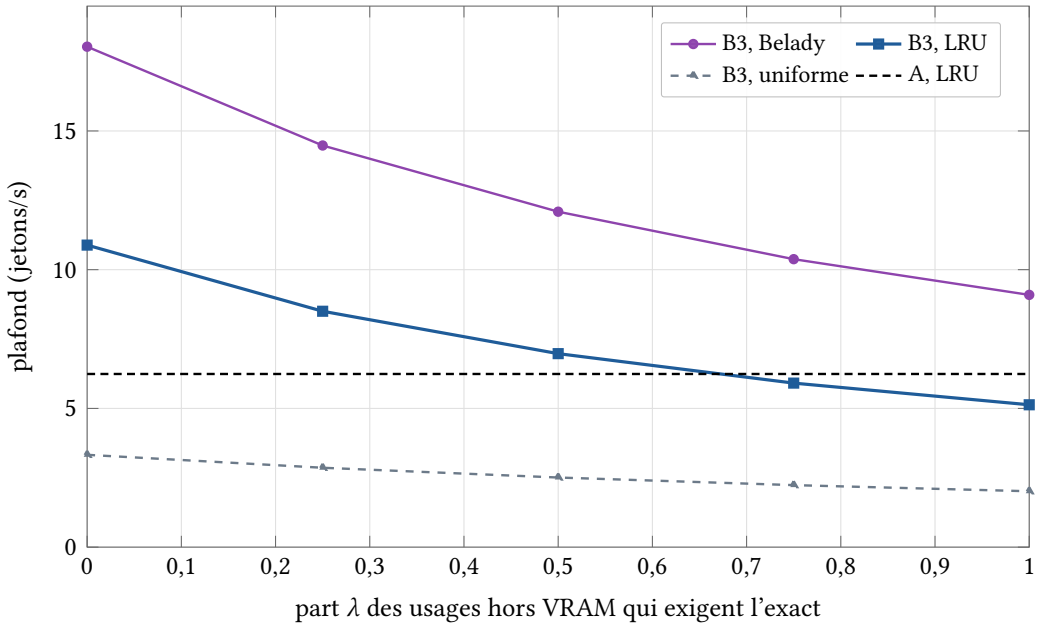


Figure 10.5 : Architecture B3 sur gpt-oss-120b : plafond selon la part λ des usages hors VRAM qui exigent l'exact [calc.]. À $\lambda = 0$, ×1,74 sur A (LRU calé) ; la courbe LRU croise A vers $\lambda \approx 0{,}67$.

Données : `decisions/journaux/csv/b3-lambda-gptoss120b.csv` (outil `outils/decisions/partisan_b.py`).

La qualité, l'inconnue qui décide. Par la linéarité de HIGGS (petites erreurs, perte ≈ somme pondérée des erreurs quadratiques relatives [litt.]), le partisan de B estime la divergence KL de B3 sans politique entre 0,05 et 0,12, soit 9 à 22 fois le budget $\kappa = 0{,}0055$ du définition 4.5 ; avec une politique qui ne sert la base qu'aux experts portant un dixième de l'énergie de routage, B1 tomberait entre 0,002 et 0,006 *(hypothèse)*. Ce ne sont pas des mesures. Elles se font sans moteur : une base « aimantée » sur des valeurs E2M1 existantes (0 ; 1,5 ; 3 ; 6) laisse un fichier MXFP4 valide que llama.cpp lit tel quel, à 5,91 % d'erreur au lieu de 4,29 % (banc Q0).

10.4 C : lire moins souvent

Un modèle dense lit tous ses poids à chaque passe, mais une passe de vérification de n jetons coûte presque le même temps tant que $n < n^\star = F\,b/(2\,W_{\text{eff}})$ (proposition 8.1). Pour Llama-3.3-70B IQ3_XXS (27 469 499 360 octets, licence Llama 3.3 Community), avec $F \approx 8$ Tflop/s utiles (hypothèse : 60 % des 13,4 mesurés en f16), $b = 26{,}9/70{,}6 = 0{,}381$ octet par paramètre et $W_{\text{eff}} = 26{,}9\,\text{Go}/3{,}79\,\text{s} = 7{,}10\,\text{Go/s}$:

$$n^\star = \frac{8 \cdot 10^{12} \times 0{,}381}{2 \times 7{,}10 \cdot 10^{9}} \approx 215 \quad \text{[calc.]}.$$

Un arbre de 64 à 128 nœuds est gratuit en calcul. Sans spéculation, 0,26 jeton/s ; avec l'arbre de SpecExec (9,7 jetons par passe, mesuré sur une RTX 4060 de bureau avec un 70B GPTQ [litt.]), **2,50** jetons/s (plancher en série 1,68) [calc.]. C'est moins que A dans le scénario LRU, pour 28,3 Go de téléchargement, un moteur d'arbre à écrire et un Llama 3 qui se quantifie mal (+76 % de perplexité à 2,07 bits [litt.], chapitre 5).

10.5 Le débat et la décision

Le cycle de décision du projet oppose deux partisans qui ne se lisent pas, puis un arbitre qui tranche avec ses propres mesures.

Le partisan de A (`decisions/partisan-A.md`) : A n'est pas une alternative à B, c'est son *plancher* (B qui sert tous ses défauts exacts *est* A) et son *juge* (le budget de qualité de B se mesure contre A ; sans un A exact au bit, une erreur ordinaire de moteur mange tout le budget). Il concède : « un A seul ferait un bon rapport d'ingénierie, pas un article de recherche ».
Le partisan de B (`decisions/partisan-B.md`) : B est le seul mécanisme qui n'existe pas encore, et c'est ce que Stone demande ; il corrige le chiffre de B1 contre son propre camp, et propose de mesurer la qualité de B d'abord, sans moteur.

La décision de l'arbitre (`decisions/ARCHITECTURE.md` ; réponse par délégation du 2026-10-15, réversible). On construit **troncons**, un moteur à nous, en Rust avec des noyaux CUDA lancés par l'API du pilote.

Table 10.3 : Les jalons décidés par l'arbitre.

Jalon	Contenu	Porte d'entrée
J1	A exact sur gpt-oss-120b : format `.trc` v1.0, lectures IOCP à la taille d'un expert, DMA depuis la mémoire épinglée, décodage rANS sur la carte, cache d'experts en VRAM, passage avant complet ; modes A0 et A	aucune
J1+	préchargement (routeur de $\ell+1$ sur le résidu de ℓ), cache SLRU	J1
Q0	qualité de B sans moteur, par fichiers statiques jugés par llama.cpp, en parallèle dès le premier jour	la carte libre 2 à 4 h
J2	B dans le moteur : format v1.1 à deux niveaux, B1 (latence), B3 (débit, politique λ)	J1 tient et Q0 n'a pas tué B
J3	disque non chiffré, spéculation si ρ le permet, portage Vulkan	décision future

Ses raisons, chiffrées : B d'abord, c'est A plus quatre modules, sans un jour gagné sur le premier programme qui marche, et B1 ne gagne plus rien au plafond ; A seul ne remplit pas l'exigence d'une contribution ; C plafonne à 2,50 contre 6,24. **CUDA plutôt que Vulkan pour J1**, par les mesures : le décodeur rANS est 14,9 % plus rapide sous CUDA (80,8 et 82,1 contre 70,2 et 71,6 Go/s, en ABBA [mes.]) ; le recouvrement copie-calcul (part cachée 0,98 à 1,01) et la mémoire épinglée (+21 %) n'ont été mesurés que par des flux CUDA ; wgpu n'offre ni import de mémoire hôte ni seconde file. Le prix, dit : le programme exige une carte NVIDIA ; les noyaux WGSL mesurés restent le second chemin.

La mesure F1 de l'arbitre : ce que coûte la découpe. Les partisans s'opposaient sur le format : un flux unique en v1 (A), ou la découpe base + complément comme format unique (B). L'arbitre l'a mesuré avec le codeur rANS réel, expert par expert, chaque flux décodé et comparé octet à octet, seuils déclarés avant le lancement (figure 10.6). Résultats [mes.] : la table choisie *par voie* selon la classe d'échelle du bloc égale les trois flux de classe de l'étude de l'entropie (3,8431 contre 3,8434 bits par poids, seuil S2 tenu) sans réassemblage ; la découpe conditionnée par la classe d'échelle coûte +0,41 % (seuil S1 de 0,40 %, le coût d'en-tête d'un flux de plus, manqué de 0,01 point) ; le complément codé par rANS fait 0,970 fois son empaquetage brut (S3) ; l'erreur de la base (4,2920 %) et celle de la base aimantée (5,913 %) sont retrouvées par ce second chemin (S4). Après la première passe, l'arbitre a déclaré une variante et son seuil S1bis (le même 0,40 %) avant la seconde : le complément conditionné aussi par la classe du demi-octet voisin et, pour le demi-octet fort, par le complément faible déjà décodé. Elle coûte +0,31 % : **S1bis tient**. D'où la règle : la v1.0 de J1 prend la table par voie (un seul niveau, le noyau le plus simple) ; **la v1.1 à deux niveaux devient le format unique de J2**, où A paie 0,31 % d'octets, sous le bruit de la lecture. Avec les bits mesurés par F1, B1 complété monte à 6,22 jetons/s (série 4,87) contre 6,24 pour A, et B3 à 11,10 (×1,78) [calc.] (`decisions/journaux/arbitre/verif-plafonds.json`, section `avec_F1`). L'arbitre le résume ainsi : le partisan de A avait raison sur l'ordre, le partisan de B sur le format final.

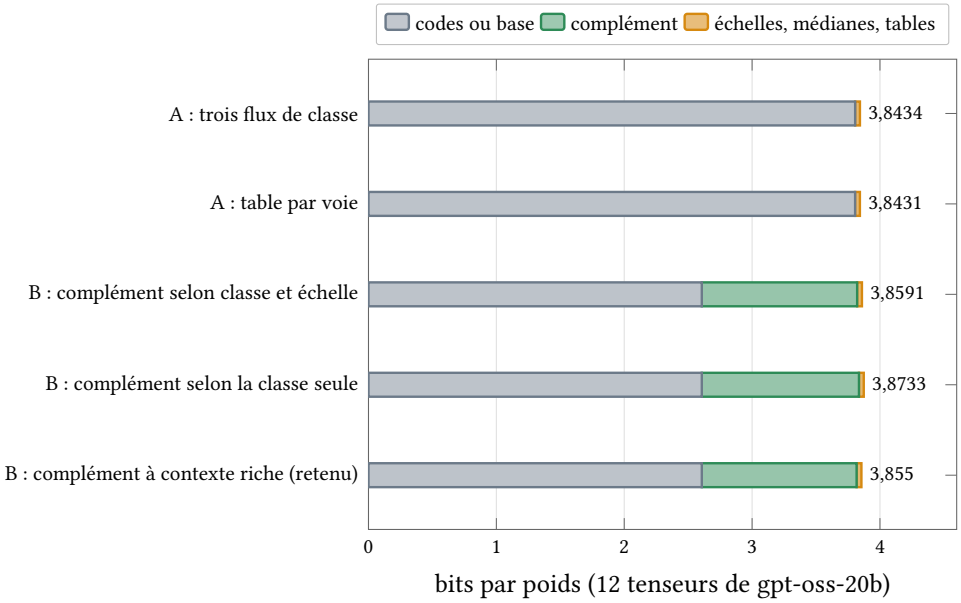
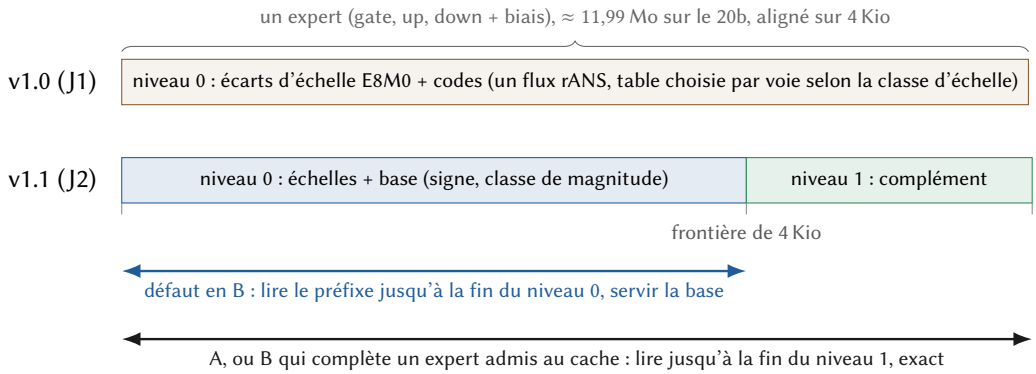


Figure 10.6 : Mesure F1 de l'arbitre : bits par poids écrits par chaque variante du format, 12 tenseurs de gpt-oss-20b, codeur rANS réel, décodage exact [mes.]. Le total est inscrit au bout de chaque barre ; la découpe base + complément coûte +0,41 %, +0,31 % avec le contexte riche retenu pour J2.

Données : `decisions/journaux/arbitre/decoupe-gptoss20b.json` (passe 2 ; outil `outils/arbitre/decoupe/`).

Le format `.trc` (figure 10.7). Un fichier autonome par modèle, petit-boutiste, aligné sur 4 096 octets (ce qu'exige la lecture sans tampon de Windows) ; **un tronçon par expert**, d'un seul tenant, pour qu'un défaut coûte une requête ; des flux rangés par **niveau de raffinement**, tout préfixe qui s'arrête à la fin d'un niveau se décodant seul. En J1, un seul niveau ; les sortes de flux de B sont réservées dès maintenant, pour que J2 ne change ni le lecteur, ni le cache, ni l'ordonnanceur. Le nom vient du projet : un tronçon est la part d'un modèle qu'on lit, décode, sert et oublie, les « chunks

comme dans minecraft » de Stone.



Le lecteur, le cache et l'ordonnanceur ne connaissent que « lire jusqu'à la fin du niveau *j* ». Tailles de la mesure F1 de l'arbitre (passe 1, gpt-oss-20b) : niveau unique 11 990 368 o en moyenne ; B : 8 261 120 + 3 780 864 o.

Figure 10.7 : Un tronçon du format `.trc` : un expert d'un seul tenant, ses niveaux de raffinement alignés sur 4 Kio. En v1.0 (J1), un seul niveau ; en v1.1 (J2), la base puis le complément, et le défaut de B ne lit que le préfixe.

Données : `decisions/ARCHITECTURE.md` (§ 4, principes) et `decisions/journaux/arbitre/decoupe-gptoss20b-passe1.json` (tailles).

Les critères de réussite de J1 , déclarés avant toute construction et dérivés de mesures nommées : logits du chemin codé identiques au bit à ceux du chemin brut du même moteur ; un moteur au moins aussi près des mathématiques exactes du modèle (référence en double précision) que llama.cpp ; RAM de pointe $\leq$ 1,9 Go en mode A0, égale à 0,5 Mo près entre gpt-oss-20b et gpt-oss-120b (le témoin de la définition 1.1) ; VRAM sous 8 188 − 909 Mio ; jetons/s au moins 0,8 fois le plancher en série recalculé avec le succès compté par le moteur.

10.6 Les premières mesures du modèle visé

L'agent de mesure travaille sur la carte pendant que ce chapitre s'écrit ; son étude n'est pas rendue, mais ses séries et ses traces sont commitées avec leurs témoins. On en tire deux figures, à lire comme provisoires.

La base llama.cpp sur gpt-oss-20b (figure 10.8). Séries ABBA de quatre passes, sur secteur, llama.cpp b11515 : experts sur le processeur, 11,38 jetons/s en génération ; placement ajusté par l'outil de llama.cpp (dix couches d'experts sur 24 en carte), 18,92 ; cache d'experts de 4 096 Mio, 10,15, à 84,59 % de succès selon le journal de llama.cpp [mes.]. L'écart du placement ajusté sur les experts au processeur est réel (+66 %, au-dessus du bruit de 0,30) ; celui du cache est dans le bruit (`mesures/base-gpt-oss/20b/resume.csv`). Sur gpt-oss-120b, une exploration d'une seule passe, non retenue par l'agent lui-même, donne 0,49 jeton/s avec les experts sur le processeur et 295 Go lus en 12 minutes et 46 secondes (`mesures/REPRISE-gpt-oss.md`) : de l'ordre du bas de la fourchette estimée pour la base (0,14 à 0,81 dans le scénario uniforme).

La localité sur gpt-oss-20b (figure 10.9). Traces de routage réelles (quatre genres de texte, lecture et génération), rejouées par l'instrument de l'étude du chargement ; témoin exact : une réplique de la

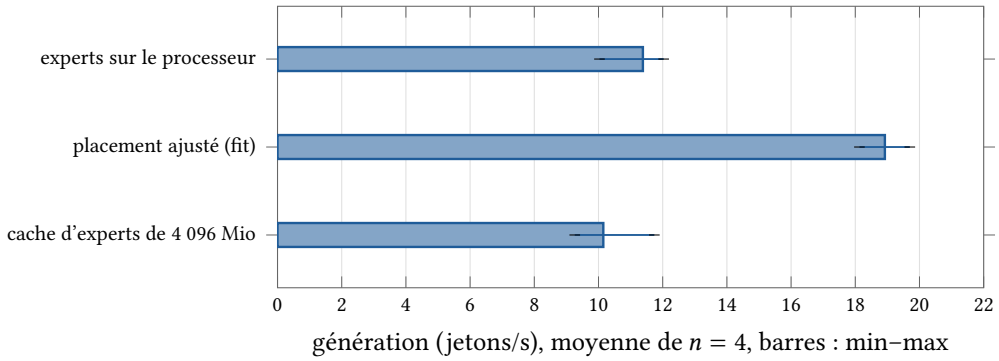


Figure 10.8 : Base llama.cpp b11515 sur gpt-oss-20b, génération, trois configurations, séries ABBA de quatre passes sur secteur [mes.]. Barres : minimum et maximum. Mesure provisoire de l'agent de mesure.

Données : `mesures/base-gpt-oss/20b/resume.csv`, tiré par `memoire/outils/donnees_figures.py`.

politique de cache de llama.cpp rejouée sur ces traces redonne au succès près les nombres du journal de llama.cpp. Sur la trace de génération :

- **La forme de l'hypothèse 4.4 ne tient pas, mais son ordre de grandeur oui.** Aux petits budgets, le succès mesuré est sous l'hypothèse (cache par couche, 15 % : LRU 0,627 contre $0{,}15^{0{,}2075} = 0{,}675$; Belady 0,686 contre 0,801) ; aux grands, au-dessus (40 % : LRU 0,882 contre 0,827 ; Belady 0,935 contre 0,899) [calc.] sur [mes.]. La courbe réelle est plus raide qu'une loi de puissance calée en un point.
- **Un cache LRU global sous $L\,k$ emplacements rend zéro** (96 emplacements pour 24 couches à 4 experts actifs) : chaque couche évince les experts de la suivante avant qu'ils ne servent. C'est une pathologie cyclique, que Belady et LFU évitent.
- **La réutilisation d'un jeton au suivant vaut $\rho = 0{,}577$**, contre 0,125 au hasard uniforme et 0,223 au hasard à popularité réelle (`mesures/localite-gpt-oss/20b/localite-gpt-oss-20b.json`) ; l'instrument la mesure comme la part des experts d'un jeton déjà choisis au jeton précédent, dans la même couche, c'est-à-dire le ρ de la proposition 4.8. Sur gpt-oss-20b, la spéculation paierait donc dès $\alpha > 0{,}42$, loin du 0,875 qu'impose l'indépendance [calc.]. Reste à le mesurer sur le 120b.

10.7 Ce qui vient

Les chapitres suivants de cette partie s'écriront avec le programme, dont la construction a commencé pendant la rédaction de celui-ci : au moment où ce mémoire est construit, le lot K a commité les noyaux du décodeur à table par voie et de la GEMV MXFP4 avec un banc indépendant, le lot E le relevé de l'ordonnancement matériel de WDDM, le lot R les compteurs du juge et leurs témoins (journal de `git`, 2026-10-15). **Implémentation** : les modules de troncons et leurs coutures, le format `.trc` au bit près, les noyaux CUDA et leurs témoins, chaque lot fini par une cassure qui tombe. **Résultats** : jetons/s, RAM et VRAM de pointe, octets lus (compteur du système), rendement physique (mesuré sur plafond) sur gpt-oss-120b, contre la base llama.cpp mesurée en ABBA ; puis, au jalon J2, la qualité de B contre A. Le chapitre 12 en donne l'ordre.

Ce qui pourrait être faux

- **Tous les plafonds de ce chapitre dépendent de l'hypothèse 4.4**, que la première mesure

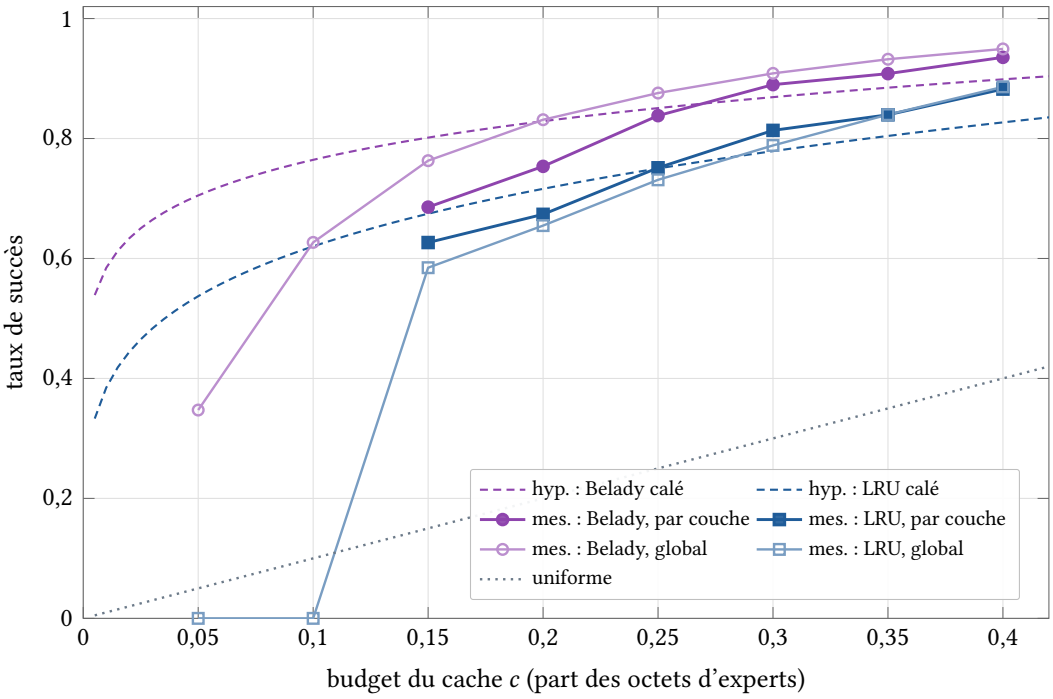


Figure 10.9 : Succès du cache d'experts sur les traces de routage réelles de gpt-oss-20b (trace de génération) [mes.], contre les scénarios calés du chapitre 4 (tirets). Le cache par couche n'a pas de point sous un expert par couche ; le LRU global rend zéro sous $L\,k = 96$ emplacements. Mesure provisoire de l'agent de mesure, sur le 20b et non sur le 120b.

Données : `mesures/localite-gpt-oss/20b/courbes-gpt-oss-20b.csv`, tiré par `memoire/outils/donnees_figures.py`.

sur gpt-oss-20b contredit dans sa forme ; ils sont à recalculer avec la courbe mesurée du 120b.

- **La décision est une réponse par délégation, réversible** : elle se change par une décision écrite et datée de `decisions/ARCHITECTURE.md`, et ce chapitre suivra.
- **La qualité de B n'est qu'estimée**, par une linéarité valable pour de petites erreurs ; 4,29 % sur un expert n'est pas petit.
- **La couche de base est mesurée sur 12 tenseurs de gpt-oss-20b**, pas sur le 120b ; les entropies des codes des deux modèles diffèrent de 0,001 bit, ce qui rend l'écart improbable, mais il n'est pas mesuré.
- **Les voisins de B** (SliceMoE, TRACE, MatGPTQ, AnyBCQ) ne sont pas tous lus en entier ; l'un d'eux peut faire, dans son corps, une partie de ce que ce chapitre appelle neuf.
- **Les mesures de la section 10.6 sont provisoires** : l'étude qui les porte n'est pas rendue, et la machine était partagée pendant les séries.

Chapitre 11

Discussion et limites

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, neuf chapitres qui mesurent, démontrent et décident chacun de leur côté. **Ce chapitre établit** ce qu'ils disent ensemble ; ce qui y est solide et ce qui y est fragile ; les idées que les nombres ont tuées ; les chiffres qu'il a fallu barrer, rassemblés au même endroit ; et les limites de la méthode elle-même. **Il reste** à le confronter au programme : chaque fragilité nommée ici a sa mesure prévue au chapitre 12.

11.1 Ce que la première partie établit

1. **Le juge est un maximum, et sur cette machine c'est le disque qui le tient.** Pour gpt-oss-120b dans le scénario de localité retenu, le disque prend 157 ms par jeton, le lien 58, la carte 36 (section 4.7). Tout levier qui ne touche pas $h_s B_e/(r_d W_{\mathrm{D}})$ est, ici, de l'ornement (corollaire 4.7).
2. **Le disque livre 47 % de son annonce**, parce que le volume est chiffré par BitLocker et passe par Intel RST VMD (chapitre 3) : le plus gros levier de la machine n'est pas un logiciel.
3. **Chaque bit retiré vaut autant que chaque octet de débit gagné** (élasticités ±0,98, proposition 4.6) ; la quantification extrême et la compression sans perte agissent sur le même terme, et la borne de Shannon dit ce qu'il reste à prendre (chapitre 5).
4. **Le taux de succès du cache décide de tout pour un mélange d'experts** (chapitre 7) : entre le routage uniforme et un LRU calé, le plafond de A change d'un facteur 3.
5. **La spéculation ne se juge pas sans la réutilisation des experts** : la condition $\alpha > 1 - \rho$ (proposition 4.8) remplace la conclusion « jamais » du chapitre 8 ; sur gpt-oss-20b, $\rho = 0{,}577$ (section 10.6).
6. **La mémoire peut cesser de dépendre du modèle** : A0 sert un modèle de 63,4 Go avec 1,9 Go de RAM de pointe, et la fenêtre minimale descend à 0,64 Go (section 4.8).
7. **Un mécanisme neuf existe, plus étroit qu'annoncé** : le raffinement exact d'un code 4 bits natif ne coûte rien en entropie (proposition 10.1) et sa base perd 0,27 bit de moins que la troncature à erreur égale ; mais une fois les compléments comptés, son gain de débit n'est réel qu'au prix d'une part réglable de précision (section 10.3.3).

11.2 Ce qui est solide, ce qui est fragile

Solide. Les débits d'étage, chacun mesuré avec un témoin qui peut tomber, en séries ABBA, et refait par un second chemin (compteur du système pour le disque, API de chiffrement pour BitLocker) ; les théorèmes et leurs démonstrations (théorèmes 3.2 et 8.3, propositions 4.6 et 10.1) ; l'exactitude du décodeur rANS sur les 108 tenseurs de gpt-oss-120b ; l'inventaire des modèles lu dans leurs en-têtes, refait sur le fichier local.

Fragile. Tout ce qui dépend de la localité : les plafonds de A et B reposent sur une loi de puissance calée en un point (hypothèse 4.4) dont la forme est déjà contredite sur gpt-oss-20b ; la base llama.cpp de gpt-oss-120b, encore estimée ; la qualité de B, seulement estimée par une linéarité valable pour de petites erreurs ; plusieurs travaux voisins, lus au résumé seulement.

11.3 Les idées tuées par les nombres

Le SOCLE demande de tuer une idée par les nombres, pas par l'opinion. Le tableau 11.1 les rassemble.

Table 11.1 : Les idées tuées par les nombres, et le nombre qui les tue.

Idée	Le nombre qui la tue	Où
Décoder dans le produit (fusion rANS + GEMV)	3,7 fois plus lent que le produit brut, pas plus rapide que deux passes [mes.]	chapitre 6
Treillis « sans multiplication » sur la carte	52 contre 121 Go/s : les comptages de bits coûtent plus que la multiplication [mes.]	chapitre 5
Décoder sur le processeur	0,075 Go/s par fil [mes.]	étude 03, § 4.4
Sparsité des activations sous vérification	l'union de 4 jetons touche 89,5 % des poids ; un masque commun (bloc de 4, 50 %) multiplie la perplexité par 16,9 [mes.]	tableau 8.5
Spéculation sur un MoE fin sous l'indépendance	il faudrait $\alpha > 0{,}969$ pour gpt-oss-120b [calc.] ; nuancé par ρ	proposition 4.8
DirectX 12 par wgpu	ni f16 ni sous-groupes ; 32 Go/s en 4 bits contre 122 [mes.]	étude 01, § 5.3
IoRing au lieu d'IOCP	écart de 0 à 5 %, sous le bruit [mes.]	chapitre 3
Noyaux plus rapides en régime disque	élasticité nulle, moins de 2 % [calc.]	corollaire 4.7
Calculer les experts tièdes sur le processeur	le lien (58 ms) ne tient pas le maximum (disque 157 ms) [calc.]	section 10.2
Base de B à 1 bit de magnitude	16,4 % d'erreur, douze fois le bruit propre du format [mes.]	tableau 10.2
C (un dense de 70B lu du disque, en arbre)	2,50 jetons/s au mieux contre 6,24 pour A, et 28,3 Go à télécharger [calc.]	section 10.4

11.4 Les chiffres barrés

Quand une mesure a contredit un chiffre publié, même le nôtre, la phrase a été barrée à sa place et l'on a dit ce qui avait changé. Ce mémoire en garde la liste (tableau 11.2) : c'est la trace que la méthode a fonctionné.

11.5 Les limites de la méthode

- **Une seule machine.** Tout est mesuré sur le portable de Stone ; c'est la cible, mais les conclusions sur l'étage limitant changent avec un autre disque (un volume non chiffré fait passer la main au lien, chapitre 12).
- **Une machine partagée.** D'autres agents mesuraient en même temps ; les débits « au mieux » sont des bornes inférieures du repos, et chaque passe journalise la charge étrangère.

Table 11.2 : Les chiffres barrés pendant les fondations, et ce qui les a remplacés.

Barré	Remplacé par, et pourquoi	Où
~~Plafond de gpt-oss-120b sans cache : 5,5 jetons/s~~	≈ 2 jetons/s : 5,5 supposait le disque au plafond de son lien, il en livre 36 %	section 3.8
~~$r^\star$ vaut 1,05 bit stocké pour 4,5 servis~~	1,17 bit : le Q4_K_M sert 5,08 bits, pas 4,5	section 3.8
~~Une matrice résidente mérite 2,6 bits de plus~~	2,18 bits (lien) et 3,24 bits (disque), avec les débits mesurés	chapitre 5
~~Témoin du cache disque : la relecture en cache dépasse le lien~~	la copie depuis le cache de Windows est bornée par un fil (3,1 Go/s) ; remplacé par le compteur du système	chapitre A
~~Justesse d'un moteur sans perte : mêmes jetons que llama.cpp~~	llama.cpp diverge de lui-même selon le moteur ; budget KL 0,0055	section 9.6
~~Taux sans perte sur le cache : +14,4 %~~	+13,1 % : la carte garde ses experts décodés	section 4.10
~~B1 : 8,88 jetons/s, ×1,42 sur A~~	6,16 (série 4,83) : les compléments n'étaient pas comptés	section 10.3.3
~~B1 bat A dès que 4,2 % des défauts sont servis par la base~~	B1 ne bat jamais A au plafond, seulement en série	section 10.3.3
~~k optimal = 0 pour un MoE lu du disque~~	dépend de ρ ; $\alpha > 0{,}42$ suffirait sur gpt-oss-20b	proposition 4.8

- **Deux machines en une.** Sur batterie, la carte, le lien et le processeur changent de régime (section 3.6.1) ; toutes les mesures retenues sont sur secteur, et l'état de l'alimentation est noté à chaque passe (chapitre A).
- **Un juge de qualité calé sur un autre modèle.** Le budget $\kappa = 0{,}0055$ vient de Qwen3.5-4B ; il doit être remesuré sur gpt-oss.
- **La littérature au résumé.** Plusieurs voisins de B (TRACE, MatGPTQ, AnyBCQ) ne sont lus qu'au résumé : rien ne se publie avant leur lecture complète.
- **Un choix d'outillage qui ferme une porte.** J1 sera en CUDA (section 10.5) : le programme exigera une carte NVIDIA jusqu'au portage Vulkan.

11.6 La lecture de « sans RAM ni rien »

La lecture retenue (définition 1.1) n'a jamais été substituée à la demande de Stone : elle est à lui confirmer. Les nombres la rendent concrète. Une RAM nulle est impossible ; une RAM qui ne dépend pas du modèle est atteignable, et son prix se lit sur la figure 4.3 : chaque Go de RAM rendu au système coûte environ 0,24 jeton/s dans le scénario LRU calé. « Sans RAM » devient un réglage, que Stone peut tourner selon ce qu'il fait d'autre sur sa machine.

Ce qui pourrait être faux

- Le bilan hérite de toutes les fragilités des chapitres : d'abord la localité, ensuite la base estimée, enfin la qualité de B.
- La liste des idées tuées vaut pour ce régime (disque chiffré, lot de un) ; plusieurs reviennent si le disque cesse de limiter.

- La liste des chiffres barrés n'est complète que pour ce que les études ont publié ; un chiffre faux encore non contredit n'y figure pas, par définition.

Chapitre 12

Travaux futurs

D'où l'on part, ce que le chapitre établit, ce qui reste

Au départ, les fragilités du chapitre 11 et les jalons du section 10.5. **Ce chapitre établit** l'ordre du travail qui vient, chaque pas avec la mesure qui le juge ; les questions de recherche ouvertes, chacune chiffrée par l'équation du jeton ; le seul geste qui revienne à Stone ; et les téléchargements qui pourraient servir, aucun n'étant nécessaire au premier programme. **Il reste** à le faire.

12.1 Les mesures qui décident, d'abord

1. **La courbe $h(c)$ et la réutilisation ρ de gpt-oss-120b**, sur ses propres traces de routage. Elles remplacent l'hypothèse 4.4 dans tous les plafonds ; la mesure du 20b (figure 10.9) dit déjà que la forme supposée est fausse. Témoin : la réplique du cache de llama.cpp rejouée sur les traces, qui doit redonner les nombres de son journal.
2. **La base llama.cpp de gpt-oss-120b en séries ABBA**, sur secteur, selon le protocole du section 9.6 : c'est la première mesure qui peut tuer A (le rapport ×2,5 à ×17 de l'étude 07 repose sur une base estimée).
3. **Le banc Q0** : la qualité de B sans moteur, par fichiers MXFP4 statiques (base aimantée sur des valeurs E2M1 existantes) jugés par llama.cpp, et la frontière des requantifications uniformes. C'est la mesure qui décide de J2.
4. **L'écart entre deux moteurs justes sur gpt-oss**, qui recale le budget de qualité κ du définition 4.5.

12.2 Le jalon J1 : construire A

L'arbitre découpe le travail en cinq lots aux fichiers disjoints, chacun fini par un témoin, sa cassure et un seuil déclaré (`decisions/ARCHITECTURE.md`, § 6) :

F, format et codeur : réécriture du GGUF d'origine en fichier `.trc` v1.0 ; témoin : chaque tronçon décodé égale l'original, octet pour octet.
E, chemin des octets : lecteur IOCP à la taille d'un expert vers la mémoire épinglée, DMA ; témoin : le débit de bout en bout ne dépasse aucun étage, et les octets arrivent exacts.
K, noyaux : décodeur rANS à table par voie, produit MXFP4, passage avant de gpt-oss ; témoin : les noyaux WGSL déjà mesurés, sur les mêmes octets (second chemin).
R, référence et juge : une référence processeur en double précision, le juge de qualité, le banc Q0.
O, cache, ordonnanceur, moteur : cache d'experts en VRAM, file des défauts, modes A0 et A ; témoin : logits identiques au bit entre le chemin codé et le chemin brut.

Le jalon se juge sur gpt-oss-120b : jetons/s, RAM et VRAM de pointe, octets lus selon le compteur du système, et rendement physique, c'est-à-dire le débit mesuré rapporté au plafond de l'équation (4.1)

recalculé avec le succès compté par le moteur. Puis J1+ ajoute le préchargement d'une couche d'avance et un cache SLRU, et se juge contre la meilleure base llama.cpp mesurée.

12.3 Les questions de recherche

Table 12.1 : Les questions de recherche ouvertes, et ce que chacune vaut d'après l'équation du jeton [calc.] (`fondations/07-ce-qui-manque.md`, § 3.2, et section 10.3.3).

	Question	Ce qu'elle vaut, et ce qui manque
R1	Le raffinement exact d'un format 4 bits natif, lié à l'étage (B)	en débit : B3, ×1,74 au prix de 48,5 % d'usages servis par la base ; en latence : B1, plancher +23 %. Manque : la qualité (Q0)
R2	L'allocation conjointe des bits et de l'étage	avec les débits de la machine, une matrice résidente mérite $\frac{1}{2}\log_2(209/2{,}58) = 3{,}2$ bits de plus qu'une matrice lue du disque ; aucun outil ne l'applique
R3	Une politique de défaut sous budget de qualité	attendre, reporter, servir la base, re-router, défaut par défaut ; le gain vit entre le plancher et le plafond (A, LRU : 3,93 → 6,24, ×1,59)
R4	La spéculation sous réutilisation réelle	condition $\alpha > 1 - \rho$ (proposition 4.8) ; $\rho = 0{,}577$ sur gpt-oss-20b ; à mesurer sur le 120b, et pour $k > 1$
R5	Un code à 2 bits de deuxième génération décodable sans CUDA	décodeur de treillis écrit et mesuré à 124 Go/s, codeur absent ; B_e divisé par 1,9 (4,25 → 2,25 bits) ; qualité inconnue sur un modèle entraîné en 4 bits
R6	Les arbres de vérification pour les hybrides à règle delta	48 Mio d'état par nœud gardé, 12 Gio pour 256 nœuds ; bloque C sur les modèles Qwen
R7	Un code à décodage en temps constant pour le régime résident	au mieux +10 % de capacité de la VRAM, +2 % de plafond ; basse priorité

12.4 Le geste qui revient à Stone

Le plus gros levier de la machine n'est pas un logiciel (section 11.1). Un volume `D:` non chiffré, créé par un script déjà écrit et que Stone lancerait lui-même en administrateur (`outils/creer-disque-modeles.ps1`), porterait le plafond de A de 6,24 à **14,1 jetons/s** dans le scénario LRU calé si le disque y atteignait les 6,0 Go/s de son annonce [calc.] (`fondations/journaux/07-synthese/csv/architectures-gptoss120b.csv`, ligne `A_sans_perte_disque_non_chiffre_6`). C'est une borne haute : Intel RST VMD reste sur le chemin, et sa part dans les 53 % perdus n'est pas séparée. Au-delà, le lien reprend la main (58 ms contre 67 pour le disque) : garder le lien en Gen 4 et la capacité de la VRAM redeviennent des leviers.

12.5 Les téléchargements possibles

Aucun n'est nécessaire pour construire et juger A et B sur gpt-oss-120b, déjà sur la machine. Tout téléchargement de plus de 500 Mo demande l'accord de Stone, avec le nom, la source, la taille et la licence (tableau 12.2).

Table 12.2 : Les téléchargements qui pourraient servir, triés par nécessité [litt.] (tailles lues par l'API Hugging Face ou NuGet, `fondations/07-ce-qui-manque.md`, § 5).

Rang	Nom	Taille	Licence	Pour
1	DirectStorage 1.4 (préversion)	3,99 Mo	Microsoft	comparer le chemin disque → carte
2	Qwen3.6-35B-A3B UD-Q4_K_M	22,13 Go	Apache-2.0	second MoE fin, trace et ρ
3	OLMoE-1B-7B Q4_K_M	≈ 4,2 Go	Apache-2.0	trace de routage rapide
4	Llama-3.3-70B IQ3_XXS + Llama-3.2-1B Q4_K_M	28,28 Go	Llama 3.3 / 3.2	architecture C
5	Qwen3.8-Flash-Next UD-Q2_K_XL	78,87 Go	qwen-community-1.0	second modèle de B
6	Qwen3.5-9B-MTP Q4_K_M	5,87 Go	Apache-2.0	accord du MTP natif

12.6 Le mémoire lui-même

Deux chapitres s'ajouteront à la troisième partie, avant la discussion : **l'implémentation** (les modules de troncons, le format `.trc` au bit près, les noyaux et leurs témoins) et **les résultats** (chaque série du jalon J1 en figure, avec barres d'erreur, témoin et seuil déclaré). ~~L'étude de l'entropie, que ce mémoire cite sans lui consacrer de chapitre parce qu'elle s'achevait chez un autre agent, recevra le sien~~ (elle l'a reçu : chapitre 6), et la mesure de gpt-oss, une fois rendue, remplacera les figures provisoires du section 10.6.

Ce qui pourrait être faux

- L'ordre proposé suppose que la base mesurée de llama.cpp ne dépasse pas le plafond de A ; si elle le dépassait, A serait à repenser avant d'être construit.
- Les gains de la tableau 12.1 sont des plafonds calculés sous l'hypothèse 4.4 ; ils bougeront avec la courbe mesurée.
- Le gain du volume non chiffré est une borne haute fondée sur une annonce de fabricant, pas sur une mesure.

Annexe A

Protocoles de mesure

Cette annexe rassemble les règles qui ont produit chaque nombre [mes.] du mémoire. Elles ont été écrites et commitées *avant* les mesures qu'elles jugent (`fondations/journaux/01-machine/00-seuils-declares.md`, commité à 01:35 le 2026-10-15, avant le premier lancement du banc) ; aucune ne s'est réglée après coup.

A.1 Les étiquettes

Chaque nombre porte sa provenance : [mes.] mesuré ici, sur la machine de Stone, par un outil et un journal commités ; [calc.] calculé par une formule écrite et un outil commité ; [litt.] littérature (chiffre d'éditeur, non reproduit). Un chiffre repris d'une étude garde son étiquette et son numéro d'étude.

A.2 Le témoin, et sa cassure

Chaque mesure a son témoin : un cas dont la réponse est connue d'avance. Tout témoin neuf doit pouvoir échouer : on force sa condition à faux (option `--fausser` ou `--casser` de l'outil), on relance, on vérifie que l'instrument tombe, puis on restaure. Exemples tirés des études :

- **disque** : chaque page de 4 Kio porte son numéro ; les pages fausses doivent valoir zéro, et toutes les pages vérifiées sous `--fausser` ;
- **RAM** : la somme lue doit égaler la somme connue du motif ; `--fausser` change l'attendu et fait échouer ;
- **carte** : aller-retour hôte → carte → hôte octet pour octet ; produit matrice-vecteur comparé à une référence processeur (écart relatif $\leq 10^{-3}$ sur 64 lignes) ;
- **plafonds physiques** : une lecture sans tampon qui dépasse le lien NVMe (7,877 Go/s), une lecture de RAM au-dessus d'un module DDR5-5200 (41,6 Go/s) prouvent une lecture servie par un cache, et font tomber l'instrument ;
- **équation du jeton** : trois chiffres publiés par d'autres études refaits à 1 % ; remplacer le débit mesuré du disque par le plafond du lien fait tomber exactement celui qui en dépend (tableau 4.3) ;
- **codeurs** : chaque flux décodé et comparé octet à octet à l'original ; une table de fréquences faussée doit rendre le décodage inexact (section 10.5).

A.3 Le seuil déclaré avant

Un seuil ne se pose pas au jugé : il dérive d'une grandeur mesurée que l'on nomme, et il se déclare avant le lancement. Ainsi le budget de qualité $\kappa = 0{,}0055$ est l'écart mesuré entre deux moteurs justes (définition 4.5) ; le seuil de la mesure F1 (0,40 %) est le coût d'en-tête mesuré d'un flux de plus ; le seuil de bridage thermique (95 % du débit initial) est plus du double du bruit attendu d'une GEMV répétée.

A.4 Les séries ABBA et le bruit

Un coût ne se fonde jamais sur une seule passe. Chaque comparaison A/B se mesure en séries A1 B1 B2 A2, répétées au moins deux fois. Le bruit d'un programme à lui-même est $|A_1 - A_2|/\overline{A}$. Une différence est déclarée réelle seulement si

$$\frac{|\overline{A} - \overline{B}|}{\overline{A}} \;\geq\; 2\max(\text{bruit}_A, \text{bruit}_B) \quad \text{et} \quad \geq 3\,\%.$$

C'est cette règle qui déclare « dans le bruit » l'écart entre IOCP et IoRing (chapitre 3) et entre le cache d'experts de llama.cpp et les experts au processeur sur gpt-oss-20b (section 10.6).

A.5 Le second chemin

Tout chiffre publié dont on dépend est refait par un chemin indépendant. Le débit du disque est refait par le compteur cumulé du système (`Win32_PerfRawData_PerfDisk_PhysicalDisk`, relevé toutes les 0,5 s ; sans tampon, le disque doit avoir vu au moins 90 % des octets de l'outil) ; le coût de BitLocker par l'API de chiffrement de Windows ; l'inventaire de gpt-oss par la lecture de l'en-tête du fichier local après la lecture de l'en-tête distant ; les plafonds des architectures par un outil de l'arbitre qui n'importe aucun outil d'un autre.

A.6 L'alimentation et la machine partagée

Sur batterie, la machine change de régime (section 3.6.1). L'état de l'alimentation est relevé au début de chaque mesure par `Win32_Battery.BatteryStatus` (2 : sur secteur ; 1 : sur batterie) et noté dans le journal ; seules les mesures sur secteur sont retenues comme débits de référence, les autres sont dites. D'autres agents travaillaient sur la même machine : chaque passe journalise la charge étrangère (compteurs du disque, `nvidia-smi`), et les débits « au mieux » sont des bornes inférieures du repos.

A.7 Le juge de qualité

Le protocole de l'étude des modèles (section 9.6) : llama.cpp b11515 tel quel comme base ; un texte réel fixé (les 3 000 premiers octets de WikiText-2 test, empreinte SHA-256 notée), 256 jetons, décodage glouton ; jetons/s du préremplissage et de la génération, mémoire de pointe du processus (toutes les 100 ms), VRAM de pointe (toutes les 250 ms), octets lus sur le disque (compteur PDH intégré, avec un repos de 10 s pour chiffrer la charge étrangère), empreinte du texte produit. La qualité se juge par la divergence KL contre la base et le taux de premier choix identique : plancher de l'instrument (même moteur) 0,000055 au 99,9^{e} centile ; écart entre deux moteurs justes 0,0055 en moyenne, 95,96 % de premier choix identique.

A.8 La phrase barrée

Quand une mesure contredit un chiffre publié, même le nôtre, la phrase est barrée à sa place (~~ainsi~~) et l'on dit ce qui a changé ; jamais de retrait silencieux. Le tableau 11.2 rassemble celles des fondations.

Annexe B

Outils

Tout nombre [mes.] ou [calc.] du mémoire sort d'un outil commité. Cette annexe les liste, rangés par étude, chacun avec la première phrase de sa propre description, citée telle quelle. La table est engendrée par `memoire/outils/annexes.py` depuis ~~`git ls-files`~~ `git ls-tree` : elle ne s'écrit pas à la main et ~~suit le dépôt~~ décrit le dépôt à la publication de ce tome, au commit `b8af154`. Corrigé le 2026-10-29 : l'outil lisait le dépôt du jour, qui a continué de grandir après ce tome ; il lit maintenant ce commit, et la table reste celle du tome publié.

B.1 La chaîne du mémoire

- `memoire/construire.sh` (et `memoire/Makefile` pour qui a `make`) : régénère les données des figures et les tables des annexes, puis construit les deux PDF par `latexmk` (lualatex, biber) ; il échoue à la première erreur, à toute référence ou citation manquante, et à toute entrée que biber ne trouve pas.
- `memoire/outils/donnees_figures.py` : tire de chaque journal commité le CSV d'une figure, sans aucun nombre écrit à la main (seules constantes : les plafonds de fiche technique, chacun avec sa dérivation).
- `memoire/outils/annexes.py` : les tables de cette annexe et de la suivante.
- `memoire/outils/bib_etudes.py` : les bibliographies des études en biblatex.

B.2 Les outils des études

Outil	Sa propre description (première ligne, citée)
01 machine	
`outils/banc/Cargo.toml`	Dépendances : toutes déjà dans le cache cargo de la machine (construction –offline). windows-sys : MIT OU Apache-2.0. wgpu : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0. bytemuck …
`outils/banc/aes_xts.py`	Débit du déchiffrement XTS-AES sur le processeur, par l'API CNG de Windows (bcrypt.dll), de 1 à N fils : le coût logiciel de BitLocker, mesuré par un second chemin.
`outils/banc/analyse_carte.py`	Lecture du journal du banc carte : tableaux par opération, taille et interface (Vulkan, DX12), série ABBA vulkan, dx12, dx12, vulkan …
`outils/banc/analyse_disque.py`	Lecture du journal du banc disque : tableaux bloc x profondeur (moyenne des passages A et B, bruit ABBA), comparaison IOCP / IoRing selon la règle déclarée, second chemin (compteur du …
`outils/banc/analyse_thermique.py`	Analyse d'une passe thermique (pilote_thermique.py) : débit de la GEMV par fenêtre de 5 s et relevé nvidia-smi par seconde.

Outil	Sa propre description (première ligne, citée)
`outils/banc/csv_disque.py`	Extrait du journal du balayage disque (disque-balayage.jsonl) un CSV pour les figures du mémoire …
`outils/banc/pilote_carte.py`	Pilote du banc carte : chaque passe de carte.exe encadrée par un relevé nvidia-smi (utilisation, VRAM occupée, horloges, température, puissance) ; témoins –fausser …
`outils/banc/pilote_disque.py`	Pilote du banc disque : témoins, balayage bloc × profondeur de file en ordre ABBA, comparaison IOCP / IoRing, second chemin par le compteur cumulé du système (CIM).
`outils/banc/pilote_llama.py`	Base de comparaison : llama.cpp b11515 (CUDA 12.4) et Qwen3.5-9B Q4_K_M, en VRAM puis avec une partie des couches sur le processeur, en projection de fichier (mmap) ou en lecture directe. …
`outils/banc/pilote_thermique.py`	Thermique sur 5 minutes : GEMV q4b en boucle (carte.exe thermique) et relevé nvidia-smi chaque seconde (température, horloges, puissance, raisons de bridage). …
`outils/banc/src/bin/carte.rs`	Banc de la carte graphique par wgpu (Vulkan ou DirectX 12), sans boîte à outils CUDA.
`outils/banc/src/bin/disque.rs`	Banc du disque : lecture NVMe sans tampon (FILE_FLAG_NO_BUFFERING), séquentielle et aléatoire, par port de complétion (IOCP) ou par IoRing, à profondeur de file fixée.
`outils/banc/src/bin/memoire.rs`	Banc de la RAM : débit de lecture, d'écriture et de copie, de 1 à N fils, et latence d'un accès dépendant (chasse de pointeurs sur un cycle aléatoire).
`outils/banc/src/lib.rs`	Pièces communes du banc : motif de données vérifiable, statistiques, arguments.
`outils/banc/suite_secteur.py`	La suite « sur secteur » : les mesures que la batterie fausse, relancées dès que la machine est branchée.
`outils/banc/toit.py`	Le toit (roofline) de troncons : plafond de jetons par seconde selon l'étage d'où viennent les poids, et facteur de compression qui rend la lecture disque aussi rapide que la PCIe.
02 quantification	
`outils/quant-bib.py`	Sources de l'étude 02 en BibTeX (biblatex) pour le mémoire : memoire/sources/quantification.bib.
`outils/quant-csv-memoire.py`	Données de l'étude 02 (quantification) en CSV, pour les figures pgfplots du mémoire.
`outils/quant-echelle.sh`	Échelle de quantification llama.cpp sur un modèle : imatrix, requantification vers chaque type, bits par poids réels, divergence KL et perplexité contre la base.
`outils/quant-gemv-treillis.cu`	Coût du décodage dans le noyau : produit matrice-vecteur (lot de 1) sur la RTX 4060, poids stockés en FP16, en scalaire 2 bits, en scalaire 4 bits, et en treillis à décalage 2 bits (code …
`outils/quant-gguf-bits.py`	Bits par poids réels d'un fichier GGUF, métadonnées comprises, et octets lus par jeton.
`outils/quant-mse-gaussien.py`	Erreur quadratique de quantificateurs à 1, 2 et 3 bits par poids sur une source gaussienne i.i.d. …
`outils/quant-mse-iq2xxs.py`	Place d'IQ2_XXS (llama.cpp) sur le banc gaussien du § 2 de fondations/02-quantification.md.

Outil	Sa propre description (première ligne, citée)
`outils/quant-mse-xor.py`	Treillis à décalage à code linéaire par parités (sans multiplication), sur source N(0,1).
`outils/quant-resume-echelle.py`	Résumé d'une échelle de quantification produite par quant-echelle.sh.
`outils/quant-vitesse.py`	Vitesse de décodage (lot de 1) de fichiers GGUF quantifiés et débit effectif des poids.
`outils/quant-vulkan-gemv/Cargo.toml`	Dépendances (mêmes versions que outils/banc, déjà dans le cache cargo) : wgpu : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0.
`outils/quant-vulkan-gemv/src/main.rs`	Coût du décodage dans un shader de calcul portable (wgpu : Vulkan ou DirectX 12) : produit matrice-vecteur 16 384 × 16 384, poids en scalaire 2 bits, en treillis à décalage 2 bits (code …
03 entropie	
`outils/03-entropie/attendre_calme.sh`	Lance une commande seulement quand la machine est calme, trois relevés de suite à 10 s d'écart. mode « processeur » : charge du processeur sous 20 % et aucun processus llama-*. …
`outils/03-entropie/champs_k.py`	Découpe des blocs quantifiés ggml en leurs champs, écrite de zéro.
`outils/03-entropie/cuda/construire.bat`	Construit rans_v4.exe (second chemin du decodeur) et dispersion.exe (reassemblage des flux de classe), CUDA, sm_89, hors du depot. …
`outils/03-entropie/cuda/dispersion.cu`	Coût de la passe de réassemblage des codes MXFP4 répartis en trois flux de classe (§1.6 de 03-entropie.md) : chaque bloc de 16 octets revient à sa place depuis le flux de sa classe. …
`outils/03-entropie/cuda/rans_v4.cu`	Second chemin pour le débit du décodeur rANS v4 : le même algorithme que decoder_v4 de decodeur.wgsl, écrit en CUDA (intrinsèques de warp …
`outils/03-entropie/debits_cpu.py`	Débits de décompression sur le processeur : zstd et lz4 contre une recopie mémoire.
`outils/03-entropie/flux_octets.py`	Ce qu'un codeur d'octets d'ordre 0 (le décodeur v4) peut prendre à chaque type K, si l'on sépare le bloc en flux d'octets de même nature (le « byte grouping » de ZipNN, appliqué aux champs …
`outils/03-entropie/gguf_lecture.py`	Lecteur GGUF minimal, écrit de zéro pour troncons.
`outils/03-entropie/mesure_entropie.py`	Entropie réelle des poids quantifiés d'un GGUF, champ par champ.
`outils/03-entropie/mxfp4_contexte.py`	Entropie des échelles E8M0 des tenseurs MXFP4, avec et sans contexte.
`outils/03-entropie/passes_carte.sh`	Passes de carte de l'étude 03, chacune dans sa propre fenêtre calme (attendre_calme.sh). 1. banc_gemv sur les codes MXFP4 réels de gpt-oss-20b : brut, fondu, décodage, deux passes ; 2. …
`outils/03-entropie/passes_secteur.sh`	Passes de carte de l'étude 03, machine sur secteur (relevé à 08:33 le 2026-10-15 : Win32_Battery = 2, horloges 2 250 / 8 101 MHz), chacune dans sa fenêtre calme.
`outils/03-entropie/proportions_classes.py`	Proportions des trois classes d'échelle (e < médiane de ligne, e = médiane, e > médiane) sur tous les tenseurs MXFP4 d'un GGUF …
`outils/03-entropie/rans/Cargo.toml`	Dépendances, toutes dans le cache cargo de la machine (construction –offline) : wgpu 30.0.1 : MIT OU Apache-2.0. pollster : MIT OU Apache-2.0. bytemuck : Zlib OU Apache-2.0 OU MIT. rayon …

Outil	Sa propre description (première ligne, citée)
`outils/03-entropie/rans/src/bin/banc_gemv.rs`	Banc de la question clé : décoder le flux entropique DANS le produit matrice-vecteur, ou JUSTE AVANT, quand les poids résident en VRAM.
`outils/03-entropie/rans/src/bin/banc_rans.rs`	Banc du rANS entrelacé sur les codes Q4_K d'un GGUF.
`outils/03-entropie/rans/src/bin/micro_cpu.rs`	Micro-banc du décodeur processeur sur données synthétiques (distribution des codes Q4_K mesurée), pour séparer le coût du code de la contention de la machine.
`outils/03-entropie/rans/src/bin/taux_mxfp4.rs`	Taux réel d'un codage MXFP4 par champs, sur tous les tenseurs MXFP4 d'un GGUF.
`outils/03-entropie/rans/src/bin/vider_flux.rs`	Écrit sur le disque un lot codé par le rANS v4 (alphabet octet), pour qu'un second décodeur, écrit dans un autre langage (CUDA, outils/03-entropie/cuda/rans_v4.cu), le décode et soit …
`outils/03-entropie/rans/src/decodeur.wgsl`	Décodeur rANS entrelacé par sous-groupe (format décrit dans lib.rs). Un sous-groupe de 32 invocations décode une tuile …
`outils/03-entropie/rans/src/gemv.wgsl`	Produit matrice-vecteur (lot 1) sur des poids MXFP4, avec et sans décodage rANS fondu.
`outils/03-entropie/rans/src/lib.rs`	rANS entrelacé par sous-groupe pour les codes 4 bits des blocs Q4_K.
`outils/03-entropie/rd_proxy.py`	Au-delà du sans-perte : à erreur égale, combien de bits gagne une quantification taillée pour un codeur entropique (ECSQ), face à une quantification à débit fixe ?
`outils/03-entropie/resume_entropie.py`	Résumé des journaux de mesure_entropie.py : par type, par rôle, et pour le modèle.
`outils/03-entropie/temoin_champs.py`	Témoin : notre découpe des champs reconstruit exactement les poids que la bibliothèque gguf (llama.cpp, licence MIT, second chemin indépendant) déquantifie.
`outils/03-entropie/vers_csv.py`	Tire des journaux de l'étude 03 les CSV des figures du mémoire.
04 chargement	
`outils/chargement_juge.py`	Le juge physique du chargement : octets lus par jeton, étage par étage, divisés par le débit de chaque étage. Décodage, un jeton à la fois (lot 1), modèle MoE ou dense.
`outils/chargement_localite.py`	Localité des experts d'un jeton au suivant, et ce qu'un cache en tire.
`outils/chargement_pcie.py`	Mesure des étages RAM -> PCIe -> VRAM de la machine, sans boîte à outils CUDA.
05 sparsité	
`outils/05-sparsite/amortissement.py`	Amortissement d'une lecture de poids sur plusieurs jetons : le calcul de l'étude 05, avec son témoin par simulation.
`outils/05-sparsite/gguf_f16.py`	Lecteur GGUF réduit au nécessaire de l'étude 05 : métadonnées et tenseurs F32 / F16 en tableaux numpy (memmap, sans copie).
`outils/05-sparsite/llama_numpy.py`	Passe avant d'un modèle d'architecture « llama » (GGUF F16), en numpy pur, avec un crochet sur l'entrée de chaque projection linéaire.
`outils/05-sparsite/mesures_sparsite.py`	Mesures de sparsité des activations sur la passe avant numpy (étude 05).

Outil	Sa propre description (première ligne, citée)
outils/05-sparsite/resume_cn.py	Résumé des séries ABBA de llama-bench (pp n, -n 0) : temps d'une passe de n jetons, médiane des séries, bruit, et verdict contre le seuil déclaré.
outils/05-sparsite/temoin_passe_avant.py	Témoin de la passe avant numpy : sa perplexité doit retrouver celle de llama-perplexity (b11515) sur les mêmes jetons, fenêtre 512, 16 fenêtres.
outils/sparsite-inventaire-gguf.py	Inventaire d'un fichier GGUF, sans dépendance : métadonnées scalaires et octets par famille de tenseurs.
06 modèles	
outils/creer-disque-modeles.ps1	creer-disque-modeles.ps1 – cree le disque D : "Modeles", non chiffre, pour les poids des modeles.
outils/inventaire_modeles.py	Inventaire des modèles lourds : paramètres totaux et actifs, lus dans les en-têtes safetensors.
outils/modeles-base.py	Banc de la base de comparaison : llama.cpp tel quel (mmap), sur un texte réel fixé, en glouton.
outils/modeles-fichiers-a-telecharger.py	Liste exacte des fichiers à demander : nom, taille, empreinte SHA-256 (champ LFS de l'API).
outils/modeles-gguf-entete-distant.py	Octets lus par jeton d'un fichier GGUF précis, lus dans son en-tête à distance (requêtes Range).
outils/modeles-gguf-tailles.py	Tailles réelles des quantifications GGUF publiées, lues dans l'arbre des dépôts Hugging Face.
outils/modeles-juge.sh	Juge de qualité léger, identique pour tous les modèles et tous les moteurs. Usage : modeles-juge.sh <modèle.gguf> <étiquette> <dossier-sortie> [base-logits.kld] Sans 4e argument …
outils/modeles-octets-par-jeton.py	Octets lus par jeton et plafonds physiques, pour chaque modèle de l'inventaire.
outils/modeles-plafonds-fichiers.py	Plafonds physiques par jeton pour les fichiers GGUF exacts candidats au téléchargement.
outils/modeles-temoin-disque.py	Témoin du compteur de disque de modeles-base.py (PDH, PhysicalDisk(_Total) / Disk Read Bytes/sec).
outils/modeles-temoin-inventaire.py	Témoin de l'inventaire : nos comptes de paramètres contre ceux que les éditeurs publient.
outils/modeles_physique.py	Plafonds physiques de la machine de Stone et modèle de coût par jeton (calculé, jamais mesuré).
07 synthèse	
outils/07-synthese/inventaire_gptoss.py	Inventaire local des fichiers gpt-oss MXFP4, par lecture de leur en-tête GGUF.
outils/07-synthese/plafonds.py	L'équation du jeton, appliquée aux architectures candidates de troncons.
outils/07-synthese/raffinement_mxfp4.py	Raffinement exact d'un format MXFP4 natif : ce que coûte et ce que vaut une couche de base.
décisions	
outils/arbitre/decoupe/Cargo.toml	Une seule dépendance, déjà dans le cache cargo de la machine (construction –offline) : rayon 1.12.0 : MIT OU Apache-2.0. …
outils/arbitre/decoupe/src/main.rs	Mesure F1 de l'arbitre (decisions/ARCHITECTURE.md) : ce que coûte, en octets réels, la découpe d'un code MXFP4 en base + complément, contre le flux unique de l'architecture A.

Outil	Sa propre description (première ligne, citée)
`outils/arbitre/verif_plafonds.py`	Second chemin de l'arbitre : les plafonds de A, A0, B1 et B3 refaits par des formules écrites ici.
`outils/arbitre/vers_csv.py`	Données des figures du mémoire tirées des journaux de l'arbitre (rien n'est recalculé ici).
`outils/decisions/partisan_a.py`	Les chiffres du partisan de l'architecture A (« l'exactitude et l'ingénierie d'abord »).
`outils/decisions/partisan_b.py`	Les chiffres du partisan de B (« la contribution d'abord ») : ce que B gagne vraiment quand on compte le trafic des compléments, et ce que vaut une base lisible par llama.cpp.
mesures gpt-oss	
`outils/gptoss-routage/construire.bat`	Construit traceur.exe contre les DLL officielles de llama.cpp b11515 (binaires CUDA 12.4). en-têtes : sources officielles du même commit (3d65c90d0, licence MIT), dossier SRC …
`outils/gptoss-routage/faire_def.py`	Tire un fichier .def (bibliothèque d'importation) de la sortie de « dumpbin /exports ».
`outils/gptoss-routage/tracer.sh`	Relève les traces de routage de gpt-oss (20b ou 120b) sur le corpus en quatre genres, puis sur l'invite fixée de la base (étude 06) avec la même génération gloutonne que la base, pour le …
`outils/gptoss-routage/traceur.cpp`	traceur.cpp : relève, jeton par jeton et couche par couche, les experts choisis par le routeur d'un modèle MoE (écrit pour gpt-oss) en faisant tourner llama.cpp b11515 tel quel.
`outils/gptoss_base_resume.py`	Résumé d'une série de la base (passes.csv de gptoss_base_serie.py) : moyenne, bruit, décision.
`outils/gptoss_base_serie.py`	Séries alternées de la base llama.cpp b11515 sur gpt-oss (protocole de fondations/06-modeles.md § 7).
`outils/gptoss_comparer_traces.py`	Compare deux traces de routage du même texte : part des choix (jeton, couche) identiques.
`outils/gptoss_corpus.py`	Corpus de texte réel pour relever le routage des experts de gpt-oss : quatre genres.
`outils/gptoss_figures.py`	Données et figures pgfplots du mémoire pour les mesures sur gpt-oss (localité et base).
`outils/gptoss_localite.py`	Localité des experts de gpt-oss sur traces réelles, et plafond de jetons/s qui en découle.
`outils/gptoss_octets.py`	Octets par expert et octets toujours lus de gpt-oss, lus dans l'entête des fichiers GGUF locaux.

Annexe C

Journaux et reproduction des figures

Ce qui produit un chiffre publié est commité : outils et journaux. Cette annexe dit où sont les journaux et quelle donnée lit chaque graphe. Les deux tables sont engendrées par `memoire/outils/annexes.py`.

C.1 Reconstruire le mémoire

Depuis `memoire/`, dans Git Bash, avec MiKTeX (lualatex, biber, latexmk) et Python 3.13 sans dépendance :

```
./construire.sh          # données des figures, annexes, puis les deux PDF
./construire.sh fr       # une seule langue
./construire.sh propre   # efface les fichiers intermédiaires
```

Chaque graphe pgfplots lit un CSV commité ; chaque schéma TikZ est un fichier de `memoire/figures/schemas/` ; les commandes des figures sont communes aux deux langues, qui ne diffèrent que par leurs textes (`\EF{anglais}{français}`). Le texte d'un mémoire se lit en parallèle de l'autre : mêmes chapitres, mêmes étiquettes, mêmes figures.

C.2 Les journaux

Dossier de journaux	Fichiers	Volume (Mo)
`decisions/journaux/`	2	0,05
`decisions/journaux/arbitre/`	9	0,05
`decisions/journaux/csv/`	2	0,00
`decisions/journaux/partisan-a/`	5	0,03
`fondations/journaux/01-machine/`	31	0,67
`fondations/journaux/02-quantification/`	106	0,89
`fondations/journaux/03-entropie/`	81	1,94
`fondations/journaux/04-chargement/`	11	0,08
`fondations/journaux/05-sparsite/`	26	0,19
`fondations/journaux/06-modeles/`	36	1,02
`fondations/journaux/07-synthese/`	10	0,11
`mesures/`	2	0,04
`mesures/base-gpt-oss/`	48	0,19
`mesures/localite-gpt-oss/`	32	3,60
`mesures/moteur/`	118	0,88

C.3 Les données de chaque graphe

Graphe (commande)	Fichiers de données qu'il lit
\GrapheArchitectures	memoire/figures/donnees/archi-plafonds.csv
\GrapheBtroisLambda	decisions/journaux/csv/b3-lambda-gptoss120b.csv
\GrapheFun	memoire/figures/donnees/archi-f1.csv
\GrapheLocaliteMesuree	memoire/figures/donnees/form-localite.csv memoire/figures/donnees/archi-localite-20b.csv
\GrapheBaseVingt	memoire/figures/donnees/archi-base-20b.csv
\GrapheJuge	memoire/figures/donnees/charge-juge.csv
\GrapheSensibiliteH	memoire/figures/donnees/charge-sensibilite-h.csv
\GrapheLocalite	memoire/figures/donnees/charge-localite-collant.csv
\GrapheHistogrammeCodes	../../fondations/journaux/03-entropie/csv/fig-histogramme-codes.csv
\GrapheEntropieGroupes	memoire/figures/donnees/entropie-groupes.csv memoire/figures/donnees/entropie-groupes-mesures.csv
\GrapheEntropiePlage	memoire/figures/donnees/entropie-plage.csv
\GrapheEchelles	memoire/figures/donnees/entropie-echelles-20b.csv memoire/figures/donnees/entropie-echelles-120b.csv
\GrapheTauxTenseurs	memoire/figures/donnees/entropie-taux-20b-down.csv memoire/figures/donnees/entropie-taux-20b-gate.csv memoire/figures/donnees/entropie-taux-20b-up.csv memoire/figures/donnees/entropie-taux-120b-down.csv memoire/figures/donnees/entropie-taux-120b-gate.csv memoire/figures/donnees/entropie-taux-120b-up.csv
\GrapheDebitsDecodeur	memoire/figures/donnees/entropie-debits-batterie.csv memoire/figures/donnees/entropie-debits-secteur.csv memoire/figures/donnees/entropie-debits-mxfp4.csv memoire/figures/donnees/entropie-debits-cuda.csv

Graphe (commande)	Fichiers de données qu'il lit
\GrapheGemvEntropie	memoire/figures/donnees/entropie-gemv-secteur.csv memoire/figures/donnees/entropie-gemv-batterie.csv
\GrapheRdProxy	../../fondations/journaux/03-entropie/csv/rd-proxy.csv
\GrapheLocaliteHypotheses	memoire/figures/donnees/form-localite.csv memoire/figures/donnees/form-localite-calage.csv
\GraphePlafondHs	fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-hs.csv
\GraphePlafondRam	fondations/journaux/07-synthese/csv/plafond-gptoss120b-contre-ram.csv
\GrapheDisque	memoire/figures/donnees/machine-disque-alea-qd1.csv memoire/figures/donnees/machine-disque-alea-qd4.csv memoire/figures/donnees/machine-disque-alea-qd16.csv memoire/figures/donnees/machine-disque-alea-qd64.csv
\GrapheEtages	memoire/figures/donnees/machine-etages.csv
\GrapheAES	memoire/figures/donnees/machine-aes-128.csv memoire/figures/donnees/machine-aes-256.csv
\GrapheLatence	memoire/figures/donnees/machine-ram-latence.csv
\GrapheThermique	memoire/figures/donnees/machine-thermique-secteur.csv memoire/figures/donnees/machine-thermique-batterie.csv
\GraphePCIe	memoire/figures/donnees/machine-pcie-generation.csv
\GrapheGEMV	memoire/figures/donnees/machine-gemv.csv
\GrapheToitR	fondations/journaux/01-machine/toit-9b-compression.csv
\GrapheToitPlacements	memoire/figures/donnees/machine-toit-placements.csv
\GrapheMoE	memoire/figures/donnees/modeles-moe.csv
\GrapheMSE	fondations/journaux/02-quantification/csv/fig-q1-lloyd.csv fondations/journaux/02-quantification/csv/fig-q1-vq8.csv fondations/journaux/02-quantification/csv/fig-q1-iq2xxs.csv fondations/journaux/02-quantification/csv/fig-q1-treillis.csv

Graphe (commande)	Fichiers de données qu'il lit
\GrapheGEMVQuant	memoire/figures/donnees/quant-gemv-vulkan.csv memoire/figures/donnees/quant-gemv-dx12.csv
\GrapheEchelle	fondations/journaux/02-quantification/csv/fig-q3-4B.csv fondations/journaux/02-quantification/csv/fig-q3-9B.csv
\GrapheVitesseQuant	fondations/journaux/02-quantification/csv/fig-q6-vitesse-4B.csv fondations/journaux/02-quantification/csv/fig-q6-vitesse-9B.csv
\GrapheIsoOctets	fondations/journaux/02-quantification/csv/fig-q4-llama2-7B.csv fondations/journaux/02-quantification/csv/fig-q4-llama2-13B.csv fondations/journaux/02-quantification/csv/fig-q4-llama2-70B.csv
\GrapheMMLU	fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-tetra.csv fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-awq.csv fondations/journaux/02-quantification/csv/fig-q4-qwen3-mmlu-fp16.csv
\GrapheAmortissement	memoire/figures/donnees/sparse-E-50.csv memoire/figures/donnees/sparse-E-60.csv memoire/figures/donnees/sparse-E-70.csv memoire/figures/donnees/sparse-E-80.csv memoire/figures/donnees/sparse-E-90.csv memoire/figures/donnees/sparse-E-95.csv
\GrapheUnions	memoire/figures/donnees/sparse-union-s25.csv memoire/figures/donnees/sparse-union-s50.csv memoire/figures/donnees/sparse-union-s70.csv memoire/figures/donnees/sparse-union-s90.csv
\GrapheCn	memoire/figures/donnees/sparse-cn.csv

Bibliographie

[1] S. Williams, A. Waterman et D. Patterson. « Roofline : An Insightful Visual Performance Model for Multicore Architectures ». In : *Communications of the ACM* 52.4 (2009-04). Origine du « toit » ; ici étendu à une chaîne d'étages de stockage (proposition 1)., p. 65-76. doi : 10.1145/1498765.1498785. (Visité le 2026-10-15).

[2] P. J. Denning. « The working set model for program behavior ». In : *Communications of the ACM* 11.5 (1968-05). Métadonnées relues par l'API de Crossref le 2026-10-15, p. 323-333. doi : 10.1145/363095.363141. (Visité le 2026-10-15).

[3] L. A. Belady. « A Study of Replacement Algorithms for a Virtual-Storage Computer ». In : *IBM Systems Journal* 5.2 (1966), p. 78-101. doi : 10.1147/sj.52.0078. (Visité le 2026-10-15).

[4] S. Barrett. *Sparse Virtual Textures*. Exposé à la Game Developers Conference 2008 ; la page ne signe que du prénom « Sean » ; le nom complet est celui du titulaire du site, connu par ailleurs, et n'est pas relu sur la page. 2008. url : https://silverspaceship.com/src/svt/ (visité le 2026-10-15).

[5] Microsoft DirectX team. *DirectStorage 1.4 Release Adds Support for Zstandard*. Dépôt github.com/microsoft/DirectStorage : MIT, GDeflate Apache-2.0, commit c53f149 du 2026-08-04. 2026-03-11. url : https://devblogs.microsoft.com/directx/directstorage-1-4-release-adds-support-for-zstandard/ (visité le 2026-10-15).

[6] lyogavin. *AirLLM*. Apache-2.0, commit 0f679cc. 2026-10-14. url : https://github.com/lyogavin/airllm (visité le 2026-10-15).

[7] DeepSpeed Team. *ZeRO-Inference : Democratizing Massive Model Inference*. 2022-09-09. url : https://www.deepspeed.ai/2022/09/09/zero-inference.html (visité le 2026-10-15).

[8] Y. Sheng et al. *FlexGen : High-Throughput Generative Inference of Large Language Models with a Single GPU*. ICML 2023 ; dernière version 2023-06-12. 2023-03-13. arXiv : 2303.06865. url : https://arxiv.org/abs/2303.06865 (visité le 2026-10-15).

[9] ggml-org. *llama.cpp*. Version commit 843e7dc ; exécutables b11515 (2026-10-09). Licence MIT. 2026-10-15. url : https://github.com/ggml-org/llama.cpp (visité le 2026-10-15).

[10] Z. Liu et al. *Deja Vu : Contextual Sparsity for Efficient LLMs at Inference Time*. ICML 2023. 2023-10-26. arXiv : 2310.17157. url : https://arxiv.org/abs/2310.17157 (visité le 2026-10-15).

[11] Y. Song et al. *PowerInfer : Fast Large Language Model Serving with a Consumer-grade GPU*. SOSP 2024 ; dernière version 2024-12-12. 2023-12-16. arXiv : 2312.12456. url : https://arxiv.org/abs/2312.12456 (visité le 2026-10-15).

[12] Z. Xue et al. *PowerInfer-2 : Fast Large Language Model Inference on a Smartphone*. dernière version 2024-12-12. 2024-06-10. arXiv : 2406.06282. url : https://arxiv.org/abs/2406.06282 (visité le 2026-10-15).

[13] K. Alizadeh et al. *LLM in a Flash : Efficient Large Language Model Inference with Limited Memory*. ACL 2024 ; dernière version 2024-07-30. 2023-12-12. arXiv : 2312.11514. URL : https://arxiv.org/abs/2312.11514 (visité le 2026-10-15).

[14] A. Eliseev et D. Mazur. *Fast Inference of Mixture-of-Experts Language Models with Offloading*. 2023-12-28. arXiv : 2312.17238. URL : https://arxiv.org/abs/2312.17238 (visité le 2026-10-15).

[15] X. Song et al. *ProMoE : Fast MoE-based LLM Serving using Proactive Caching*. dernière version 2025-09-01. 2024-10-29. arXiv : 2410.22134. URL : https://arxiv.org/abs/2410.22134 (visité le 2026-10-15).

[16] P. Tang et al. *HOBBIT : A Mixed Precision Expert Offloading System for Fast MoE Inference*. dernière version 2024-11-06. 2024-11-03. arXiv : 2411.01433. URL : https://arxiv.org/abs/2411.01433 (visité le 2026-10-15).

[17] X. He et al. *ExpertFlow : Optimized Expert Activation and Token Allocation for Efficient Mixture-of-Experts Inference*. DAC 2026 ; dernière version 2026-04-02. 2024-10-23. arXiv : 2410.17954. URL : https://arxiv.org/abs/2410.17954 (visité le 2026-10-15).

[18] Wang et al. *SeqMoE : Sequence-level Expert Prefetching for Offloaded MoE Inference*. 2026-09-11. arXiv : 2609.12978. URL : https://arxiv.org/abs/2609.12978 (visité le 2026-10-15).

[19] Lin et al. *The Other Half of the Memory Wall (Edge0)*. 2026-09-17. arXiv : 2609.18063. URL : https://arxiv.org/abs/2609.18063 (visité le 2026-10-15).

[20] K. Kamahori et al. *Fiddler : CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models*. ICLR 2025 ; dernière version 2025-05-01. 2024-02-10. arXiv : 2402.07033. URL : https://arxiv.org/abs/2402.07033 (visité le 2026-10-15).

[21] H. Chen et al. *KTransformers : Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models*. SOSP 2025, DOI 10.1145/3731569.3764843. 2025-10. URL : https://madsys.cs.tsinghua.edu.cn/publication/ktransformers-unleashing-the-full-potential-of-cpu/gpu-hybrid-inference-for-moe-models/SOSP25-chen.pdf (visité le 2026-10-15).

[22] ggml-org. *llama.cpp pull request 25294 : experts read from disk with an explicit cache*. ouverte au 2026-10-15. 2026-07-04. URL : https://github.com/ggml-org/llama.cpp/pull/25294 (visité le 2026-10-15).

[23] Wok3b. *moe-stream*. licence MIT d'après la page du dépôt. 2026. URL : https://github.com/Wok3b/moe-stream (visité le 2026-10-15).

[24] Liang et al. *SSD-LLaMA : SSD, RAM and VRAM as One Hierarchy for Trillion-Parameter MoE Inference*. 2026-09-16. arXiv : 2609.18110. URL : https://arxiv.org/abs/2609.18110 (visité le 2026-10-15).

[25] M. Chen, W. Shao, P. Xu et al. *EfficientQAT : Efficient Quantization-Aware Training for Large Language Models*. Première version 2024-07-10, dernière 2025-05-19 (2407.11062v3). ACL 2025 Main, camera ready version. 2024-07-10. arXiv : 2407.11062. URL : https://arxiv.org/abs/2407.11062 (visité le 2026-10-15).

[26] V. Malinovskii, A. Panferov, I. Ilin, H. Guo, P. Richtárik et D. Alistarh. *Pushing the Limits of Large Language Model Quantization via the Linearity Theorem*. Première version 2024-11-26, dernière 2024-11-26 (2411.17525v1). 2024-11-26. arXiv : 2411.17525. URL : https://arxiv.org/abs/2411.17525 (visité le 2026-10-15).

[27] A. Tseng, J. Chee, Q. Sun, V. Kuleshov et C. D. Sa. *QuIP# : Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks*. Première version 2024-02-06, dernière 2024-06-04 (2402.04396v2). ICML 2024. 2024-02-06. arXiv : 2402.04396. URL : https://arxiv.org/abs/2402.04396 (visité le 2026-10-15).

[28] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko et D. Alistarh. *Extreme Compression of Large Language Models via Additive Quantization*. Première version 2024-01-11, dernière 2024-09-11 (2401.06118v4). ICML, 2024. 2024-01-11. arXiv : 2401.06118. URL : https://arxiv.org/abs/2401.06118 (visité le 2026-10-15).

[29] V. Malinovskii, D. Mazur, I. Ilin et al. *PV-Tuning : Beyond Straight-Through Estimation for Extreme LLM Compression*. Première version 2024-05-23, dernière 2024-05-30 (2405.14852v2). Preprint. 2024-05-23. arXiv : 2405.14852. URL : https://arxiv.org/abs/2405.14852 (visité le 2026-10-15).

[30] Y. Liu, J. Wen, Y. Wang et al. *VPTQ : Extreme Low-bit Vector Post-Training Quantization for Large Language Models*. Première version 2024-09-25, dernière 2024-10-22 (2409.17066v2). EMNLP 2024, Main, Poster. 2024-09-25. arXiv : 2409.17066. URL : https://arxiv.org/abs/2409.17066 (visité le 2026-10-15).

[31] A. Tseng, Q. Sun, D. Hou et C. D. Sa. *QTIP : Quantization with Trellises and Incoherence Processing*. Première version 2024-06-17, dernière 2025-06-18 (2406.11235v4). NeurIPS 2024 Spotlight. 2024-06-17. arXiv : 2406.11235. URL : https://arxiv.org/abs/2406.11235 (visité le 2026-10-15).

[32] G. Ding, Z. Wang, R. Li et J. Liu. *CurveTQ : Rotation-Free Trellis Quantization of LLM Weights via Curvature-Weighted Search*. Première version 2026-10-06, dernière 2026-10-06 (2610.09212v1). 2026-10-06. arXiv : 2610.09212. URL : https://arxiv.org/abs/2610.09212 (visité le 2026-10-15).

[33] X. Que, N. Elkayam, S. Pyakurel, S. Pan et D. Gope. *XOR-Trellis : Ultra-Low-Complexity Dequantization and Curvature-Aware Hadamard-Free LLM Quantization*. Première version 2026-09-30, dernière 2026-09-30 (2610.00432v1). 2026-09-30. arXiv : 2610.00432. URL : https://arxiv.org/abs/2610.00432 (visité le 2026-10-15).

[34] P.-J. Malandrino. *Tetra : Serving Leech-Lattice Quantized LLMs at 2.7 Bits per Parameter*. Première version 2026-09-28, dernière 2026-09-28 (2609.35465v1). 15 pages, 4 figures, 8 tables. Code, measurement logs and preregistrations : https://github.com/pjmalandrino/llvq. 2026-09-28. arXiv : 2609.35465. URL : https://arxiv.org/abs/2609.35465 (visité le 2026-10-15).

[35] S. Ma, H. Wang, L. Ma et al. *The Era of 1-bit LLMs : All Large Language Models are in 1.58 Bits*. Première version 2024-02-27, dernière 2024-02-27 (2402.17764v1). Work in progress. 2024-02-27. arXiv : 2402.17764. URL : https://arxiv.org/abs/2402.17764 (visité le 2026-10-15).

[36] Z. Liu, C. Zhao, H. Huang et al. *ParetoQ : Improving Scaling Laws in Extremely Low-bit LLM Quantization*. Première version 2025-02-04, dernière 2025-10-13 (2502.02631v2). NeurIPS 2025. Model weights are available at https://huggingface.co/collections/facebook/mobilellm-6722be18cb86c20ebe113e95. 2025-02-04. arXiv : 2502.02631. URL : https://arxiv.org/abs/2502.02631 (visité le 2026-10-15).

[37] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. URL : https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visité le 2026-10-15).

[38] NVIDIA, F. Abecassis, A. Agrusa et al. *Pretraining Large Language Models with NVFP4*. Première version 2025-09-29, dernière 2026-03-04 (2509.25149v2). Update includes : (1) fixing a typo in eq. 2 (2) updating author list, and (3) adding a related work. 2025-09-29. arXiv : 2509.25149. URL : https://arxiv.org/abs/2509.25149 (visité le 2026-10-15).

[39] S. Ashkboos, A. Mohtashami, M. L. Croci et al. *QuaRot : Outlier-Free 4-Bit Inference in Rotated LLMs*. Première version 2024-03-30, dernière 2024-10-29 (2404.00456v2). 21 pages, 7 figures. 2024-03-30. arXiv : 2404.00456. URL : https://arxiv.org/abs/2404.00456 (visité le 2026-10-15).

[40] Z. Liu, C. Zhao, I. Fedorov et al. *SpinQuant : LLM quantization with learned rotations*. Première version 2024-05-26, dernière 2025-02-20 (2405.16406v4). ICLR 2025. 2024-05-26. arXiv : 2405.16406. URL : https://arxiv.org/abs/2405.16406 (visité le 2026-10-15).

[41] T. Zhang, M. Hariri, S. Zhong et al. *70% Size, 100% Accuracy : Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11)*. NeurIPS 2025 ; v3 du 2026-01-01. Code : github.com/LeanModels/DFloat11, Apache-2.0, commit 4577338 du 2025-11-24. 2025-04-15. arXiv : 2504.11651. URL : https://arxiv.org/abs/2504.11651 (visité le 2026-10-15).

[42] M. Hershcovitch, A. Wood, L. Choshen et al. *ZipNN : Lossless Compression for AI Models*. v2 du 2025-06-04. Code : github.com/zipnn/zipnn, MIT, commit 6009704 du 2026-04-11. 2024. arXiv : 2411.05239. URL : https://arxiv.org/abs/2411.05239 (visité le 2026-10-15).

[43] P. Yubeaton, T. Mahmoud, S. Naga et al. *Huff-LLM : End-to-End Lossless Compression for Efficient LLM Inference*. Aucun code publié trouvé. 2025-02-02. arXiv : 2502.00922. URL : https://arxiv.org/abs/2502.00922 (visité le 2026-10-15).

[44] Z. Yang, T. Zhang, J. Xie, C. Li, Z. Xu et A. Shrivastava. *To Compress or Not ? Pushing the Frontier of Lossless GenAI Model Weights Compression with Exponent Concentration*. ECF8 ; aucun code lié sur la page. 2025-10-03. arXiv : 2510.02676. URL : https://arxiv.org/abs/2510.02676 (visité le 2026-10-15).

[45] R. Fan, X. Yu, X. Pan et al. « ZipServ : Fast and Memory-Efficient LLM Inference with Hardware-Aware Lossless Compression ». In : *Proceedings of ASPLOS 2026*. Code : github.com/HPMLL/ZipServ_ASPLOS26, Apache-2.0, commit 6f8a209 du 2025-12-19. 2026-03-18. arXiv : 2603.17435. URL : https://arxiv.org/abs/2603.17435 (visité le 2026-10-15).

[46] H. Tan, Y. Chen, G. Alonso, W.-F. Wong et B. He. « Approaching Shannon Bound with Lossless LLM Weight Compression ». In : *Proceedings of ISCA 2026*. Code non trouvé. 2026-06-14. arXiv : 2606.15789. URL : https://arxiv.org/abs/2606.15789 (visité le 2026-10-15).

[47] A. Sanyal, G. Datta, P. Mukherjee, S. P. Chinchali et M. Orshansky. *EntroLLM : Entropy Encoded Weight Compression for Efficient Large Language Model Inference on Edge Devices*. v4 du 2026-01-26. 2025-05-05. arXiv : 2505.02380. URL : https://arxiv.org/abs/2505.02380 (visité le 2026-10-15).

[48] H. Zhang, Z. Duan et Y. Chen. *EntroPack : Fast and Accurate Entropy-Coded Weight Compression at Arbitrary Bitrates*. Code : github.com/modelscope/entropack, non cloné, licence non vérifiée. 2026-09-28. arXiv : 2609.34185. URL : https://arxiv.org/abs/2609.34185 (visité le 2026-10-15).

[49] P. Putzky, M. Genzel, M. Mollenhauer, S. Schulze, T. Wollmann et S. Dietzel. « Float8@2bits : Entropy Coding Enables Data-Free Model Compression ». In : *Proceedings of ICML 2026*. EntQuant ; v2 du 2026-05-29. Code : github.com/merantix-momentum/entquant (licence non vérifiée). 2026-01-30. arXiv : 2601.22787. URL : https://arxiv.org/abs/2601.22787 (visité le 2026-10-15).

[50] E. Lifar, S. Savkin, O. Ordentlich et Y. Polyanskiy. *WaterSIC : Information-Theoretically (Near) Optimal Linear Layer Quantization*. v2 du 2026-06-01. Code : github.com/egorlifar/watersic (licence non vérifiée). 2026-03-05. arXiv : 2603.04956. URL : https://arxiv.org/abs/2603.04956 (visité le 2026-10-15).

[51] S. Nuri. *vram-compression : Entropy-coded LLM weight quantization with a fused rANS decode GEMV kernel*. Un seul commit (1af9a93, 2026-09-23). Aucun fichier de licence : tous droits réservés par défaut, lu seulement. 2026-09-23. URL : https://github.com/snuri00/vram-compression (visité le 2026-10-15).

[52] J. Duda. *Asymmetric numeral systems : entropy coding combining speed of Huffman coding with compression rate of arithmetic coding*. 2013. arXiv : 1311.2540. URL : https://arxiv.org/abs/1311.2540 (visité le 2026-10-15).

[53] F. Giesen. *Interleaved entropy coders*. 2014. arXiv : 1402.3392. URL : https://arxiv.org/abs/1402.3392 (visité le 2026-10-15).

[54] A. Weissenberger et B. Schmidt. « Massively Parallel ANS Decoding on GPUs ». In : *Proceedings of the 48th International Conference on Parallel Processing (ICPP)*. Code : github.com/weissenberger/multians, LGPL-3.0, commit 55515d8 du 2019-07-26. 2019. DOI : 10.1145/3337821.3337888. (Visité le 2026-10-15).

[55] F. Lin, K. Arunruangsirilert, H. Sun et J. Katto. « Recoil : Parallel rANS Decoding with Decoder-Adaptive Scalability ». In : *Proceedings of the 52nd International Conference on Parallel Processing (ICPP)*. Code : github.com/lin-toto/recoil, GPL-3.0, commit c36e25d du 2026-07-12. 2023. DOI : 10.1145/3605573.3605588. arXiv : 2306.12141. (Visité le 2026-10-15).

[56] NVIDIA. *nvCOMP Release Notes and Algorithm Selection*. Version 6.0.0, non datée ; binaire propriétaire. 2026. URL : https://docs.nvidia.com/cuda/nvcomp/release_notes.html (visité le 2026-10-15).

[57] Meta (facebookresearch). *DietGPU : GPU-based lossless compression for numerical data*. MIT, commit a4d70a1 du 2026-03-18. URL : https://github.com/facebookresearch/dietgpu (visité le 2026-10-15).

[58] Meta (facebook). *Zstandard*. README, zstd 1.5.7 et lz4 1.10.0 mesurés par lzbench sur Silesia, i7-9700K. URL : https://github.com/facebook/zstd (visité le 2026-10-15).

[59] Y. Leviathan, M. Kalman et Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML 2023 ; dernière version 2023-05-18. 2022-11-30. arXiv : 2211.17192. URL : https://arxiv.org/abs/2211.17192 (visité le 2026-10-15).

[60] C. Chen et al. *Accelerating Large Language Model Decoding with Speculative Sampling*. 2023-02-02. arXiv : 2302.01318. URL : https://arxiv.org/abs/2302.01318 (visité le 2026-10-15).

[61] T. Cai et al. *Medusa : Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. dernière version 2024-06-14. 2024-01-19. arXiv : 2401.10774. URL : https://arxiv.org/abs/2401.10774 (visité le 2026-10-15).

[62] Y. Li et al. *EAGLE-2 : Faster Inference of Language Models with Dynamic Draft Trees*. dernière version 2024-06-30. 2024-06-24. arXiv : 2406.16858. URL : https://arxiv.org/abs/2406.16858 (visité le 2026-10-15).

[63] Y. Li et al. *EAGLE-3 : Scaling up Inference Acceleration of Large Language Models via Training-Time Test*. dernière version 2025-04-23. 2025-03-03. arXiv : 2503.01840. URL : https://arxiv.org/abs/2503.01840 (visité le 2026-10-15).

[64] F. Gloeckle et al. *Better and Faster Large Language Models via Multi-token Prediction*. 2024-04-30. arXiv : 2404.19737. URL : https://arxiv.org/abs/2404.19737 (visité le 2026-10-15).

[65] DeepSeek-AI. *DeepSeek-V3 Technical Report*. dernière version 2025-02-18. 2024-12-27. arXiv : 2412.19437. URL : https://arxiv.org/abs/2412.19437 (visité le 2026-10-15).

[66] Chen, Liang et Liu. *DFlash : Block Diffusion Drafting for Speculative Decoding*. ICML 2026 ; dernière version 2026-05-28. 2026-02-05. arXiv : 2602.06036. URL : https://arxiv.org/abs/2602.06036 (visité le 2026-10-15).

[67] Y. Fu et al. *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. 2024-02-03. arXiv : 2402.02057. URL : https://arxiv.org/abs/2402.02057 (visité le 2026-10-15).

[68] X. Miao et al. *SpecInfer : Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification*. 2023-05-16. arXiv : 2305.09781. URL : https://arxiv.org/abs/2305.09781 (visité le 2026-10-15).

[69] Z. Chen et al. *Sequoia : Scalable, Robust, and Hardware-aware Speculative Decoding*. dernière version 2025-07-05. 2024-02-19. arXiv : 2402.12374. URL : https://arxiv.org/abs/2402.12374 (visité le 2026-10-15).

[70] R. Svirschevski et al. *SpecExec : Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices*. dernière version 2024-11-30. 2024-06-04. arXiv : 2406.02532. URL : https://arxiv.org/abs/2406.02532 (visité le 2026-10-15).

[71] Wang et al. *Substitute Speculative Decoding for Offloaded LLMs (SubSpec)*. NeurIPS 2025 ; dernière version 2025-10-08. 2025-09-22. arXiv : 2509.18344. URL : https://arxiv.org/abs/2509.18344 (visité le 2026-10-15).

[72] Saxena et al. *Utility-Driven Speculative Decoding for Mixture-of-Experts (Cascade)*. 2025-06-17. arXiv : 2506.20675. URL : https://arxiv.org/abs/2506.20675 (visité le 2026-10-15).

[73] McDanel et al. *MoE-Spec : Expert Budgeting for Speculative Decoding of Mixture-of-Experts*. 2026-02-17. arXiv : 2602.16052. URL : https://arxiv.org/abs/2602.16052 (visité le 2026-10-15).

[74] Liang et al. *AcceptMoE : Acceptance-aware Expert Budgets for Offloaded Speculative Decoding*. 2026-08-04. arXiv : 2608.02989. URL : https://arxiv.org/abs/2608.02989 (visité le 2026-10-15).

[75] Xie et al. *Less Experts, Faster Decoding (EcoSpec)*. 2026-07-14. arXiv : 2607.12696. URL : https://arxiv.org/abs/2607.12696 (visité le 2026-10-15).

[76] Samsung Electronics. *Samsung Electronics Unveils High-Performance PC SSD That Raises Everyday Computing and Gaming to a New Level*. PM9C1a : 6 000 Mo/s en lecture séquentielle, 900 k IOPS en lecture aléatoire (éditeur, non reproduit). Date : janvier 2023, d'après les reprises datées du 2023-01-13 ; la page primaire n'a pas répondu à la consultation. L'identification du disque MZVMA1T0HCLD au PM9C1a est probable, non prouvée. 2023-01. URL : https://news.samsungsemiconductor.com/global/samsung-electronics-unveils-high-performance-pc-ssd-that-raises-everyday-computing-and-gaming-to-a-new-level/ (visité le 2026-10-15).

[77] StorageReview. *Samsung PM9C1a SSD*. Reprise des chiffres de l'éditeur. URL : https://storagereview.com/?p=119560 (visité le 2026-10-15).

[78] Microsoft. *BypassIO for Filter Drivers*. Cas « Volume stack bypass is disabled (fvevol.sys) » quand BitLocker est actif ; chemin complet réservé à StorNVMe. 2025-11-05. URL : https://learn.microsoft.com/en-us/windows-hardware/drivers/ifs/bypassio (visité le 2026-10-15).

[79] Microsoft. *CNG Algorithm Identifiers (Bcrypt.h)*. BCRYPT_XTS_AES_ALGORITHM, L"XTS-AES", SP 800-38E et IEEE 1619-2007, depuis Windows 10. 2026-05-30. URL : https://learn.microsoft.com/en-us/windows/win32/seccng/cng-algorithm-identifiers (visité le 2026-10-15).

[80] *IEEE Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices*. Annexe B, vecteur 1 : témoin de aes_xts.py. IEEE, 2008. DOI : 10.1109/IEEESTD.2008.4493450. (Visité le 2026-10-15).

[81] Microsoft. *ioringapi.h header*. IoRing ; version 4 relevée par QueryIoRingCapabilities sur la machine. URL : https://learn.microsoft.com/en-us/windows/win32/api/ioringapi/ (visité le 2026-10-15).

[82] Microsoft. *Microsoft.Direct3D.DirectStorage (paquet NuGet), index des versions*. 1.3.0 (2025-06-27) : 2 816 824 octets ; 1.4.0-preview2-2606.904 (2026-06-16) : 3 986 072 octets (requêtes HEAD, rien téléchargé). URL : https://api.nuget.org/v3/registration5-gz-semver2/microsoft.direct3d.directstorage/index.json (visité le 2026-10-15).

[83] Microsoft DirectX Team. *DirectStorage 1.4 Release Adds Support for Zstandard*. 2026-03-11. URL : https://devblogs.microsoft.com/directx/directstorage-1-4-release-adds-support-for-zstandard/ (visité le 2026-10-15).

[84] gfx-rs. *wgpu*. Version 30.0.1. Licence MIT ou Apache-2.0. Dépendance du banc carte (Vulkan, DirectX 12). URL : https://github.com/gfx-rs/wgpu (visité le 2026-10-15).

[85] ggml-org. *llama.cpp*. Version b11515 (3d65c90d0). Licence MIT. Base de comparaison (llama-bench, moteur CUDA 12.4). URL : https://github.com/ggml-org/llama.cpp (visité le 2026-10-15).

[86] *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. 2019. DOI : 10.1109/IEEESTD.2019.8766229. (Visité le 2026-10-15).

[87] B. D. Rouhani, R. Zhao, A. More et al. *Microscaling Data Formats for Deep Learning*. Première version 2023-10-16, dernière 2023-10-19 (2310.10537v3). 2023-10-16. arXiv : 2310.10537. URL : https://arxiv.org/abs/2310.10537 (visité le 2026-10-15).

[88] P. Micikevicius et al. *FP8 Formats for Deep Learning*. Formats E4M3 et E5M2. 2022-09-12. arXiv : 2209.05433. URL : https://arxiv.org/abs/2209.05433 (visité le 2026-10-15).

[89] D. Kalamkar et al. *A Study of BFLOAT16 for Deep Learning Training*. 2019-05-29. arXiv : 1905.12322. URL : https://arxiv.org/abs/1905.12322 (visité le 2026-10-15).

[90] C. E. Shannon. « Coding Theorems for a Discrete Source with a Fidelity Criterion ». In : *IRE National Convention Record*. T. 7. 4. Fonction distorsion-débit ; borne de la source gaussienne. 1959, p. 142-163.

[91] T. M. Cover et J. A. Thomas. *Elements of Information Theory*. 2e éd. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ : Wiley-Interscience, 2006. ISBN : 978-0-471-24195-9. DOI : 10.1002/047174882X. (Visité le 2026-10-15).

[92] S. P. Lloyd. « Least Squares Quantization in PCM ». In : *IEEE Transactions on Information Theory* 28.2 (1982). Rapport interne des Bell Labs de 1957, publié en 1982., p. 129-137. DOI : 10.1109/TIT.1982.1056489. (Visité le 2026-10-15).

[93] J. Max. « Quantizing for Minimum Distortion ». In : *IRE Transactions on Information Theory* 6.1 (1960), p. 7-12. DOI : 10.1109/TIT.1960.1057548. (Visité le 2026-10-15).

[94] P. F. Panter et W. Dite. « Quantization Distortion in Pulse-Count Modulation with Nonuniform Spacing of Levels ». In : *Proceedings of the IRE* 39.1 (1951), p. 44-48. DOI : 10.1109/JRPROC.1951.230419.

[95] M. W. Marcellin et T. R. Fischer. « Trellis Coded Quantization of Memoryless and Gauss-Markov Sources ». In : *IEEE Transactions on Communications* 38.1 (1990), p. 82-93. DOI : 10.1109/26.46532. (Visité le 2026-10-15).

[96] A. J. Viterbi. « Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm ». In : *IEEE Transactions on Information Theory* 13.2 (1967), p. 260-269. DOI : 10.1109/TIT.1967.1054010. (Visité le 2026-10-15).

[97] D. Lee et H. O. Song. *Q-Palette : Fractional-Bit Quantizers Toward Optimal Bit Allocation for Efficient LLM Deployment*. Première version 2025-09-24, dernière 2025-10-22 (2509.20214v2). NeurIPS 2025. 2025-09-24. arXiv : 2509.20214. URL : https://arxiv.org/abs/2509.20214 (visité le 2026-10-15).

[98] A. Fristrup, S. Pandey et A. Kariryaa. *ApexQuant : Data-Free Elastic Quantization by Residual Re-Isotropization*. Première version 2026-10-06, dernière 2026-10-06 (2610.07904v1). 25 pages, 6 figures. 2026-10-06. arXiv : 2610.07904. URL : https://arxiv.org/abs/2610.07904 (visité le 2026-10-15).

[99] A. Dadgarnia, S. Tabesh, M. Nikdan et al. *GSQ : Highly-Accurate Low-Precision Scalar Quantization for LLMs via Gumbel-Softmax Sampling*. Première version 2026-04-20, dernière 2026-05-15 (2604.18556v2). 2026-04-20. arXiv : 2604.18556. URL : https://arxiv.org/abs/2604.18556 (visité le 2026-10-15).

[100] *ml-explore/mlx issue 3043*. Fermée le 2026-01-26 par son auteur ; méthode non décrite. 2026-01-22. URL : https://github.com/ml-explore/mlx/issues/3043 (visité le 2026-10-15).

[101] H. Gish et J. N. Pierce. « Asymptotically efficient quantizing ». In : *IEEE Transactions on Information Theory* 14.5 (1968), p. 676-683. DOI : 10.1109/TIT.1968.1054193.

[102] F. Cheng, C. Guo, C. Wei et al. « Ecco : Improving Memory Bandwidth and Capacity for LLMs via Entropy-aware Cache Compression ». In : *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*. 2025-05-11, p. 793-807. arXiv : 2505.06901. URL : https://arxiv.org/abs/2505.06901 (visité le 2026-10-15).

[103] Suram. *Cacheable by Design ? Training Routers for Expert Locality*. un seul auteur ; incohérences internes relevées par l'étude 04. 2026-08-18. arXiv : 2608.18261. URL : https://arxiv.org/abs/2608.18261 (visité le 2026-10-15).

[104] ggml-org. *llama.cpp discussion 27149 : reading only the active experts*. communautaire. 2026-08-15. URL : https://github.com/ggml-org/llama.cpp/discussions/27149 (visité le 2026-10-15).

[105] S. Zhong et al. *AdapMoE : Adaptive Sensitivity-based Expert Gating and Management for Efficient MoE Inference*. 2024-08-19. arXiv : 2408.10284. URL : https://arxiv.org/abs/2408.10284 (visité le 2026-10-15).

[106] SmallThinker Team. *SmallThinker : A Family of Efficient Large Language Models Natively Trained for Local Deployment*. 2025-07. arXiv : 2507.20984. URL : https://arxiv.org/abs/2507.20984 (visité le 2026-10-15).

[107] Hoang et al. *SpecMD : Expert Caching for Mixture-of-Experts Decoding*. 2026-02-03. arXiv : 2602.03921. URL : https://arxiv.org/abs/2602.03921 (visité le 2026-10-15).

[108] Kim et al. *FlashMoE : Learned Expert Cache Replacement*. dernière version 2026-10-05. 2026-01-22. arXiv : 2601.17063. URL : https://arxiv.org/abs/2601.17063 (visité le 2026-10-15).

[109] A. Skliar et al. *Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference*. TMLR ; dernière version 2025-06-24. 2024-11-27. arXiv : 2412.00099. URL : https://arxiv.org/abs/2412.00099 (visité le 2026-10-15).

[110] A. Q. Jiang et al. *Mixtral of Experts*. 2024-01-08. arXiv : 2401.04088. URL : https://arxiv.org/abs/2401.04088 (visité le 2026-10-15).

[111] M. Elhoushi et al. *LayerSkip : Enabling Early Exit Inference and Self-Speculative Decoding*. ACL 2024 ; dernière version 2024-10-18. 2024-04-25. arXiv : 2404.16710. URL : https://arxiv.org/abs/2404.16710 (visité le 2026-10-15).

[112] J. Zhao et al. *QSpec : Speculative Decoding with Complementary Quantization Schemes*. EMNLP 2025 ; dernière version 2025-10-02. 2024-10-15. arXiv : 2410.11305. URL : https://arxiv.org/abs/2410.11305 (visité le 2026-10-15).

[113] Wu et al. *STree : Speculative Tree Decoding for Hybrid State-Space Models*. NeurIPS 2025 ; dernière version 2025-10-27. 2025-05-20. arXiv : 2505.14969. URL : https://arxiv.org/abs/2505.14969 (visité le 2026-10-15).

[114] Pan et al. *Making Every Verified Token Count (EVICT)*. dernière version 2026-09-05. 2026-05-01. arXiv : 2605.00342. URL : https://arxiv.org/abs/2605.00342 (visité le 2026-10-15).

[115] Han. *DraftExpert : Distilled Draft Experts for On-Demand MoE Inference*. 2026-07-27. arXiv : 2607.24434. URL : https://arxiv.org/abs/2607.24434 (visité le 2026-10-15).

[116] J.-Y. Lee et al. *CATS : Contextually-Aware Thresholding for Sparsity in Large Language Models*. dernière version 2024-11-03. 2024-04-12. arXiv : 2404.08763. URL : https://arxiv.org/abs/2404.08763 (visité le 2026-10-15).

[117] J. Liu et al. *Training-Free Activation Sparsity in Large Language Models (TEAL)*. ICLR 2025 ; dernière version 2025-02-25. 2024-08-26. arXiv : 2408.14690. URL : https://arxiv.org/abs/2408.14690 (visité le 2026-10-15).

[118] Zhang et al. *R-Sparse : Rank-Aware Activation Sparsity for Efficient LLM Inference*. ICLR 2025. 2025-04-28. arXiv : 2504.19449. URL : https://arxiv.org/abs/2504.19449 (visité le 2026-10-15).

[119] Y. Song et al. *Turbo Sparse : Achieving LLM SOTA Performance with Minimal Activated Parameters*. dernière version 2024-06-11. 2024-06-10. arXiv : 2406.05955. URL : https://arxiv.org/abs/2406.05955 (visité le 2026-10-15).

[120] ggml-org. *GGUF file format specification (docs/gguf.md)*. Spécification publique lue par l'étude 06. URL : https://github.com/ggml-org/ggml/blob/master/docs/gguf.md (visité le 2026-10-15).

[121] DeepSeek-AI. *DeepSeek-V4 : Towards Highly Efficient Million-Token Context Intelligence*. 2026-04-26. arXiv : 2606.19348. URL : https://arxiv.org/abs/2606.19348 (visité le 2026-10-15).

[122] OpenAI. *gpt-oss-120b and gpt-oss-20b Model Card*. 2025-08-08. arXiv : 2508.10925. URL : https://arxiv.org/abs/2508.10925 (visité le 2026-10-15).

[123] DeepSeek-AI. *Conditional Memory via Scalable Lookup (Engram)*. dernière version 2026-07-12. 2026-01-12. arXiv : 2601.07372. URL : https://arxiv.org/abs/2601.07372 (visité le 2026-10-15).

[124] Kimi Team. *Kimi K2 : Open Agentic Intelligence*. dernière version 2026-02-03. 2025-07-28. arXiv : 2507.20534. URL : https://arxiv.org/abs/2507.20534 (visité le 2026-10-15).

[125] GLM Team. *GLM-4.5 : Agentic, Reasoning, and Coding Foundation Models*. 2025-08-08. arXiv : 2508.06471. URL : https://arxiv.org/abs/2508.06471 (visité le 2026-10-15).

[126] Qwen Team. *Qwen3 Technical Report*. 2025-05-14. arXiv : 2505.09388. URL : https://arxiv.org/abs/2505.09388 (visité le 2026-10-15).

[127] ggml-org. *gpt-oss-20b-GGUF (gpt-oss-20b-MXFP4.gguf, 12 109 566 624 octets)*. Apache-2.0 ; empreinte SHA-256 vérifiée à la réception (REPRISE.md). 2025-08. URL : https://huggingface.co/ggml-org/gpt-oss-20b-GGUF (visité le 2026-10-15).

[128] ggml-org. *gpt-oss-120b-GGUF (gpt-oss-120b-MXFP4.gguf, 63 387 346 208 octets)*. Apache-2.0 ; empreinte SHA-256 vérifiée à la réception (REPRISE.md). 2025-08. URL : https://huggingface.co/ggml-org/gpt-oss-120b-GGUF (visité le 2026-10-15).

[129] W. H. R. Equitz et T. M. Cover. « Successive Refinement of Information ». In : *IEEE Transactions on Information Theory* 37.2 (1991), p. 269-275.

[130] L. Babaoglu, S. Chen et A. Khisti. *Multi-Bitwidth Quantization for LLMs Using Additive Codebooks*. v2 du 2026-09-26. Résumé seul lu : la source gaussienne est raffinable sans pénalité de débit sous une erreur quadratique pondérée ; un seul modèle sert plusieurs largeurs de bits. 2026-06-11. arXiv : 2606.12876. (Visité le 2026-10-15).

[131] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim et H.-J. Yoo. *SliceMoE : Bit-Sliced Expert Caching under Miss-Rate Constraints for Efficient MoE Inference*. v5 du 2026-09-08. Corps lu (HTML) : INT asymétrique G32, tranche haute par décalage (AMAT, troncature), experts critiques selon les scores du routeur, tranches basses évincées en premier, aucun codage entropique, flash UFS 3.1 et LPDDR4, accélérateur simulé ; DeepSeek-V2-Lite et Qwen1.5-MoE (éditeur, non reproduit). 2025-12-15. arXiv : 2512.12990. URL : https://arxiv.org/html/2512.12990 (visité le 2026-10-15).

[132] P. Tang, J. Liu, X. Hou et al. *HOBBIT : A Mixed Precision Expert Offloading System for Fast MoE Inference*. v2 du 2024-11-06. Corps lu (HTML) : deux copies par expert (FP16 et Int4, ou Int8 et Int2), choix selon la somme des poids de routage des experts mieux classés (seuils 0,6 et 0,9 sur Mixtral), 67/30/3 % ; RTX 4090 et Jetson AGX Orin ; perte au plus 1 point (éditeur, non reproduit). 2024-11-03. arXiv : 2411.01433. URL : https://arxiv.org/html/2411.01433v2 (visité le 2026-10-15).